```
Created on....10.04.2019    Last changed on/by........ 10.04.2019 VIVEK
Author........VIVEK

Title.........DPR Report

Type..................1     Reporting
Status................
Application...........
Class.................
Logical database......
  from application....
Selection screen......
Authorization group...
Development class.....ZABAP

  Start via variants
  Deactivate editor lock
X Fixed point arithmetic active
```

```
*&---------------------------------------------------------------------*
*& Report  ZPRA_DPR_REPORT
*&
*&---------------------------------------------------------------------*
*&
*&
*&---------------------------------------------------------------------*
REPORT ZPRA_DPR_REPORT.
INCLUDE zpra_dpr_report_top .
INCLUDE zpra_dpr_report_f01 .
START-OF-SELECTION .
  PERFORM clear_variables .
  PERFORM fetch_data .
  PERFORM process_gas_records  .
  PERFORM process_data .

```

```
*&---------------------------------------------------------------------*
*&  Include           ZPRA_DPR_REPORT_F01
*&---------------------------------------------------------------------*
*&---------------------------------------------------------------------*
*&      Form  FETCH_DATA
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*

FORM fetch_data .
*  DATA : lt_zpra_t_dly_prd  TYPE STANDARD TABLE OF zpra_t_dly_prd .
  DATA : lv_mrec_start_date  TYPE sy-datum,
         lv_monat            TYPE t009b-poper,
         lv_date1            TYPE sy-datum,
         lv_date2            TYPE sy-datum,
         lv_gjahr            TYPE gjahr,
         lv_index            TYPE sy-tabix,
         lv_combine_tar_date TYPE sy-datum,
         lv_min_date         TYPE sy-datum,
         lv_max_date         TYPE sy-datum,
         lv_year             TYPE gjahr.
  DATA : lt_zpra_c_prd_prof TYPE STANDARD TABLE OF ty_zpra_c_prd_prof .

  gv_rep_date = p_date .
  p_date = p_date - 1 .

  CALL FUNCTION 'CONVERSION_EXIT_SDATE_OUTPUT'
    EXPORTING
      input  = gv_rep_date
    IMPORTING
      output = gv_repdate_e.

  PERFORM add_product_range USING  :  c_prod_oil ,
                                      c_prod_con ,
                                      c_prod_lng ,
                                      c_prod_gas .
  PERFORM add_vl_type_range USING  :  'NET_PROD'  ,
                                      'GROSS_PROD',
                                      'GAS_INJ'   .
  PERFORM buiild_tar_code_range .
* Select those assets which are configured for DPR Report .
  SELECT *
    FROM zpra_c_dpr_prof
    INTO TABLE gt_zpra_c_dpr_prof
   WHERE product IN r_product[]
     AND display_opt NE ' ' .
  IF sy-subrc IS NOT INITIAL .
    MESSAGE 'No assets configured for DPR Report' TYPE 'E' .
  ENDIF.
  SELECT product
         asset
         block
    FROM zpra_c_prd_prof
    INTO TABLE          gt_zpra_c_prd_prof
     FOR ALL ENTRIES IN gt_zpra_c_dpr_prof
   WHERE product     EQ gt_zpra_c_dpr_prof-product
     AND asset       EQ gt_zpra_c_dpr_prof-asset .
  IF sy-subrc IS NOT INITIAL.
```

```
  62      MESSAGE 'No Assets found in product profile' TYPE 'E' .
  63    ENDIF.
  64    SORT gt_zpra_c_prd_prof .
  65    DELETE ADJACENT DUPLICATES FROM gt_zpra_c_prd_prof COMPARING ALL FIELD
      S .
  66    lv_year = p_date(4) .
  67    lv_year = lv_year - 5 .
  68    CONCATENATE lv_year '0401' INTO lv_min_date .
  69    lv_max_date  = p_date .
  70
  71    SELECT SINGLE periv
  72             FROM t001
  73             INTO gv_periv
  74            WHERE bukrs EQ 'OVL' .
  75    IF sy-subrc IS NOT INITIAL.
  76      MESSAGE 'Company Code OVL does not exist' TYPE 'E' .
  77    ENDIF.
  78
  79    CALL FUNCTION 'DATE_TO_PERIOD_CONVERT'
  80      EXPORTING
  81        i_date         = p_date
  82      " I_MONMIT       = 00
  83        i_periv        = gv_periv
  84      IMPORTING
  85        e_buper        = lv_monat
  86        e_gjahr        = gv_current_gjahr
  87      EXCEPTIONS
  88        input_false    = 01
  89        t009_notfound  = 02
  90        t009b_notfound = 03.
  91
  92    IF sy-subrc <> 0.
  93      MESSAGE 'Error getting Period' TYPE 'E' .
  94    ENDIF.
  95
  96    CALL FUNCTION 'FIRST_AND_LAST_DAY_IN_YEAR_GET'
  97      EXPORTING
  98        i_gjahr        = gv_current_gjahr
  99        i_periv        = gv_periv
 100      IMPORTING
 101        e_first_day    = gv_year_start_date
 102        e_last_day     = gv_year_end_date
 103      EXCEPTIONS
 104        input_false    = 1
 105        t009_notfound  = 2
 106        t009b_notfound = 3
 107        OTHERS         = 4.
 108    IF sy-subrc <> 0.
 109      MESSAGE 'Unexpected Internal Error' TYPE 'E'.
 110    ENDIF.
 111    lv_max_date = gv_year_end_date .
 112    PERFORM remove_expired_blocks TABLES gt_zpra_c_prd_prof USING  lv_min_
      date lv_max_date .
 113
 114    CALL FUNCTION 'CCM_GO_BACK_MONTHS'
 115      EXPORTING
 116        currdate   = p_date
 117        backmonths = '001'
 118      IMPORTING
 119        newdate    = gv_month_back_datum.
 120
```

```
  CALL FUNCTION 'HR_JP_MONTH_BEGIN_END_DATE'
    EXPORTING
      iv_date             = p_date
    IMPORTING
      ev_month_begin_date = gv_month_begin_datum
      ev_month_end_date   = gv_month_end_datum.
  gv_month_days_till_today = p_date - gv_month_begin_datum + 1 .

  CALL FUNCTION 'HR_JP_MONTH_BEGIN_END_DATE'
    EXPORTING
      iv_date             = gv_month_back_datum
    IMPORTING
      ev_month_begin_date = gv_month_back_begin_datum
      ev_month_end_date   = gv_month_back_end_datum.

  gv_current_month_days = gv_month_end_datum - gv_month_begin_datum + 1.
  SELECT SINGLE ltx
           FROM t247
           INTO gv_month_name
          WHERE spras = sy-langu
            AND mnr EQ p_date+4(2).

  SELECT SINGLE ltx
           FROM t247
           INTO gv_last_month_name
          WHERE spras = sy-langu
            AND mnr EQ gv_month_back_begin_datum+4(2).

* SELECT *
*   FROM zdpr_gas_combine
*   INTO TABLE gt_zdpr_gas_combine
*  WHERE xfeld EQ 'X' .
  LOOP AT gt_zpra_c_dpr_prof INTO gs_zpra_c_dpr_prof WHERE product EQ c_prod_gas
                                                           AND display_opt EQ 'C' .
    gs_zdpr_gas_combine-asset =  gs_zpra_c_dpr_prof-asset .
    APPEND gs_zdpr_gas_combine TO gt_zdpr_gas_combine.
  ENDLOOP.
  SORT gt_zdpr_gas_combine BY asset .

  REFRESH r_combine_asset[] .
  LOOP AT gt_zdpr_gas_combine INTO gs_zdpr_gas_combine.
    r_combine_asset-sign   = 'I' .
    r_combine_asset-option = 'EQ' .
    r_combine_asset-low    = gs_zdpr_gas_combine-asset .
    APPEND r_combine_asset .
  ENDLOOP.

  SELECT *
    FROM zpra_t_dly_prd
    INTO TABLE gt_zpra_t_dly_prd
     FOR ALL ENTRIES IN gt_zpra_c_prd_prof
   WHERE production_date LE p_date
     AND production_date GE gv_month_back_datum
     AND product     EQ gt_zpra_c_prd_prof-product
     AND asset       EQ gt_zpra_c_prd_prof-asset
     AND block       EQ gt_zpra_c_prd_prof-block
     AND prd_vl_type IN r_prd_vl_type[] .

  IF sy-subrc IS NOT INITIAL.
```

```
      MESSAGE 'No data found for the selection' TYPE 'E' .
    ENDIF.


    SORT  gt_zpra_t_dly_prd BY production_date product asset .

  * lt_zpra_t_dly_prd = gt_zpra_t_dly_prd .
  * SORT lt_zpra_t_dly_prd BY asset block product prd_vl_type.
  * DELETE ADJACENT DUPLICATES FROM lt_zpra_t_dly_prd COMPARING asset bloc
    k product prd_vl_type .

    SELECT asset
          block
          vld_frm
          vld_to
          pi
          prod_start_date
     FROM zpra_t_prd_pi
     INTO TABLE gt_zpra_t_prd_pi
      FOR ALL ENTRIES IN gt_zpra_c_prd_prof
    WHERE asset EQ gt_zpra_c_prd_prof-asset
      AND block EQ gt_zpra_c_prd_prof-block
      AND vld_frm LE p_date
      AND vld_to  GE gv_month_back_datum .

    SORT gt_zpra_t_prd_pi BY asset block vld_frm vld_to .

    CALL FUNCTION 'CCM_GO_BACK_MONTHS'
      EXPORTING
        currdate   = gv_month_back_datum
        backmonths = '020'
      IMPORTING
        newdate    = lv_mrec_start_date.

    SELECT SINGLE periv
             FROM t001
             INTO gv_periv
            WHERE bukrs EQ 'OVL' .
    IF sy-subrc IS NOT INITIAL.
      MESSAGE 'Company Code OVL does not exist' TYPE 'E' .
    ENDIF.
    CALL FUNCTION 'DATE_TO_PERIOD_CONVERT'
      EXPORTING
        i_date         = p_date
      " I_MONMIT       = 00
        i_periv        = gv_periv
      IMPORTING
        e_buper        = lv_monat
        e_gjahr        = gv_current_gjahr
      EXCEPTIONS
        input_false    = 01
        t009_notfound  = 02
        t009b_notfound = 03.

    IF sy-subrc <> 0.
      MESSAGE 'Error getting Period' TYPE 'E' .
    ENDIF.
    gv_current_calendar_gjahr = p_date(4) .
    gv_next_gjahr   = gv_current_gjahr + 1 .
    gv_last_gjahr   = gv_current_gjahr - 1 .
    gv_5_back_gjahr = gv_current_gjahr - 5 .
```

```
240
241    CALL FUNCTION 'FIRST_AND_LAST_DAY_IN_YEAR_GET'
242      EXPORTING
243        i_gjahr        = gv_current_gjahr
244        i_periv        = gv_periv
245      IMPORTING
246        e_first_day    = gv_year_start_date
247        e_last_day     = gv_year_end_date
248      EXCEPTIONS
249        input_false    = 1
250        t009_notfound  = 2
251        t009b_notfound = 3
252        OTHERS         = 4.
253    IF sy-subrc <> 0.
254      MESSAGE 'Unexpected Internal Error' TYPE 'E'.
255    ENDIF.
256    gv_total_days_in_year = gv_year_end_date - gv_year_start_date + 1 .
257    gv_current_gjahr_days = gv_total_days_in_year .
258    gv_days_gone_in_year  =  p_date - gv_year_start_date + 1 .
259  *  gv_days_left_in_year  =  gv_year_end_date - p_date   + 1 .
260    gv_days_left_in_year  =  gv_year_end_date - p_date   .
261    gv_current_monat = lv_monat+1(2) .
262
263    lv_gjahr = gv_year_start_date(4) - 1 .
264    CONCATENATE lv_gjahr gv_year_start_date+4(4) INTO gv_last_year_s_date
     .
265
266    lv_gjahr = p_date(4) - 1 .
267    CONCATENATE lv_gjahr p_date+4(4) INTO gv_last_year_t_date .
268
269    gv_days_gone_in_last_year  =  gv_last_year_t_date - gv_last_year_s_dat
     e + 1 .
270
271    CALL FUNCTION 'FIRST_AND_LAST_DAY_IN_YEAR_GET'
272      EXPORTING
273        i_gjahr        = gv_last_gjahr
274        i_periv        = gv_periv
275      IMPORTING
276        e_first_day    = lv_date1
277        e_last_day     = lv_date2
278      EXCEPTIONS
279        input_false    = 1
280        t009_notfound  = 2
281        t009b_notfound = 3
282        OTHERS         = 4.
283    IF sy-subrc <> 0.
284      MESSAGE 'Unexpected Internal Error' TYPE 'E'.
285    ENDIF.
286    gv_last_gjahr_days = lv_date2 - lv_date1 + 1 .
287
288    CALL FUNCTION 'DATE_TO_PERIOD_CONVERT'
289      EXPORTING
290        i_date         = lv_mrec_start_date
291      " I_MONMIT       = 00
292        i_periv        = gv_periv
293      IMPORTING
294        e_buper        = lv_monat
295        e_gjahr        = gv_mrec_gjahr_start
296      EXCEPTIONS
297        input_false    = 01
298        t009_notfound  = 02
```

```
      t009b_notfound = 03.

  IF sy-subrc <> 0.
    MESSAGE 'Error getting MREC Date' TYPE 'E' .
  ENDIF.
  gv_mrec_monat_start = lv_monat+1(2) .

  CALL FUNCTION 'DATE_TO_PERIOD_CONVERT'
    EXPORTING
      i_date         = gv_month_back_datum
    " I_MONMIT       = 00
      i_periv        = gv_periv
    IMPORTING
      e_buper        = lv_monat
      e_gjahr        = gv_mrec_gjahr_end
    EXCEPTIONS
      input_false    = 01
      t009_notfound  = 02
      t009b_notfound = 03.

  IF sy-subrc <> 0.
    MESSAGE 'Error getting MREC Date' TYPE 'E' .
  ENDIF.
  gv_month_back_calendar_gjahr = gv_month_back_datum(4) .
  gv_month_back_gjahr = gv_mrec_gjahr_end .
  gv_month_back_monat = lv_monat+1(2) .
  gv_mrec_monat_end = lv_monat+1(2) .
  IF gv_mrec_gjahr_start EQ gv_mrec_gjahr_end .
    SELECT gjahr
           monat
           asset
           block
           product
           prd_vl_type
           prod_vl_qty1
           prod_vl_uom1
           prod_vl_qty2
           prod_vl_uom2
      FROM zpra_t_mrec_prd
      INTO TABLE gt_zpra_t_mrec_prd
       FOR ALL ENTRIES IN gt_zpra_c_prd_prof
     WHERE asset        EQ gt_zpra_c_prd_prof-asset
       AND block        EQ gt_zpra_c_prd_prof-block
       AND product      EQ gt_zpra_c_prd_prof-product
       AND prd_vl_type  IN r_prd_vl_type[]
       AND gjahr EQ gv_mrec_gjahr_start
       AND monat GE gv_mrec_monat_start
       AND monat LE gv_mrec_monat_end .
  ELSE.
    SELECT gjahr
           monat
           asset
           block
           product
           prd_vl_type
           prod_vl_qty1
           prod_vl_uom1
           prod_vl_qty2
           prod_vl_uom2
      FROM zpra_t_mrec_prd
      INTO TABLE gt_zpra_t_mrec_prd
```

```
        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
      WHERE asset        EQ gt_zpra_c_prd_prof-asset
        AND block        EQ gt_zpra_c_prd_prof-block
        AND product      EQ gt_zpra_c_prd_prof-product
        AND prd_vl_type  IN r_prd_vl_type[]
*     AND ( ( gjahr EQ gv_mrec_gjahr_start AND
*           monat GE gv_mrec_monat_start ) OR
*           ( gjahr EQ gv_mrec_gjahr_end AND
*           monat LE gv_mrec_monat_end ) ) .
        AND ( gjahr GE gv_mrec_gjahr_start AND
              gjahr LE gv_mrec_gjahr_end ) .
    DELETE gt_zpra_t_mrec_prd WHERE gjahr EQ gv_mrec_gjahr_end   AND monat GT gv_mrec_monat_end   .
    DELETE gt_zpra_t_mrec_prd WHERE gjahr EQ gv_mrec_gjahr_start AND monat LT gv_mrec_monat_start .
  ENDIF.

  SELECT dn_no
         dn_de
    FROM zoiu_pr_dn
    INTO TABLE gt_asset_desc
     FOR ALL ENTRIES IN gt_zpra_c_prd_prof
   WHERE dn_no EQ gt_zpra_c_prd_prof-asset .

  SORT gt_asset_desc BY asset .

  DELETE gt_zpra_t_mrec_prd WHERE product EQ c_prod_gas .
  SORT gt_zpra_t_mrec_prd BY product ASCENDING asset ASCENDING gjahr DESCENDING monat DESCENDING block ASCENDING prd_vl_type ASCENDING .
  IF r_tar_code[] IS NOT INITIAL.
    SELECT asset
           block
           tar_code
           vld_frm
           vld_to
           pi
      FROM zpra_t_tar_pi
      INTO TABLE gt_zpra_t_tar_pi
        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
      WHERE asset EQ gt_zpra_c_prd_prof-asset
        AND block EQ gt_zpra_c_prd_prof-block
        AND vld_frm LE gv_month_end_datum
        AND vld_to  GE gv_month_begin_datum .
    SORT gt_zpra_t_tar_pi BY asset block tar_code vld_frm .

    SELECT tar_code
           gjahr
           monat
           asset
           block
           product
           prod_vl_type_cd
           tar_qty
           uom
      FROM zpra_t_prd_tar
      INTO TABLE gt_zpra_t_prd_tar
        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
      WHERE gjahr EQ gv_current_gjahr
        AND monat EQ gv_current_monat
        AND asset EQ gt_zpra_c_prd_prof-asset
        AND block EQ gt_zpra_c_prd_prof-block
```

```
        AND product EQ gt_zpra_c_prd_prof-product
        AND prod_vl_type_cd IN r_prd_vl_type[]
        AND tar_code IN r_tar_code[] .
     SORT gt_zpra_t_prd_tar BY gjahr monat asset block product prod_vl_type_cd tar_code .

     SELECT *
       FROM zpra_t_tar_cf
       INTO TABLE gt_zpra_t_tar_cf
        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
      WHERE gjahr EQ gv_current_gjahr
        AND asset EQ gt_zpra_c_prd_prof-asset
        AND block EQ gt_zpra_c_prd_prof-block
        AND product EQ gt_zpra_c_prd_prof-product .

     SORT gt_zpra_t_tar_cf BY gjahr asset block product .
   ENDIF.

   SELECT *
     FROM zpra_t_tar_cf
     INTO TABLE gt_cf
      FOR ALL ENTRIES IN gt_zpra_c_prd_prof
    WHERE   asset EQ gt_zpra_c_prd_prof-asset
      AND   block EQ gt_zpra_c_prd_prof-block
      AND product EQ gt_zpra_c_prd_prof-product .

   SORT gt_cf BY gjahr product asset block  .

   PERFORM fetch_section2a2_data .

   SELECT asset
          tar_code
          MIN( vld_frm )
     FROM zpra_t_tar_pi
     INTO TABLE gt_tar_start_dates
    WHERE tar_code IN r_tar_code[]
    GROUP BY asset tar_code .
   SORT gt_tar_start_dates BY asset tar_code .

   lt_zpra_c_prd_prof = gt_zpra_c_prd_prof .
   SORT lt_zpra_c_prd_prof BY asset  .

   LOOP AT gt_tar_start_dates INTO gs_tar_start_dates.
     lv_index = sy-tabix .
     READ TABLE lt_zpra_c_prd_prof TRANSPORTING NO FIELDS WITH KEY asset = gs_tar_start_dates-asset BINARY SEARCH .
     IF sy-subrc IS NOT INITIAL.
       DELETE gt_tar_start_dates INDEX lv_index .
     ENDIF.
   ENDLOOP.
 ENDFORM.
 *&———————————————————————————————————————————————————————————*
 *&      Form  PROCESS_DATA
 *&———————————————————————————————————————————————————————————*
 *       text
 *———————————————————————————————————————————————————————————*
 *  —>  p1        text
 *  <—  p2        text
 *———————————————————————————————————————————————————————————*
 FORM process_data .
   PERFORM start_excel      .
```

```
  PERFORM process_sec1_data .
  PERFORM display_section1 .

  PERFORM process_sec2_data .

  PERFORM process_sec3_data .

  PERFORM process_sec4_data .

  CALL METHOD OF go_worksheet2 'ACTIVATE'.

  PERFORM process_sec5_data .

  PERFORM process_sec6_data .

  CALL METHOD OF go_worksheet 'ACTIVATE'.

  PERFORM set_columns_width .

  PERFORM border_cells .
  PERFORM set_cell_formats .
  PERFORM finalize_worksheet .
ENDFORM.
*&———————————————————————————————————————————————————————*
*&      Form  PROCESS_SEC1_DATA
*&———————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————*
FORM process_sec1_data .
  gt_zpra_t_dly_prd2 = gt_zpra_t_dly_prd.
  SORT gt_zpra_t_dly_prd2 BY product asset  .
  DELETE ADJACENT DUPLICATES FROM gt_zpra_t_dly_prd2 COMPARING product asset .
  PERFORM calculated_wtd_pi .
  PERFORM calculated_wtd_cf .
  PERFORM prepare_dynamic_table_sec1 .
  PERFORM fill_dynamic_table_sec1    .

ENDFORM.
*&———————————————————————————————————————————————————————*
*&      Form  PROCESS_SEC2_DATA
*&———————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————*
FORM process_sec2_data .

  PERFORM process_sec2a_data .
* Section 2b is same as 2a1 only difference being section 2a1 takes only
* current month data, 2b shows whole year data
  PERFORM fetch_data_section2b .
  PERFORM prepare_dynamic_table_sec2b .
  PERFORM process_sec2b_data .
* Section 2c is same as 2b only difference being section  2b shows whole year data
```

```
* 2c shows YTD data
  PERFORM fetch_data_section2c .
  PERFORM prepare_dynamic_table_sec2c .
  PERFORM process_sec2c_data .

  PERFORM fetch_data_section2d .
  PERFORM process_gas_records_2d .
  PERFORM prepare_dynamic_table_sec2d .
  PERFORM fill_dynamic_table_sec2d .
  PERFORM display_section2d .

  IF p_bbd IS NOT INITIAL OR
     p_tmd IS NOT INITIAL OR
     p_bmd IS NOT INITIAL .
    PERFORM prepare_dynamic_table_sec2e .
    PERFORM fill_dynamic_table_sec2e .
    PERFORM display_section2e .
  ENDIF.
  PERFORM fetch_data_section2f .
  PERFORM process_gas_records_2f .
  PERFORM prepare_dynamic_table_sec2f .
  PERFORM fill_dynamic_table_sec2f .
  PERFORM display_section2f .

ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  PROCESS_SEC3_DATA
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM process_sec3_data .

  PERFORM display_section3_header .

  PERFORM prepare_dynamic_table_sec3a .
  PERFORM fill_dynamic_table_sec3a .
  PERFORM display_section3a .

  PERFORM prepare_dynamic_table_sec3b .
  PERFORM process_sec3b_data .
  PERFORM fill_dynamic_table_sec3b .
  PERFORM display_section3b .

  PERFORM fetch_data_section3c .
  PERFORM process_gas_records_3c .
  PERFORM prepare_dynamic_table_sec3c .
  PERFORM fill_dynamic_table_sec3c .
  PERFORM display_section3c .

  PERFORM prepare_dynamic_table_sec3d .
  PERFORM fill_dynamic_table_sec3d .
  PERFORM display_section3d .

  PERFORM prepare_dynamic_table_sec3e .
  PERFORM fill_dynamic_table_sec3e .
  PERFORM display_section3e .

  PERFORM fetch_data_section3f .
```

```
  PERFORM process_gas_records_3f .
  PERFORM prepare_dynamic_table_sec3f .
  PERFORM fill_dynamic_table_sec3f .
  PERFORM display_section3f .

  gv_sec3_end_row = gv_row .
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  PROCESS_SEC4_DATA
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM process_sec4_data .
  PERFORM fetch_data_section4a .
  PERFORM process_gas_records_4a .
  PERFORM prepare_dynamic_table_sec4a .
  PERFORM fill_dynamic_table_sec4a .

  PERFORM display_section4a .
  PERFORM display_section4b .

  PERFORM display_run_date_time .

ENDFORM.
FORM process_sec5_data .
  PERFORM process_sec5a_data .
ENDFORM .
FORM process_sec5a_data .
  PERFORM prepare_dynamic_table_sec5a .
  PERFORM fetch_data_section5a .
  PERFORM process_gas_records_5a .
  PERFORM fill_dynamic_table_sec5a .
  PERFORM display_section5a .
  PERFORM show_progress USING '90' .
ENDFORM .
FORM process_sec6_data .
  PERFORM prepare_dynamic_table_sec6 .
  PERFORM fill_dynamic_table_sec6 .
  PERFORM display_section6 .
ENDFORM .

*&---------------------------------------------------------------------*
*&      Form  PREPARE_DYNAMIC_TABLE
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM prepare_dynamic_table_sec1 .
  DATA : lv_col_name       TYPE lvc_fname,
         lv_product        TYPE zpra_t_dly_prd-product,
         lv_no_column      TYPE c,
         lv_combine_column TYPE c.
  DATA : lt_zpra_c_prd_prof  TYPE STANDARD TABLE OF ty_zpra_c_prd_prof,
         lt_zpra_c_prd_prof2 TYPE STANDARD TABLE OF ty_zpra_c_prd_prof.
  CLEAR gv_pos .
  REFRESH gt_dyn_fcat .
```

```
  PERFORM add_dyn_field USING 'COL01' 'Column1' 30 .
  PERFORM add_dyn_field USING 'COL02' 'Column1' 30 .

  lt_zpra_c_prd_prof = gt_zpra_c_prd_prof .
  SORT lt_zpra_c_prd_prof BY product asset.
  DELETE ADJACENT DUPLICATES FROM lt_zpra_c_prd_prof COMPARING product asset .

  lt_zpra_c_prd_prof2 = lt_zpra_c_prd_prof .
  DELETE lt_zpra_c_prd_prof  WHERE product EQ c_prod_gas .
  DELETE lt_zpra_c_prd_prof2 WHERE product NE c_prod_gas .
  APPEND LINES OF lt_zpra_c_prd_prof2 TO lt_zpra_c_prd_prof .

  LOOP AT lt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof.
    CLEAR lv_no_column .
    IF lv_product IS NOT INITIAL.
      IF lv_product NE gs_zpra_c_prd_prof-product.
        IF lv_product EQ '722000004' AND lv_combine_column IS NOT INITIAL.
          gv_len = strlen( lv_product ) .
          CONCATENATE lv_product(gv_len) '-' 'COMBINE' INTO lv_col_name .
          PERFORM add_dyn_field USING lv_col_name lv_col_name 35 .
        ENDIF.
        gv_len = strlen( lv_product ) .
        CONCATENATE lv_product(gv_len) '-' 'TOTAL' INTO lv_col_name .
        PERFORM add_dyn_field USING lv_col_name lv_col_name 35 .
      ENDIF.
    ENDIF.
    IF gs_zpra_c_prd_prof-product EQ c_prod_gas.
      READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_c_prd_prof-asset BINARY SEARCH .
      IF sy-subrc IS INITIAL .
        lv_no_column = 'X' .
        lv_combine_column = 'X' .
      ENDIF.
    ENDIF.
    IF lv_no_column IS INITIAL .
      gv_len = strlen( gs_zpra_c_prd_prof-product ) .
      CONCATENATE gs_zpra_c_prd_prof-product(gv_len) '-' gs_zpra_c_prd_prof-asset INTO lv_col_name .
      PERFORM add_dyn_field USING lv_col_name lv_col_name 35 .
    ENDIF.
    lv_product = gs_zpra_c_prd_prof-product .
  ENDLOOP.

  IF lv_product EQ c_prod_gas AND lv_combine_column IS NOT INITIAL.
    gv_len = strlen( lv_product ) .
    CONCATENATE lv_product(gv_len) '-' 'COMBINE' INTO lv_col_name .
    PERFORM add_dyn_field USING lv_col_name lv_col_name 35 .
  ENDIF.

  gv_len = strlen( gs_zpra_c_prd_prof-product ) .
  CONCATENATE gs_zpra_c_prd_prof-product(gv_len) '-' 'TOTAL' INTO lv_col_name .
  PERFORM add_dyn_field USING lv_col_name lv_col_name 35 .

  CONCATENATE 'GRAND' '-' 'TOTAL' INTO lv_col_name .
  PERFORM add_dyn_field USING lv_col_name lv_col_name 35 .

  gv_table_columns = gv_pos .
```

```
** Create a dynamic internal table with this structure.
  CALL METHOD cl_alv_table_create=>create_dynamic_table
    EXPORTING
*     i_style_table             = #X#
      it_fieldcatalog           = gt_dyn_fcat
    IMPORTING
      ep_table                  = gt_dyn_table
    EXCEPTIONS
      generate_subpool_dir_full = 1
      OTHERS                    = 2.
  IF sy-subrc NE 0.
    MESSAGE 'Unexpected Internal Error' TYPE 'E' .
  ELSE.
* Assign the new table to field symbol
    ASSIGN gt_dyn_table->* TO <gfs_dyn_table>.
  ENDIF.
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  ADD_DYN_FIELD
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      -->P_COLPOS  text
*      -->P_COL01  text
*      -->P_COLTEXT  text
*      -->P_OUTPUTLEN  text
*----------------------------------------------------------------------*
FORM add_dyn_field  USING    p_colname
                             p_coltext
                             p_outputlen.

  gv_pos = gv_pos + 1.
  gs_dyn_fcat-fieldname = p_colname .
  gs_dyn_fcat-outputlen = p_outputlen.
*gs_dyn_fcat-tabname   = #IT_DEMO#.
  gs_dyn_fcat-coltext   = p_coltext.
  gs_dyn_fcat-col_pos   = gv_pos.
*gs_dyn_fcat-key = #X#.
*gs_dyn_fcat-key_sel = #X#.
  APPEND gs_dyn_fcat TO gt_dyn_fcat.
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  FILL_DYNAMIC_TABLE
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM fill_dynamic_table_sec1 .
  DATA : lv_col_name      TYPE lvc_fname,
         lv_combine_field TYPE c,
         lv_index         TYPE sy-tabix,
         lv_product       TYPE char100,
         lv_asset         TYPE char100.
  DATA : lt_zpra_t_dly_prd     TYPE STANDARD TABLE OF zpra_t_dly_prd.

  SORT gt_wtd_pi BY production_date product asset .
  PERFORM add_dummy_data TABLES lt_zpra_t_dly_prd .
*  BREAK JVUSER01 .
  LOOP AT lt_zpra_t_dly_prd INTO gs_zpra_t_dly_prd.
```

```
774      lv_index = sy-tabix .
775      CLEAR lv_combine_field .
776      AT NEW production_date .
777        IF lv_index NE 1 .
778          PERFORM fill_null_values_with_previous .
779        ENDIF.
780        APPEND INITIAL LINE TO <gfs_dyn_table> ASSIGNING <gfs_dyn_line> .
781        ASSIGN COMPONENT 'COL01' OF STRUCTURE <gfs_dyn_line> TO <gfs_field
     > .
782        CALL FUNCTION 'CONVERSION_EXIT_SDATE_OUTPUT'
783          EXPORTING
784            input  = gs_zpra_t_dly_prd-production_date
785          IMPORTING
786            output = <gfs_field>.
787        IF sy-subrc <> 0.
788          MESSAGE 'Unexpteced Internal Error' TYPE 'E' .
789        ENDIF.
790        REPLACE ALL OCCURRENCES OF '.' IN <gfs_field> WITH '-' .
791      ENDAT .
792      IF gs_zpra_t_dly_prd-comments EQ 'DUMMYDPRDATA'.
793        CONTINUE.
794      ENDIF.
795  *    PERFORM convert_non_gas_units CHANGING gs_zpra_t_dly_prd.
796      PERFORM convert_non_gas_units_2a2 CHANGING gs_zpra_t_dly_prd.
797
798      IF gs_zpra_t_dly_prd-product = c_prod_gas.
799        READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY a
     sset = gs_zpra_t_dly_prd-asset BINARY SEARCH .
800        IF  sy-subrc IS INITIAL.
801          lv_combine_field = 'X' .
802        ENDIF.
803      ENDIF.
804      PERFORM get_constorium_multipliers USING  gs_zpra_t_dly_prd lv_combi
     ne_field.
805      PERFORM get_constorium_multipliers_2a2 USING '1' .
806
807  * Individual Column..
808      gv_len = strlen( gs_zpra_t_dly_prd-product ) .
809      IF lv_combine_field IS INITIAL.
810        CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' gs_zpra_t_dly_pr
     d-asset INTO lv_col_name .
811      ELSE.
812        CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'COMBINE'
             INTO lv_col_name .
813      ENDIF.
814      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
     ld> .
815      IF <gfs_field> IS ASSIGNED.
816        <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_in
     dividual_mul.
817        UNASSIGN <gfs_field> .
818      ENDIF.
819  * Product Total..
820      CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'TOTAL'
             INTO lv_col_name .
821      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
     ld> .
822      IF <gfs_field> IS ASSIGNED.
823        <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_to
     tal_mul .
824        UNASSIGN <gfs_field> .
```

```
    ENDIF.
* Grand Total
    CONCATENATE 'GRAND' '-' 'TOTAL'
         INTO lv_col_name .
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_grand_total_mul.
      UNASSIGN <gfs_field> .
    ENDIF.
  ENDLOOP.
  PERFORM fill_null_values_with_previous .

  LOOP AT <gfs_dyn_table> ASSIGNING <gfs_dyn_line> .
    DO .
      lv_index = sy-index .
      ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      IF sy-subrc IS NOT INITIAL.
        EXIT .
      ELSE.
        IF lv_index GT 2.
          IF p_mb IS NOT INITIAL.
            <gfs_field> = <gfs_field> / 1000000 .
          ENDIF.
          IF p_bmd IS NOT INITIAL.
            READ TABLE gt_dyn_fcat INTO gs_dyn_fcat WITH KEY col_pos = lv_index BINARY SEARCH .
            IF sy-subrc IS INITIAL.
              SPLIT gs_dyn_fcat-fieldname AT '-' INTO lv_product lv_asset .
              IF lv_product EQ c_prod_gas.
                <gfs_field> = <gfs_field> / 6290 .
              ENDIF.
            ENDIF.
          ENDIF.
          IF <gfs_field> LT 0 .
            <gfs_field> = <gfs_field> * -1 .
            CONDENSE <gfs_field>.
            CONCATENATE '-' <gfs_field> INTO <gfs_field> .
          ENDIF.
        ENDIF.
      ENDIF.
    ENDDO.
  ENDLOOP.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  DISPLAY_SECTION1
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM display_section1 .
  PERFORM display_section1_header .
  PERFORM display_section1_data   .
  PERFORM formatting_section1 .
ENDFORM.
FORM display_section2a1 .
```

```
  PERFORM display_sec2a_targets .
ENDFORM.
FORM display_section2a2 .
  DATA lv_lines TYPE sy-tabix .
  DESCRIBE TABLE <gfs_sec2a2_table> LINES lv_lines .
  gv_row = gv_row + 1 .
  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row + lv_lines - 1 .
  gv_e_col = gv_s_col + gv_table_columns - 1.
  PERFORM prepare_section2a2_paste_data .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM paste_data .

  GET PROPERTY OF go_range 'interior' = go_interior .
  SET PROPERTY OF go_interior 'Color' = gv_sec2a2_colour .
  gv_row = gv_e_row .

  PERFORM select_range USING gv_s_row 2 gv_e_row 2  .
  PERFORM set_numberformat USING 'mmm yyyy'.

  PERFORM select_range USING gv_s_row 1 gv_e_row 1  .
  CALL METHOD OF go_range 'Merge' .
  PERFORM set_range USING 'Prod. : MTD Actual'  0.

ENDFORM.
FORM display_section2a3 .
  DATA lv_lines TYPE sy-tabix .
  DESCRIBE TABLE <gfs_sec2a3_table> LINES lv_lines .
  gv_row = gv_row + 1 .
  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row + lv_lines - 1 .
  gv_e_col = gv_s_col + gv_table_columns - 1.
  PERFORM prepare_section2a3_paste_data .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM paste_data .

  GET PROPERTY OF go_range 'interior' = go_interior .
  SET PROPERTY OF go_interior 'Color' = gv_sec2a3_colour .
  gv_row = gv_e_row .

  PERFORM select_range USING gv_row 2 gv_row 2  .
  PERFORM set_numberformat USING 'mmm yyyy'.

*  PERFORM select_range USING gv_s_row 1 gv_e_row 1 .
*  PERFORM set_range USING 'Monthly Actual' 0.

ENDFORM.
FORM display_section2d .
  DATA lv_lines TYPE sy-tabix .
  DESCRIBE TABLE <gfs_sec2d_table> LINES lv_lines .
  gv_row = gv_row + 1 .
  gv_sec2d_start_row = gv_row .
  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row + lv_lines - 1 .
  gv_e_col = gv_s_col + gv_table_columns - 1.
  PERFORM prepare_section2d_paste_data .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM paste_data .
```

```
941
942    GET PROPERTY OF go_range 'interior' = go_interior .
943    SET PROPERTY OF go_interior 'Color' = gv_sec2d_colour .
944    gv_row = gv_e_row .
945
946  *  PERFORM select_range USING gv_row 2 gv_row 2  .
947  *  PERFORM set_numberformat USING 'mmmm yyyy'.
948
949    PERFORM select_range USING gv_s_row 1 gv_e_row 1 .
950    CALL METHOD OF go_range 'Merge' .
951    PERFORM set_range USING 'YTD Actual Prod.' 0.
952  ENDFORM.
953  FORM display_section2f .
954    DATA lv_lines TYPE sy-tabix .
955    DESCRIBE TABLE <gfs_sec2f_table> LINES lv_lines .
956    gv_row = gv_row + 1 .
957    gv_sec2f_start_row = gv_row .
958    gv_s_row = gv_row .
959    gv_s_col = 1 .
960    gv_e_row = gv_s_row + lv_lines - 1 .
961    gv_e_col = gv_s_col + gv_table_columns - 1.
962    PERFORM prepare_section2f_paste_data .
963    PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
964    PERFORM paste_data .
965
966    GET PROPERTY OF go_range 'interior' = go_interior .
967    SET PROPERTY OF go_interior 'Color' = gv_sec2f_colour .
968    gv_row = gv_e_row .
969    gv_sec2_end_row = gv_row .
970  *  PERFORM select_range USING gv_row 2 gv_row 2  .
971  *  PERFORM set_numberformat USING 'mmmm yyyy'.
972  ENDFORM.
973  FORM display_section3a .
974    DATA lv_lines TYPE sy-tabix .
975    DESCRIBE TABLE <gfs_sec3a_table> LINES lv_lines .
976  *————————————————Changes to show blank line if target data is not availa
     ble————————*
977  *  CHECK lv_lines IS NOT INITIAL .
978    IF lv_lines IS INITIAL.
979      lv_lines = 1.
980    ENDIF.
981  *————————————————End of changes by Abhishek——————————TR OCDK904738———————
     ————————————————*
982    gv_row = gv_row + 1 .
983    gv_sec3a_start_row = gv_row .
984    gv_s_row = gv_row .
985    gv_s_col = 1 .
986    gv_e_row = gv_s_row + lv_lines - 1 .
987    gv_e_col = gv_s_col + gv_table_columns - 1.
988    PERFORM prepare_section3a_paste_data .
989    PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
990  *————————————————Changes to show blank line if target data is not availa
     ble————————*
991    IF gt_paste IS NOT INITIAL.
992      PERFORM paste_data .
993    ENDIF.
994  *————————————————End of changes by Abhishek——————————TR OCDK904738———————
     ————————————————*
995
996    GET PROPERTY OF go_range 'interior' = go_interior .
997    SET PROPERTY OF go_interior 'Color' = gv_sec3a_colour .
```

```
  gv_row = gv_e_row .

  PERFORM select_range USING gv_s_row 1 gv_e_row 1 .
  CALL METHOD OF go_range 'Merge' .
  CONCATENATE 'Target :' gv_current_gjahr '-' gv_next_gjahr+2(2) INTO gv_txt SEPARATED BY space .
  PERFORM set_range USING gv_txt 0.

ENDFORM.

FORM display_section3b .
  DATA lv_lines TYPE sy-tabix .
  DESCRIBE TABLE <gfs_sec3b_table> LINES lv_lines .
*————————————————Changes to show blank line if target data is not available————————*
*  CHECK lv_lines IS NOT INITIAL .
  IF lv_lines IS INITIAL.
    lv_lines = 1.
  ENDIF.
*————————————————End of changes by Abhishek————————TR OCDK904738————————————————*
  gv_row   = gv_row + 1 .
  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row + lv_lines - 1 .
  gv_e_col = gv_s_col + gv_table_columns - 1.
  PERFORM prepare_section3b_paste_data .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
*————————————————Changes to show blank line if target data is not available————————*
  IF gt_paste IS NOT INITIAL.
    PERFORM paste_data .
  ENDIF.
*————————————————End of changes by Abhishek————————TR OCDK904738————————————————*

  GET PROPERTY OF go_range 'interior' = go_interior .
  SET PROPERTY OF go_interior 'Color' = gv_sec3b_colour .
  gv_row = gv_e_row .

  PERFORM select_range USING gv_s_row 1 gv_e_row 1 .
  CALL METHOD OF go_range 'Merge' .
  CONCATENATE 'YTD Target :' gv_current_gjahr '-' gv_next_gjahr+2(2) INTO gv_txt SEPARATED BY space .
  PERFORM set_range USING gv_txt 0.

ENDFORM.
FORM display_section2e .
  DATA lv_lines TYPE sy-tabix .
  DATA lv_txt   TYPE char50 .
  DESCRIBE TABLE <gfs_sec2e_table> LINES lv_lines .
*————————————————Changes to show blank line if asking rate is not available————————*
*  CHECK lv_lines IS NOT INITIAL .
  IF lv_lines IS INITIAL.
    lv_lines = 1.
  ENDIF.
*————————————————End of changes by Abhishek————————TR OCDK904754————————————————*
  gv_row = gv_row + 1 .
  gv_s_row = gv_row .
```

```
  gv_s_col = 1 .
  gv_e_row = gv_s_row + lv_lines - 1 .
  gv_e_col = gv_s_col + gv_table_columns - 1.
  PERFORM prepare_section2e_paste_data .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
*————————————————Changes to show blank line if asking rate is not available————————*
  IF gt_paste IS NOT INITIAL.
    PERFORM paste_data .
  ENDIF.
*————————————————End of changes by Abhishek————————TR OCDK904754————————————————*

  GET PROPERTY OF go_range 'interior' = go_interior .
  SET PROPERTY OF go_interior 'Color' = gv_sec2e_colour .
  gv_row = gv_e_row .

*  PERFORM select_range USING gv_row 2 gv_row 2  .
*  PERFORM set_numberformat USING 'mmmm yyyy'.

  PERFORM select_range USING gv_s_row 1 gv_e_row 1 .
  CALL METHOD OF go_range 'Merge' .
  CONCATENATE 'Asking Rate :' gv_current_gjahr '-' gv_next_gjahr+2(2) INTO lv_txt SEPARATED BY space.
  PERFORM set_range USING lv_txt 1.

ENDFORM.
FORM display_section3c .
  DATA lv_lines TYPE sy-tabix .
  DESCRIBE TABLE <gfs_sec3c_table> LINES lv_lines .
  CHECK lv_lines IS NOT INITIAL .
  gv_row   = gv_row + 1 .
  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row + lv_lines - 1 .
  gv_e_col = gv_s_col + gv_table_columns - 1.
  PERFORM prepare_section3c_paste_data .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM paste_data .

  GET PROPERTY OF go_range 'interior' = go_interior .
  SET PROPERTY OF go_interior 'Color' = gv_sec3c_colour .
  gv_row = gv_e_row .

ENDFORM.
FORM display_section4a .
  DATA lv_lines TYPE sy-tabix .
  DESCRIBE TABLE <gfs_sec4a_table> LINES lv_lines .
  CHECK lv_lines IS NOT INITIAL .
  gv_row   = gv_row + 2 .
  gv_sec4_start_row = gv_row .
  PERFORM select_range USING gv_row 1 gv_row 2  .
  CALL METHOD OF go_range 'Merge' .
  PERFORM set_range USING 'Gas Production (YTD)' 0.

  PERFORM set_range_interior USING gv_header_gas_colour.

  gv_row   = gv_row + 1 .
  PERFORM select_range USING gv_row 1 gv_row 2  .
  CALL METHOD OF go_range 'Merge' .
  PERFORM set_range USING 'Unit of Measurement: MMSCM' 0.
```

```
1109
1110    PERFORM set_range_interior USING gv_header_gas_colour.
1111
1112    gv_row   = gv_row + 1 .
1113    gv_s_row = gv_row .
1114    gv_s_col = 1 .
1115    gv_e_row = gv_s_row + lv_lines - 1 .
1116    gv_e_col = 2 .
1117    PERFORM prepare_section4a_paste_data .
1118    PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
1119    PERFORM paste_data .
1120
1121    PERFORM select_range USING gv_s_row gv_e_col gv_e_row gv_e_col  .
1122    PERFORM set_numberformat USING '0.000' .
1123
1124    gv_row = gv_e_row .
1125    gv_sec4_end_row = gv_row .
1126
1127  ENDFORM.
1128  FORM display_section4b .
1129    DATA lv_lines TYPE sy-tabix .
1130
1131    gv_row   = gv_row + 2 .
1132    PERFORM select_range USING gv_row 1 gv_row 1  .
1133    PERFORM set_range USING 'Remarks' 0.
1134    PERFORM set_range_font  USING 13 1 .
1135
1136    PERFORM prepare_section4b_paste_data .
1137    DESCRIBE TABLE gt_paste LINES lv_lines .
1138    CHECK lv_lines GT 0 .
1139
1140    gv_row   = gv_row + 1 .
1141    PERFORM select_range USING gv_row 1 gv_row 1  .
1142    PERFORM set_range USING 'Asset' 0.
1143
1144    PERFORM select_range USING gv_row 2 gv_row 2  .
1145    PERFORM set_range USING 'Date' 0.
1146
1147    PERFORM select_range USING gv_row 3 gv_row 9  .
1148    CALL METHOD OF go_range 'Merge' .
1149    PERFORM set_range USING 'Comments' 0.
1150
1151    PERFORM select_range USING gv_row 1 gv_row 9  .
1152    PERFORM set_range_font  USING 13 1 .
1153    PERFORM set_all_borders_range .
1154
1155    gv_row   = gv_row + 1 .
1156    gv_s_row = gv_row .
1157    gv_s_col = 1 .
1158    gv_e_row = gv_s_row + lv_lines - 1 .
1159    gv_e_col = 3 .
1160
1161    PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
1162    PERFORM paste_data .
1163
1164    PERFORM select_range USING gv_s_row 1 gv_e_row 2  .
1165    PERFORM set_all_borders_range .
1166
1167    PERFORM select_range USING gv_s_row 3 gv_e_row 9  .
1168    PERFORM set_thin_border USING 1 1 1 1 .
1169    CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '12'.
```

```
  SET PROPERTY OF go_border 'LineStyle' = '1' .
  CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '10'.
  SET PROPERTY OF go_border 'LineStyle' = '1' .


  gv_row = gv_e_row .

ENDFORM.
FORM display_section5a .
  DATA lv_lines TYPE sy-tabix .
  gv_row   =  1 .

  DESCRIBE TABLE <gfs_sec5a_table> LINES lv_lines .
  CHECK lv_lines IS NOT INITIAL .
  gv_5a_rows = lv_lines .

  gv_row   = gv_row + 1 .
  gv_s_row = gv_row .
  gv_s_col = 2 .
  gv_e_row = gv_s_row + gv_5a_cols - 1 .
  gv_e_col = gv_s_col + gv_5a_rows .

  PERFORM prepare_section5a_paste_data .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM paste_data_sheet2 .

  PERFORM create_chart .

ENDFORM.
FORM create_chart .
  DATA : lv_chart_title TYPE char50,
         lv_unit        TYPE char10.
  CASE abap_true.
    WHEN p_bb.
      lv_unit   = 'BOE' .
    WHEN p_bbd.
      lv_unit   = 'BOEPD' .
    WHEN p_tm.
      lv_unit   = 'TOE' .
    WHEN p_tmd.
      lv_unit   = 'TOEPD' .
    WHEN p_mb.
      lv_unit   = 'MMTOE' .
    WHEN p_bmd.
      lv_unit   = 'BOEPD' .
    WHEN OTHERS.
  ENDCASE.
  CONCATENATE 'Production Performance' gv_current_gjahr '-' gv_next_gjah
r INTO lv_chart_title SEPARATED BY space .
  GET PROPERTY OF go_application 'Charts' = go_charts .
  CALL METHOD OF go_charts 'Add' = go_chart .
  CALL METHOD OF go_chart 'Activate' .
  SET PROPERTY OF go_chart 'HasTitle' = 1.
  GET PROPERTY OF go_chart 'ChartTitle' = go_title.
  GET PROPERTY OF go_title 'Characters' = go_titlechar.
  SET PROPERTY OF go_titlechar 'Text' = lv_chart_title.

  CALL METHOD OF go_chart 'Axes' = go_axes
    EXPORTING    #2 = 2.
  SET PROPERTY OF go_axes 'HasTitle' = 1.
  GET PROPERTY OF go_axes 'AxisTitle' = go_axestitle.
```

```
  GET PROPERTY OF go_axestitle 'Characters' = go_axestitle.
  SET PROPERTY OF go_axestitle 'Text' = lv_unit.

  SET PROPERTY OF go_chart 'HasLegend' = 1.
  GET PROPERTY OF go_chart 'Legend'  = go_legend.
  CALL METHOD OF go_legend 'Select'.
  SET PROPERTY OF go_legend 'Position'  =  '-4160'.

  SET PROPERTY OF go_chart 'ChartType' = '65' .
  CALL METHOD OF go_chart 'SetSourceData'
    EXPORTING
      #1 = go_range
      #2 = 1.
  CALL METHOD OF go_worksheet3 'ACTIVATE'.

  CALL METHOD OF go_chart 'Location'
    EXPORTING
      #1 = 2
      #2 = gv_sheet3_name.
  CALL METHOD OF go_worksheet3 'ChartObjects' = go_chartobjects .
  SET PROPERTY OF go_chartobjects 'Left' = 1 .
  SET PROPERTY OF go_chartobjects 'Top' = 30 .
  SET PROPERTY OF go_chartobjects 'Height' = 600 .
  SET PROPERTY OF go_chartobjects 'Width' = 1000 .

ENDFORM.
FORM display_section6 .
  DATA lv_lines TYPE sy-tabix .
  DATA : lv_ind_unit   TYPE char50,
         lv_total_unit TYPE char50,
         lv_gt_unit    TYPE char50 . "grand total

  gv_row   = 44 .
  gv_col   = 1  .

  PERFORM prepare_section6_paste_data .
  DESCRIBE TABLE gt_paste LINES lv_lines .
  CHECK lv_lines GT 0 .

  gv_row   = gv_row + 1 .
  gv_s_row = gv_row .
  gv_e_row = gv_s_row + 1 .
  gv_s_col = 1 .
  gv_e_col = 1 .

  CONCATENATE 'ONGC Videsh' gv_current_gjahr '-' gv_next_gjahr+2(2) INTO gv_txt .

  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  CALL METHOD OF go_range 'Merge' .
  PERFORM set_range USING gv_txt 0.

  gv_e_row = gv_s_row .
  gv_s_col = gv_s_col + 1 .
  gv_e_col = gv_s_col + 1 .

  PERFORM get_unit_desc USING c_prod_oil CHANGING lv_ind_unit lv_total_unit .
  CONCATENATE 'Oil, LNG & Condensate (' lv_ind_unit ')' INTO gv_txt SEPARATED BY space.
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
```

```
  CALL METHOD OF go_range 'Merge' .
  PERFORM set_range USING gv_txt 0.

  gv_e_row = gv_s_row .
  gv_s_col = gv_e_col + 1 .
  gv_e_col = gv_s_col + 1 .

  PERFORM get_unit_desc USING c_prod_gas CHANGING lv_ind_unit lv_total_unit .
  CONCATENATE 'Gas (' lv_ind_unit ')' INTO gv_txt SEPARATED BY space.
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  CALL METHOD OF go_range 'Merge' .
  PERFORM set_range USING gv_txt 0.

  CASE abap_true.
    WHEN p_bb.
      lv_gt_unit   = 'BOE' .
    WHEN p_bbd.
      lv_gt_unit   = 'BOEPD' .
    WHEN p_tm.
      lv_gt_unit   = 'TOE' .
    WHEN p_tmd.
      lv_gt_unit   = 'TOEPD' .
    WHEN p_mb.
      lv_gt_unit   = 'MMTOE' .
    WHEN p_bmd.
      lv_gt_unit   = 'BOEPD' .
    WHEN OTHERS.
  ENDCASE.

  gv_e_row = gv_s_row .
  gv_s_col = gv_e_col + 1 .
  gv_e_col = gv_s_col + 1 .

  CONCATENATE 'Total (O+OEG) (' lv_gt_unit ')' INTO gv_txt SEPARATED BY space.
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  CALL METHOD OF go_range 'Merge' .
  PERFORM set_range USING gv_txt 0.

  gv_s_row = gv_s_row + 1 .
  gv_e_row = gv_s_row .
  gv_s_col =  2 .
  gv_e_col =  2 .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM set_range USING 'Annual' 0.

  gv_s_col = gv_s_col +  1 .
  gv_e_col = gv_e_col +  1 .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM set_range USING 'YTD' 0.

  gv_s_col = gv_s_col +  1 .
  gv_e_col = gv_e_col +  1 .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM set_range USING 'Annual' 0.

  gv_s_col = gv_s_col +  1 .
  gv_e_col = gv_e_col +  1 .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM set_range USING 'YTD' 0.
```

```

  gv_s_col = gv_s_col +  1 .
  gv_e_col = gv_e_col +  1 .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM set_range USING 'Annual' 0.

  gv_s_col = gv_s_col +  1 .
  gv_e_col = gv_e_col +  1 .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM set_range USING 'YTD' 0.

  PERFORM select_range USING gv_row 1 gv_e_row gv_e_col  .
  PERFORM set_range_font  USING 13 1 .
  GET PROPERTY OF go_range 'interior' = go_interior .
  SET PROPERTY OF go_interior 'Color' = 12611584 .
  GET PROPERTY OF go_range 'FONT' = go_font .
  SET PROPERTY OF go_font 'COLOR' = '-460552' .
  PERFORM set_range_formatting USING  0 'C' 'C' .
  PERFORM set_all_borders_range .

  gv_row   = gv_row + 2 .
  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row + lv_lines - 1 .
  gv_e_col = 7 .

  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM paste_data_sheet3 .
  PERFORM set_all_borders_range .

  CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '12'.
  SET PROPERTY OF go_border 'LineStyle' = '1' .
  CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '10'.
  SET PROPERTY OF go_border 'LineStyle' = '1' .

  gv_row = gv_e_row .
  gv_s_col = gv_s_col + 1 .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM set_range_formatting USING  0 'C' 'C' .
  PERFORM set_numberformat USING '0.000' .

  PERFORM col_width USING 1 1 '27'  .
  PERFORM col_width USING 2 7 '16'  .

  PERFORM select_range USING 1 1 1 1  .

ENDFORM.
FORM display_section3f .
  DATA lv_lines TYPE sy-tabix .
  DESCRIBE TABLE <gfs_sec3f_table> LINES lv_lines .
  CHECK lv_lines IS NOT INITIAL .
  gv_row   = gv_row + 1 .
  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row + lv_lines - 1 .
  gv_e_col = gv_s_col + gv_table_columns - 1.
  PERFORM prepare_section3f_paste_data .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM paste_data .

  GET PROPERTY OF go_range 'interior' = go_interior .
```

```
  SET PROPERTY OF go_interior 'Color' = gv_sec3f_colour .
  gv_row = gv_e_row .

  PERFORM select_range USING gv_s_row 1 gv_e_row 1 .
  CALL METHOD OF go_range 'Merge' .
  gv_txt = 'Actual Production'.
  PERFORM set_range USING gv_txt 1.

ENDFORM.

FORM display_section3d .
  DATA lv_lines TYPE sy-tabix .
  DESCRIBE TABLE <gfs_sec3d_table> LINES lv_lines .
  CHECK lv_lines IS NOT INITIAL .
  gv_row   = gv_row + 1 .
  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row + lv_lines - 1 .
  gv_e_col = gv_s_col + gv_table_columns - 1.
  PERFORM prepare_section3d_paste_data .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM paste_data .

  GET PROPERTY OF go_range 'interior' = go_interior .
  SET PROPERTY OF go_interior 'Color' = gv_sec3d_colour .
  gv_row = gv_e_row .

  PERFORM select_range USING gv_s_row 1 gv_e_row 1 .
  CALL METHOD OF go_range 'Merge' .
  gv_txt = '% Achiev. wrt YTD Target' .
  PERFORM set_range USING gv_txt 1.

ENDFORM.
FORM display_section3e .
  DATA lv_lines TYPE sy-tabix .
  DESCRIBE TABLE <gfs_sec3e_table> LINES lv_lines .
  CHECK lv_lines IS NOT INITIAL .
  gv_row   = gv_row + 1 .
  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row + lv_lines - 1 .
  gv_e_col = gv_s_col + gv_table_columns - 1.
  PERFORM prepare_section3e_paste_data .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM paste_data .

  GET PROPERTY OF go_range 'interior' = go_interior .
  SET PROPERTY OF go_interior 'Color' = gv_sec3e_colour .
  gv_row = gv_e_row .

  PERFORM select_range USING gv_s_row 1 gv_e_row 1 .
  CALL METHOD OF go_range 'Merge' .
  gv_txt = '% Achiev. wrt Yearly Target' .
  PERFORM set_range USING gv_txt 1.

  PERFORM show_progress USING '40' .
ENDFORM.

*&---------------------------------------------------------------------*
*&      Form  START_EXCEL
*&---------------------------------------------------------------------*
```

```
*       text
*———————————————————————————————————————————————————————————————————*
*  ——>  p1        text
*  <——  p2        text
*———————————————————————————————————————————————————————————————————*
FORM start_excel .
  CREATE OBJECT go_excel 'excel.application' .
  SET PROPERTY OF go_excel 'VISIBLE' = 0 .
  CALL METHOD OF go_excel 'WORKBOOKS' = go_workbooks .

  gv_sheet1_name = '1' .
  gv_sheet2_name = '2' .
  gv_sheet3_name = 'Production Performance' .

  CALL METHOD OF go_workbooks 'ADD' = go_workbook .
  GET PROPERTY OF go_workbook 'Application' = go_application .
  CALL METHOD OF go_excel 'WORKSHEETS' = go_worksheet3
  EXPORTING
    #1 =  1.
  GET PROPERTY OF go_excel 'Sheets' = go_sheets .
  CALL METHOD OF go_sheets 'Add' = go_worksheet2 .
  CALL METHOD OF go_sheets 'Add' = go_worksheet .
  SET PROPERTY OF go_worksheet  'Name' = gv_sheet1_name .
  SET PROPERTY OF go_worksheet2 'Name' = gv_sheet2_name .
  SET PROPERTY OF go_worksheet3 'Name' = gv_sheet3_name .

  CALL METHOD OF go_worksheet 'ACTIVATE'.
  SET PROPERTY OF go_excel 'PrintCommunication' = abap_false.
  CALL METHOD OF go_worksheet 'PAGESETUP' = go_pagesetup .
  SET PROPERTY OF go_pagesetup 'FitToPagesWide' = 1 .
  SET PROPERTY OF go_pagesetup 'FitToPagesTall' = 0 .
  SET PROPERTY OF go_pagesetup 'Orientation' = 2 .
  SET PROPERTY OF go_excel 'PrintCommunication' = abap_true.

  SET PROPERTY OF go_excel 'PrintCommunication' = abap_false.
  CALL METHOD OF go_worksheet3 'PAGESETUP' = go_pagesetup .
  SET PROPERTY OF go_pagesetup 'FitToPagesWide' = 1 .
  SET PROPERTY OF go_pagesetup 'FitToPagesTall' = 0 .
  SET PROPERTY OF go_pagesetup 'Orientation' = 2 .
  SET PROPERTY OF go_excel 'PrintCommunication' = abap_true.


  gv_row = 1 .
ENDFORM.
*&———————————————————————————————————————————————————————————————————*
*&      Form  DISPLAY_SECTION1_HEADER
*&———————————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————————*
*  ——>  p1        text
*  <——  p2        text
*———————————————————————————————————————————————————————————————————*
FORM display_section1_header .
  PERFORM display_logo .
  PERFORM display_report_date .
  PERFORM display_product_names .
  PERFORM display_asset_names .
  PERFORM display_pi          .
  PERFORM display_cf          .
  PERFORM display_consortium_level .
  PERFORM display_units .
```

```
  PERFORM join_header_total_cells .
  PERFORM join_header1_column_1_2 .
  PERFORM set_section1_header_colors .
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  DISPLAY_SECTION1_DATA
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM display_section1_data .
  DATA : lv_lines TYPE sy-tabix,
         lv_col   TYPE sy-tabix.
  DESCRIBE TABLE <gfs_dyn_table> LINES lv_lines .
  gv_row = gv_row + 1 .
  gv_sec1_data_start_row = gv_row .
  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row + lv_lines - 1 .
  gv_e_col = gv_s_col + gv_table_columns - 1.
  PERFORM prepare_section1_paste_data .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM paste_data .
  PERFORM colour_alternate_rows .
  PERFORM colour_yellow_cells .
  PERFORM colour_dates .
  PERFORM colour_sec1_data_totals .
  gv_row = gv_e_row .
  gv_sec2_start_row = gv_row + 1.

  lv_col = 3 .
  PERFORM select_range USING gv_sec1_data_start_row lv_col gv_row lv_lin
es  .
  PERFORM set_numberformat USING '0.000' .
ENDFORM.
FORM formatting_section1 .
  PERFORM merge_col_1_2_section1 .
  PERFORM format_sec1_header .
  PERFORM format_sec1_data_col_1_2 .
ENDFORM .
*&———————————————————————————————————————————————————————————————*
*&      Form  SELECT_CELL
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*      —>P_GV_ROW  text
*      —>P_GV_COL  text
*———————————————————————————————————————————————————————————————*
FORM select_cell  USING    p_row
                           p_col.
  CALL METHOD OF go_excel 'Cells' = go_cell
                         EXPORTING #1 = p_row #2 = p_col.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  CELL_VALUE
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*      —>P_0393   text
```

```
*———————————————————————————————————————————————————————————————————*
FORM set_cell  USING    p_cell_value
                        p_wraptext  .
  SET PROPERTY OF go_cell 'Value' = p_cell_value .
  SET PROPERTY OF go_cell 'WrapText' = p_wraptext .
ENDFORM.

*&———————————————————————————————————————————————————————————————————*
*&      Form  SELECT_RANGE
*&———————————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————————*
*      ——>P_GV_ROW  text
*      ——>P_1      text
*      ——>P_GV_SEC1_LINES  text
*      ——>P_30     text
*———————————————————————————————————————————————————————————————————*
FORM select_range  USING    p_s_row
                            p_s_col
                            p_e_row
                            p_e_col .

  CALL METHOD OF go_excel 'Cells' = go_cell_from
   EXPORTING
   #1 = p_s_row
   #2 = p_s_col .

  CALL METHOD OF go_excel 'Cells' = go_cell_to
   EXPORTING
   #1 = p_e_row
   #2 = p_e_col .

  CALL METHOD OF go_excel 'Range' = go_range
    EXPORTING
    #1 = go_cell_from
    #2 = go_cell_to.

  CALL METHOD OF go_range 'Select' .

ENDFORM.
FORM set_range  USING    p_cell_value
                        p_wraptext  .
  SET PROPERTY OF go_range 'Value' = p_cell_value .
  SET PROPERTY OF go_range 'WrapText' = p_wraptext .
ENDFORM.
*&———————————————————————————————————————————————————————————————————*
*&      Form  SET_RANGE_FONT
*&———————————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————————*
*      ——>P_1263   text
*———————————————————————————————————————————————————————————————————*
FORM set_range_font  USING    p_size
                              p_bold.
  GET PROPERTY OF go_range 'FONT' = go_font .
  SET PROPERTY OF go_font  'BOLD' = p_bold .
  SET PROPERTY OF go_font  'Size' = p_size .
ENDFORM.

FORM set_range_formatting USING p_wraptext
                                p_horizontal
```

```
                                  p_vertical .
  SET PROPERTY OF go_range 'WrapText' = p_wraptext .
  IF p_horizontal EQ 'C'.
    SET PROPERTY OF go_range 'HorizontalAlignment' =  -4108 .
  ELSEIF  p_horizontal EQ 'L'.
    SET PROPERTY OF go_range 'HorizontalAlignment' =  -4131 .
  ELSEIF  p_horizontal EQ 'R'.
    SET PROPERTY OF go_range 'HorizontalAlignment' =  -4152 .
  ENDIF.
  IF p_vertical EQ 'C' .
    SET PROPERTY OF go_range 'VerticalAlignment' = -4108 .
  ELSEIF p_vertical EQ 'T' .
    SET PROPERTY OF go_range 'VerticalAlignment' = -4160 .
  ELSEIF p_vertical EQ 'B' .
    SET PROPERTY OF go_range 'VerticalAlignment' = -4107 .
  ENDIF.

ENDFORM .
*&---------------------------------------------------------------------*
*&      Form  SET_THIN_BORDER
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      -->P_1      text
*      -->P_1      text
*      -->P_1      text
*      -->P_1      text
*----------------------------------------------------------------------*
FORM set_thin_border   USING    p_left
                                p_right
                                p_top
                                p_bottom .
  IF p_left EQ 1 .
    CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '7' .
    SET PROPERTY OF go_border 'LineStyle' = '1'  .
  ENDIF.
  IF p_right EQ 1 .
    CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '10'.
    SET PROPERTY OF go_border 'LineStyle' = '1' .
  ENDIF.
  IF p_top EQ 1 .
    CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '8'.
    SET PROPERTY OF go_border 'LineStyle' = '1' .
  ENDIF.
  IF p_bottom EQ 1 .
    CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '9'.
    SET PROPERTY OF go_border 'LineStyle' = '1' .
  ENDIF.
ENDFORM.

*&---------------------------------------------------------------------*
*&      Form  ROW_HEIGHT
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      -->P_7      text
*      -->P_15     text
*----------------------------------------------------------------------*
FORM row_height  USING  p_row
                        p_height .
  CALL METHOD OF go_excel 'ROWS' = go_row
```

```
    EXPORTING
      #1 = p_row .
*CALL METHOD OF go_row 'Select' .
  SET PROPERTY OF go_row 'Rowheight' = p_height.

ENDFORM.
FORM merge_col_1_2_section1 .
  DATA lv_row TYPE sy-tabix .
  PERFORM select_range USING gv_sec1_h_start_row 1 gv_sec1_h_start_row 2 .
  go_range0 = go_range .
  CALL METHOD OF go_range0 'Copy' .
  lv_row = gv_sec1_h_start_row + 1 .
  PERFORM select_range USING lv_row 1 gv_row 2 .
  CALL METHOD OF go_range 'PasteSpecial'
    EXPORTING
      #1 = -4122.

  PERFORM select_range USING lv_row 1 gv_row 2  .
  PERFORM set_numberformat USING 'dd-mmm-yyyy'.

ENDFORM.
FORM format_sec1_header .
  DATA : lv_row  TYPE sy-tabix .
  lv_row = gv_sec1_h_start_row + 1 .
  PERFORM select_range USING gv_sec1_h_start_row 1 lv_row gv_table_columns .
  PERFORM set_range_formatting USING  1 'C' 'C' .
  PERFORM row_height USING gv_sec1_h_start_row 15 .
  PERFORM row_height USING lv_row 45 .
ENDFORM .
FORM format_sec1_data_col_1_2 .
  PERFORM select_range USING gv_sec1_data_start_row 1 gv_row 2 .
  PERFORM set_range_formatting USING  0 'C' 'C' .
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  PREPARE_SECTION1_PASTE_DATA
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM prepare_section1_paste_data .
  DATA lv_index TYPE sy-tabix .
  CLEAR gs_paste.
  REFRESH gt_paste .
  LOOP AT <gfs_dyn_table> ASSIGNING <gfs_dyn_line>.
    CLEAR gs_paste .
    DO .
      lv_index = sy-index .
      ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      IF sy-subrc IS INITIAL.
        IF lv_index NE 1.
          CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab INTO gs_paste .
        ENDIF.
        CONCATENATE gs_paste <gfs_field> INTO gs_paste .
      ELSE.
        EXIT .
```

```
      ENDIF.
    ENDDO.
    APPEND gs_paste TO gt_paste .
  ENDLOOP.
ENDFORM.
FORM prepare_section2a2_paste_data .
  DATA lv_index TYPE sy-tabix .
  CLEAR gs_paste.
  REFRESH gt_paste .
  LOOP AT <gfs_sec2a2_table> ASSIGNING <gfs_dyn_line>.
    CLEAR gs_paste .
    DO .
      lv_index = sy-index .
      ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      IF sy-subrc IS INITIAL.
        IF lv_index NE 1.
          CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab INTO gs_paste .
        ENDIF.
        CONCATENATE gs_paste <gfs_field> INTO gs_paste .
      ELSE.
        EXIT .
      ENDIF.
    ENDDO.
    APPEND gs_paste TO gt_paste .
  ENDLOOP.
ENDFORM.
FORM prepare_section2a3_paste_data .
  DATA lv_index TYPE sy-tabix .
  CLEAR gs_paste.
  REFRESH gt_paste .
  LOOP AT <gfs_sec2a3_table> ASSIGNING <gfs_dyn_line>.
    CLEAR gs_paste .
    DO .
      lv_index = sy-index .
      ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      IF sy-subrc IS INITIAL.
        IF lv_index NE 1.
          CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab INTO gs_paste .
        ENDIF.
        CONCATENATE gs_paste <gfs_field> INTO gs_paste .
      ELSE.
        EXIT .
      ENDIF.
    ENDDO.
    APPEND gs_paste TO gt_paste .
  ENDLOOP.
ENDFORM.
FORM prepare_section2d_paste_data .
  DATA lv_index TYPE sy-tabix .
  CLEAR gs_paste.
  REFRESH gt_paste .
  LOOP AT <gfs_sec2d_table> ASSIGNING <gfs_dyn_line>.
    CLEAR gs_paste .
    DO .
      lv_index = sy-index .
      ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
```

```
1825        IF sy-subrc IS INITIAL.
1826          IF lv_index NE 1.
1827            CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab IN
      TO gs_paste .
1828          ENDIF.
1829          CONCATENATE gs_paste <gfs_field> INTO gs_paste .
1830        ELSE.
1831          EXIT .
1832        ENDIF.
1833      ENDDO.
1834      APPEND gs_paste TO gt_paste .
1835    ENDLOOP.
1836  ENDFORM.
1837  FORM prepare_section2f_paste_data .
1838    DATA lv_index TYPE sy-tabix .
1839    CLEAR gs_paste.
1840    REFRESH gt_paste .
1841    LOOP AT <gfs_sec2f_table> ASSIGNING <gfs_dyn_line>.
1842      CLEAR gs_paste .
1843      DO .
1844        lv_index = sy-index .
1845        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_fiel
      d> .
1846        IF sy-subrc IS INITIAL.
1847          IF lv_index NE 1.
1848            CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab IN
      TO gs_paste .
1849          ENDIF.
1850          CONCATENATE gs_paste <gfs_field> INTO gs_paste .
1851        ELSE.
1852          EXIT .
1853        ENDIF.
1854      ENDDO.
1855      APPEND gs_paste TO gt_paste .
1856    ENDLOOP.
1857  ENDFORM.
1858  FORM prepare_section2e_paste_data .
1859    DATA lv_index TYPE sy-tabix .
1860    CLEAR gs_paste.
1861    REFRESH gt_paste .
1862    LOOP AT <gfs_sec2e_table> ASSIGNING <gfs_dyn_line>.
1863      CLEAR gs_paste .
1864      DO .
1865        lv_index = sy-index .
1866        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_fiel
      d> .
1867        IF sy-subrc IS INITIAL.
1868          IF lv_index NE 1.
1869            CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab IN
      TO gs_paste .
1870          ENDIF.
1871          CONCATENATE gs_paste <gfs_field> INTO gs_paste .
1872        ELSE.
1873          EXIT .
1874        ENDIF.
1875      ENDDO.
1876      APPEND gs_paste TO gt_paste .
1877    ENDLOOP.
1878  ENDFORM.
1879  *&———————————————————————————————————————————————————————————*
1880  *&      Form  ADD_PRODUCT_RANGE
```

```
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
*      ---->P_0688   text
*---------------------------------------------------------------------*
FORM add_product_range  USING  p_product.

  CLEAR r_product .

  r_product-sign   = 'I' .
  r_product-option = 'EQ' .
  r_product-low    = p_product .

  APPEND r_product TO r_product[] .
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  ADD_VL_TYPE_RANGE
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
*      ---->P_0704   text
*---------------------------------------------------------------------*
FORM add_vl_type_range  USING  p_vl_type .

  CLEAR r_prd_vl_type .

  r_prd_vl_type-sign   = 'I' .
  r_prd_vl_type-option = 'EQ' .
  r_prd_vl_type-low    = p_vl_type .

  APPEND r_prd_vl_type TO r_prd_vl_type[] .
ENDFORM.
FORM buiild_tar_code_range .
  REFRESH r_tar_code[] .
  IF p_t_be IS NOT INITIAL.
    PERFORM add_tar_code_range USING 'TAR_BE' .
  ENDIF.
  IF p_t_in IS NOT INITIAL.
    PERFORM add_tar_code_range USING 'TAR_IN' .
  ENDIF.
  IF p_t_ex IS NOT INITIAL.
    PERFORM add_tar_code_range USING 'TAR_EX' .
  ENDIF.
  IF p_t_vg IS NOT INITIAL.
    PERFORM add_tar_code_range USING 'TAR_VG' .
  ENDIF.
  IF p_t_pc IS NOT INITIAL.
    PERFORM add_tar_code_range USING 'TAR_PC' .
  ENDIF.
  IF p_t_re IS NOT INITIAL.
    PERFORM add_tar_code_range USING 'TAR_RE' .
  ENDIF.
ENDFORM .
*&---------------------------------------------------------------------*
*&      Form  ADD_VL_TAR_CODE_RANGE
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
*      ---->P_0704   text
*---------------------------------------------------------------------*
FORM add_tar_code_range  USING  p_tar_code .
```

```
  CLEAR r_tar_code .

  r_tar_code-sign   = 'I' .
  r_tar_code-option = 'EQ' .
  r_tar_code-low    = p_tar_code .

  APPEND r_tar_code TO r_tar_code[] .
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  CALCULATED_WTD_PI
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  ——>  p1        text
*  <——  p2        text
*———————————————————————————————————————————————————————————————*
FORM calculated_wtd_pi .

  DATA : lt_dly_prd         TYPE STANDARD TABLE OF zpra_t_dly_prd,
         lt_wtd_pi          TYPE STANDARD TABLE OF ty_wtd_pi,
         lt_zpra_c_prd_prof TYPE STANDARD TABLE OF ty_zpra_c_prd_prof.
  DATA : ls_dly_prd         TYPE                   zpra_t_dly_prd,
         ls_wtd_pi          TYPE                   ty_wtd_pi,
         ls_wtd_pi_t        TYPE                   ty_wtd_pi,
         ls_dly_prd_temp    TYPE                   zpra_t_dly_prd,
         ls_zpra_c_prd_prof TYPE                   ty_zpra_c_prd_prof.
  DATA : lv_index       TYPE                   sy-tabix,
         lv_numerator   TYPE                   p LENGTH 16 DECIMALS 9,
         lv_denominator TYPE                   p LENGTH 16 DECIMALS 9,
         lv_pi          TYPE                   zpra_t_prd_pi-pi,
         lv_index1      TYPE                   sy-tabix,
         lv_date1       TYPE                   sy-datum,
         lv_date2       TYPE                   sy-datum.

  FIELD-SYMBOLS : <fs_wtd_pi> TYPE ty_wtd_pi .
* Here we are calculating weighted PI for each asset for each day. This will be used to convert quantities to OVL level
* Formula used is wtd pi = sum of all block of assets' pi * quantity  divided by sum of all blocks quantity
  REFRESH gt_wtd_pi .
  lt_dly_prd = gt_zpra_t_dly_prd .

  PERFORM populate_no_data_entries TABLES lt_dly_prd USING gv_month_back_datum p_date .

  SORT lt_dly_prd BY production_date DESCENDING product ASCENDING asset ASCENDING block ASCENDING .

  LOOP AT lt_dly_prd INTO ls_dly_prd.
    lv_index = sy-tabix .
    PERFORM convert_non_gas_units_2a2 CHANGING ls_dly_prd.

****    IF ls_dly_prd-prod_vl_qty1 IS INITIAL.
***** When some blocks are shut down, then quantity comes as zero; but due to this, PI is calculated as 0
***** to avoid this making the quantity as 1 so that PI is calculated rightly
****     ls_dly_prd-prod_vl_qty1 = 1 .
****    ENDIF.
    IF ( ls_dly_prd-product         EQ ls_dly_prd_temp-product
```

```
      AND
1997           ls_dly_prd-asset           EQ ls_dly_prd_temp-asset
      AND
1998           ls_dly_prd-production_date EQ  ls_dly_prd_temp-production_date
      AND
1999           ls_dly_prd-block           NE ls_dly_prd_temp-block )
2000          OR lv_index = 1 .
2001        IF ls_dly_prd-prod_vl_qty1 IS NOT INITIAL .
2002          CLEAR lv_pi .
2003          LOOP AT gt_zpra_t_prd_pi INTO gs_zpra_t_prd_pi WHERE asset   EQ
      ls_dly_prd-asset
2004                                                          AND block   EQ
      ls_dly_prd-block
2005                                                          AND vld_frm LE
      ls_dly_prd-production_date
2006                                                          AND vld_to  GE
      ls_dly_prd-production_date .
2007            lv_pi = gs_zpra_t_prd_pi-pi .
2008            EXIT .
2009          ENDLOOP .
2010          lv_numerator   = lv_numerator   + ( ls_dly_prd-prod_vl_qty1  * l
      v_pi ).
2011          lv_denominator = lv_denominator + ls_dly_prd-prod_vl_qty1 .
2012        ENDIF.
2013      ENDIF.
2014      IF ( ls_dly_prd-production_date NE ls_dly_prd_temp-production_date O
      R
2015           ls_dly_prd-product         NE ls_dly_prd_temp-product   OR
2016           ls_dly_prd-asset           NE ls_dly_prd_temp-asset  ) AND
2017           lv_index NE 1.
2018
2019        gs_wtd_pi-production_date     = ls_dly_prd_temp-production_date
      .
2020        gs_wtd_pi-product             = ls_dly_prd_temp-product
      .
2021        gs_wtd_pi-asset               = ls_dly_prd_temp-asset
      .
2022        gs_wtd_pi-numerator           = lv_numerator
      .
2023        gs_wtd_pi-denominator         = lv_denominator
      .
2024
2025        IF lv_denominator IS INITIAL.
2026          gs_wtd_pi-pi = 0 .
2027        ELSE.
2028          gs_wtd_pi-pi = lv_numerator / lv_denominator .
2029        ENDIF.
2030        APPEND gs_wtd_pi TO gt_wtd_pi .
2031        CLEAR : lv_numerator   ,
2032                lv_denominator .
2033
2034        IF ls_dly_prd-prod_vl_qty1 IS NOT INITIAL .
2035          CLEAR lv_pi .
2036          LOOP AT gt_zpra_t_prd_pi INTO gs_zpra_t_prd_pi WHERE asset   EQ
      ls_dly_prd-asset
2037                                                          AND block   EQ
      ls_dly_prd-block
2038                                                          AND vld_frm LE
      ls_dly_prd-production_date
2039                                                          AND vld_to  GE
      ls_dly_prd-production_date .
```

```
          lv_pi = gs_zpra_t_prd_pi-pi .
          EXIT .
        ENDLOOP .
        lv_numerator   = lv_numerator   + ( ls_dly_prd-prod_vl_qty1 * lv_pi ).
        lv_denominator = lv_denominator + ls_dly_prd-prod_vl_qty1 .
      ENDIF.
    ENDIF.
    ls_dly_prd_temp = ls_dly_prd .
  ENDLOOP.

  gs_wtd_pi-production_date = ls_dly_prd_temp-production_date  .
  gs_wtd_pi-product = ls_dly_prd_temp-product  .
  gs_wtd_pi-asset   = ls_dly_prd_temp-asset    .
  gs_wtd_pi-numerator   = lv_numerator         .
  gs_wtd_pi-denominator = lv_denominator       .

  IF lv_denominator IS INITIAL.
    gs_wtd_pi-pi = 0 .
  ELSE.
    gs_wtd_pi-pi = lv_numerator / lv_denominator .
  ENDIF.
  APPEND gs_wtd_pi TO gt_wtd_pi .
* Putting previous day's PI if it is not there
  SORT gt_wtd_pi BY production_date product asset .

  lt_zpra_c_prd_prof = gt_zpra_c_prd_prof .
  SORT lt_zpra_c_prd_prof BY product asset .
  DELETE ADJACENT DUPLICATES FROM lt_zpra_c_prd_prof COMPARING product asset .

  LOOP AT lt_zpra_c_prd_prof INTO ls_zpra_c_prd_prof..
    lv_date1 = gv_month_back_datum .
    DO .
      READ TABLE gt_wtd_pi INTO ls_wtd_pi_t WITH KEY production_date = lv_date1
                                                     product = ls_zpra_c_prd_prof-product
                                                     asset   = ls_zpra_c_prd_prof-asset BINARY SEARCH.
      IF sy-subrc IS NOT INITIAL OR ls_wtd_pi_t-pi IS INITIAL.
        lv_date2 = lv_date1 - 1.
        READ TABLE gt_wtd_pi INTO ls_wtd_pi WITH KEY production_date = lv_date2
                                                     product = ls_zpra_c_prd_prof-product
                                                     asset   = ls_zpra_c_prd_prof-asset BINARY SEARCH.
        IF sy-subrc IS INITIAL.
          ls_wtd_pi-production_date = ls_wtd_pi-production_date + 1 .
          APPEND ls_wtd_pi TO gt_wtd_pi .
          SORT gt_wtd_pi BY production_date product asset .
        ENDIF.
      ENDIF.
      IF lv_date1 GE p_date.
        EXIT .
      ENDIF.
      lv_date1 = lv_date1 + 1 .
    ENDDO.
  ENDLOOP.

```

```
  DELETE ADJACENT DUPLICATES FROM gt_wtd_pi COMPARING ALL FIELDS.
* If on last date no data of any block of asset is maintained, PI at top will not be shown. Calcuating that
  LOOP AT gt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof.
    READ TABLE gt_wtd_pi INTO gs_wtd_pi WITH KEY production_date = p_date
                                                  product = gs_zpra_c_prd_prof-product
                                                  asset   = gs_zpra_c_prd_prof-asset.
*    IF sy-subrc IS NOT INITIAL.
*      gs_wtd_pi-production_date  = p_date .
*      gs_wtd_pi-product          = gs_zpra_c_prd_prof-product .
*      gs_wtd_pi-asset            = gs_zpra_c_prd_prof-asset .
*      CLEAR : lv_numerator   ,
*              lv_denominator ,
*              lv_pi          .
*      LOOP AT gt_zpra_t_prd_pi INTO gs_zpra_t_prd_pi WHERE asset   EQ gs_wtd_pi-asset
*                                                     AND vld_frm LE gs_wtd_pi-production_date
*                                                     AND vld_to  GE gs_wtd_pi-production_date .
*        lv_pi = gs_zpra_t_prd_pi-pi .
*        lv_numerator   = lv_numerator   + lv_pi .
*        lv_denominator = lv_denominator + 1 .
*      ENDLOOP .
*      IF lv_denominator IS NOT INITIAL.
*        gs_wtd_pi-pi               = lv_numerator / lv_denominator.
*        gs_wtd_pi-numerator        = lv_numerator   .
*        gs_wtd_pi-denominator      = lv_denominator .
*        APPEND gs_wtd_pi TO gt_wtd_pi .
*      ENDIF.
    IF sy-subrc IS NOT INITIAL .
      SELECT SINGLE pi
               FROM zpra_t_ast_pi
               INTO lv_pi
              WHERE asset   EQ gs_zpra_c_prd_prof-asset
                AND vld_frm LE p_date
                AND vld_to  GE p_date .
      IF sy-subrc IS INITIAL.
        gs_wtd_pi-production_date  = p_date .
        gs_wtd_pi-product          = gs_zpra_c_prd_prof-product .
        gs_wtd_pi-asset            = gs_zpra_c_prd_prof-asset .
        gs_wtd_pi-pi = lv_pi .
        APPEND gs_wtd_pi TO gt_wtd_pi .
      ENDIF.
    ELSEIF   gs_wtd_pi-pi IS INITIAL .
      lv_index = sy-tabix .
      SELECT SINGLE pi
               FROM zpra_t_ast_pi
               INTO lv_pi
              WHERE asset   EQ gs_wtd_pi-asset
                AND vld_frm LE gs_wtd_pi-production_date
                AND vld_to  GE gs_wtd_pi-production_date .
      IF sy-subrc IS INITIAL.
        gs_wtd_pi-pi = lv_pi .
        MODIFY gt_wtd_pi FROM gs_wtd_pi INDEX lv_index .
      ENDIF.
    ENDIF.
  ENDLOOP.
```

```
* We have some assets in GAS shown as combinded, so need to calculate combined PI of all those assets
  IF r_combine_asset[] IS NOT INITIAL.
    CLEAR : lv_numerator   ,
            lv_denominator .
    lt_wtd_pi = gt_wtd_pi .
    DELETE gt_wtd_pi WHERE product EQ c_prod_gas
                       AND asset   IN r_combine_asset[] .
    DELETE lt_wtd_pi WHERE product NE c_prod_gas .
    DELETE lt_wtd_pi WHERE asset  NOT IN r_combine_asset[] .
    SORT lt_wtd_pi BY production_date .
    LOOP AT lt_wtd_pi INTO ls_wtd_pi.
      lv_index = sy-tabix .
      AT NEW production_date .
        IF lv_index NE 1 .
          IF lv_denominator IS INITIAL.
            <fs_wtd_pi>-pi = 0 .
          ELSE.
            <fs_wtd_pi>-pi = lv_numerator / lv_denominator .
            <fs_wtd_pi>-numerator   = lv_numerator .
            <fs_wtd_pi>-denominator = lv_denominator .
          ENDIF.
        ENDIF.
        CLEAR : lv_numerator   ,
                lv_denominator .
        APPEND INITIAL LINE TO gt_wtd_pi ASSIGNING <fs_wtd_pi> .
        <fs_wtd_pi>-production_date = ls_wtd_pi-production_date  .
        <fs_wtd_pi>-product         = c_prod_gas  .
        <fs_wtd_pi>-asset           = 'COMBINE'    .
      ENDAT .
      lv_numerator   = lv_numerator   + ls_wtd_pi-numerator .
      lv_denominator = lv_denominator + ls_wtd_pi-denominator .
    ENDLOOP.
    IF lv_denominator IS INITIAL.
      <fs_wtd_pi>-pi = 0 .
    ELSE.
      <fs_wtd_pi>-pi = lv_numerator / lv_denominator .
      <fs_wtd_pi>-numerator   = lv_numerator .
      <fs_wtd_pi>-denominator = lv_denominator .
    ENDIF.

  ENDIF.


ENDFORM.
*&————————————————————————————————————————————————————————————————*
*&      Form  CALCULATED_WTD_cf
*&————————————————————————————————————————————————————————————————*
*       text
*————————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*————————————————————————————————————————————————————————————————*
FORM calculated_wtd_cf .
  DATA : ls_zpra_t_mrec_prd_temp TYPE            ty_zpra_t_mrec_prd .
  DATA : lv_index       TYPE                   sy-tabix,
         lv_numerator   TYPE                   zpra_t_dly_prd-prod_vl_qty1,
         lv_denominator TYPE                   zpra_t_dly_prd-prod_vl_qty1,
         lv_count       TYPE                   sy-tabix.
```

```
2205
2206    LOOP AT gt_zpra_t_mrec_prd INTO gs_zpra_t_mrec_prd.
2207      lv_index = sy-tabix .
2208      IF lv_index NE 1.
2209        IF ls_zpra_t_mrec_prd_temp-product EQ gs_zpra_t_mrec_prd-product A
      ND
2210           ls_zpra_t_mrec_prd_temp-asset   EQ gs_zpra_t_mrec_prd-asset   A
      ND
2211           ( ls_zpra_t_mrec_prd_temp-monat NE gs_zpra_t_mrec_prd-monat OR
2212           ls_zpra_t_mrec_prd_temp-gjahr   NE gs_zpra_t_mrec_prd-gjahr ).
2213          CONTINUE .
2214        ENDIF.
2215      ENDIF.
2216      IF ( gs_zpra_t_mrec_prd-product EQ ls_zpra_t_mrec_prd_temp-product A
      ND
2217           gs_zpra_t_mrec_prd-asset   EQ ls_zpra_t_mrec_prd_temp-asset   )
2218           OR lv_index = 1 .
2219        IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL .
2220          lv_numerator   = lv_numerator   + gs_zpra_t_mrec_prd-prod_vl_qty
      1 .
2221          lv_denominator = lv_denominator + gs_zpra_t_mrec_prd-prod_vl_qty
      2.
2222        ENDIF.
2223      ENDIF.
2224      IF ( gs_zpra_t_mrec_prd-product NE ls_zpra_t_mrec_prd_temp-product O
      R
2225           gs_zpra_t_mrec_prd-asset   NE ls_zpra_t_mrec_prd_temp-asset ) A
      ND
2226           lv_index NE 1.
2227        gs_wtd_cf-product     = ls_zpra_t_mrec_prd_temp-product  .
2228        gs_wtd_cf-asset       = ls_zpra_t_mrec_prd_temp-asset    .
2229
2230        IF lv_denominator IS INITIAL.
2231          gs_wtd_cf-cf = 0 .
2232        ELSE.
2233          gs_wtd_cf-cf = lv_numerator / lv_denominator .
2234        ENDIF.
2235        APPEND gs_wtd_cf TO gt_wtd_cf .
2236        CLEAR : lv_numerator   ,
2237                lv_denominator .
2238
2239        IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL .
2240          lv_numerator   = lv_numerator   + gs_zpra_t_mrec_prd-prod_vl_qty
      1 .
2241          lv_denominator = lv_denominator + gs_zpra_t_mrec_prd-prod_vl_qty
      2.
2242        ENDIF.
2243      ENDIF.
2244      ls_zpra_t_mrec_prd_temp = gs_zpra_t_mrec_prd .
2245    ENDLOOP.
2246
2247    gs_wtd_cf-product     = ls_zpra_t_mrec_prd_temp-product  .
2248    gs_wtd_cf-asset       = ls_zpra_t_mrec_prd_temp-asset    .
2249
2250    IF lv_denominator IS INITIAL.
2251      gs_wtd_cf-cf = 0 .
2252    ELSE.
2253      gs_wtd_cf-cf = lv_numerator / lv_denominator .
2254    ENDIF.
2255    APPEND gs_wtd_cf TO gt_wtd_cf .
2256
```

```
  SORT gt_wtd_cf BY product asset .
  LOOP AT gt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof.
    READ TABLE gt_wtd_cf INTO gs_wtd_cf WITH KEY product = gs_zpra_c_prd_prof-product
                                                  asset = gs_zpra_c_prd_prof-asset BINARY SEARCH .
    IF sy-subrc IS NOT INITIAL.
      CLEAR lv_count .
      CLEAR gs_wtd_cf .
      LOOP AT gt_zpra_t_tar_cf INTO gs_zpra_t_tar_cf WHERE   gjahr EQ gv_current_gjahr
                                                         AND product EQ gs_zpra_c_prd_prof-product
                                                         AND   asset EQ gs_zpra_c_prd_prof-asset
                                                         AND   block EQ gs_zpra_c_prd_prof-block .
        lv_count = lv_count + 1 .
        gs_wtd_cf-product = gs_zpra_c_prd_prof-product .
        gs_wtd_cf-asset   = gs_zpra_c_prd_prof-asset   .
        gs_wtd_cf-cf      = gs_wtd_cf-cf + gs_zpra_t_tar_cf-conv_factor .
      ENDLOOP .
      IF lv_count IS NOT INITIAL.
        gs_wtd_cf-cf = gs_wtd_cf-cf / lv_count .
        APPEND gs_wtd_cf TO gt_wtd_cf .
        SORT gt_wtd_cf BY product asset .
      ENDIF.
    ENDIF.
  ENDLOOP.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  PROCESS_GAS_RECORDS
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM process_gas_records .
  DATA : lt_zpra_t_dly_prd  TYPE STANDARD TABLE OF zpra_t_dly_prd  .

  DATA : ls_zpra_t_dly_prd   TYPE                  zpra_t_dly_prd  .
  FIELD-SYMBOLS : <fs_dly_prd> TYPE zpra_t_dly_prd  .

  DATA : lv_index TYPE sy-tabix .
  lt_zpra_t_dly_prd  = gt_zpra_t_dly_prd .
  SORT  lt_zpra_t_dly_prd BY production_date product asset block prd_vl_type .

  DELETE gt_zpra_t_dly_prd  WHERE product EQ c_prod_gas .
  DELETE lt_zpra_t_dly_prd  WHERE product NE c_prod_gas .

  IF lt_zpra_t_dly_prd IS NOT INITIAL.
    LOOP AT lt_zpra_t_dly_prd INTO ls_zpra_t_dly_prd .
      lv_index = sy-tabix .
      IF lv_index EQ 1 OR
        <fs_dly_prd>-production_date NE ls_zpra_t_dly_prd-production_date  OR
        <fs_dly_prd>-product         NE ls_zpra_t_dly_prd-product
      OR
```

```
2308          <fs_dly_prd>-asset           NE ls_zpra_t_dly_prd-asset
        OR
2309          <fs_dly_prd>-block           NE ls_zpra_t_dly_prd-block .
2310
2311          APPEND INITIAL LINE TO gt_zpra_t_dly_prd ASSIGNING <fs_dly_prd>
      .
2312          <fs_dly_prd>-mandt           = ls_zpra_t_dly_prd-mandt .
2313          <fs_dly_prd>-production_date = ls_zpra_t_dly_prd-production_date
       .
2314          <fs_dly_prd>-product         = ls_zpra_t_dly_prd-product .
2315          <fs_dly_prd>-asset           = ls_zpra_t_dly_prd-asset .
2316          <fs_dly_prd>-block           = ls_zpra_t_dly_prd-block .
2317          <fs_dly_prd>-prod_vl_uom1    = ls_zpra_t_dly_prd-prod_vl_uom1 .
2318          <fs_dly_prd>-prd_vl_type     = 'NET_PROD' .
2319        ENDIF.
2320        PERFORM convert_gas_units CHANGING ls_zpra_t_dly_prd.
2321        IF ls_zpra_t_dly_prd-prd_vl_type EQ 'GAS_INJ' .
2322          ls_zpra_t_dly_prd-prod_vl_qty1 = ls_zpra_t_dly_prd-prod_vl_qty1
      * -1 .
2323        ENDIF.
2324        <fs_dly_prd>-prod_vl_qty1    = <fs_dly_prd>-prod_vl_qty1 + ls_zpra
      _t_dly_prd-prod_vl_qty1 .
2325      ENDLOOP.
2326    ENDIF.
2327
2328    lt_zpra_t_dly_prd = gt_zpra_t_dly_prd .
2329    *gt_zdpr_gas_combine
2330    LOOP AT  lt_zpra_t_dly_prd INTO ls_zpra_t_dly_prd.
2331      lv_index = sy-tabix .
2332      IF ls_zpra_t_dly_prd-product NE c_prod_gas.
2333        DELETE lt_zpra_t_dly_prd INDEX lv_index .
2334        CONTINUE .
2335      ENDIF.
2336      READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY ls_
      zpra_t_dly_prd-asset BINARY SEARCH .
2337      IF sy-subrc IS NOT INITIAL.
2338        DELETE lt_zpra_t_dly_prd INDEX lv_index .
2339      ENDIF.
2340    ENDLOOP.
2341
2342    LOOP AT  gt_zpra_t_dly_prd INTO ls_zpra_t_dly_prd.
2343      lv_index = sy-tabix .
2344      IF ls_zpra_t_dly_prd-product EQ c_prod_gas.
2345        READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY l
      s_zpra_t_dly_prd-asset BINARY SEARCH .
2346        IF sy-subrc IS INITIAL.
2347          DELETE gt_zpra_t_dly_prd INDEX lv_index .
2348        ENDIF.
2349      ENDIF.
2350    ENDLOOP.
2351
2352    PERFORM populate_no_data_entries TABLES lt_zpra_t_dly_prd USING gv_mon
      th_back_datum p_date .
2353
2354    APPEND LINES OF lt_zpra_t_dly_prd TO gt_zpra_t_dly_prd .
2355
2356    SORT  gt_zpra_t_dly_prd BY production_date product asset .
2357
2358
2359    lt_zpra_t_dly_prd  = gt_zpra_t_dly_prd_mb .
2360    SORT  lt_zpra_t_dly_prd BY production_date product asset block prd_vl_
```

```
type .

  DELETE gt_zpra_t_dly_prd_mb  WHERE product EQ c_prod_gas .
  DELETE lt_zpra_t_dly_prd     WHERE product NE c_prod_gas .
  IF lt_zpra_t_dly_prd IS NOT INITIAL.

    LOOP AT lt_zpra_t_dly_prd INTO ls_zpra_t_dly_prd .
      lv_index = sy-tabix .
      IF lv_index EQ 1 OR
        <fs_dly_prd>-production_date NE ls_zpra_t_dly_prd-production_date  OR
        <fs_dly_prd>-product         NE ls_zpra_t_dly_prd-product  OR
        <fs_dly_prd>-asset           NE ls_zpra_t_dly_prd-asset  OR
        <fs_dly_prd>-block           NE ls_zpra_t_dly_prd-block .

        APPEND INITIAL LINE TO gt_zpra_t_dly_prd_mb ASSIGNING <fs_dly_prd> .
        <fs_dly_prd>-mandt           = ls_zpra_t_dly_prd-mandt .
        <fs_dly_prd>-production_date = ls_zpra_t_dly_prd-production_date .
        <fs_dly_prd>-product         = ls_zpra_t_dly_prd-product .
        <fs_dly_prd>-asset           = ls_zpra_t_dly_prd-asset .
        <fs_dly_prd>-block           = ls_zpra_t_dly_prd-block .
        <fs_dly_prd>-prod_vl_uom1    = ls_zpra_t_dly_prd-prod_vl_uom1 .
        <fs_dly_prd>-prd_vl_type     = 'NET_PROD' .
      ENDIF.
      PERFORM convert_gas_units CHANGING ls_zpra_t_dly_prd.
      IF ls_zpra_t_dly_prd-prd_vl_type EQ 'GAS_INJ' .
        ls_zpra_t_dly_prd-prod_vl_qty1 = ls_zpra_t_dly_prd-prod_vl_qty1 * -1 .
      ENDIF.
      <fs_dly_prd>-prod_vl_qty1    = <fs_dly_prd>-prod_vl_qty1 + ls_zpra_t_dly_prd-prod_vl_qty1 .
    ENDLOOP.
  ENDIF.
  SORT  gt_zpra_t_dly_prd_mb BY production_date product asset .

ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  FILL_NULL_VALUES_WITH_PREVIOUS
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM fill_null_values_with_previous .

  DATA : ls_zpra_t_dly_prd TYPE zpra_t_dly_prd .
  DATA  :lv_col_index     TYPE sy-tabix,
         lv_lines         TYPE sy-tabix,
         lv_combine_field TYPE c,
         lv_date          TYPE sy-datum,
         lv_col_name      TYPE lvc_fname.

  FIELD-SYMBOLS : <lfs_dyn_line> ,
                  <lfs_field> ,
                  <lfs_field_previous> .
  UNASSIGN : <lfs_dyn_line> ,
```

```
              <lfs_field>    ,
              <lfs_field_previous> .
    DESCRIBE TABLE <gfs_dyn_table> LINES lv_lines .
    lv_lines = lv_lines - 1 .
    IF lv_lines GT 0.
      READ TABLE <gfs_dyn_table> ASSIGNING <lfs_dyn_line> INDEX lv_lines .
    ENDIF.
    DO .
      lv_col_index = sy-index .
      ASSIGN COMPONENT lv_col_index OF STRUCTURE <gfs_dyn_line> TO <lfs_field> .
      IF sy-subrc IS NOT INITIAL.
        EXIT .
      ENDIF.
      READ TABLE gt_dyn_fcat INTO gs_dyn_fcat WITH KEY col_pos = lv_col_index BINARY SEARCH .
      IF sy-subrc IS INITIAL.
        SPLIT gs_dyn_fcat-fieldname AT '-' INTO ls_zpra_t_dly_prd-product ls_zpra_t_dly_prd-asset .
        IF NOT ( gs_dyn_fcat-fieldname CS 'COL01' OR
                 gs_dyn_fcat-fieldname CS 'COL02' OR
                 gs_dyn_fcat-fieldname CS 'TOTAL' ).
          IF <lfs_field> EQ ' '.
            IF <lfs_dyn_line> IS ASSIGNED.
              ASSIGN COMPONENT lv_col_index OF STRUCTURE <lfs_dyn_line> TO <lfs_field_previous> .
              <lfs_field> = <lfs_field_previous> .
              IF ls_zpra_t_dly_prd-product = c_prod_gas.
                CLEAR lv_combine_field .
                READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = ls_zpra_t_dly_prd-asset BINARY SEARCH .
                IF  sy-subrc IS INITIAL.
                  lv_combine_field = 'X' .
                ENDIF.
              ENDIF.
              lv_date = gs_zpra_t_dly_prd-production_date - 1 .
              ls_zpra_t_dly_prd-production_date = lv_date .
              IF p_c_jv IS NOT INITIAL .
                PERFORM get_pi_for_null_values USING ls_zpra_t_dly_prd lv_combine_field.
              ELSE.
                gv_total_mul       = 1 .
                gv_grand_total_mul = 1 .
                IF p_bmd IS NOT INITIAL AND ls_zpra_t_dly_prd-product EQ c_prod_gas.
*                   gv_grand_total_mul = gv_grand_total_mul * 6290 .
                ENDIF.
              ENDIF.
* Product Total..
              gv_len = strlen( ls_zpra_t_dly_prd-product ) .
              CONCATENATE ls_zpra_t_dly_prd-product(gv_len) '-' 'TOTAL'
                  INTO lv_col_name .
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <lfs_field> .
              IF <lfs_field> IS ASSIGNED.
                <lfs_field> = <lfs_field> + <lfs_field_previous> * gv_total_mul .
                UNASSIGN <lfs_field> .
              ENDIF.
*Grand Total
              CONCATENATE 'GRAND' '-' 'TOTAL'
```

```
                     INTO lv_col_name .
            ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <lfs_field> .
            IF <lfs_field> IS ASSIGNED.
              <lfs_field> = <lfs_field> + <lfs_field_previous> * gv_grand_total_mul .
              UNASSIGN <lfs_field> .
            ENDIF.
          ELSE.
            <lfs_field> = '0.00' .
* Product Total..
            gv_len = strlen( ls_zpra_t_dly_prd-product ) .
            CONCATENATE ls_zpra_t_dly_prd-product(gv_len) '-' 'TOTAL'
                INTO lv_col_name .
            ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <lfs_field> .
            IF <lfs_field> IS ASSIGNED.
              <lfs_field> = <lfs_field> + '0.00' .
              UNASSIGN <lfs_field> .
            ENDIF.
*Grand Total
            CONCATENATE 'GRAND' '-' 'TOTAL'
                     INTO lv_col_name .
            ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <lfs_field> .
            IF <lfs_field> IS ASSIGNED.
              <lfs_field> = <lfs_field> + '0.00' .
              UNASSIGN <lfs_field> .
            ENDIF.

          ENDIF.
          gs_copied_cells-row = lv_lines + 1 .
          gs_copied_cells-col = lv_col_index .
          APPEND gs_copied_cells TO gt_copied_cells .
        ENDIF.
      ENDIF.
    ENDIF.
    UNASSIGN : <lfs_field> ,
               <lfs_field_previous> .
  ENDDO.
  UNASSIGN : <lfs_dyn_line> .
ENDFORM.
*&———————————————————————————————————————————————————————————*
*&      Form  PASTE_DATA
*&———————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————*
FORM paste_data .
  DO 10 TIMES .
    CALL METHOD cl_gui_frontend_services=>clipboard_export
      IMPORTING
        data         = gt_paste
      CHANGING
        rc           = gv_rc
      EXCEPTIONS
        cntl_error   = 1
        error_no_gui = 2
        OTHERS       = 4.
```

```
      IF sy-subrc IS INITIAL AND gv_rc IS INITIAL.
        EXIT .
      ENDIF.
    ENDDO.
    DO 10 TIMES .
      CALL METHOD OF go_worksheet 'Paste'.
      IF sy-subrc IS INITIAL.
        EXIT.
      ENDIF.
    ENDDO.
  ENDFORM.
  *&---------------------------------------------------------------------*
  *&      Form  PASTE_DATA
  *&---------------------------------------------------------------------*
  *       text
  *----------------------------------------------------------------------*
  *  -->  p1        text
  *  <--  p2        text
  *----------------------------------------------------------------------*
  FORM paste_data_sheet3 .
    DO 10 TIMES .
      CALL METHOD cl_gui_frontend_services=>clipboard_export
        IMPORTING
          data         = gt_paste
        CHANGING
          rc           = gv_rc
        EXCEPTIONS
          cntl_error   = 1
          error_no_gui = 2
          OTHERS       = 4.
      IF sy-subrc IS INITIAL AND gv_rc IS INITIAL.
        EXIT .
      ENDIF.
    ENDDO.
    DO 10 TIMES .
      CALL METHOD OF go_worksheet3 'Paste'.
      IF sy-subrc IS INITIAL.
        EXIT.
      ENDIF.
    ENDDO.

  ENDFORM.
  *&---------------------------------------------------------------------*
  *&      Form  PASTE_DATA_sheet2
  *&---------------------------------------------------------------------*
  *       text
  *----------------------------------------------------------------------*
  *  -->  p1        text
  *  <--  p2        text
  *----------------------------------------------------------------------*
  FORM paste_data_sheet2 .
    DO 10 TIMES .
      CALL METHOD cl_gui_frontend_services=>clipboard_export
        IMPORTING
          data         = gt_paste
        CHANGING
          rc           = gv_rc
        EXCEPTIONS
          cntl_error   = 1
          error_no_gui = 2
          OTHERS       = 4.
```

```
      IF sy-subrc IS INITIAL AND gv_rc IS INITIAL.
        EXIT .
      ENDIF.
    ENDDO.
    DO 10 TIMES .
      CALL METHOD OF go_worksheet2 'Paste'.
      IF sy-subrc IS INITIAL.
        EXIT.
      ENDIF.
    ENDDO.
  ENDFORM.
  *&———————————————————————————————————————————————————————————————*
  *&      Form  COLOUR_YELLOW_CELLS
  *&———————————————————————————————————————————————————————————————*
  *       text
  *———————————————————————————————————————————————————————————————*
  *  —>  p1        text
  *  <—  p2        text
  *———————————————————————————————————————————————————————————————*
  FORM colour_yellow_cells .
    DATA : lv_row TYPE sy-tabix .
    LOOP AT gt_copied_cells INTO gs_copied_cells.
      lv_row = gv_row + gs_copied_cells-row - 1 .
      CALL METHOD OF go_excel 'Cells' = go_cell
       EXPORTING
       #1 = lv_row
       #2 = gs_copied_cells-col .

      GET PROPERTY OF go_cell 'interior' = go_interior .
      SET PROPERTY OF go_interior 'Color' = 65535 .

      GET PROPERTY OF go_cell 'FONT' = go_font .
      SET PROPERTY OF go_font 'COLOR' = '-16776961' .
    ENDLOOP.
  ENDFORM.
  *&———————————————————————————————————————————————————————————————*
  *&      Form  DISPLAY_LOGO
  *&———————————————————————————————————————————————————————————————*
  *       text
  *———————————————————————————————————————————————————————————————*
  *  —>  p1        text
  *  <—  p2        text
  *———————————————————————————————————————————————————————————————*
  FORM display_logo .
    PERFORM download_image .
    PERFORM display_image  .
    PERFORM delete_image   .

    PERFORM show_progress USING '0' .
  ENDFORM.
  *&———————————————————————————————————————————————————————————————*
  *&      Form  DOWNLOAD_IMAGE
  *&———————————————————————————————————————————————————————————————*
  *       text
  *———————————————————————————————————————————————————————————————*
  *  —>  p1        text
  *  <—  p2        text
  *———————————————————————————————————————————————————————————————*
  FORM download_image .

    DATA : l_bytecount TYPE i,
```

```
          l_tdbtype   LIKE stxbitmaps-tdbtype,
          l_content   TYPE STANDARD TABLE OF bapiconten INITIAL SIZE 0.

   DATA: graphic_size TYPE i.

   DATA: BEGIN OF graphic_table OCCURS 0,
           line(255) TYPE x,
         END OF graphic_table.

   CONCATENATE p_fname '\' sy-datum sy-uzeit '.bmp' INTO gv_image_name .
   CALL FUNCTION 'SAPSCRIPT_GET_GRAPHIC_BDS'
     EXPORTING
       i_object       = 'GRAPHICS'
       i_name         = 'OVL_FULL'
       i_id           = 'BMAP'
       i_btype        = 'BCOL'
     IMPORTING
       e_bytecount    = l_bytecount
     TABLES
       content        = l_content
     EXCEPTIONS
       not_found      = 1
       bds_get_failed = 2
       bds_no_content = 3
       OTHERS         = 4.

   CALL FUNCTION 'SAPSCRIPT_CONVERT_BITMAP'
     EXPORTING
       old_format               = 'BDS'
       new_format               = 'BMP'
       bitmap_file_bytecount_in = l_bytecount
     IMPORTING
       bitmap_file_bytecount    = graphic_size
     TABLES
       bds_bitmap_file          = l_content
       bitmap_file              = graphic_table
     EXCEPTIONS
       OTHERS                   = 1.

   CALL FUNCTION 'WS_DOWNLOAD'
     EXPORTING
       bin_filesize            = graphic_size
       filename                = gv_image_name
       filetype                = 'BIN'
     TABLES
       data_tab                = graphic_table
     EXCEPTIONS
       invalid_filesize        = 1
       invalid_table_width     = 2
       invalid_type            = 3
       no_batch                = 4
       unknown_error           = 5
       gui_refuse_filetransfer = 6.

   IF sy-subrc <> 0.
     MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
             WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
   ENDIF.
 ENDFORM.
 *&———————————————————————————————————————————————————————————————*
 *&      Form  DISPLAY_IMAGE
```

```
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*---------------------------------------------------------------------*
FORM display_image .
  DATA lo_shapes TYPE ole2_object .


  GET PROPERTY OF go_worksheet 'Shapes' = lo_shapes.

  CALL METHOD OF lo_shapes 'AddPicture'
    EXPORTING
      #1 = gv_image_name "image file name on presentation server
      #2 = '1'
      #3 = '1'
      #4 = 1      "left
      #5 = 1      "top
      #6 = 390    "right
      #7 = 45.    "bottom

  gv_row = gv_row + 2 .
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  DELETE_IMAGE
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*---------------------------------------------------------------------*
FORM delete_image .
  DATA : lv_file TYPE string,
         lv_rc   TYPE i.
  lv_file = gv_image_name .

  CALL METHOD cl_gui_frontend_services=>file_delete
    EXPORTING
      filename             = lv_file
    CHANGING
      rc                   = lv_rc
    EXCEPTIONS
      file_delete_failed   = 1
      cntl_error           = 2
      error_no_gui         = 3
      file_not_found       = 4
      access_denied        = 5
      unknown_error        = 6
      not_supported_by_gui = 7
      wrong_parameter      = 8
      OTHERS               = 9.
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  DISPLAY_REPORT_DATE
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*---------------------------------------------------------------------*
```

```
FORM display_report_date .
  DATA : lv_date    TYPE char50,
         lv_heading TYPE char100.

  CALL FUNCTION 'CONVERSION_EXIT_SDATE_OUTPUT'
    EXPORTING
      input  = gv_rep_date
    IMPORTING
      output = lv_date.
  REPLACE ALL OCCURRENCES OF '.' IN lv_date WITH '-' .

  CONCATENATE 'Daily Production Report :' lv_date INTO lv_heading SEPARATED BY space .

  gv_row = gv_row + 1 .
  PERFORM select_range USING gv_row 1 gv_row 5  .
  CALL METHOD OF go_range 'Merge' .
  PERFORM set_range USING lv_heading 0.

  PERFORM set_range_formatting USING  1 'L' 'C' .
  PERFORM set_range_interior USING gv_header_colour .
  GET PROPERTY OF go_range 'FONT' = go_font .
  SET PROPERTY OF go_font  'BOLD' = 1 .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  DISPLAY_PRODUCT_NAMES
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM display_product_names .
  DATA : lv_separator TYPE char10,
         lv_no_comma  TYPE c.

  CLEAR gv_grand_total_desc .
  gv_row = gv_row + 1 .
  gv_start_row = gv_row .
  gv_sec1_h_start_row = gv_row .
  gv_col = 1 .
  gv_s_row = gv_row .
  gv_s_col = gv_col .
  gv_e_row = gv_s_row .
  gv_e_col = gv_col .
  PERFORM select_cell USING gv_s_row gv_s_col .
  PERFORM set_cell USING 'Product' 0.
  CLEAR gv_txt .
  gv_col = gv_col + 1 .

  LOOP AT r_product.
    CASE r_product-low .
      WHEN c_prod_oil.
        gv_txt =  c_oil_desc.
      WHEN c_prod_con.
        gv_txt = c_con_desc .
      WHEN c_prod_lng.
        gv_txt = c_lng_desc .
      WHEN c_prod_gas.
        gv_txt = c_gas_desc .
```

```
      WHEN OTHERS.
    ENDCASE.
    PERFORM write_product_name USING r_product-low.
  ENDLOOP.
  LOOP AT gt_product_col INTO gs_product_col.
    AT FIRST .
      gv_grand_total_desc = 'Total' .
      lv_no_comma  = 'X' .
    ENDAT .
    AT LAST .
      IF sy-tabix NE 1 .
        CONCATENATE gv_grand_total_desc 'and' INTO gv_grand_total_desc SEPARATED BY space .
      ENDIF.
      lv_no_comma  = 'X' .
    ENDAT .
    IF lv_no_comma IS INITIAL.
      CONCATENATE gv_grand_total_desc  ',' INTO gv_grand_total_desc .
    ENDIF.
    CONCATENATE gv_grand_total_desc gs_product_col-desc INTO gv_grand_total_desc SEPARATED BY space .
    CLEAR lv_no_comma .
  ENDLOOP.
*  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
ENDFORM.
*&———————————————————————————————————————————————————————————————————*
*&      Form  DISPLAY_ASSET_NAMES
*&———————————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————————*
FORM display_asset_names .
  DATA : lv_product    TYPE zpra_t_dly_prd-product,
         lv_asset      TYPE zpra_t_dly_prd-asset,
         lv_asset_desc TYPE char512,
         lv_asset_pre  TYPE char512,
         lv_index      TYPE sy-tabix.
  DATA : v_expdt_cnt TYPE I.
  CLEAR   gs_paste .
  REFRESH gt_paste .

  LOOP AT gt_dyn_fcat INTO gs_dyn_fcat .
    IF gs_dyn_fcat-fieldname EQ 'COL01'.
      lv_asset_desc = 'Asset' .
    ELSEIF gs_dyn_fcat-fieldname EQ 'COL02'.
    ELSEIF gs_dyn_fcat-fieldname EQ 'GRAND-TOTAL' .
      lv_asset_desc = gv_grand_total_desc .
    ELSEIF gs_dyn_fcat-fieldname CS 'TOTAL' .
      SPLIT gs_dyn_fcat-fieldname AT '-' INTO lv_product lv_asset .
      CASE lv_product .
        WHEN c_prod_oil.
          lv_asset_desc =  c_oil_desc.
        WHEN c_prod_con.
          lv_asset_desc = c_con_desc .
        WHEN c_prod_lng.
          lv_asset_desc = c_lng_desc .
        WHEN c_prod_gas.
          lv_asset_desc = c_gas_desc .
        WHEN OTHERS.
```

```
      ENDCASE.
      CONCATENATE 'Total' lv_asset_desc INTO lv_asset_desc SEPARATED BY space.
    ELSE .
      SPLIT gs_dyn_fcat-fieldname AT '-' INTO lv_product lv_asset .
      IF lv_asset NE 'COMBINE'.
        READ TABLE gt_asset_desc INTO gs_asset_desc WITH KEY asset = lv_asset BINARY SEARCH .
        IF sy-subrc IS INITIAL.
          lv_asset_desc = gs_asset_desc-desc .
*          SELECT SINGLE LISCENSE_EXP_DT from ZPRA_T_PRD_PI INTO @data(v_expdt)
*            WHERE ASSET = @lv_asset.
*            IF  v_expdt is NOT INITIAL and v_expdt LT P_DATE.
*            CONCATENATE lv_asset_desc '*' INTO lv_asset_desc.
*            ENDIF
*************************************Changes by hrishikesh nikam on 31.03.2022*************************************
          SELECT ASSET, BLOCK, LISCENSE_EXP_DT from ZPRA_T_PRD_PI INTO TABLE @data(v_expdt)
            WHERE ASSET = @lv_asset.
           IF  v_expdt is NOT INITIAL.
            DESCRIBE TABLE v_expdt LINES DATA(v_expdt_lines).
              CLEAR v_expdt_cnt.
              LOOP AT v_expdt INTO DATA(ls_v_expdt) WHERE ASSET = LV_ASSET AND LISCENSE_EXP_DT LT P_DATE .
               IF ls_v_expdt-liscense_exp_dt GT '00000000'.
                v_expdt_cnt = v_expdt_cnt + 1.
               ENDIF.
               CLEAR ls_v_expdt.
              ENDLOOP.
            IF  v_expdt_cnt = v_expdt_lines.
             CONCATENATE lv_asset_desc '*' INTO lv_asset_desc.
            ENDIF.
           ENDIF.
*****************************************************************************************************************
        ENDIF.
      ELSE.

        LOOP AT r_combine_asset.
          lv_index = sy-tabix .
          READ TABLE gt_asset_desc INTO gs_asset_desc WITH KEY asset = r_combine_asset-low BINARY SEARCH .
          IF sy-subrc IS INITIAL.
            SPLIT gs_asset_desc-desc AT ',' INTO lv_asset_pre gs_asset_desc-desc .
            CONDENSE gs_asset_desc-desc .
            IF lv_index NE 1.
              CONCATENATE lv_asset_desc ', ' INTO lv_asset_desc .
            ENDIF.
            CONCATENATE lv_asset_desc gs_asset_desc-desc INTO lv_asset_desc .
          ENDIF.
        ENDLOOP.
      ENDIF.
    ENDIF.
    IF gs_dyn_fcat-col_pos NE 1.
      CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab INTO gs_paste .
    ENDIF.
```

```
    CONCATENATE gs_paste lv_asset_desc INTO gs_paste .
    CLEAR : gs_asset_desc , lv_asset_desc .
  ENDLOOP.

  APPEND gs_paste TO gt_paste .

  gv_row = gv_row + 1 .
  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row .
  gv_e_col = gv_s_col + gv_table_columns - 1.

  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM paste_data .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  DISPLAY_PI
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM display_pi .

  DATA : lv_product TYPE zpra_t_dly_prd-product,
         lv_asset   TYPE zpra_t_dly_prd-asset,
         lv_pi      TYPE char50,
         lv_col     TYPE sy-tabix.
  CLEAR   gs_paste .
  REFRESH gt_paste .
  SORT gt_wtd_pi BY product ASCENDING asset ASCENDING production_date DESCENDING .
  LOOP AT gt_dyn_fcat INTO gs_dyn_fcat .
    IF gs_dyn_fcat-fieldname EQ 'COL01'.
      lv_pi = 'ONGC VideshShare (%)' .
    ELSEIF gs_dyn_fcat-fieldname EQ 'COL01'.
    ELSEIF gs_dyn_fcat-fieldname CS 'TOTAL' .
*      gs_asset_desc-desc = '' .
    ELSE .
      SPLIT gs_dyn_fcat-fieldname AT '-' INTO lv_product lv_asset .
      IF lv_asset NE 'COMBINE'.
        CLEAR gs_wtd_pi .
        READ TABLE gt_wtd_pi INTO gs_wtd_pi WITH KEY product = lv_product
                                                      asset   = lv_asset .
        IF sy-subrc IS INITIAL.
          lv_pi = gs_wtd_pi-pi .
        ENDIF.
      ELSE.
        CLEAR gs_wtd_pi .
        READ TABLE gt_wtd_pi INTO gs_wtd_pi WITH KEY product = lv_product
                                                      asset   = 'COMBINE' .
        IF sy-subrc IS INITIAL.
          lv_pi = gs_wtd_pi-pi .
        ENDIF.
      ENDIF.
```

```
      ENDIF.
      IF gs_dyn_fcat-col_pos NE 1.
        CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab INTO gs_paste .
      ENDIF.
      CONCATENATE gs_paste lv_pi INTO gs_paste .
      CLEAR lv_pi .
    ENDLOOP.

    APPEND gs_paste TO gt_paste .

    gv_row = gv_row + 1 .
    gv_s_row = gv_row .
    gv_s_col = 1 .
    gv_e_row = gv_s_row .
    gv_e_col = gv_s_col + gv_table_columns - 1.

    PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
    PERFORM paste_data .

    lv_col = 3 .
    PERFORM select_range USING gv_s_row lv_col gv_e_row gv_e_col  .
    PERFORM set_numberformat USING '0.000' .

  ENDFORM.
  *&---------------------------------------------------------------------*
  *&      Form  DISPLAY_CF
  *&---------------------------------------------------------------------*
  *       text
  *----------------------------------------------------------------------*
  *  -->  p1        text
  *  <--  p2        text
  *----------------------------------------------------------------------*
  FORM display_cf .

    DATA : lv_product TYPE zpra_t_dly_prd-product,
           lv_asset   TYPE zpra_t_dly_prd-asset,
           lv_cf      TYPE char50,
           lv_col     TYPE sy-tabix.
    CLEAR   gs_paste .
    REFRESH gt_paste .

    LOOP AT gt_dyn_fcat INTO gs_dyn_fcat .
      IF gs_dyn_fcat-fieldname EQ 'COL01'.
        lv_cf = 'Conversion Factor (bbl/tonnes)' .
      ELSEIF gs_dyn_fcat-fieldname CS 'COL02' .
      ELSEIF gs_dyn_fcat-fieldname CS 'TOTAL' .
*       gs_asset_desc-desc = '' .
      ELSE .
        SPLIT gs_dyn_fcat-fieldname AT '-' INTO lv_product lv_asset .
        IF lv_asset NE 'COMBINE'.
          CLEAR gs_wtd_pi .
          READ TABLE gt_wtd_cf INTO gs_wtd_cf WITH KEY product = lv_product
                                                        asset   = lv_asset BINARY SEARCH .
          IF sy-subrc IS INITIAL.
            lv_cf = gs_wtd_cf-cf .
          ENDIF.
        ENDIF.
      ENDIF.
```

```
      IF gs_dyn_fcat-col_pos NE 1.
        CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab INTO g
    s_paste .
      ENDIF.
      CONCATENATE gs_paste lv_cf INTO gs_paste .
      CLEAR lv_cf .
    ENDLOOP.

    APPEND gs_paste TO gt_paste .

    gv_row = gv_row + 1 .
    gv_s_row = gv_row .
    gv_s_col = 1 .
    gv_e_row = gv_s_row .
    gv_e_col = gv_s_col + gv_table_columns - 1.

    PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
    PERFORM paste_data .

    lv_col = 3 .

    PERFORM select_range USING gv_s_row lv_col gv_e_row gv_e_col  .
    PERFORM set_numberformat USING '0.000' .

  ENDFORM.
  FORM display_consortium_level .
    CONSTANTS : c_ongc TYPE char50 VALUE 'ONGC Videsh',
                c_jvl  TYPE char50 VALUE 'JV'.
    DATA : lv_consortium_txt TYPE char50 .

    IF p_c_jv IS NOT INITIAL.
      lv_consortium_txt = c_jvl  .
    ELSE .
      lv_consortium_txt = c_ongc .
    ENDIF.

    gv_row = gv_row + 1 .
    gv_col = 1 .
    gv_s_row = gv_row .
    gv_s_col = gv_col .
    gv_e_row = gv_s_row .
    gv_e_col = gv_col .
    PERFORM select_cell USING gv_s_row gv_s_col .
    PERFORM set_cell USING 'Consortium Level' 0.
    CLEAR gv_txt .
    gv_col = gv_col + 2 .

    LOOP AT gt_product_col INTO gs_product_col.
      gv_s_col = gs_product_col-s_col .
      gv_e_col = gs_product_col-e_col - 1 .
      PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
      CALL METHOD OF go_range 'Merge' .
      PERFORM set_range USING lv_consortium_txt 0.
      PERFORM set_range_formatting USING 0 'C' 'C' .

      gv_s_col = gs_product_col-e_col .
      gv_e_col = gv_s_col .

      PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
      PERFORM set_range USING c_ongc  0.
      PERFORM set_range_formatting USING 0 'C' 'C' .
```

```
3106
3107    ENDLOOP.
3108
3109    gv_s_col = gv_s_col + 1 .
3110    gv_e_col = gv_s_col .
3111
3112    PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
3113    PERFORM set_range USING c_ongc  0.
3114    PERFORM set_range_formatting USING 0 'C' 'C' .
3115
3116  ENDFORM .
3117  FORM display_units .
3118    DATA : lv_ind_unit   TYPE char50 , "non gas
3119           lv_total_unit TYPE char50 , "gas
3120           lv_gt_unit    TYPE char50 . "grand total
3121
3122    gv_row = gv_row + 1 .
3123    gv_col = 1 .
3124    gv_s_row = gv_row .
3125    gv_s_col = gv_col .
3126    gv_e_row = gv_s_row .
3127    gv_e_col = gv_col .
3128    PERFORM select_cell USING gv_s_row gv_s_col .
3129    PERFORM set_cell USING 'Date/Unit' 0.
3130    CLEAR gv_txt .
3131    gv_col = gv_col + 2 .
3132
3133    LOOP AT gt_product_col INTO gs_product_col.
3134      gv_s_col = gs_product_col-s_col .
3135      gv_e_col = gs_product_col-e_col - 1 .
3136      PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
3137      CALL METHOD OF go_range 'Merge' .
3138
3139      PERFORM get_unit_desc USING gs_product_col-product CHANGING lv_ind_u
      nit lv_total_unit .
3140
3141      PERFORM set_range USING lv_ind_unit 0.
3142      PERFORM set_range_formatting USING 0 'C' 'C' .
3143      gv_s_col = gs_product_col-e_col .
3144      gv_e_col = gv_s_col .
3145
3146      PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
3147      PERFORM set_range USING lv_total_unit  0.
3148      PERFORM set_range_formatting USING 0 'C' 'C' .
3149
3150    ENDLOOP.
3151
3152    gv_s_col = gv_s_col + 1 .
3153    gv_e_col = gv_s_col .
3154
3155    CASE abap_true.
3156      WHEN p_bb.
3157        lv_gt_unit   = 'BOE' .
3158      WHEN p_bbd.
3159        lv_gt_unit   = 'BOEPD' .
3160      WHEN p_tm.
3161        lv_gt_unit   = 'TOE' .
3162      WHEN p_tmd.
3163        lv_gt_unit   = 'TOEPD' .
3164      WHEN p_mb.
3165        lv_gt_unit   = 'MMTOE' .
```

```
3166      WHEN p_bmd.
3167        lv_gt_unit   = 'BOEPD' .
3168      WHEN OTHERS.
3169    ENDCASE.
3170
3171    PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
3172    PERFORM set_range USING lv_gt_unit  0.
3173    PERFORM set_range_formatting USING 0 'C' 'C' .
3174
3175  ENDFORM .
3176  *&———————————————————————————————————————————————————————————————*
3177  *&      Form  GET_UNIT_DESC
3178  *&———————————————————————————————————————————————————————————————*
3179  *       text
3180  *———————————————————————————————————————————————————————————————*
3181  *      ——>P_PRODUCT  text
3182  *      <——P_IND_UNIT  text
3183  *      <——P_TOTAL_UNIT  text
3184  *———————————————————————————————————————————————————————————————*
3185  FORM get_unit_desc  USING    p_product
3186                      CHANGING p_ind_unit
3187                               p_total_unit.
3188    CASE p_product.
3189      WHEN c_prod_oil.
3190        CASE abap_true.
3191          WHEN p_bb.
3192            p_ind_unit   = 'BBL' .
3193            p_total_unit = 'BBL' .
3194          WHEN p_bbd.
3195            p_ind_unit   = 'BOPD' .
3196            p_total_unit = 'BOPD' .
3197          WHEN p_tm.
3198            p_ind_unit   = 'Tons' .
3199            p_total_unit = 'Tons' .
3200          WHEN p_tmd.
3201            p_ind_unit   = 'TPD' .
3202            p_total_unit = 'TPD' .
3203          WHEN p_mb.
3204            p_ind_unit   = 'MMT' .
3205            p_total_unit = 'MMT' .
3206          WHEN p_bmd.
3207            p_ind_unit   = 'BOPD' .
3208            p_total_unit = 'BOPD' .
3209          WHEN OTHERS.
3210        ENDCASE.
3211      WHEN c_prod_con.
3212        CASE abap_true.
3213          WHEN p_bb.
3214            p_ind_unit   = 'BBL' .
3215            p_total_unit = 'BBL' .
3216          WHEN p_bbd.
3217            p_ind_unit   = 'BCPD' .
3218            p_total_unit = 'BCPD' .
3219          WHEN p_tm.
3220            p_ind_unit   = 'Tons' .
3221            p_total_unit = 'Tons' .
3222          WHEN p_tmd.
3223            p_ind_unit   = 'TPD' .
3224            p_total_unit = 'TPD' .
3225          WHEN p_mb.
3226            p_ind_unit   = 'MMT' .
```

```
3227            p_total_unit = 'MMT' .
3228          WHEN p_bmd.
3229            p_ind_unit   = 'BCPD' .
3230            p_total_unit = 'BCPD' .
3231          WHEN OTHERS.
3232        ENDCASE.
3233      WHEN c_prod_lng.
3234        CASE abap_true.
3235          WHEN p_bb.
3236            p_ind_unit   = 'BOE' .
3237            p_total_unit = 'BOE' .
3238          WHEN p_bbd.
3239            p_ind_unit   = 'BOEPD' .
3240            p_total_unit = 'BOEPD' .
3241          WHEN p_tm.
3242            p_ind_unit   = 'Tons' .
3243            p_total_unit = 'Tons' .
3244          WHEN p_tmd.
3245            p_ind_unit   = 'TPD' .
3246            p_total_unit = 'TPD' .
3247          WHEN p_mb.
3248            p_ind_unit   = 'MMT' .
3249            p_total_unit = 'MMT' .
3250          WHEN p_bmd.
3251            p_ind_unit   = 'MMSCMD' .
3252            p_total_unit = 'MMSCMD' .
3253          WHEN OTHERS.
3254        ENDCASE.
3255      WHEN c_prod_gas.
3256        CASE abap_true.
3257          WHEN p_bb.
3258            p_ind_unit   = 'BOE' .
3259            p_total_unit = 'BOE' .
3260          WHEN p_bbd.
3261            p_ind_unit   = 'BOEPD' .
3262            p_total_unit = 'BOEPD' .
3263          WHEN p_tm.
3264            p_ind_unit   = 'MSCM' .
3265            p_total_unit = 'MSCM' .
3266          WHEN p_tmd.
3267            p_ind_unit   = 'MSCMD' .
3268            p_total_unit = 'MSCMD' .
3269          WHEN p_mb.
3270            p_ind_unit   = 'BCM' .
3271            p_total_unit = 'BCM' .
3272          WHEN p_bmd.
3273            p_ind_unit   = 'MMSCMD' .
3274            p_total_unit = 'MMSCMD' .
3275          WHEN OTHERS.
3276        ENDCASE.
3277      WHEN OTHERS.
3278    ENDCASE.
3279
3280  ENDFORM.
3281  *&---------------------------------------------------------------------*
3282  *&      Form  WRITE_PRODUCT_NAME
3283  *&---------------------------------------------------------------------*
3284  *       text
3285  *---------------------------------------------------------------------*
3286  *      -->P_R_PRODUCT_LOW  text
3287  *---------------------------------------------------------------------*
```

```
FORM write_product_name  USING    p_product.
  DATA : lv_product   TYPE zpra_t_dly_prd-product,
         lv_asset     TYPE zpra_t_dly_prd-asset,
         lv_col_count TYPE sy-tabix.

  LOOP AT gt_dyn_fcat INTO gs_dyn_fcat.
    IF gs_dyn_fcat-fieldname EQ 'COL01'.
    ELSEIF gs_dyn_fcat-fieldname EQ 'COL02'.
    ELSE.
      SPLIT gs_dyn_fcat-fieldname AT '-' INTO lv_product lv_asset .
      IF lv_product EQ p_product .
        IF NOT lv_asset CS 'TOTAL'.
          lv_col_count = lv_col_count + 1.
        ENDIF.
      ENDIF.
    ENDIF.
  ENDLOOP.
  IF lv_col_count IS NOT INITIAL.
    gv_col   = gv_col + 1 .
    gv_s_col = gv_col .
    gv_e_col = gv_col + lv_col_count - 1 .
    gv_col   = gv_e_col .
    PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .

    CALL METHOD OF go_range 'Merge' .
    PERFORM set_range USING gv_txt 0.
    gv_col   = gv_col + 1 .
    gs_product_col-product = p_product .
    gs_product_col-s_col   = gv_s_col  .
    gs_product_col-e_col   = gv_e_col + 1 .
    gs_product_col-desc    = gv_txt    .
    APPEND gs_product_col TO gt_product_col .

  ENDIF.

ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  COLOUR_ALTERNATE_ROWS
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM colour_alternate_rows .
  DATA : lv_lines       TYPE sy-tabix,
         lv_switch_flag TYPE c.
  DESCRIBE TABLE <gfs_dyn_table> LINES lv_lines .
  DO lv_lines TIMES.
    IF lv_switch_flag EQ 'X'.
      CLEAR lv_switch_flag .
      gv_s_row = gv_s_row + 1 .
      CONTINUE .
    ENDIF.
    PERFORM select_range USING gv_s_row gv_s_col gv_s_row gv_e_col  .
    PERFORM set_range_interior USING 14540253 .
    lv_switch_flag = 'X'.
    gv_s_row = gv_s_row + 1 .
  ENDDO.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
```

```
*&      Form  SET_RANGE_INTERIOR
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
*      -->P_1263   text
*---------------------------------------------------------------------*
FORM set_range_interior  USING    p_color.
  GET PROPERTY OF go_range 'interior' = go_interior .
  SET PROPERTY OF go_interior 'Color' = p_color .
ENDFORM.
FORM set_numberformat USING p_format.
  SET PROPERTY OF go_range 'NumberFormat' = p_format .
ENDFORM.

*&---------------------------------------------------------------------*
*&      Form  SET_SECTION1_HEADER_COLORS
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*---------------------------------------------------------------------*
FORM set_section1_header_colors .
  DATA lv_color TYPE sy-tabix .
  PERFORM select_range USING gv_start_row 1 gv_row 2 .
  PERFORM set_range_interior USING gv_dates_colour .

  LOOP AT gt_product_col INTO gs_product_col.
    PERFORM select_range USING gv_start_row gs_product_col-s_col gv_row gs_product_col-e_col .
    CASE  gs_product_col-product.
      WHEN '722000001'.
        lv_color = gv_header_oil_colour .
      WHEN '722000003'.
        lv_color = gv_header_cond_colour .
      WHEN '722000005'.
        lv_color = gv_header_lng_colour .
      WHEN '722000004'.
        lv_color = gv_header_gas_colour .
      WHEN OTHERS.
    ENDCASE.
    PERFORM set_range_interior USING lv_color .
  ENDLOOP.
  PERFORM select_range USING gv_start_row gv_table_columns gv_row gv_table_columns .
  PERFORM set_range_interior USING gv_header_gt_colour .
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  CONVERT_GAS_UNITS
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
*      <--P_ZPRA_T_DLY_PRD  text
*---------------------------------------------------------------------*
FORM convert_gas_units  CHANGING p_zpra_t_dly_prd TYPE zpra_t_dly_prd.
  DATA : lv_qty TYPE char50 .
* First Convert to MCM

  CASE p_zpra_t_dly_prd-prod_vl_uom1 .
    WHEN 'MCM'.
      lv_qty = p_zpra_t_dly_prd-prod_vl_qty1.
```

```
      WHEN 'M3'.
        lv_qty = p_zpra_t_dly_prd-prod_vl_qty1 / 1000000 .
      WHEN 'MCF'.
        lv_qty = p_zpra_t_dly_prd-prod_vl_qty1 / '35.3'.
      WHEN OTHERS.
    ENDCASE.

  * Convert to display UoM
    CASE abap_true.
      WHEN p_bb.
        p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty * 6290 .
      WHEN p_bbd.
        p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty * 6290 .
      WHEN p_tm.
        p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty * 1000 .
      WHEN p_tmd.
        p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty * 1000 .
      WHEN p_mb.
        p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty * 1000 .
      WHEN p_bmd.
        p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty * 6290.
      WHEN OTHERS.
    ENDCASE.
  ENDFORM.
  *&---------------------------------------------------------------------*
  *&      Form  CONVERT_GAS_UNITS2
  *&---------------------------------------------------------------------*
  *       text
  *----------------------------------------------------------------------*
  *      <--P_ZPRA_T_DLY_PRD  text
  *----------------------------------------------------------------------*
  FORM convert_gas_units2  CHANGING p_zpra_t_dly_prd TYPE zpra_t_dly_prd.
    DATA : lv_qty TYPE char50 .
  * First Convert to MCM

    CASE p_zpra_t_dly_prd-prod_vl_uom1 .
      WHEN 'MCM'.
        lv_qty = p_zpra_t_dly_prd-prod_vl_qty1.
      WHEN 'M3'.
        lv_qty = p_zpra_t_dly_prd-prod_vl_qty1 / 1000000 .
      WHEN 'MCF'.
        lv_qty = p_zpra_t_dly_prd-prod_vl_qty1 / '35.3'.
      WHEN OTHERS.
    ENDCASE.

  * Convert to display UoM
    CASE abap_true.
      WHEN p_bb.
        p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty .              "* 6290 .
      WHEN p_bbd.
        p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty .              "* 6290 .
      WHEN p_tm.
        p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty * 1000 .
      WHEN p_tmd.
        p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty * 1000 .
      WHEN p_mb.
        p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty * 1000 .
      WHEN p_bmd.
        p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty .              "* 6290.
      WHEN OTHERS.
    ENDCASE.
```

```
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  CONVERT_GAS_UNITS_TO_BOPD
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      <--P_ZPRA_T_DLY_PRD  text
*----------------------------------------------------------------------*
FORM convert_gas_units_to_bopd  CHANGING p_zpra_t_dly_prd TYPE zpra_t_dly_prd.
  DATA : lv_qty TYPE char50 .
* First Convert to MCM

  CASE p_zpra_t_dly_prd-prod_vl_uom1 .
    WHEN 'MCM'.
      lv_qty = p_zpra_t_dly_prd-prod_vl_qty1.
    WHEN 'M3'.
      lv_qty = p_zpra_t_dly_prd-prod_vl_qty1 / 1000000 .
    WHEN 'MCF'.
      lv_qty = p_zpra_t_dly_prd-prod_vl_qty1 / '35.3'.
    WHEN OTHERS.
  ENDCASE.

  p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty * 6290 .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  CONVERT_GAS_UNITS_TO_BCM
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      <--P_ZPRA_T_DLY_PRD  text
*----------------------------------------------------------------------*
FORM convert_gas_units_to_bcm  CHANGING p_zpra_t_dly_prd TYPE zpra_t_dly_prd.
  DATA : lv_qty TYPE char50 .
* First Convert to MCM

  CASE p_zpra_t_dly_prd-prod_vl_uom1 .
    WHEN 'MCM'.
      lv_qty = p_zpra_t_dly_prd-prod_vl_qty1.
    WHEN 'M3'.
      lv_qty = p_zpra_t_dly_prd-prod_vl_qty1 / 1000000 .
    WHEN 'MCF'.
      lv_qty = p_zpra_t_dly_prd-prod_vl_qty1 / '35.3'.
    WHEN OTHERS.
  ENDCASE.
* BCM being small unit, dividing here makes decimals error.
* Will divide at end..
* p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty / 1000 .
  p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty * 1000 .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  CONVERT_GAS_UNITS_TO_MMSCM
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      <--P_ZPRA_T_DLY_PRD  text
*----------------------------------------------------------------------*
FORM convert_gas_units_to_mmscm  CHANGING p_zpra_t_dly_prd TYPE zpra_t_d
```

```
ly_prd.
  DATA : lv_qty TYPE char50 .
* First Convert to MCM
* Multiplying by 1000 due to decimal precision, will divide later
  CASE p_zpra_t_dly_prd-prod_vl_uom1 .
    WHEN 'MCM'.
      lv_qty = 1000 * p_zpra_t_dly_prd-prod_vl_qty1 .
    WHEN 'M3'.
      lv_qty = 1000 * p_zpra_t_dly_prd-prod_vl_qty1 / 1000000 .
    WHEN 'MCF'.
      lv_qty = 1000 * p_zpra_t_dly_prd-prod_vl_qty1 / '35.3'.
    WHEN OTHERS.
  ENDCASE.

* p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty * 6290 .
  p_zpra_t_dly_prd-prod_vl_qty1 = lv_qty  .

ENDFORM.

*&———————————————————————————————————————————————————————————*
*&      Form  CONVERT_MREC_GAS_UNITS
*&———————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————*
*      <——P_ZPRA_T_MREC_PRD  text
*———————————————————————————————————————————————————————————*
FORM convert_mrec_gas_units  CHANGING p_zpra_t_mrec_prd TYPE ty_zpra_t_m
rec_prd.
  DATA : lv_qty TYPE char50 .
* First Convert to MCM

  CASE p_zpra_t_mrec_prd-prod_vl_uom1 .
    WHEN 'MCM'.
      lv_qty = p_zpra_t_mrec_prd-prod_vl_qty1.
    WHEN 'M3'.
      lv_qty = p_zpra_t_mrec_prd-prod_vl_qty1 / 1000000 .
    WHEN 'MCF'.
      lv_qty = p_zpra_t_mrec_prd-prod_vl_qty1 / '35.3'.
    WHEN OTHERS.
  ENDCASE.

* Convert to display UoM
  CASE abap_true.
    WHEN p_bb.
      p_zpra_t_mrec_prd-prod_vl_qty1 = lv_qty * 6290 .
    WHEN p_bbd.
      p_zpra_t_mrec_prd-prod_vl_qty1 = lv_qty * 6290 .
    WHEN p_tm.
      p_zpra_t_mrec_prd-prod_vl_qty1 = lv_qty * 1000 .
    WHEN p_tmd.
      p_zpra_t_mrec_prd-prod_vl_qty1 = lv_qty * 1000 .
    WHEN p_mb.
      p_zpra_t_mrec_prd-prod_vl_qty1 = lv_qty * 1000 .
    WHEN p_bmd.
      p_zpra_t_mrec_prd-prod_vl_qty1 = lv_qty * 6290.
    WHEN OTHERS.
  ENDCASE.
ENDFORM.
*&———————————————————————————————————————————————————————————*
*&      Form  CONVERT_MREC_TO_BCM
*&———————————————————————————————————————————————————————————*
```

```
*       text
*————————————————————————————————————————————————————————————————————*
*      <——P_ZPRA_T_MREC_PRD  text
*————————————————————————————————————————————————————————————————————*
FORM convert_mrec_to_bcm  CHANGING p_zpra_t_mrec_prd TYPE ty_zpra_t_mrec_prd.
  DATA : lv_qty TYPE char50 .
* First Convert to MCM

  CASE p_zpra_t_mrec_prd-prod_vl_uom1 .
    WHEN 'MCM'.
      lv_qty = p_zpra_t_mrec_prd-prod_vl_qty1.
    WHEN 'M3'.
      lv_qty = p_zpra_t_mrec_prd-prod_vl_qty1 / 1000000 .
    WHEN 'MCF'.
      lv_qty = p_zpra_t_mrec_prd-prod_vl_qty1 / '35.3'.
    WHEN OTHERS.
  ENDCASE.

  p_zpra_t_mrec_prd-prod_vl_qty1 = lv_qty * 1000 .
ENDFORM.
*&————————————————————————————————————————————————————————————————————*
*&      Form  CONVERT_MREC_GAS_UNITS_TO_BOE
*&————————————————————————————————————————————————————————————————————*
*       text
*————————————————————————————————————————————————————————————————————*
*      <——P_ZPRA_T_MREC_PRD  text
*————————————————————————————————————————————————————————————————————*
FORM convert_mrec_gas_to_mmscm  CHANGING p_zpra_t_mrec_app TYPE ty_zpra_t_mrec_app.
  DATA : lv_qty TYPE char50 .
* First Convert to MCM
* Mutiplying by 1000 for decimal error precision, will divide later
* lv_qty = p_zpra_t_mrec_app-app_vl_qty * 1000 .
  p_zpra_t_mrec_app-app_vl_qty = 1000 * p_zpra_t_mrec_app-app_vl_qty * 1000 .

* Convert to BOE
* p_zpra_t_mrec_app-app_vl_qty = lv_qty * 6290 .

ENDFORM.

*&————————————————————————————————————————————————————————————————————*
*&      Form  CONVERT_NON_GAS_UNITS
*&————————————————————————————————————————————————————————————————————*
*       text
*————————————————————————————————————————————————————————————————————*
*      <——P_GS_ZPRA_T_DLY_PRD  text
*————————————————————————————————————————————————————————————————————*
FORM convert_non_gas_units  CHANGING p_zpra_t_dly_prd TYPE zpra_t_dly_prd.
  CHECK p_zpra_t_dly_prd-product NE '722000004' .
  CLEAR gs_wtd_cf .
  READ TABLE gt_wtd_cf INTO gs_wtd_cf WITH KEY product = p_zpra_t_dly_prd-product
                                                asset   = p_zpra_t_dly_prd-asset BINARY SEARCH .
  CASE abap_true.
    WHEN p_bb.
    WHEN p_bbd.
    WHEN p_tm.
```

```
      IF gs_wtd_cf-cf IS NOT INITIAL.
        p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / gs_wtd_cf-cf .
      ENDIF.
    WHEN p_tmd.
      IF gs_wtd_cf-cf IS NOT INITIAL.
        p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / gs_wtd_cf-cf .
      ENDIF.
    WHEN p_mb .
      IF gs_wtd_cf-cf IS NOT INITIAL.
        p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / gs_wtd_cf-cf .
*      p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / 1000000 .
* Converting to TONS now. will convert to MMT at end
      ENDIF.
    WHEN p_bmd.
    WHEN OTHERS.
  ENDCASE.

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  CONVERT_NON_GAS_UNITS2
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      <--P_GS_ZPRA_T_DLY_PRD  text
*----------------------------------------------------------------------*
FORM convert_non_gas_units2  USING p_days CHANGING p_zpra_t_dly_prd TYPE zpra_t_dly_prd .
  DATA : lv_cf TYPE zpra_t_tar_cf-conv_factor .
  DATA : lv_buper TYPE t009b-poper,
         lv_monat TYPE zpra_t_mrec_prd-monat,
         lv_gjahr TYPE t009b-bdatj.

  CHECK p_zpra_t_dly_prd-product NE c_prod_gas .
  CLEAR gs_zpra_t_mrec_prd .

  CALL FUNCTION 'DATE_TO_PERIOD_CONVERT'
    EXPORTING
      i_date         = p_zpra_t_dly_prd-production_date
    " I_MONMIT       = 00
      i_periv        = gv_periv
    IMPORTING
      e_buper        = lv_buper
      e_gjahr        = lv_gjahr
    EXCEPTIONS
      input_false    = 01
      t009_notfound  = 02
      t009b_notfound = 03.
  lv_monat = lv_buper+1(2) .


  LOOP AT gt_zpra_t_mrec_prd INTO gs_zpra_t_mrec_prd WHERE  product = p_zpra_t_dly_prd-product
                                                        AND  asset   = p_zpra_t_dly_prd-asset
                                                        AND  block   = p_zpra_t_dly_prd-block
                                                        AND  prd_vl_type
```

```
= p_zpra_t_dly_prd-prd_vl_type . " No binary search
    IF gs_zpra_t_mrec_prd-gjahr GT lv_gjahr.
      CLEAR gs_zpra_t_mrec_prd .
      CONTINUE .
    ENDIF.
    IF gs_zpra_t_mrec_prd-gjahr EQ lv_gjahr AND
       gs_zpra_t_mrec_prd-monat GT lv_monat .
      CLEAR gs_zpra_t_mrec_prd .
      CONTINUE .
    ENDIF.
    EXIT .
  ENDLOOP.
  CASE abap_true.
    WHEN p_bb.
    WHEN p_bbd.
      p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / p_days .
    WHEN p_tm.
      IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL.
        lv_cf = gs_zpra_t_mrec_prd-prod_vl_qty1 /  gs_zpra_t_mrec_prd-prod_vl_qty2 .
      ELSE.
        PERFORM get_cf_from_date USING  p_zpra_t_dly_prd-production_date
                                        p_zpra_t_dly_prd-product
                                        p_zpra_t_dly_prd-asset
                                        p_zpra_t_dly_prd-block
                                CHANGING lv_cf.
      ENDIF.
      IF lv_cf IS NOT INITIAL.
        p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / lv_cf .
      ENDIF.
    WHEN p_tmd.
      IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL.
        lv_cf = gs_zpra_t_mrec_prd-prod_vl_qty1 /  gs_zpra_t_mrec_prd-prod_vl_qty2 .
      ELSE.
        PERFORM get_cf_from_date USING  p_zpra_t_dly_prd-production_date
                                        p_zpra_t_dly_prd-product
                                        p_zpra_t_dly_prd-asset
                                        p_zpra_t_dly_prd-block
                                CHANGING lv_cf.
      ENDIF.
      IF lv_cf IS NOT INITIAL.
        p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / lv_cf .
      ENDIF.
      p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / p_days .

    WHEN p_mb .
      IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL.
        lv_cf = gs_zpra_t_mrec_prd-prod_vl_qty1 /  gs_zpra_t_mrec_prd-prod_vl_qty2 .
      ELSE.
        PERFORM get_cf_from_date USING  p_zpra_t_dly_prd-production_date
                                        p_zpra_t_dly_prd-product
                                        p_zpra_t_dly_prd-asset
                                        p_zpra_t_dly_prd-block
                                CHANGING lv_cf.
      ENDIF.
```

```
      IF lv_cf IS NOT INITIAL.
        p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / lv_cf .
*       p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / 1000000 .
      ENDIF.
    WHEN p_bmd.
      p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / p_days .
    WHEN OTHERS.
  ENDCASE.

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  CONVERT_NON_GAS_UNITS3C
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      <--P_GS_ZPRA_T_DLY_PRD  text
*----------------------------------------------------------------------*
FORM convert_non_gas_units3c  USING p_monat TYPE zpra_t_mrec_prd-monat CHANGING p_zpra_t_dly_prd TYPE zpra_t_dly_prd .
  DATA : lv_cf TYPE zpra_t_mrec_prd-prod_vl_qty1 .
  DATA : lv_buper TYPE t009b-poper,
         lv_monat TYPE zpra_t_mrec_prd-monat,
         lv_gjahr TYPE t009b-bdatj.

  CHECK p_zpra_t_dly_prd-product NE c_prod_gas .
  CLEAR gs_zpra_t_mrec_prd .

  CALL FUNCTION 'DATE_TO_PERIOD_CONVERT'
    EXPORTING
      i_date         = p_zpra_t_dly_prd-production_date
    " I_MONMIT       = 00
      i_periv        = gv_periv
    IMPORTING
      e_buper        = lv_buper
      e_gjahr        = lv_gjahr
    EXCEPTIONS
      input_false    = 01
      t009_notfound  = 02
      t009b_notfound = 03.
  lv_monat = lv_buper+1(2) .



  LOOP AT gt_zpra_t_mrec_prd_3c INTO gs_zpra_t_mrec_prd WHERE product EQ p_zpra_t_dly_prd-product
                                                              AND asset   EQ p_zpra_t_dly_prd-asset
                                                              AND block   EQ p_zpra_t_dly_prd-block
                                                              AND prd_vl_type EQ p_zpra_t_dly_prd-prd_vl_type . " No binary search
    IF gs_zpra_t_mrec_prd-gjahr GT lv_gjahr.
      CLEAR gs_zpra_t_mrec_prd .
      CONTINUE .
    ENDIF.
    IF gs_zpra_t_mrec_prd-gjahr EQ lv_gjahr AND
       gs_zpra_t_mrec_prd-monat GT lv_monat .
      CLEAR gs_zpra_t_mrec_prd .
```

```
      CONTINUE .
    ENDIF.
    EXIT .
  ENDLOOP .
  IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL.
    lv_cf = gs_zpra_t_mrec_prd-prod_vl_qty1 / gs_zpra_t_mrec_prd-prod_vl_qty2 .
  ELSE .
    PERFORM get_cf_from_date USING  p_zpra_t_dly_prd-production_date
                                    p_zpra_t_dly_prd-product
                                    p_zpra_t_dly_prd-asset
                                    p_zpra_t_dly_prd-block
                             CHANGING lv_cf.
  ENDIF.
  IF lv_cf IS NOT INITIAL.
    p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / lv_cf .
*   p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / 1000 .
  ELSE.
*   CLEAR p_zpra_t_dly_prd-prod_vl_qty1 .
  ENDIF.
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  CONVERT_RPRD_UNITS
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      -->P_ZPRA_T_DLY_RPRD  text
*----------------------------------------------------------------------*
FORM convert_rprd_units_mb  USING    p_zpra_t_dly_rprd TYPE zpra_t_dly_rprd.

  IF  p_zpra_t_dly_rprd-product NE c_prod_gas.
*  CASE abap_true.
*    WHEN p_bb OR p_bbd OR p_bmd.
*      IF p_c_jv IS NOT INITIAL.
*        p_zpra_t_dly_rprd-jv_prd_vl_qty1  = p_zpra_t_dly_rprd-jv_rcn_vl_qty2 .
*        p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty2 .
*      ELSE.
*        p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty2 .
*        p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty2 .
*      ENDIF.
*    WHEN p_tm OR p_tmd.
*      IF p_c_jv IS NOT INITIAL.
*        p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-jv_rcn_vl_qty3 .
*        p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 .
*      ELSE.
*        p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 .
*        p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 .
*      ENDIF.
*    WHEN p_mb.
***      IF p_c_jv IS NOT INITIAL.
```

```
3850  ***         p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-jv_rcn_v
      l_qty3 / 1000000 .
3851  ***         p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn
      _vl_qty3 / 1000000 .
3852  ***       ELSE.
3853      p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3
        .
3854      p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty
      3  .
3855  ***       ENDIF.
3856  *     WHEN OTHERS.
3857  *   ENDCASE.
3858    ELSE .
3859  * CASE abap_true.
3860  *    WHEN p_bb OR p_bbd.
3861  *     IF p_c_jv IS NOT INITIAL.
3862  *       p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-jv_rcn_vl_qt
      y3 * 6290 .
3863  *       p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_
      qty3 * 6290  .
3864  *     ELSE.
3865  *       p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_q
      ty3 * 6290 .
3866  *       p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_
      qty3 * 6290  .
3867  *     ENDIF.
3868  *    WHEN p_tm OR p_tmd.
3869  *     IF p_c_jv IS NOT INITIAL.
3870  *       p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-jv_rcn_vl_qt
      y3 * 1000 .
3871  *       p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_
      qty3 * 1000  .
3872  *     ELSE.
3873  *       p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_q
      ty3 * 1000 .
3874  *       p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_
      qty3 * 1000  .
3875  *     ENDIF.
3876  *    WHEN p_mb.
3877  ***     IF p_c_jv IS NOT INITIAL.
3878  ***       p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-jv_rcn_vl_
      qty3 / 1000 .
3879  ***       p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_v
      l_qty3 / 1000  .
3880  ***     ELSE.
3881      p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3
        * 1000.
3882      p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty
      3  * 1000 .
3883  ***     ENDIF.
3884  *    WHEN p_bmd.
3885  *     IF p_c_jv IS NOT INITIAL.
3886  *       p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-jv_rcn_vl_qt
      y3 .
3887  *       p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_
      qty3 .
3888  *     ELSE.
3889  *       p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_q
      ty3 .
3890  *       p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_
      qty3 .
```

```
*    ENDIF.
*   WHEN OTHERS.
* ENDCASE.

  ENDIF.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  CONVERT_RPRD_UNITS
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*      ——>P_ZPRA_T_DLY_RPRD  text
*———————————————————————————————————————————————————————————————*
FORM convert_rprd_units  USING    p_zpra_t_dly_rprd TYPE zpra_t_dly_rprd
.

  IF  p_zpra_t_dly_rprd-product NE c_prod_gas.
    CASE abap_true.
      WHEN p_bb OR p_bbd OR p_bmd.
        IF p_c_jv IS NOT INITIAL.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1  = p_zpra_t_dly_rprd-jv_rcn_v
l_qty2 .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_
vl_qty2 .
        ELSE.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_v
l_qty2 .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_
vl_qty2 .
        ENDIF.
      WHEN p_tm OR p_tmd.
        IF p_c_jv IS NOT INITIAL.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-jv_rcn_vl
_qty3 .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_
vl_qty3 .
        ELSE.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_v
l_qty3 .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_
vl_qty3 .
        ENDIF.
      WHEN p_mb.
        IF p_c_jv IS NOT INITIAL.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-jv_rcn_vl
_qty3 .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_
vl_qty3 .
        ELSE.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_v
l_qty3   .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_
vl_qty3  .
        ENDIF.
      WHEN OTHERS.
    ENDCASE.
  ELSE .
    CASE abap_true.
      WHEN p_bb OR p_bbd.
        IF p_c_jv IS NOT INITIAL.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-jv_rcn_vl
```

```
_qty3 * 6290 .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 * 6290  .
        ELSE.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 * 6290 .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 * 6290  .
        ENDIF.
      WHEN p_tm OR p_tmd.
        IF p_c_jv IS NOT INITIAL.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-jv_rcn_vl_qty3 * 1000 .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 * 1000  .
        ELSE.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 * 1000 .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 * 1000  .
        ENDIF.
      WHEN p_mb.
        IF p_c_jv IS NOT INITIAL.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-jv_rcn_vl_qty3 * 1000 .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 * 1000  .
        ELSE.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3  * 1000.
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3  * 1000 .
        ENDIF.
      WHEN p_bmd.
        IF p_c_jv IS NOT INITIAL.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-jv_rcn_vl_qty3 * 6290 .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 * 6290 .
        ELSE.
          p_zpra_t_dly_rprd-jv_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 * 6290 .
          p_zpra_t_dly_rprd-ovl_prd_vl_qty1 = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3 * 6290 .
        ENDIF.
      WHEN OTHERS.
    ENDCASE.

  ENDIF.
ENDFORM.

*&---------------------------------------------------------------------*
*&      Form  CONVERT_NON_GAS_UNITS3F
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      <--P_GS_ZPRA_T_DLY_PRD  text
*----------------------------------------------------------------------*
FORM convert_non_gas_units3f  USING p_days CHANGING p_zpra_t_dly_prd TYPE zpra_t_dly_prd .
  DATA : lv_cf TYPE zpra_t_tar_cf-conv_factor .
```

```
3983
3984    CHECK p_zpra_t_dly_prd-product NE c_prod_gas.
3985    CLEAR gs_zpra_t_mrec_prd .
3986
3987    DATA : lv_buper TYPE t009b-poper,
3988           lv_monat TYPE zpra_t_mrec_prd-monat,
3989           lv_gjahr TYPE t009b-bdatj.
3990
3991    CALL FUNCTION 'DATE_TO_PERIOD_CONVERT'
3992      EXPORTING
3993        i_date         = p_zpra_t_dly_prd-production_date
3994      " I_MONMIT       = 00
3995        i_periv        = gv_periv
3996      IMPORTING
3997        e_buper        = lv_buper
3998        e_gjahr        = lv_gjahr
3999      EXCEPTIONS
4000        input_false    = 01
4001        t009_notfound  = 02
4002        t009b_notfound = 03.
4003    lv_monat = lv_buper+1(2) .
4004
4005    LOOP AT gt_zpra_t_mrec_prd_3f INTO gs_zpra_t_mrec_prd WHERE product =
      p_zpra_t_dly_prd-product
4006                                                            AND       a
      sset   = p_zpra_t_dly_prd-asset
4007                                                            AND       b
      lock   = p_zpra_t_dly_prd-block
4008                                                            AND   prd_v
      l_type = p_zpra_t_dly_prd-prd_vl_type . " No binary search
4009      IF gs_zpra_t_mrec_prd-gjahr GT lv_gjahr.
4010        CLEAR gs_zpra_t_mrec_prd .
4011        CONTINUE .
4012      ENDIF.
4013      IF gs_zpra_t_mrec_prd-gjahr EQ lv_gjahr AND
4014         gs_zpra_t_mrec_prd-monat GT lv_monat .
4015        CLEAR gs_zpra_t_mrec_prd .
4016        CONTINUE .
4017      ENDIF.
4018      EXIT .
4019    ENDLOOP .
4020    IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL.
4021      lv_cf = gs_zpra_t_mrec_prd-prod_vl_qty1 /  gs_zpra_t_mrec_prd-prod_v
      l_qty2 .
4022    ELSE.
4023      PERFORM get_cf_from_date USING  p_zpra_t_dly_prd-production_date
4024                                      p_zpra_t_dly_prd-product
4025                                      p_zpra_t_dly_prd-asset
4026                                      p_zpra_t_dly_prd-block
4027                              CHANGING lv_cf.
4028    ENDIF.
4029    IF lv_cf IS NOT INITIAL.
4030      p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / lv_c
      f .
4031  *      p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / 1
      000000 .
4032    ENDIF.
4033  ENDFORM.
4034  *&———————————————————————————————————————————————————————————*
4035  *&      Form  CONVERT_NON_GAS_MREC_UNITS
4036  *&———————————————————————————————————————————————————————————*
```

```
*       text
*————————————————————————————————————————————————————————————————————*
*      <——P_GS_ZPRA_T_DLY_PRD  text
*————————————————————————————————————————————————————————————————————*
FORM convert_non_gas_mrec_units  USING p_days CHANGING p_zpra_t_mrec_prd TYPE ty_zpra_t_mrec_prd.
  CHECK p_zpra_t_mrec_prd-product NE c_prod_gas.
  CASE abap_true.
    WHEN p_bb.
    WHEN p_bbd.
      p_zpra_t_mrec_prd-prod_vl_qty1 = p_zpra_t_mrec_prd-prod_vl_qty1 / p_days .
    WHEN p_tm.
      p_zpra_t_mrec_prd-prod_vl_qty1 = p_zpra_t_mrec_prd-prod_vl_qty2 .
    WHEN p_tmd.
      p_zpra_t_mrec_prd-prod_vl_qty1 = p_zpra_t_mrec_prd-prod_vl_qty2 / p_days.
    WHEN p_mb .
*     IF gs_wtd_cf-cf IS NOT INITIAL.
      p_zpra_t_mrec_prd-prod_vl_qty1  = p_zpra_t_mrec_prd-prod_vl_qty2 .
*      p_zpra_t_mrec_prd-prod_vl_qty1  = p_zpra_t_mrec_prd-prod_vl_qty1 / 1000000 .
*     ENDIF.
    WHEN p_bmd.
      p_zpra_t_mrec_prd-prod_vl_qty1 = p_zpra_t_mrec_prd-prod_vl_qty1 / p_days .
    WHEN OTHERS.
  ENDCASE.
ENDFORM.
FORM convert_non_gas_units_2a2  CHANGING p_zpra_t_dly_prd TYPE zpra_t_dly_prd.

  DATA : lv_cf TYPE zpra_t_tar_cf-conv_factor .
  CHECK p_zpra_t_dly_prd-product NE c_prod_gas .
  CLEAR gs_zpra_t_mrec_prd .
  READ TABLE gt_zpra_t_mrec_prd INTO gs_zpra_t_mrec_prd WITH KEY product = p_zpra_t_dly_prd-product
                                                                 asset = p_zpra_t_dly_prd-asset
                                                                 block = p_zpra_t_dly_prd-block
                                                                 prd_vl_type = p_zpra_t_dly_prd-prd_vl_type . " No binary search
  CASE abap_true.
    WHEN p_bb.
    WHEN p_bbd.
      p_zpra_t_dly_prd-prod_vl_qty1 =  p_zpra_t_dly_prd-prod_vl_qty1 ." / ( p_date - gv_month_begin_datum + 1 ) .
    WHEN p_tm.
      IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL.
        lv_cf = gs_zpra_t_mrec_prd-prod_vl_qty1 /  gs_zpra_t_mrec_prd-prod_vl_qty2 .
      ELSE.
        PERFORM get_cf_from_date USING  p_zpra_t_dly_prd-production_date
                                        p_zpra_t_dly_prd-product
                                        p_zpra_t_dly_prd-asset
                                        p_zpra_t_dly_prd-block
                                CHANGING lv_cf.
      ENDIF.
      IF lv_cf IS NOT INITIAL.
```

```
        p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / lv_cf .
      ENDIF.
    WHEN p_tmd.
      IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL.
        lv_cf = gs_zpra_t_mrec_prd-prod_vl_qty1 /  gs_zpra_t_mrec_prd-prod_vl_qty2 .
      ELSE.
        PERFORM get_cf_from_date USING  p_zpra_t_dly_prd-production_date
                                        p_zpra_t_dly_prd-product
                                        p_zpra_t_dly_prd-asset
                                        p_zpra_t_dly_prd-block
                                CHANGING lv_cf.
      ENDIF.
      IF lv_cf IS NOT INITIAL.
        p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / lv_cf .
*      p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / gs_zpra_t_mrec_prd-prod_vl_qty1 *  gs_zpra_t_mrec_prd-prod_vl_qty2 .
*      p_zpra_t_dly_prd-prod_vl_qty1 =  p_zpra_t_dly_prd-prod_vl_qty1 ." / ( p_date - gv_month_begin_datum + 1 ) .
      ENDIF.
    WHEN p_mb .
      IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL.
        lv_cf = gs_zpra_t_mrec_prd-prod_vl_qty1 /  gs_zpra_t_mrec_prd-prod_vl_qty2 .
      ELSE.
        PERFORM get_cf_from_date USING  p_zpra_t_dly_prd-production_date
                                        p_zpra_t_dly_prd-product
                                        p_zpra_t_dly_prd-asset
                                        p_zpra_t_dly_prd-block
                                CHANGING lv_cf.
      ENDIF.
      IF lv_cf IS NOT INITIAL.
        p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / lv_cf .
*      p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / gs_zpra_t_mrec_prd-prod_vl_qty1 *  gs_zpra_t_mrec_prd-prod_vl_qty2 .
*      p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / 1000000 .
      ENDIF.
    WHEN p_bmd.
      p_zpra_t_dly_prd-prod_vl_qty1 =  p_zpra_t_dly_prd-prod_vl_qty1 ." / ( p_date - gv_month_begin_datum + 1 ) .
    WHEN OTHERS.
  ENDCASE.

ENDFORM.
FORM convert_sec2a_units  CHANGING p_zpra_t_prd_tar TYPE ty_zpra_t_prd_tar.
  CLEAR gs_zpra_t_tar_cf .
  READ TABLE gt_zpra_t_tar_cf INTO gs_zpra_t_tar_cf WITH KEY gjahr = gv_current_gjahr
                                                             asset = p_zpra_t_prd_tar-asset
                                                             block = p_zpra_t_prd_tar-block
                                                             product = p_zpra_t_prd_tar-product BINARY SEARCH .
  IF p_zpra_t_prd_tar-product NE c_prod_gas .
    CASE abap_true.
```

```
      WHEN p_bb.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 * gs_zpra_t_tar_cf-conv_factor.
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 * gs_zpra_t_tar_cf-conv_factor.
      WHEN p_bbd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 * gs_zpra_t_tar_cf-conv_factor .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 * gs_zpra_t_tar_cf-conv_factor .
      WHEN p_tm.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 .
      WHEN p_tmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 .
      WHEN p_mb .
      WHEN p_bmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty   * 1000000 * gs_zpra_t_tar_cf-conv_factor .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2  * gs_zpra_t_tar_cf-conv_factor * 1000000 .
      WHEN OTHERS.
    ENDCASE.
  ELSE.
    CASE abap_true.
      WHEN p_bb.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty  * 1000 * 6290 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000 * 6290 .
      WHEN p_bbd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty  * 1000 * 6290 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000 * 6290 .
      WHEN p_tm.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 .
      WHEN p_tmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 .
      WHEN p_mb .
      WHEN p_bmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty  * 1000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000 .
      WHEN OTHERS.
    ENDCASE.
  ENDIF.
ENDFORM.
FORM convert_sec2b_units  CHANGING p_zpra_t_prd_tar TYPE ty_zpra_t_prd_tar.
  CLEAR gs_zpra_t_tar_cf .
  READ TABLE gt_zpra_t_tar_cf INTO gs_zpra_t_tar_cf WITH KEY gjahr = gv_current_gjahr
                                                             asset = p_z
```

```
pra_t_prd_tar-asset
                                                                    block = p_z
pra_t_prd_tar-block
                                                                    product = p
_zpra_t_prd_tar-product BINARY SEARCH .
  IF p_zpra_t_prd_tar-product NE c_prod_gas .
    CASE abap_true.
      WHEN p_bb.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000
* gs_zpra_t_tar_cf-conv_factor.
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000
* gs_zpra_t_tar_cf-conv_factor.
      WHEN p_bbd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000
* gs_zpra_t_tar_cf-conv_factor. " / gv_current_gjahr_days.
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000
* gs_zpra_t_tar_cf-conv_factor. " / gv_current_gjahr_days.
      WHEN p_tm.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000
.
      WHEN p_tmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
" / gv_current_gjahr_days.
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000
. "/ gv_current_gjahr_days .
      WHEN p_mb .
      WHEN p_bmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000
* gs_zpra_t_tar_cf-conv_factor ."/ gv_current_gjahr_days.
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000
* gs_zpra_t_tar_cf-conv_factor ."/ gv_current_gjahr_days.
      WHEN OTHERS.
    ENDCASE.
  ELSE.
    CASE abap_true.
      WHEN p_bb.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty  * 1000 * 6
290 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000 * 6
290 .
      WHEN p_bbd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty  * 1000 * 6
290 ."/ gv_current_gjahr_days.
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000 * 6
290 ."/ gv_current_gjahr_days.
      WHEN p_tm.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000
.
      WHEN p_tmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000.
" / gv_current_gjahr_days.
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000.
" / gv_current_gjahr_days .
      WHEN p_mb .
      WHEN p_bmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty  * 1000 ."
/ gv_current_gjahr_days.
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000 ."/
 gv_current_gjahr_days.
```

```
      WHEN OTHERS.
    ENDCASE.
  ENDIF.
ENDFORM.
FORM convert_sec2c_units  CHANGING p_zpra_t_prd_tar TYPE ty_zpra_t_prd_tar.
  CLEAR gs_zpra_t_tar_cf .
  READ TABLE gt_zpra_t_tar_cf INTO gs_zpra_t_tar_cf WITH KEY gjahr = gv_current_gjahr
                                                             asset = p_zpra_t_prd_tar-asset
                                                             block = p_zpra_t_prd_tar-block
                                                             product = p_zpra_t_prd_tar-product BINARY SEARCH .
  IF p_zpra_t_prd_tar-product NE c_prod_gas .
    CASE abap_true.
      WHEN p_bb.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 * gs_zpra_t_tar_cf-conv_factor.
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 * gs_zpra_t_tar_cf-conv_factor.
      WHEN p_bbd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 * gs_zpra_t_tar_cf-conv_factor .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 * gs_zpra_t_tar_cf-conv_factor .
      WHEN p_tm.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 .
      WHEN p_tmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 .
      WHEN p_mb .
      WHEN p_bmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 * gs_zpra_t_tar_cf-conv_factor .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 * gs_zpra_t_tar_cf-conv_factor .
      WHEN OTHERS.
    ENDCASE.
  ELSE.
    CASE abap_true.
      WHEN p_bb.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty  * 1000 * 6290 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000 * 6290 .
      WHEN p_bbd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty  * 1000 * 6290 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000 * 6290 .
      WHEN p_tm.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 .
      WHEN p_tmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
```

```
4258          p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000
      .
4259        WHEN p_mb .
4260        WHEN p_bmd.
4261          p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty  * 1000 .
4262          p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000 .
4263        WHEN OTHERS.
4264      ENDCASE.
4265    ENDIF.
4266  ENDFORM.
4267  FORM convert_sec5a_units  CHANGING p_zpra_t_prd_tar TYPE ty_zpra_t_prd_t
      ar.
4268    CLEAR gs_zpra_t_tar_cf .
4269    READ TABLE gt_zpra_t_tar_cf INTO gs_zpra_t_tar_cf WITH KEY gjahr = gv_
      current_gjahr
4270                                                               asset = p_z
      pra_t_prd_tar-asset
4271                                                               block = p_z
      pra_t_prd_tar-block
4272                                                               product = p
      _zpra_t_prd_tar-product BINARY SEARCH .
4273    IF p_zpra_t_prd_tar-product NE c_prod_gas .
4274      CASE abap_true.
4275        WHEN p_bb.
4276          p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000
      * gs_zpra_t_tar_cf-conv_factor.
4277          p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000
      * gs_zpra_t_tar_cf-conv_factor.
4278        WHEN p_bbd.
4279          p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000
      * gs_zpra_t_tar_cf-conv_factor .
4280          p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000
      * gs_zpra_t_tar_cf-conv_factor .
4281        WHEN p_tm.
4282          p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
4283          p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000
      .
4284        WHEN p_tmd.
4285          p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
4286          p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000
      .
4287        WHEN p_mb .
4288        WHEN p_bmd.
4289          p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000
      * gs_zpra_t_tar_cf-conv_factor .
4290          p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000
      * gs_zpra_t_tar_cf-conv_factor .
4291        WHEN OTHERS.
4292      ENDCASE.
4293    ELSE.
4294      CASE abap_true.
4295        WHEN p_bb.
4296          p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty  * 1000 * 6
      290 .
4297          p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000 * 6
      290 .
4298        WHEN p_bbd.
4299          p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty  * 1000 * 6
      290 .
4300          p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000 * 6
      290 .
```

```
      WHEN p_tm.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 .
      WHEN p_tmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty * 1000000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000000 .
      WHEN p_mb .
      WHEN p_bmd.
        p_zpra_t_prd_tar-tar_qty  = p_zpra_t_prd_tar-tar_qty  * 1000 .
        p_zpra_t_prd_tar-tar_qty2 = p_zpra_t_prd_tar-tar_qty2 * 1000 .
      WHEN OTHERS.
    ENDCASE.
  ENDIF.
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  GET_PI_FOR_NULL_VALUES
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM get_pi_for_null_values USING p_zpra_t_dly_prd TYPE zpra_t_dly_prd
                                  p_combine.
  CLEAR gs_wtd_cf .
  IF p_combine IS NOT INITIAL.
    READ TABLE gt_wtd_pi INTO gs_wtd_pi WITH KEY production_date = p_zpra_t_dly_prd-production_date
                                                  product = p_zpra_t_dly_prd-product
                                                    asset = 'COMBINE' BINARY SEARCH  .

  ELSE.
    READ TABLE gt_wtd_pi INTO gs_wtd_pi WITH KEY production_date = p_zpra_t_dly_prd-production_date
                                                  product = p_zpra_t_dly_prd-product
                                                    asset = p_zpra_t_dly_prd-asset BINARY SEARCH .
  ENDIF.
  IF sy-subrc IS INITIAL.
    CASE abap_true .
      WHEN p_c_jv.
        gv_individual_mul   = 1 .
        gv_total_mul        = gs_wtd_pi-pi / 100 .
        gv_total_mul        = gs_wtd_pi-pi / 100 .
      WHEN p_c_olv .
        gv_individual_mul = gs_wtd_pi-pi / 100  .
        gv_total_mul      = gs_wtd_pi-pi / 100  ..
      WHEN OTHERS.
    ENDCASE.
    gv_grand_total_mul = gv_total_mul .
    IF p_bmd IS NOT INITIAL AND p_zpra_t_dly_prd-product EQ c_prod_gas.
*      gv_grand_total_mul = gv_grand_total_mul * 6290 .
    ENDIF.
  ELSE.
    gv_individual_mul   = 0 .
    gv_total_mul        = 0 .
```

```
    gv_grand_total_mul  = 0 .
  ENDIF.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  GET_CONSTORIUM_MULTIPLIERS
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  ——>  p1        text
*  <——  p2        text
*———————————————————————————————————————————————————————————————*
FORM get_constorium_multipliers USING p_zpra_t_dly_prd TYPE zpra_t_dly_prd
                                      p_combine.
*  CLEAR gs_wtd_cf .
*  IF p_combine IS NOT INITIAL.
*    READ TABLE gt_wtd_pi INTO gs_wtd_pi WITH KEY production_date = p_zpra_t_dly_prd-production_date
*                                                  product = p_zpra_t_dly_prd-product
*                                                    asset = 'COMBINE' BINARY SEARCH  .
*
*  ELSE.
*    READ TABLE gt_wtd_pi INTO gs_wtd_pi WITH KEY production_date = p_zpra_t_dly_prd-production_date
*                                                  product = p_zpra_t_dly_prd-product
*                                                    asset = p_zpra_t_dly_prd-asset BINARY SEARCH .
*  ENDIF.
*  IF sy-subrc IS INITIAL.
*    CASE abap_true .
*      WHEN p_c_jv.
*        gv_individual_mul   = 1 .
*        gv_total_mul        = gs_wtd_pi-pi / 100 .
*        gv_total_mul        = gs_wtd_pi-pi / 100 .
*      WHEN p_c_olv .
*        gv_individual_mul = gs_wtd_pi-pi / 100  .
*        gv_total_mul      = gs_wtd_pi-pi / 100  ..
*      WHEN OTHERS.
*    ENDCASE.
*    gv_grand_total_mul = gv_total_mul .
*    IF p_bmd IS NOT INITIAL AND p_zpra_t_dly_prd-product EQ c_prod_gas.
*      gv_grand_total_mul = gv_grand_total_mul * 6290 .
*    ENDIF.
*  ELSE.
*    gv_individual_mul   = 0 .
*    gv_total_mul        = 0 .
*    gv_grand_total_mul  = 0 .
*  ENDIF.
  LOOP AT gt_zpra_t_prd_pi INTO gs_zpra_t_prd_pi WHERE asset   EQ p_zpra_t_dly_prd-asset
                                                       AND block   EQ p_zpra_t_dly_prd-block
                                                       AND vld_frm LE p_zpra_t_dly_prd-production_date
                                                       AND vld_to  GE p_zpra_t_dly_prd-production_date .
    EXIT .
  ENDLOOP .
```

```
  CASE abap_true .
    WHEN p_c_jv.
      gv_individual_mul = 1 .
      gv_total_mul      = gs_zpra_t_prd_pi-pi / 100 .
    WHEN p_c_olv .
      gv_individual_mul = gs_zpra_t_prd_pi-pi / 100  .
      gv_total_mul      = gs_zpra_t_prd_pi-pi / 100  .
    WHEN OTHERS.
  ENDCASE.
  gv_grand_total_mul = gv_total_mul .
  IF p_bmd IS NOT INITIAL AND p_zpra_t_dly_prd-product EQ c_prod_gas..
*    gv_grand_total_mul = gv_grand_total_mul * 6290 .
  ENDIF.

ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  GET_CONSTORIUM_MULTIPLIERS_2A2
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM get_constorium_multipliers_2a2 USING p_table.
  CLEAR gs_zpra_t_prd_pi .
  DATA: lv_numerator   TYPE                  zpra_t_dly_prd-prod_vl_qty 1,
        lv_denominator TYPE                  zpra_t_dly_prd-prod_vl_qty 1,
        lv_prod_gas    TYPE                   c,
        lv_low_date    TYPE                  sy-datum   VALUE '99991231',
        lv_high_date   TYPE                  sy-datum   VALUE '19000101'.

  IF p_table EQ '1' .
    LOOP AT gt_zpra_t_prd_pi INTO gs_zpra_t_prd_pi WHERE asset   EQ gs_zpra_t_dly_prd-asset
                                                     AND block   EQ gs_zpra_t_dly_prd-block
                                                     AND vld_frm LE gs_zpra_t_dly_prd-production_date
                                                     AND vld_to  GE gs_zpra_t_dly_prd-production_date .
      EXIT .
    ENDLOOP .
    IF gs_zpra_t_dly_prd-product EQ c_prod_gas .
      lv_prod_gas = 'X' .
    ENDIF.
  ELSEIF p_table EQ '2' .
    LOOP AT gt_zpra_t_prd_pi_mb INTO gs_zpra_t_prd_pi WHERE asset   EQ gs_zpra_t_dly_prd-asset
                                                     AND block   EQ gs_zpra_t_dly_prd-block
                                                     AND vld_frm LE gs_zpra_t_dly_prd-production_date
                                                     AND vld_to  GE gs_zpra_t_dly_prd-production_date .
      EXIT .
    ENDLOOP .
    IF gs_zpra_t_dly_prd-product EQ c_prod_gas .
```

```
      lv_prod_gas = 'X' .
    ENDIF.
  ELSEIF p_table EQ '3' .
    LOOP AT gt_zpra_t_prd_pi_lm INTO gs_zpra_t_prd_pi WHERE asset   EQ gs_zpra_t_mrec_prd-asset
                                                            AND block   EQ gs_zpra_t_mrec_prd-block
                                                            AND vld_frm LE gv_month_back_end_datum
                                                            AND vld_to  GE gv_month_back_begin_datum .
      IF gs_zpra_t_prd_pi-vld_frm LT gv_month_back_begin_datum .
        gs_zpra_t_prd_pi-vld_frm = gv_month_back_begin_datum  .
      ENDIF.
      IF gs_zpra_t_prd_pi-vld_to GT gv_month_back_end_datum.
        gs_zpra_t_prd_pi-vld_to = gv_month_back_end_datum .
      ENDIF.
      IF gs_zpra_t_prd_pi-vld_frm LT lv_low_date.
        lv_low_date = gs_zpra_t_prd_pi-vld_frm .
      ENDIF.
      IF gs_zpra_t_prd_pi-vld_to GT lv_high_date.
        lv_high_date = gs_zpra_t_prd_pi-vld_to .
      ENDIF.

      lv_numerator = lv_numerator + gs_zpra_t_prd_pi-pi * ( gs_zpra_t_prd_pi-vld_to - gs_zpra_t_prd_pi-vld_frm + 1 ) .
    ENDLOOP .
*    gs_zpra_t_prd_pi-pi = lv_numerator / ( gv_month_back_end_datum - gv_month_back_begin_datum + 1 ) .
    gs_zpra_t_prd_pi-pi = lv_numerator / ( lv_high_date - lv_low_date + 1 ) .

    IF gs_zpra_t_mrec_prd-product EQ c_prod_gas .
      lv_prod_gas = 'X' .
    ENDIF.
  ELSEIF p_table EQ '4' .
    LOOP AT gt_zpra_t_prd_pi_lm INTO gs_zpra_t_prd_pi WHERE asset   EQ gs_zpra_t_dly_prd-asset
                                                            AND block   EQ gs_zpra_t_dly_prd-block
                                                            AND vld_frm LE gs_zpra_t_dly_prd-production_date
                                                            AND vld_to  GE gs_zpra_t_dly_prd-production_date .
      EXIT .
    ENDLOOP .
    IF gs_zpra_t_dly_prd-product EQ c_prod_gas .
      lv_prod_gas = 'X' .
    ENDIF.
  ELSEIF p_table EQ '5' .
    LOOP AT gt_zpra_t_prd_pi_2d INTO gs_zpra_t_prd_pi WHERE asset   EQ gs_zpra_t_mrec_prd-asset
                                                            AND block   EQ gs_zpra_t_mrec_prd-block
                                                            AND vld_frm LE gv_search_end_datum
                                                            AND vld_to  GE gv_search_begin_datum .
      IF gs_zpra_t_prd_pi-vld_frm LT gv_search_begin_datum .
        gs_zpra_t_prd_pi-vld_frm = gv_search_begin_datum  .
      ENDIF.
```

```
4499        IF gs_zpra_t_prd_pi-vld_to GT gv_search_end_datum.
4500          gs_zpra_t_prd_pi-vld_to = gv_search_end_datum .
4501        ENDIF.
4502        IF gs_zpra_t_prd_pi-vld_frm LT lv_low_date.
4503          lv_low_date = gs_zpra_t_prd_pi-vld_frm .
4504        ENDIF.
4505        IF gs_zpra_t_prd_pi-vld_to GT lv_high_date.
4506          lv_high_date = gs_zpra_t_prd_pi-vld_to .
4507        ENDIF.
4508        lv_numerator = lv_numerator + gs_zpra_t_prd_pi-pi * ( gs_zpra_t_pr
      d_pi-vld_to - gs_zpra_t_prd_pi-vld_frm + 1 ) .
4509      ENDLOOP .
4510  *    gs_zpra_t_prd_pi-pi = lv_numerator / ( gv_search_end_datum - gv_sea
      rch_begin_datum + 1 ) .
4511      gs_zpra_t_prd_pi-pi = lv_numerator / ( lv_high_date - lv_low_date +
      1 ) .
4512      IF gs_zpra_t_mrec_prd-product EQ c_prod_gas .
4513        lv_prod_gas = 'X' .
4514      ENDIF.
4515    ELSEIF p_table EQ '6' .
4516      LOOP AT gt_zpra_t_prd_pi_2f INTO gs_zpra_t_prd_pi WHERE asset   EQ g
      s_zpra_t_mrec_prd-asset
4517                                                              AND block   EQ g
      s_zpra_t_mrec_prd-block
4518                                                              AND vld_frm LE g
      v_search_end_datum
4519                                                              AND vld_to  GE g
      v_search_begin_datum .
4520        IF gs_zpra_t_prd_pi-vld_frm LT gv_search_begin_datum .
4521          gs_zpra_t_prd_pi-vld_frm = gv_search_begin_datum  .
4522        ENDIF.
4523        IF gs_zpra_t_prd_pi-vld_to GT gv_search_end_datum.
4524          gs_zpra_t_prd_pi-vld_to = gv_search_end_datum .
4525        ENDIF.
4526        IF gs_zpra_t_prd_pi-vld_frm LT lv_low_date.
4527          lv_low_date = gs_zpra_t_prd_pi-vld_frm .
4528        ENDIF.
4529        IF gs_zpra_t_prd_pi-vld_to GT lv_high_date.
4530          lv_high_date = gs_zpra_t_prd_pi-vld_to .
4531        ENDIF.
4532
4533        lv_numerator = lv_numerator + gs_zpra_t_prd_pi-pi * ( gs_zpra_t_pr
      d_pi-vld_to - gs_zpra_t_prd_pi-vld_frm + 1 ) .
4534      ENDLOOP .
4535  *    gs_zpra_t_prd_pi-pi = lv_numerator / ( gv_search_end_datum - gv_sea
      rch_begin_datum + 1 ) .
4536      gs_zpra_t_prd_pi-pi = lv_numerator / ( lv_high_date - lv_low_date +
      1 ) .
4537
4538      IF gs_zpra_t_mrec_prd-product EQ c_prod_gas .
4539        lv_prod_gas = 'X' .
4540      ENDIF.
4541    ELSEIF p_table EQ '7' .
4542
4543      LOOP AT gt_zpra_t_prd_pi_3c INTO gs_zpra_t_prd_pi WHERE asset   EQ g
      s_zpra_t_dly_prd-asset
4544                                                              AND block   EQ g
      s_zpra_t_dly_prd-block
4545                                                              AND vld_frm LE g
      s_zpra_t_dly_prd-production_date
4546                                                              AND vld_to  GE g
```

```
s_zpra_t_dly_prd-production_date .
      EXIT .
    ENDLOOP .
    IF gs_zpra_t_dly_prd-product EQ c_prod_gas .
      lv_prod_gas = 'X' .
    ENDIF.
  ELSEIF p_table EQ '8' .

    LOOP AT gt_zpra_t_prd_pi_3f INTO gs_zpra_t_prd_pi WHERE asset   EQ g
s_zpra_t_dly_prd-asset
                                                        AND block   EQ g
s_zpra_t_dly_prd-block
                                                        AND vld_frm LE g
s_zpra_t_dly_prd-production_date
                                                        AND vld_to  GE g
s_zpra_t_dly_prd-production_date .
      EXIT .
    ENDLOOP .
    IF gs_zpra_t_dly_prd-product EQ c_prod_gas .
      lv_prod_gas = 'X' .
    ENDIF.
  ELSEIF p_table EQ '9' .

    LOOP AT gt_zpra_t_prd_pi_4a INTO gs_zpra_t_prd_pi WHERE asset   EQ g
s_zpra_t_dly_prd-asset
                                                        AND block   EQ g
s_zpra_t_dly_prd-block
                                                        AND vld_frm LE g
s_zpra_t_dly_prd-production_date
                                                        AND vld_to  GE g
s_zpra_t_dly_prd-production_date .
      EXIT .
    ENDLOOP .
    IF gs_zpra_t_dly_prd-product EQ c_prod_gas .
      lv_prod_gas = 'X' .
    ENDIF.
  ENDIF.
  CASE abap_true .
    WHEN p_c_jv.
      gv_individual_mul = 1 .
      gv_total_mul      = gs_zpra_t_prd_pi-pi / 100 .
    WHEN p_c_olv .
      gv_individual_mul = gs_zpra_t_prd_pi-pi / 100  .
      gv_total_mul      = gs_zpra_t_prd_pi-pi / 100  .
    WHEN OTHERS.
  ENDCASE.
  gv_grand_total_mul = gv_total_mul .
  IF p_bmd IS NOT INITIAL AND lv_prod_gas IS NOT INITIAL.
*    gv_grand_total_mul = gv_grand_total_mul * 6290 .
  ENDIF.
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  SET_COLUMN_WIDTH
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM set_columns_width .
  DATA lv_end_col TYPE sy-tabix .
```

```
  lv_end_col = gv_table_columns - 1 .
  PERFORM col_width USING 1 1 '25'  .
  PERFORM col_width USING 2 2 '11.15'  .
  PERFORM col_width USING 3 lv_end_col '11.5'  .
  LOOP AT gt_product_col INTO gs_product_col.
    PERFORM col_width USING gs_product_col-e_col gs_product_col-e_col '12'  .
  ENDLOOP.
  PERFORM col_width USING gv_table_columns gv_table_columns '12'  .

ENDFORM.
FORM col_width  USING p_col_start
                      p_col_end
                      p_width.
*CALL METHOD OF go_excel 'Columns' = go_column
*  EXPORTING
*    #1 = p_col.

  PERFORM select_range USING 1 p_col_start 1 p_col_end  .
  SET PROPERTY OF go_range 'ColumnWidth' = p_width.

* PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  COLOUR_DATES
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM colour_dates .
  DATA : lv_lines   TYPE sy-tabix,
         lv_end_row TYPE sy-tabix.
  DESCRIBE TABLE <gfs_dyn_table> LINES lv_lines .

  lv_end_row = gv_sec1_data_start_row + lv_lines - 1 .

  PERFORM select_range USING gv_sec1_data_start_row 1 lv_end_row 2  .

  GET PROPERTY OF go_range 'interior' = go_interior .
  SET PROPERTY OF go_interior 'Color' = gv_dates_colour .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  JOIN_HEADER_TOTAL_CELLS
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM join_header_total_cells .
  DATA : lv_end_row   TYPE sy-tabix .
  lv_end_row = gv_sec1_h_start_row + 1 .
  LOOP AT gt_product_col INTO gs_product_col.
    PERFORM select_range USING gv_sec1_h_start_row gs_product_col-e_col lv_end_row gs_product_col-e_col .
    CALL METHOD OF go_range 'Merge' .
  ENDLOOP.
```

```
* Grand Total
  gs_product_col-e_col = gs_product_col-e_col + 1.
  PERFORM select_range USING gv_sec1_h_start_row gs_product_col-e_col lv_end_row gs_product_col-e_col .
  CALL METHOD OF go_range 'Merge' .

ENDFORM.
FORM join_header1_column_1_2.
  PERFORM select_range USING gv_sec1_h_start_row 1 gv_sec1_h_start_row 2 .
  CALL METHOD OF go_range 'Merge' .
ENDFORM .

*&———————————————————————————————————————————————————————————————————*
*&      Form  COLOUR_SEC1_DATA_TOTALS
*&———————————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————————*
FORM colour_sec1_data_totals .

  DATA : lv_end_row TYPE sy-tabix,
         lv_color   TYPE sy-tabix,
         lv_lines   TYPE sy-tabix.

  DESCRIBE TABLE <gfs_dyn_table> LINES lv_lines .

  lv_end_row = gv_sec1_data_start_row + lv_lines - 1 .

  LOOP AT gt_product_col INTO gs_product_col.
    PERFORM select_range USING gv_sec1_data_start_row gs_product_col-e_col  lv_end_row gs_product_col-e_col.
    CASE  gs_product_col-product.
      WHEN '722000001'.
        lv_color = gv_header_oil_colour .
      WHEN '722000003'.
        lv_color = gv_header_cond_colour .
      WHEN '722000005'.
        lv_color = gv_header_lng_colour .
      WHEN '722000004'.
        lv_color = gv_header_gas_colour .
      WHEN OTHERS.
    ENDCASE.
    PERFORM set_range_interior USING lv_color .
  ENDLOOP.
  gs_product_col-e_col = gs_product_col-e_col + 1.
  PERFORM select_range USING gv_sec1_data_start_row gs_product_col-e_col lv_end_row gs_product_col-e_col .
  PERFORM set_range_interior USING gv_header_gt_colour .

ENDFORM.
*&———————————————————————————————————————————————————————————————————*
*&      Form  PROCESS_SEC2A_DATA
*&———————————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————————*
```

```
FORM process_sec2a_data .
  SORT gt_zpra_t_mrec_prd BY product ASCENDING asset ASCENDING block ASCENDING prd_vl_type ASCENDING  gjahr DESCENDING monat DESCENDING .

  PERFORM convert_sec2a_to_jvl.
  PERFORM prepare_dynamic_table_sec2a1 .
  PERFORM fill_dynamic_table_sec2a1    .
  gv_colour = gv_sec2a_tgt_colour .
  CONCATENATE 'Target' ':' gv_month_name gv_current_calendar_gjahr INTO gv_txt SEPARATED BY space.
  PERFORM display_section2a1 .

  PERFORM prepare_dynamic_table_sec2a2 .
  PERFORM populate_no_data_entries TABLES gt_zpra_t_dly_prd USING gv_month_back_datum p_date .
  PERFORM fill_dynamic_table_sec2a2    .
  PERFORM display_section2a2 .

  PERFORM fetch_data_section2a3 .
  PERFORM process_gas_records_2a3 .
  PERFORM prepare_dynamic_table_sec2a3 .
  PERFORM fill_dynamic_table_sec2a3    .
  PERFORM display_section2a3 .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  CONVERT_SEC2A_TO_JVL
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM convert_sec2a_to_jvl .
  FIELD-SYMBOLS : <lfs_zpra_t_prd_tar> TYPE ty_zpra_t_prd_tar .
  DATA:  lv_numerator                   TYPE ty_zpra_t_prd_tar-tar_qty .
  LOOP AT gt_zpra_t_prd_tar ASSIGNING <lfs_zpra_t_prd_tar>.
    CLEAR lv_numerator .
    <lfs_zpra_t_prd_tar>-tar_qty2 = <lfs_zpra_t_prd_tar>-tar_qty .
    IF p_c_jv IS NOT INITIAL.
      LOOP AT gt_zpra_t_tar_pi INTO gs_zpra_t_tar_pi WHERE asset     EQ <lfs_zpra_t_prd_tar>-asset
                                                           AND block     EQ <lfs_zpra_t_prd_tar>-block
                                                           AND tar_code  EQ <lfs_zpra_t_prd_tar>-tar_code.
        IF gs_zpra_t_tar_pi-vld_frm LT gv_month_begin_datum.
          gs_zpra_t_tar_pi-vld_frm = gv_month_begin_datum .
        ENDIF.
        IF gs_zpra_t_tar_pi-vld_to GT gv_month_end_datum.
          gs_zpra_t_tar_pi-vld_to = gv_month_end_datum .
        ENDIF.
        IF gs_zpra_t_tar_pi-pi IS NOT INITIAL.
          lv_numerator = lv_numerator + ( gs_zpra_t_tar_pi-vld_to - gs_zpra_t_tar_pi-vld_frm + 1 ) / gs_zpra_t_tar_pi-pi  .
        ENDIF.
      ENDLOOP.
      <lfs_zpra_t_prd_tar>-tar_qty = <lfs_zpra_t_prd_tar>-tar_qty * lv_numerator * 100 / gv_current_month_days .
    ENDIF.
  ENDLOOP.
```

```
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  CONVERT_SEC2B_TO_JVL
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM convert_sec2b_to_jvl .
  FIELD-SYMBOLS : <lfs_zpra_t_prd_tar> TYPE ty_zpra_t_prd_tar .
  DATA :lv_numerator TYPE                    p LENGTH 16 DECIMALS 9,
        lv_first_day TYPE                    sy-datum,
        lv_last_day  TYPE                    sy-datum.

  LOOP AT gt_zpra_t_prd_tar ASSIGNING <lfs_zpra_t_prd_tar>.
    CLEAR lv_numerator .
    <lfs_zpra_t_prd_tar>-tar_qty2 = <lfs_zpra_t_prd_tar>-tar_qty .
    CALL FUNCTION 'PERIOD_DAY_DETERMINE'
      EXPORTING
        i_gjahr              = <lfs_zpra_t_prd_tar>-gjahr
        i_monat              = <lfs_zpra_t_prd_tar>-monat
        i_periv              = gv_periv
      IMPORTING
        e_fday               = lv_first_day
        e_lday               = lv_last_day
*       E_SPERIOD            =
      EXCEPTIONS
        error_period         = 1
        error_period_version = 2
        firstday_not_defined = 3
        period_not_defined   = 4
        year_invalid         = 5
        OTHERS               = 6.
    IF sy-subrc <> 0.
      MESSAGE 'Internal Date error' TYPE 'E' .
    ENDIF.

    IF p_c_jv IS NOT INITIAL.
      LOOP AT gt_zpra_t_tar_pi INTO gs_zpra_t_tar_pi WHERE asset     EQ <lfs_zpra_t_prd_tar>-asset
                                                           AND block     EQ <lfs_zpra_t_prd_tar>-block
                                                           AND tar_code  EQ <lfs_zpra_t_prd_tar>-tar_code .
        IF gs_zpra_t_tar_pi-vld_to LT lv_first_day OR gs_zpra_t_tar_pi-vld_frm GT lv_last_day.
          CONTINUE.
        ENDIF.
        IF gs_zpra_t_tar_pi-vld_frm LT lv_first_day.
          gs_zpra_t_tar_pi-vld_frm = lv_first_day .
        ENDIF.
        IF gs_zpra_t_tar_pi-vld_to GT lv_last_day.
          gs_zpra_t_tar_pi-vld_to = lv_last_day .
        ENDIF.
        IF gs_zpra_t_tar_pi-pi IS NOT INITIAL.
          lv_numerator = lv_numerator + ( gs_zpra_t_tar_pi-vld_to - gs_zpra_t_tar_pi-vld_frm + 1 ) / gs_zpra_t_tar_pi-pi  .
        ENDIF.
      ENDLOOP.
      <lfs_zpra_t_prd_tar>-tar_qty = <lfs_zpra_t_prd_tar>-tar_qty * lv_n
```

```
umerator * 100 / ( lv_last_day - lv_first_day + 1 ) .
    ENDIF.
  ENDLOOP.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  CONVERT_SEC2C_TO_JVL
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM convert_sec2c_to_jvl .
  FIELD-SYMBOLS : <lfs_zpra_t_prd_tar> TYPE ty_zpra_t_prd_tar .
  DATA :lv_numerator TYPE                  p LENGTH 16 DECIMALS 9,
        lv_first_day TYPE                  sy-datum,
        lv_last_day  TYPE                  sy-datum.
  LOOP AT gt_zpra_t_prd_tar_2c ASSIGNING <lfs_zpra_t_prd_tar>.
    CLEAR lv_numerator .

    CALL FUNCTION 'PERIOD_DAY_DETERMINE'
      EXPORTING
        i_gjahr              = <lfs_zpra_t_prd_tar>-gjahr
        i_monat              = <lfs_zpra_t_prd_tar>-monat
        i_periv              = gv_periv
      IMPORTING
        e_fday               = lv_first_day
        e_lday               = lv_last_day
*       E_SPERIOD            =
      EXCEPTIONS
        error_period         = 1
        error_period_version = 2
        firstday_not_defined = 3
        period_not_defined   = 4
        year_invalid         = 5
        OTHERS               = 6.
    IF sy-subrc <> 0.
      MESSAGE 'Internal Date error' TYPE 'E' .
    ENDIF.

    IF <lfs_zpra_t_prd_tar>-monat EQ gv_current_monat.
      <lfs_zpra_t_prd_tar>-tar_qty = <lfs_zpra_t_prd_tar>-tar_qty * ( p_date - lv_first_day + 1 ) / ( lv_last_day - lv_first_day + 1 ) .
      lv_last_day = p_date .
    ENDIF.
    <lfs_zpra_t_prd_tar>-tar_qty2 = <lfs_zpra_t_prd_tar>-tar_qty .

    IF p_c_jv IS NOT INITIAL.
      LOOP AT gt_zpra_t_tar_pi INTO gs_zpra_t_tar_pi WHERE asset     EQ <lfs_zpra_t_prd_tar>-asset
                                                           AND block     EQ <lfs_zpra_t_prd_tar>-block
                                                           AND tar_code  EQ <lfs_zpra_t_prd_tar>-tar_code .
        IF gs_zpra_t_tar_pi-vld_to LT lv_first_day OR gs_zpra_t_tar_pi-vld_frm GT lv_last_day.
          CONTINUE.
        ENDIF.
        IF gs_zpra_t_tar_pi-vld_frm LT lv_first_day.
          gs_zpra_t_tar_pi-vld_frm = lv_first_day .
        ENDIF.
```

```
        IF gs_zpra_t_tar_pi-vld_to GT lv_last_day.
          gs_zpra_t_tar_pi-vld_to = lv_last_day .
        ENDIF.
        IF gs_zpra_t_tar_pi-pi IS NOT INITIAL.
          lv_numerator = lv_numerator + ( gs_zpra_t_tar_pi-vld_to - gs_zpra_t_tar_pi-vld_frm + 1 ) / gs_zpra_t_tar_pi-pi  .
        ENDIF.
      ENDLOOP.
      <lfs_zpra_t_prd_tar>-tar_qty = <lfs_zpra_t_prd_tar>-tar_qty * lv_numerator * 100 / ( lv_last_day - lv_first_day + 1 ) .
    ENDIF.
  ENDLOOP.
ENDFORM.

*&———————————————————————————————————————————————————————————————*
*&      Form  FILL_DYNAMIC_TABLE_SEC2A1
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM fill_dynamic_table_sec2a1 .
  DATA : lv_col_name      TYPE lvc_fname,
         lv_combine_field TYPE c,
         lv_index         TYPE sy-tabix,
         lv_target_name   TYPE char25,
         lv_mmscmd_mul    TYPE sy-tabix,
         lv_days          TYPE sy-tabix,
         lv_start_date    TYPE sy-tabix,
         lv_tar_code      TYPE zpra_t_prd_tar-tar_code.

  SORT gt_zpra_t_prd_tar BY tar_code product asset.
  LOOP AT gt_zpra_t_prd_tar INTO gs_zpra_t_prd_tar .
    CLEAR lv_combine_field .
    AT NEW tar_code .
      APPEND INITIAL LINE TO <gfs_sec2_table> ASSIGNING <gfs_dyn_line> .
      ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      PERFORM get_target_type_name USING gs_zpra_t_prd_tar-tar_code CHANGING lv_target_name.
      <gfs_field> = lv_target_name.
    ENDAT .
    PERFORM convert_sec2a_units CHANGING gs_zpra_t_prd_tar.

    IF gs_zpra_t_prd_tar-product = c_prod_gas.
      READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_t_prd_tar-asset BINARY SEARCH .
      IF  sy-subrc IS INITIAL.
        lv_combine_field = 'X' .
      ENDIF.
    ENDIF.
* Individual Column..
    gv_len = strlen( gs_zpra_t_prd_tar-product ) .
    IF lv_combine_field IS INITIAL.
      CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' gs_zpra_t_prd_tar-asset INTO lv_col_name .
    ELSE.
      CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' 'COMBINE'
        INTO lv_col_name .
    ENDIF.
```

```
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty .
      UNASSIGN <gfs_field> .
    ENDIF.
* Product Total..
    CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' 'TOTAL'
        INTO lv_col_name .
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty2 .
      UNASSIGN <gfs_field> .
    ENDIF.
* Grand Total
    CONCATENATE 'GRAND' '-' 'TOTAL'
         INTO lv_col_name .
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      lv_mmscmd_mul = 1 .
      IF gs_zpra_t_prd_tar-product EQ c_prod_gas AND p_bmd IS NOT INITIAL.
        lv_mmscmd_mul = 6290 .
      ENDIF.
      <gfs_field> = <gfs_field> + ( gs_zpra_t_prd_tar-tar_qty2 * lv_mmscmd_mul ).
      UNASSIGN <gfs_field> .
    ENDIF.
  ENDLOOP.

* Now dividing by no of days in case of per day units
  IF p_bbd IS NOT INITIAL OR
     p_tmd IS NOT INITIAL OR
     p_bmd IS NOT INITIAL .
    LOOP AT <gfs_sec2_table> ASSIGNING <gfs_dyn_line> .
      DO .
        lv_index = sy-index .
        IF lv_index EQ 2.
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
          IF sy-subrc IS INITIAL.
            lv_target_name = <gfs_field> .
            PERFORM get_target_from_name USING lv_target_name CHANGING lv_tar_code.
          ENDIF.
        ENDIF.
        IF lv_index GT 2.
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
          IF sy-subrc IS NOT INITIAL.
            EXIT .
          ENDIF.

          lv_days = gv_month_end_datum - gv_month_begin_datum + 1 .
          PERFORM get_target_start_date USING lv_index lv_tar_code CHANGING lv_start_date .
          IF lv_start_date GT gv_month_begin_datum .
            lv_days =  gv_month_end_datum - lv_start_date + 1 .
          ENDIF.
```

```
          <gfs_field> = <gfs_field> / lv_days .
          UNASSIGN <gfs_field> .
        ENDIF.
      ENDDO.
    ENDLOOP .
  ENDIF.

ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  FILL_DYNAMIC_TABLE_SEC2B
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM fill_dynamic_table_sec2b .
  DATA : lv_col_name      TYPE lvc_fname,
         lv_combine_field TYPE c,
         lv_index         TYPE sy-tabix,
         lv_target_name   TYPE char25,
         lv_mmscmd_mul    TYPE sy-tabix,
         lv_days          TYPE sy-tabix,
         lv_start_date    TYPE sy-tabix,
         lv_tar_code      TYPE zpra_t_prd_tar-tar_code.

  SORT gt_zpra_t_prd_tar BY tar_code .
  LOOP AT gt_zpra_t_prd_tar INTO gs_zpra_t_prd_tar .
    CLEAR lv_combine_field .
    AT NEW tar_code .
      APPEND INITIAL LINE TO <gfs_sec2_table> ASSIGNING <gfs_dyn_line> .
      ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      PERFORM get_target_type_name USING gs_zpra_t_prd_tar-tar_code CHANGING lv_target_name.
      <gfs_field> = lv_target_name.
    ENDAT .
    PERFORM convert_sec2b_units CHANGING gs_zpra_t_prd_tar.

    IF gs_zpra_t_prd_tar-product = c_prod_gas.
      READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_t_prd_tar-asset BINARY SEARCH .
      IF  sy-subrc IS INITIAL.
        lv_combine_field = 'X' .
      ENDIF.
    ENDIF.
* Individual Column..
    gv_len = strlen( gs_zpra_t_prd_tar-product ) .
    IF lv_combine_field IS INITIAL.
      CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' gs_zpra_t_prd_tar-asset INTO lv_col_name .
    ELSE.
      CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' 'COMBINE'
        INTO lv_col_name .
    ENDIF.
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty .
      UNASSIGN <gfs_field> .
    ENDIF.
```

```
* Product Total..
    CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' 'TOTAL'
        INTO lv_col_name .
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty2 .
      UNASSIGN <gfs_field> .
    ENDIF.
* Grand Total
    CONCATENATE 'GRAND' '-' 'TOTAL'
         INTO lv_col_name .
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      lv_mmscmd_mul = 1 .
      IF gs_zpra_t_prd_tar-product EQ c_prod_gas AND p_bmd IS NOT INITIAL.
        lv_mmscmd_mul = 6290 .
      ENDIF.
      <gfs_field> = <gfs_field> + ( gs_zpra_t_prd_tar-tar_qty2 * lv_mmscmd_mul ).
      UNASSIGN <gfs_field> .
    ENDIF.
  ENDLOOP.
* Now dividing by no of days in case of per day units
  IF p_bbd IS NOT INITIAL OR
     p_tmd IS NOT INITIAL OR
     p_bmd IS NOT INITIAL .
    LOOP AT <gfs_sec2_table> ASSIGNING <gfs_dyn_line> .
      DO .
        lv_index = sy-index .
        IF lv_index EQ 2.
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
          IF sy-subrc IS INITIAL.
            lv_target_name = <gfs_field> .
            PERFORM get_target_from_name USING lv_target_name CHANGING lv_tar_code.
          ENDIF.
        ENDIF.
        IF lv_index GT 2.
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
          IF sy-subrc IS NOT INITIAL.
            EXIT .
          ENDIF.

          lv_days = gv_year_end_date - gv_year_start_date + 1 .
          PERFORM get_target_start_date USING lv_index lv_tar_code CHANGING lv_start_date .
          IF lv_start_date GT gv_year_start_date .
            lv_days =  gv_year_end_date - lv_start_date + 1 .
          ENDIF.
          IF <gfs_field> IS NOT INITIAL.
            <gfs_field> = <gfs_field> / lv_days .
          ENDIF.
          UNASSIGN <gfs_field> .
        ENDIF.
      ENDDO.
    ENDLOOP .
```

```
  ENDIF.

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  FILL_DYNAMIC_TABLE_SEC2C
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM fill_dynamic_table_sec2c .
  DATA : lv_col_name      TYPE lvc_fname,
         lv_combine_field TYPE c,
         lv_index         TYPE sy-tabix,
         lv_target_name   TYPE char25,
         lv_mmscmd_mul    TYPE sy-tabix,
         lv_days          TYPE sy-tabix,
         lv_start_date    TYPE sy-tabix,
         lv_tar_code      TYPE zpra_t_prd_tar-tar_code.

  SORT gt_zpra_t_prd_tar_2c BY tar_code .
  LOOP AT gt_zpra_t_prd_tar_2c INTO gs_zpra_t_prd_tar .
    CLEAR lv_combine_field .
    AT NEW tar_code .
      APPEND INITIAL LINE TO <gfs_sec2_table> ASSIGNING <gfs_dyn_line> .
      ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      PERFORM get_target_type_name USING gs_zpra_t_prd_tar-tar_code CHANGING lv_target_name.
      <gfs_field> = lv_target_name.
    ENDAT .
    PERFORM convert_sec2c_units CHANGING gs_zpra_t_prd_tar.

    IF gs_zpra_t_prd_tar-product = c_prod_gas.
      READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_t_prd_tar-asset BINARY SEARCH .
      IF  sy-subrc IS INITIAL.
        lv_combine_field = 'X' .
      ENDIF.
    ENDIF.
* Individual Column..
    gv_len = strlen( gs_zpra_t_prd_tar-product ) .
    IF lv_combine_field IS INITIAL.
      CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' gs_zpra_t_prd_tar-asset INTO lv_col_name .
    ELSE.
      CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' 'COMBINE'
        INTO lv_col_name .
    ENDIF.
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty .
      UNASSIGN <gfs_field> .
    ENDIF.
* Product Total..
    CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' 'TOTAL'
        INTO lv_col_name .
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
```

```
    IF <gfs_field> IS ASSIGNED.
      <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty2 .
      UNASSIGN <gfs_field> .
    ENDIF.
* Grand Total
    CONCATENATE 'GRAND' '-' 'TOTAL'
         INTO lv_col_name .
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      lv_mmscmd_mul = 1 .
      IF gs_zpra_t_prd_tar-product EQ c_prod_gas AND p_bmd IS NOT INITIAL.
        lv_mmscmd_mul = 6290 .
      ENDIF.
      <gfs_field> = <gfs_field> + ( gs_zpra_t_prd_tar-tar_qty2 * lv_mmscmd_mul ).
      UNASSIGN <gfs_field> .
    ENDIF.
  ENDLOOP.
* Now dividing by no of days in case of per day units
  IF p_bbd IS NOT INITIAL OR
     p_tmd IS NOT INITIAL OR
     p_bmd IS NOT INITIAL .
    LOOP AT <gfs_sec2_table> ASSIGNING <gfs_dyn_line> .
      DO .
        lv_index = sy-index .
        IF lv_index EQ 2.
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
          IF sy-subrc IS INITIAL.
            lv_target_name = <gfs_field> .
            PERFORM get_target_from_name USING lv_target_name CHANGING lv_tar_code.
          ENDIF.
        ENDIF.
        IF lv_index GT 2.
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
          IF sy-subrc IS NOT INITIAL.
            EXIT .
          ENDIF.
          lv_days = p_date - gv_year_start_date + 1 .
          PERFORM get_target_start_date USING lv_index lv_tar_code CHANGING lv_start_date .
          IF lv_start_date GT gv_year_start_date .
            lv_days =  p_date - lv_start_date + 1 .
          ENDIF.
          <gfs_field> = <gfs_field> / lv_days .
          UNASSIGN <gfs_field> .
        ENDIF.
      ENDDO.
    ENDLOOP .
  ENDIF.

ENDFORM.

*&———————————————————————————————————————————————————————————*
*&      Form  FILL_DYNAMIC_TABLE_SEC2A2
*&———————————————————————————————————————————————————————————*
*       text
```

```
*————————————————————————————————————————————————————————————————————*
*  ——>  p1        text
*  <——  p2        text
*————————————————————————————————————————————————————————————————————*
FORM fill_dynamic_table_sec2a2 .
  DATA : lv_col_name      TYPE lvc_fname,
         lv_combine_field TYPE c,
         lv_index         TYPE sy-tabix,
         lv_year_index    TYPE sy-tabix,
         lv_last_gjahr    TYPE gjahr,
         lv_days          TYPE sy-tabix,
         lv_days2         TYPE sy-tabix,
         lv_start_date    TYPE sy-datum,
         lv_product       TYPE char100,
         lv_asset         TYPE char100.

  DATA : lv_debug              TYPE c         .

  IF lv_debug IS NOT INITIAL.
    SORT gt_zpra_t_dly_prd BY production_date .
  ENDIF.

  lv_days  = 1 .
  lv_days2 =  p_date - gv_month_begin_datum + 1 .

  REFRESH <gfs_sec2a2_table> .
  APPEND INITIAL LINE TO <gfs_sec2a2_table> ASSIGNING <gfs_dyn_line> .
  ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
  CONCATENATE gv_month_name  gv_current_calendar_gjahr INTO <gfs_field> SEPARATED BY space .
  LOOP AT gt_zpra_t_dly_prd INTO gs_zpra_t_dly_prd WHERE production_date GE gv_month_begin_datum.
    CLEAR lv_combine_field .

    PERFORM convert_non_gas_units_2a2 CHANGING gs_zpra_t_dly_prd.
    PERFORM convert_gas_per_day_units USING lv_days CHANGING gs_zpra_t_dly_prd .
    IF gs_zpra_t_dly_prd-product = c_prod_gas.
      READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_t_dly_prd-asset BINARY SEARCH .
      IF  sy-subrc IS INITIAL.
        lv_combine_field = 'X' .
      ENDIF.
    ENDIF.
    PERFORM get_constorium_multipliers_2a2 USING '1' .
* Individual Column..
    gv_len = strlen( gs_zpra_t_dly_prd-product ) .
    IF lv_combine_field IS INITIAL.
      CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' gs_zpra_t_dly_prd-asset INTO lv_col_name .
    ELSE.
      CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'COMBINE'
        INTO lv_col_name .
    ENDIF.
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_individual_mul.
      UNASSIGN <gfs_field> .
    ENDIF.
```

```
* Product Total..
    CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'TOTAL'
        INTO lv_col_name .
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_total_mul .
      UNASSIGN <gfs_field> .
    ENDIF.
* Grand Total
    CONCATENATE 'GRAND' '-' 'TOTAL'
         INTO lv_col_name .
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_grand_total_mul .
      UNASSIGN <gfs_field> .
    ENDIF.
  ENDLOOP.
* Last Year Same Month Data
  lv_last_gjahr = gv_current_calendar_gjahr - 1 .
  APPEND INITIAL LINE TO <gfs_sec2a2_table> ASSIGNING <gfs_dyn_line> .
  ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
  CONCATENATE  ' ' gv_month_name  INTO <gfs_field>  .
  CONCATENATE <gfs_field> lv_last_gjahr INTO <gfs_field> SEPARATED BY space .
*  CONCATENATE gv_month_name lv_last_gjahr INTO <gfs_field> SEPARATED BY space .
  LOOP AT gt_zpra_t_dly_prd_mb INTO gs_zpra_t_dly_prd .
    CLEAR lv_combine_field .

    PERFORM convert_non_gas_units_2a2 CHANGING gs_zpra_t_dly_prd.
    PERFORM convert_gas_per_day_units USING lv_days CHANGING gs_zpra_t_dly_prd .
    IF gs_zpra_t_dly_prd-product = c_prod_gas .
      READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_t_dly_prd-asset BINARY SEARCH .
      IF  sy-subrc IS INITIAL.
        lv_combine_field = 'X' .
      ENDIF.
    ENDIF.
    PERFORM get_constorium_multipliers_2a2 USING '2'.
* Individual Column..
    gv_len = strlen( gs_zpra_t_dly_prd-product ) .
    IF lv_combine_field IS INITIAL.
      CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' gs_zpra_t_dly_prd-asset INTO lv_col_name .
    ELSE.
      CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'COMBINE'
        INTO lv_col_name .
    ENDIF.
    ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED.
      <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_individual_mul.
      UNASSIGN <gfs_field> .
    ENDIF.
* Product Total..
```

```
5295      CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'TOTAL'
              INTO lv_col_name .
5296      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
      ld> .
5297      IF <gfs_field> IS ASSIGNED.
5298        <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_to
      tal_mul .
5299        UNASSIGN <gfs_field> .
5300      ENDIF.
5301  * Grand Total
5302      CONCATENATE 'GRAND' '-' 'TOTAL'
               INTO lv_col_name .
5303      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
      ld> .
5304      IF <gfs_field> IS ASSIGNED.
5305        <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_gr
      and_total_mul .
5306        UNASSIGN <gfs_field> .
5307      ENDIF.
5308    ENDLOOP.
5309
5310  * Now dividing by no of days in case of per day units
5311    IF p_bbd IS NOT INITIAL OR
5312       p_tmd IS NOT INITIAL OR
5313       p_bmd IS NOT INITIAL .
5314      LOOP AT <gfs_sec2a2_table> ASSIGNING <gfs_dyn_line> .
5315        lv_year_index = sy-tabix .
5316        DO .
5317          lv_index = sy-index .
5318          lv_days2 =  p_date - gv_month_begin_datum + 1 .
5319          IF lv_index GT 2.
5320            ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_
      field> .
5321            IF sy-subrc IS NOT INITIAL.
5322              EXIT .
5323            ENDIF.
5324            PERFORM get_asset_start_date USING lv_index CHANGING lv_start_
      date .
5325            IF lv_year_index = 1.
5326              IF lv_start_date GT gv_month_begin_datum .
5327                lv_days2 =  p_date - lv_start_date + 1 .
5328              ENDIF.
5329            ELSE.
5330              IF lv_start_date GT gv_year_back_begin_datum .
5331                lv_days2 = lv_days2 + gv_year_back_begin_datum - lv_start_
      date .
5332              ENDIF.
5333            ENDIF.
5334            <gfs_field> = <gfs_field> / lv_days2 .
5335            UNASSIGN <gfs_field> .
5336          ENDIF.
5337        ENDDO.
5338      ENDLOOP .
5339    ENDIF.
5340
5341    IF p_mb IS NOT INITIAL.
5342      LOOP AT <gfs_sec2a2_table> ASSIGNING <gfs_dyn_line> .
5343        DO .
5344          lv_index = sy-index .
5345          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_fi
      eld> .
```

```
        IF sy-subrc IS NOT INITIAL.
          EXIT .
        ELSE.
          IF lv_index GT 2.
            <gfs_field> = <gfs_field> / 1000000 .
          ENDIF.
        ENDIF.
      ENDDO.
    ENDLOOP.
  ENDIF.

  IF p_bmd IS NOT INITIAL.
    LOOP AT <gfs_sec2a2_table> ASSIGNING <gfs_dyn_line> .
      DO .
        lv_index = sy-index .
        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
        IF sy-subrc IS NOT INITIAL.
          EXIT .
        ELSE.
          IF lv_index GT 2.
            READ TABLE gt_dyn_fcat INTO gs_dyn_fcat WITH KEY col_pos = lv_index BINARY SEARCH .
            IF sy-subrc IS INITIAL.
              SPLIT gs_dyn_fcat-fieldname AT '-' INTO lv_product lv_asset .
              IF lv_product EQ c_prod_gas.
                <gfs_field> = <gfs_field> / 6290 .
              ENDIF.
            ENDIF.
          ENDIF.
        ENDIF.
      ENDDO.
    ENDLOOP.
  ENDIF.

ENDFORM.
*&———————————————————————————————————————————————————————————————————*
*&      Form  FILL_DYNAMIC_TABLE_SEC2A3
*&———————————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————————*
*  ——>  p1        text
*  <——  p2        text
*———————————————————————————————————————————————————————————————————*
FORM fill_dynamic_table_sec2a3 .
  DATA : lt_zpra_t_dly_prd     TYPE STANDARD TABLE OF zpra_t_dly_prd .
  DATA : ls_zpra_t_dly_prd     TYPE                   zpra_t_dly_prd .
  DATA : lv_col_name      TYPE lvc_fname,
         lv_combine_field TYPE c,
         lv_index         TYPE sy-tabix,
         lv_days          TYPE sy-tabix,
         lv_start_date    TYPE sy-datum,
         lv_product       TYPE char100,
         lv_asset         TYPE char100.

*  lv_days = gv_month_back_end_datum - gv_month_back_begin_datum + 1 .
  lv_days = 1 .
  gt_zpra_t_mrec_prd =  gt_zpra_t_mrec_prd_2a3 .
  lt_zpra_t_dly_prd = gt_zpra_t_dly_prd .
  SORT lt_zpra_t_dly_prd BY product asset block .
```

```
  DELETE ADJACENT DUPLICATES FROM lt_zpra_t_dly_prd COMPARING product as set block .

  APPEND INITIAL LINE TO <gfs_sec2a3_table> ASSIGNING <gfs_dyn_line> .
  ASSIGN COMPONENT 'COL01' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
  <gfs_field> = 'Prod. : Montly Actual' .
  ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
  CONCATENATE gv_last_month_name gv_month_back_calendar_gjahr INTO <gfs_field> .
  LOOP AT gt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof.
    lv_index = sy-tabix .
    CLEAR lv_combine_field .
    READ TABLE gt_zpra_t_mrec_app_2a3 INTO gs_zpra_t_mrec_app WITH KEY gjahr   = gv_month_back_gjahr
                                                                       monat   = gv_month_back_monat
                                                                       product = gs_zpra_c_prd_prof-product
                                                                       asset   = gs_zpra_c_prd_prof-asset
                                                                       block   = gs_zpra_c_prd_prof-block BINARY SEARCH .
    IF sy-subrc IS INITIAL. "if found in MREC APP
      gs_zpra_t_mrec_app-app_vl_qty = gs_zpra_t_mrec_app-app_vl_qty  * 1000000 .
      IF gs_zpra_t_mrec_app-product = c_prod_gas.
        READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_t_mrec_app-asset BINARY SEARCH .
        IF  sy-subrc IS INITIAL.
          lv_combine_field = 'X' .
        ENDIF.
      ENDIF.

*   Individual Column..
      gv_len = strlen( gs_zpra_t_mrec_app-product ) .
      IF lv_combine_field IS INITIAL.
        CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' gs_zpra_t_mrec_app-asset INTO lv_col_name .
      ELSE.
        CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' 'COMBINE'
           INTO lv_col_name .
      ENDIF.
      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      IF <gfs_field> IS ASSIGNED.
        <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty .
        UNASSIGN <gfs_field> .
      ENDIF.
*   Product Total..
      CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' 'TOTAL'
           INTO lv_col_name .
      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      IF <gfs_field> IS ASSIGNED.
        <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty .
        UNASSIGN <gfs_field> .
      ENDIF.
*   Grand Total
      CONCATENATE 'GRAND' '-' 'TOTAL'
           INTO lv_col_name .
      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_f
```

```
ield> .
      IF <gfs_field> IS ASSIGNED.
        <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty.
        UNASSIGN <gfs_field> .
      ENDIF.
    ELSE. "Now work out from MREC

      READ TABLE gt_zpra_t_mrec_prd_2a3 INTO gs_zpra_t_mrec_prd WITH KEY
 product = gs_zpra_c_prd_prof-product

 asset   = gs_zpra_c_prd_prof-asset

 block   = gs_zpra_c_prd_prof-block

 gjahr   = gv_month_back_gjahr

 monat   = gv_month_back_monat BINARY SEARCH .
      IF sy-subrc IS INITIAL. "if found in MREC
        gs_zpra_t_dly_prd = ls_zpra_t_dly_prd .
        PERFORM convert_non_gas_mrec_units USING lv_days CHANGING gs_zpr
a_t_mrec_prd.
        IF gs_zpra_t_mrec_prd-product = c_prod_gas.
          READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH K
EY asset = gs_zpra_t_mrec_prd-asset BINARY SEARCH .
          IF  sy-subrc IS INITIAL.
            lv_combine_field = 'X' .
          ENDIF.
        ENDIF.
        PERFORM get_constorium_multipliers_2a2 USING '3'.

*   Individual Column..
        gv_len = strlen( gs_zpra_t_mrec_prd-product ) .
        IF lv_combine_field IS INITIAL.
          CONCATENATE gs_zpra_t_mrec_prd-product(gv_len) '-' gs_zpra_t_m
rec_prd-asset INTO lv_col_name .
        ELSE.
          CONCATENATE gs_zpra_t_mrec_prd-product(gv_len) '-' 'COMBINE'
             INTO lv_col_name .
        ENDIF.
        ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs
_field> .
        IF <gfs_field> IS ASSIGNED.
          <gfs_field> = <gfs_field> + gs_zpra_t_mrec_prd-prod_vl_qty1 *
gv_individual_mul.
          UNASSIGN <gfs_field> .
        ENDIF.
*   Product Total..
        CONCATENATE gs_zpra_t_mrec_prd-product(gv_len) '-' 'TOTAL'
             INTO lv_col_name .
        ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs
_field> .
        IF <gfs_field> IS ASSIGNED.
          <gfs_field> = <gfs_field> + gs_zpra_t_mrec_prd-prod_vl_qty1 *
gv_total_mul .
          UNASSIGN <gfs_field> .
        ENDIF.
*   Grand Total
        CONCATENATE 'GRAND' '-' 'TOTAL'
             INTO lv_col_name .
        ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs
_field> .
```

```
5494          IF <gfs_field> IS ASSIGNED.
5495            <gfs_field> = <gfs_field> + gs_zpra_t_mrec_prd-prod_vl_qty1 *
      gv_grand_total_mul.
5496            UNASSIGN <gfs_field> .
5497          ENDIF.
5498
5499        ELSE . "Now work out from daily production
5500          LOOP AT gt_zpra_t_dly_prd_2a3 INTO gs_zpra_t_dly_prd WHERE produ
      ct EQ gs_zpra_c_prd_prof-product
5501                                                                  AND asset
         EQ gs_zpra_c_prd_prof-asset
5502                                                                  AND block
         EQ gs_zpra_c_prd_prof-block .
5503            lv_index = sy-tabix .
5504            CLEAR lv_combine_field .
5505
5506            PERFORM convert_non_gas_units2 USING lv_days CHANGING gs_zpra_
      t_dly_prd .
5507
5508            IF gs_zpra_t_dly_prd-product = c_prod_gas.
5509              READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH
       KEY asset = gs_zpra_t_dly_prd-asset BINARY SEARCH .
5510              IF  sy-subrc IS INITIAL.
5511                lv_combine_field = 'X' .
5512              ENDIF.
5513            ENDIF.
5514            PERFORM get_constorium_multipliers_2a2 USING '4'.
5515  *       Individual Column..
5516            gv_len = strlen( gs_zpra_t_dly_prd-product ) .
5517            IF lv_combine_field IS INITIAL.
5518              CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' gs_zpra_t_
      dly_prd-asset INTO lv_col_name .
5519            ELSE.
5520              CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'COMBINE'
                  INTO lv_col_name .
5521            ENDIF.
5522            ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <g
      fs_field> .
5523            IF <gfs_field> IS ASSIGNED.
5524              <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 *
       gv_individual_mul.
5525              UNASSIGN <gfs_field> .
5526            ENDIF.
5527  *       Product Total..
5528            CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'TOTAL'
                  INTO lv_col_name .
5529            ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <g
      fs_field> .
5530            IF <gfs_field> IS ASSIGNED.
5531              <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 *
       gv_total_mul .
5532              UNASSIGN <gfs_field> .
5533            ENDIF.
5534  *       Grand Total
5535            CONCATENATE 'GRAND' '-' 'TOTAL'
                  INTO lv_col_name .
5536            ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <g
      fs_field> .
5537            IF <gfs_field> IS ASSIGNED.
5538              <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 *
       gv_grand_total_mul.
```

```
            UNASSIGN <gfs_field> .
          ENDIF.
        ENDLOOP.
      ENDIF.
    ENDIF.
  ENDLOOP.

* Now dividing by no of days in case of per day units
  IF p_bbd IS NOT INITIAL OR
     p_tmd IS NOT INITIAL OR
     p_bmd IS NOT INITIAL .
    LOOP AT <gfs_sec2a3_table> ASSIGNING <gfs_dyn_line> .
      DO .
        lv_index = sy-index .
        lv_days = gv_month_back_end_datum - gv_month_back_begin_datum + 1 .
        IF lv_index GT 2.
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
          IF sy-subrc IS NOT INITIAL.
            EXIT .
          ENDIF.
          PERFORM get_asset_start_date USING lv_index CHANGING lv_start_date .

          IF lv_start_date GT gv_month_back_begin_datum .
            lv_days =  gv_month_back_end_datum - lv_start_date + 1 .
          ENDIF.

          <gfs_field> = <gfs_field> / lv_days .
          UNASSIGN <gfs_field> .
        ENDIF.
      ENDDO.
    ENDLOOP .
  ENDIF.
  IF p_mb IS NOT INITIAL.
    LOOP AT <gfs_sec2a3_table> ASSIGNING <gfs_dyn_line> .
      DO .
        lv_index = sy-index .
        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
        IF sy-subrc IS NOT INITIAL.
          EXIT .
        ELSE.
          IF lv_index GT 2.
            <gfs_field> = <gfs_field> / 1000000 .
          ENDIF.
        ENDIF.
      ENDDO.
    ENDLOOP.
  ENDIF.

  IF p_bmd IS NOT INITIAL.
    LOOP AT <gfs_sec2a3_table> ASSIGNING <gfs_dyn_line> .
      DO .
        lv_index = sy-index .
        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
        IF sy-subrc IS NOT INITIAL.
          EXIT .
        ELSE.
```

```
          IF lv_index GT 2.
            READ TABLE gt_dyn_fcat INTO gs_dyn_fcat WITH KEY col_pos = lv_index BINARY SEARCH .
            IF sy-subrc IS INITIAL.
              SPLIT gs_dyn_fcat-fieldname AT '-' INTO lv_product lv_asset .
              IF lv_product EQ c_prod_gas.
                <gfs_field> = <gfs_field> / 6290 .
              ENDIF.
            ENDIF.
          ENDIF.
        ENDIF.
      ENDDO.
    ENDLOOP.
  ENDIF.

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  PREPARE_DYNAMIC_TABLE_SEC2
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM prepare_dynamic_table_sec2a1 .
  CALL METHOD cl_alv_table_create=>create_dynamic_table
    EXPORTING
*     i_style_table             = #X#
      it_fieldcatalog           = gt_dyn_fcat
    IMPORTING
      ep_table                  = gt_sec2a1_table
    EXCEPTIONS
      generate_subpool_dir_full = 1
      OTHERS                    = 2.
  IF sy-subrc NE 0.
    MESSAGE 'Unexpected Internal Error' TYPE 'E' .
  ELSE.
* Assign the new table to field symbol
    ASSIGN gt_sec2a1_table->* TO <gfs_sec2_table>.
  ENDIF.
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  PREPARE_DYNAMIC_TABLE_SEC2A2
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM prepare_dynamic_table_sec2a2 .
  CALL METHOD cl_alv_table_create=>create_dynamic_table
    EXPORTING
*     i_style_table             = #X#
      it_fieldcatalog           = gt_dyn_fcat
    IMPORTING
      ep_table                  = gt_sec2a2_table
    EXCEPTIONS
      generate_subpool_dir_full = 1
      OTHERS                    = 2.
  IF sy-subrc NE 0.
```

```
5654      MESSAGE 'Unexpected Internal Error' TYPE 'E' .
5655    ELSE.
5656  * Assign the new table to field symbol
5657      ASSIGN gt_sec2a2_table->* TO <gfs_sec2a2_table>.
5658    ENDIF.
5659  ENDFORM.
5660  *&———————————————————————————————————————————————————————————————*
5661  *&      Form  PREPARE_DYNAMIC_TABLE_SEC2A2
5662  *&———————————————————————————————————————————————————————————————*
5663  *       text
5664  *———————————————————————————————————————————————————————————————*
5665  *  —>  p1        text
5666  *  <—  p2        text
5667  *———————————————————————————————————————————————————————————————*
5668  FORM prepare_dynamic_table_sec2b .
5669    CALL METHOD cl_alv_table_create=>create_dynamic_table
5670      EXPORTING
5671  *     i_style_table             = #X#
5672        it_fieldcatalog           = gt_dyn_fcat
5673      IMPORTING
5674        ep_table                  = gt_sec2b_table
5675      EXCEPTIONS
5676        generate_subpool_dir_full = 1
5677        OTHERS                    = 2.
5678    IF sy-subrc NE 0.
5679      MESSAGE 'Unexpected Internal Error' TYPE 'E' .
5680    ELSE.
5681  * Assign the new table to field symbol
5682      ASSIGN gt_sec2b_table->* TO <gfs_sec2b_table>.
5683    ENDIF.
5684  ENDFORM.
5685  FORM prepare_dynamic_table_sec2c .
5686    CALL METHOD cl_alv_table_create=>create_dynamic_table
5687      EXPORTING
5688  *     i_style_table             = #X#
5689        it_fieldcatalog           = gt_dyn_fcat
5690      IMPORTING
5691        ep_table                  = gt_sec2c_table
5692      EXCEPTIONS
5693        generate_subpool_dir_full = 1
5694        OTHERS                    = 2.
5695    IF sy-subrc NE 0.
5696      MESSAGE 'Unexpected Internal Error' TYPE 'E' .
5697    ELSE.
5698  * Assign the new table to field symbol
5699      ASSIGN gt_sec2c_table->* TO <gfs_sec2c_table>.
5700    ENDIF.
5701  ENDFORM.
5702  *&———————————————————————————————————————————————————————————————*
5703  *&      Form  PREPARE_DYNAMIC_TABLE_SEC2A3
5704  *&———————————————————————————————————————————————————————————————*
5705  *       text
5706  *———————————————————————————————————————————————————————————————*
5707  *  —>  p1        text
5708  *  <—  p2        text
5709  *———————————————————————————————————————————————————————————————*
5710  FORM prepare_dynamic_table_sec2a3 .
5711    CALL METHOD cl_alv_table_create=>create_dynamic_table
5712      EXPORTING
5713  *     i_style_table             = #X#
5714        it_fieldcatalog           = gt_dyn_fcat
```

```
      IMPORTING
        ep_table                  = gt_sec2a3_table
      EXCEPTIONS
        generate_subpool_dir_full = 1
        OTHERS                    = 2.
    IF sy-subrc NE 0.
      MESSAGE 'Unexpected Internal Error' TYPE 'E' .
    ELSE.
  * Assign the new table to field symbol
      ASSIGN gt_sec2a3_table->* TO <gfs_sec2a3_table>.
    ENDIF.
  ENDFORM.
  *&---------------------------------------------------------------------*
  *&      Form  PREPARE_DYNAMIC_TABLE_SEC2D
  *&---------------------------------------------------------------------*
  *       text
  *----------------------------------------------------------------------*
  *  -->  p1        text
  *  <--  p2        text
  *----------------------------------------------------------------------*
  FORM prepare_dynamic_table_sec2d .
    CALL METHOD cl_alv_table_create=>create_dynamic_table
      EXPORTING
  *     i_style_table             = #X#
        it_fieldcatalog           = gt_dyn_fcat
      IMPORTING
        ep_table                  = gt_sec2d_table
      EXCEPTIONS
        generate_subpool_dir_full = 1
        OTHERS                    = 2.
    IF sy-subrc NE 0.
      MESSAGE 'Unexpected Internal Error' TYPE 'E' .
    ELSE.
  * Assign the new table to field symbol
      ASSIGN gt_sec2d_table->* TO <gfs_sec2d_table>.
    ENDIF.
  ENDFORM.
  *&---------------------------------------------------------------------*
  *&      Form  PREPARE_DYNAMIC_TABLE_SEC2F
  *&---------------------------------------------------------------------*
  *       text
  *----------------------------------------------------------------------*
  *  -->  p1        text
  *  <--  p2        text
  *----------------------------------------------------------------------*
  FORM prepare_dynamic_table_sec2f .
    CALL METHOD cl_alv_table_create=>create_dynamic_table
      EXPORTING
  *     i_style_table             = #X#
        it_fieldcatalog           = gt_dyn_fcat
      IMPORTING
        ep_table                  = gt_sec2f_table
      EXCEPTIONS
        generate_subpool_dir_full = 1
        OTHERS                    = 2.
    IF sy-subrc NE 0.
      MESSAGE 'Unexpected Internal Error' TYPE 'E' .
    ELSE.
  * Assign the new table to field symbol
      ASSIGN gt_sec2f_table->* TO <gfs_sec2f_table>.
    ENDIF.
```

```
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  PREPARE_DYNAMIC_TABLE_SEC2F
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM prepare_dynamic_table_sec3a .
  CALL METHOD cl_alv_table_create=>create_dynamic_table
    EXPORTING
*     i_style_table             = #X#
      it_fieldcatalog           = gt_dyn_fcat
    IMPORTING
      ep_table                  = gt_sec3a_table
    EXCEPTIONS
      generate_subpool_dir_full = 1
      OTHERS                    = 2.
  IF sy-subrc NE 0.
    MESSAGE 'Unexpected Internal Error' TYPE 'E' .
  ELSE.
* Assign the new table to field symbol
    ASSIGN gt_sec3a_table->* TO <gfs_sec3a_table>.
  ENDIF.
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  PREPARE_DYNAMIC_TABLE_SEC2F
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM prepare_dynamic_table_sec3b .
  CALL METHOD cl_alv_table_create=>create_dynamic_table
    EXPORTING
*     i_style_table             = #X#
      it_fieldcatalog           = gt_dyn_fcat
    IMPORTING
      ep_table                  = gt_sec3b_table
    EXCEPTIONS
      generate_subpool_dir_full = 1
      OTHERS                    = 2.
  IF sy-subrc NE 0.
    MESSAGE 'Unexpected Internal Error' TYPE 'E' .
  ELSE.
* Assign the new table to field symbol
    ASSIGN gt_sec3b_table->* TO <gfs_sec3b_table>.
  ENDIF.
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  PREPARE_DYNAMIC_TABLE_SEC3C
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM prepare_dynamic_table_sec3c .
  CALL METHOD cl_alv_table_create=>create_dynamic_table
```

```
      EXPORTING
*       i_style_table             = #X#
        it_fieldcatalog           = gt_dyn_fcat
      IMPORTING
        ep_table                  = gt_sec3c_table
      EXCEPTIONS
        generate_subpool_dir_full = 1
        OTHERS                    = 2.
    IF sy-subrc NE 0.
      MESSAGE 'Unexpected Internal Error' TYPE 'E' .
    ELSE.
* Assign the new table to field symbol
      ASSIGN gt_sec3c_table->* TO <gfs_sec3c_table>.
    ENDIF.
  ENDFORM.
  *&---------------------------------------------------------------------*
  *&      Form  PREPARE_DYNAMIC_TABLE_SEC3D
  *&---------------------------------------------------------------------*
  *       text
  *----------------------------------------------------------------------*
  *  -->  p1        text
  *  <--  p2        text
  *----------------------------------------------------------------------*
  FORM prepare_dynamic_table_sec3d .
    CALL METHOD cl_alv_table_create=>create_dynamic_table
      EXPORTING
*       i_style_table             = #X#
        it_fieldcatalog           = gt_dyn_fcat
      IMPORTING
        ep_table                  = gt_sec3d_table
      EXCEPTIONS
        generate_subpool_dir_full = 1
        OTHERS                    = 2.
    IF sy-subrc NE 0.
      MESSAGE 'Unexpected Internal Error' TYPE 'E' .
    ELSE.
* Assign the new table to field symbol
      ASSIGN gt_sec3d_table->* TO <gfs_sec3d_table>.
    ENDIF.
  ENDFORM.
  *&---------------------------------------------------------------------*
  *&      Form  PREPARE_DYNAMIC_TABLE_SEC3E
  *&---------------------------------------------------------------------*
  *       text
  *----------------------------------------------------------------------*
  *  -->  p1        text
  *  <--  p2        text
  *----------------------------------------------------------------------*
  FORM prepare_dynamic_table_sec3e .
    CALL METHOD cl_alv_table_create=>create_dynamic_table
      EXPORTING
*       i_style_table             = #X#
        it_fieldcatalog           = gt_dyn_fcat
      IMPORTING
        ep_table                  = gt_sec3e_table
      EXCEPTIONS
        generate_subpool_dir_full = 1
        OTHERS                    = 2.
    IF sy-subrc NE 0.
      MESSAGE 'Unexpected Internal Error' TYPE 'E' .
    ELSE.
```

```
* Assign the new table to field symbol
    ASSIGN gt_sec3e_table->* TO <gfs_sec3e_table>.
  ENDIF.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  PREPARE_DYNAMIC_TABLE_SEC3F
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM prepare_dynamic_table_sec3f .
  CALL METHOD cl_alv_table_create=>create_dynamic_table
    EXPORTING
*     i_style_table             = #X#
      it_fieldcatalog           = gt_dyn_fcat
    IMPORTING
      ep_table                  = gt_sec3f_table
    EXCEPTIONS
      generate_subpool_dir_full = 1
      OTHERS                    = 2.
  IF sy-subrc NE 0.
    MESSAGE 'Unexpected Internal Error' TYPE 'E' .
  ELSE.
* Assign the new table to field symbol
    ASSIGN gt_sec3f_table->* TO <gfs_sec3f_table>.
  ENDIF.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  PREPARE_DYNAMIC_TABLE_SEC4A
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM prepare_dynamic_table_sec4a .
  CALL METHOD cl_alv_table_create=>create_dynamic_table
    EXPORTING
*     i_style_table             = #X#
      it_fieldcatalog           = gt_dyn_fcat
    IMPORTING
      ep_table                  = gt_sec4a_table
    EXCEPTIONS
      generate_subpool_dir_full = 1
      OTHERS                    = 2.
  IF sy-subrc NE 0.
    MESSAGE 'Unexpected Internal Error' TYPE 'E' .
  ELSE.
* Assign the new table to field symbol
    ASSIGN gt_sec4a_table->* TO <gfs_sec4a_table>.
  ENDIF.
ENDFORM.

*&———————————————————————————————————————————————————————————————*
*&      Form  PREPARE_DYNAMIC_TABLE_SEC2E
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
```

```
*  <——  p2        text
*———————————————————————————————————————————————————————————————————————*
FORM prepare_dynamic_table_sec2e .
  CALL METHOD cl_alv_table_create=>create_dynamic_table
    EXPORTING
*     i_style_table             = #X#
      it_fieldcatalog           = gt_dyn_fcat
    IMPORTING
      ep_table                  = gt_sec2e_table
    EXCEPTIONS
      generate_subpool_dir_full = 1
      OTHERS                    = 2.
  IF sy-subrc NE 0.
    MESSAGE 'Unexpected Internal Error' TYPE 'E' .
  ELSE.
* Assign the new table to field symbol
    ASSIGN gt_sec2e_table->* TO <gfs_sec2e_table>.
  ENDIF.
ENDFORM.
*&———————————————————————————————————————————————————————————————————————*
*&      Form  GET_TARGET_TYPE_NAME
*&———————————————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————————————*
*      ——>P_GS_ZPRA_T_PRD_TAR_TAR_CODE  text
*      <——P_<GFS_FIELD>  text
*———————————————————————————————————————————————————————————————————————*
FORM get_target_type_name  USING    p_tar_code
                           CHANGING p_target_name.
  CASE p_tar_code.
    WHEN 'TAR_BE'.
      p_target_name = 'BE' .
    WHEN 'TAR_IN'.
      p_target_name = 'IN' .
    WHEN 'TAR_EX'.
      p_target_name = 'MOU-Ex' .
    WHEN 'TAR_VG'.
      p_target_name = 'MOU VG' .
    WHEN 'TAR_PC'.
      p_target_name = 'PC' .
    WHEN 'TAR_RE'.
      p_target_name = 'RE' .
    WHEN OTHERS.
  ENDCASE.
ENDFORM.
*&———————————————————————————————————————————————————————————————————————*
*&      Form  GET_TARGET_FROM_NAME
*&———————————————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————————————*
*      ——>P_GS_ZPRA_T_PRD_TAR_TAR_CODE  text
*      <——P_<GFS_FIELD>  text
*———————————————————————————————————————————————————————————————————————*
FORM get_target_from_name  USING    p_target_name
                           CHANGING p_tar_code.
  CASE p_target_name.
    WHEN 'BE'.
      p_tar_code = 'TAR_BE' .
    WHEN 'IN'.
      p_tar_code = 'TAR_IN' .
    WHEN 'MOU-Ex'.
```

```
      p_tar_code = 'TAR_EX' .
    WHEN 'MOU VG'.
      p_tar_code = 'TAR_VG' .
    WHEN 'PC'.
      p_tar_code = 'TAR_PC' .
    WHEN 'RE'.
      p_tar_code = 'TAR_RE' .
    WHEN OTHERS.
  ENDCASE.
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  DISPLAY_SEC2A_TARGE2TS
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  ---->  p1        text
*  <----  p2        text
*----------------------------------------------------------------------*
FORM display_sec2a_targets .

  DATA lv_lines TYPE sy-tabix .
  DESCRIBE TABLE <gfs_sec2_table> LINES lv_lines .
*-----------------Changes to show blank line if target data is not available----------*
*  CHECK lv_lines IS NOT INITIAL .
  IF lv_lines IS INITIAL.
    lv_lines = 1.
  ENDIF.
*-----------------End of changes by Abhishek----------TR OCDK904738----------------*
  gv_row = gv_row + 1 .
  gv_sec2a_tgt_start_row = gv_row .

  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row + lv_lines - 1 .
  gv_e_col = gv_s_col + gv_table_columns - 1.
  PERFORM prepare_section2a_paste_data .
  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
*-----------------Changes to show blank line if target data is not available----------*
  IF gt_paste IS NOT INITIAL.
    PERFORM paste_data .
  ENDIF.
*-----------------End of changes by Abhishek----------TR OCDK904738----------------*

  GET PROPERTY OF go_range 'interior' = go_interior .
  SET PROPERTY OF go_interior 'Color' = gv_colour .
  gv_row = gv_e_row .

  PERFORM select_range USING gv_s_row 1 gv_e_row 1 .
  CALL METHOD OF go_range 'Merge' .
  PERFORM set_range USING gv_txt 0.


ENDFORM.
FORM prepare_section2a_paste_data .
  DATA lv_index TYPE sy-tabix .
  CLEAR gs_paste.
  REFRESH gt_paste .
```

```
  LOOP AT <gfs_sec2_table> ASSIGNING <gfs_dyn_line>.
    CLEAR gs_paste .
    DO .
      lv_index = sy-index .
      ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      IF sy-subrc IS INITIAL.
        IF lv_index NE 1.
          CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab INTO gs_paste .
        ENDIF.
        CONCATENATE gs_paste <gfs_field> INTO gs_paste .
      ELSE.
        EXIT .
      ENDIF.
    ENDDO.
    APPEND gs_paste TO gt_paste .
  ENDLOOP.
ENDFORM.
FORM prepare_section3a_paste_data .
  DATA lv_index TYPE sy-tabix .
  CLEAR gs_paste.
  REFRESH gt_paste .
  LOOP AT <gfs_sec3a_table> ASSIGNING <gfs_dyn_line>.
    CLEAR gs_paste .
    DO .
      lv_index = sy-index .
      ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      IF sy-subrc IS INITIAL.
        IF lv_index NE 1.
          CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab INTO gs_paste .
        ENDIF.
        CONCATENATE gs_paste <gfs_field> INTO gs_paste .
      ELSE.
        EXIT .
      ENDIF.
    ENDDO.
    APPEND gs_paste TO gt_paste .
  ENDLOOP.
ENDFORM.
FORM prepare_section3b_paste_data .
  DATA lv_index TYPE sy-tabix .
  CLEAR gs_paste.
  REFRESH gt_paste .
  LOOP AT <gfs_sec3b_table> ASSIGNING <gfs_dyn_line>.
    CLEAR gs_paste .
    DO .
      lv_index = sy-index .
      ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      IF sy-subrc IS INITIAL.
        IF lv_index NE 1.
          CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab INTO gs_paste .
        ENDIF.
        CONCATENATE gs_paste <gfs_field> INTO gs_paste .
      ELSE.
        EXIT .
      ENDIF.
```

```
      ENDDO.
      APPEND gs_paste TO gt_paste .
    ENDLOOP.
  ENDFORM.
  FORM prepare_section3c_paste_data .
    DATA lv_index TYPE sy-tabix .
    CLEAR gs_paste.
    REFRESH gt_paste .
    LOOP AT <gfs_sec3c_table> ASSIGNING <gfs_dyn_line>.
      CLEAR gs_paste .
      DO .
        lv_index = sy-index .
        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_fiel
      d> .
        IF sy-subrc IS INITIAL.
          IF lv_index NE 1.
            CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab IN
      TO gs_paste .
          ENDIF.
          CONCATENATE gs_paste <gfs_field> INTO gs_paste .
        ELSE.
          EXIT .
        ENDIF.
      ENDDO.
      APPEND gs_paste TO gt_paste .
    ENDLOOP.
  ENDFORM.
  FORM prepare_section3f_paste_data .
    DATA lv_index TYPE sy-tabix .
    CLEAR gs_paste.
    REFRESH gt_paste .
    LOOP AT <gfs_sec3f_table> ASSIGNING <gfs_dyn_line>.
      CLEAR gs_paste .
      DO .
        lv_index = sy-index .
        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_fiel
      d> .
        IF sy-subrc IS INITIAL.
          IF lv_index NE 1.
            CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab IN
      TO gs_paste .
          ENDIF.
          CONCATENATE gs_paste <gfs_field> INTO gs_paste .
        ELSE.
          EXIT .
        ENDIF.
      ENDDO.
      APPEND gs_paste TO gt_paste .
    ENDLOOP.
  ENDFORM.
  FORM prepare_section4a_paste_data .
    DATA lv_index TYPE sy-tabix .
    CLEAR gs_paste.
    REFRESH gt_paste .
    LOOP AT <gfs_sec4a_table> ASSIGNING <gfs_dyn_line>.
      CLEAR gs_paste .
      DO 2 TIMES.
        lv_index = sy-index .
        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_fiel
      d> .
        IF sy-subrc IS INITIAL.
```

```
6188          IF lv_index NE 1.
6189            CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab IN
      TO gs_paste .
6190          ENDIF.
6191          CONCATENATE gs_paste <gfs_field> INTO gs_paste .
6192        ELSE.
6193          EXIT .
6194        ENDIF.
6195      ENDDO.
6196      APPEND gs_paste TO gt_paste .
6197    ENDLOOP.
6198  ENDFORM.
6199  FORM prepare_section4b_paste_data .
6200    DATA : lv_date       TYPE char20,
6201           lv_asset_desc TYPE char512,
6202           lv_expcom TYPE char100,
6203           lv_expdt TYPE char20.
6204
6205    CLEAR gs_paste.
6206    REFRESH gt_paste .
6207    LOOP AT gt_zpra_t_dly_prd INTO gs_zpra_t_dly_prd WHERE production_date
       EQ p_date AND comments IS NOT INITIAL.
6208      CALL FUNCTION 'CONVERSION_EXIT_SDATE_OUTPUT'
6209        EXPORTING
6210          input  = gs_zpra_t_dly_prd-production_date
6211        IMPORTING
6212          output = lv_date.
6213      READ TABLE gt_asset_desc INTO gs_asset_desc WITH KEY asset = gs_zpra
      _t_dly_prd-asset BINARY SEARCH .
6214      IF sy-subrc IS INITIAL.
6215        lv_asset_desc = gs_asset_desc-desc .
6216      ENDIF.
6217      CONCATENATE lv_asset_desc
6218                  cl_abap_char_utilities=>horizontal_tab
6219                  lv_date
6220                  cl_abap_char_utilities=>horizontal_tab
6221                  gs_zpra_t_dly_prd-comments
6222                  INTO gs_paste .
6223
6224      APPEND gs_paste TO gt_paste .
6225      CLEAR gs_paste.
6226    ENDLOOP.
6227
6228  ***********************************Changes by hrishikesh nikam on 23.05.
      22**********
6229    SELECT * from zpra_t_prd_pi INTO TABLE @data(lt_exp_asset)
6230      WHERE LISCENSE_EXP_DT LT @sy-datum.
6231
6232    SELECT ASSET FROM zpra_t_prd_pi INTO TABLE @DATA(LT_EXP_ASSET1)
6233                   FOR ALL ENTRIES IN @lt_exp_asset
6234                   WHERE ASSET = @lt_exp_asset-asset
6235                    AND LISCENSE_EXP_DT GT @SY-DATUM .
6236
6237
6238  *    DELETE lt_exp_asset WHERE LISCENSE_EXP_DT GT SY-DATUM. "Todays Date
6239
6240      LOOP AT lt_exp_asset1 INTO DATA(ls_exp_asset1).
6241  *      IF ls_exp_asset1-asset GT SY-DATUM.
6242         DELETE lt_exp_asset WHERE ASSET = ls_exp_asset1-asset .
6243  *      ENDIF.
6244      ENDLOOP.
```

```

    DELETE lt_exp_asset WHERE LISCENSE_EXP_DT is INITIAL.
    SORT lt_exp_asset BY ASSET LISCENSE_EXP_DT  DESCENDING.
    DELETE ADJACENT DUPLICATES FROM lt_exp_asset COMPARING ASSET.   ""changes by hrishikesh nikam on 30.03.2022
    SORT lt_exp_asset BY ASSET ASCENDING.                           ""changes by hrishikesh nikam on 30.03.2022


***********************************end of Changes by hrishikesh nikam on 23.05.22**********
    LOOP AT lt_exp_asset INTO DATA(ls_exp_asset).                   ""changes by hrishikesh nikam
      if ls_exp_asset-liscense_exp_dt LT p_date.                    ""changes by hrishikesh nikam

         CALL FUNCTION 'CONVERSION_EXIT_SDATE_OUTPUT'
      EXPORTING
        input  = ls_exp_asset-liscense_exp_dt
      IMPORTING
        output = lv_expdt.

        CONCATENATE 'The respective asset has been relinquished on' lv_expdt INTO lv_expcom SEPARATED BY space.

       CALL FUNCTION 'CONVERSION_EXIT_SDATE_OUTPUT'
      EXPORTING
        input  = p_date
      IMPORTING
        output = lv_date.


    READ TABLE gt_asset_desc INTO gs_asset_desc WITH KEY asset = ls_exp_asset-asset BINARY SEARCH .
    IF sy-subrc IS INITIAL.
      lv_asset_desc = gs_asset_desc-desc .
    ENDIF.
    CONCATENATE lv_asset_desc
                cl_abap_char_utilities=>horizontal_tab
                lv_date
                cl_abap_char_utilities=>horizontal_tab
                lv_expcom
                INTO gs_paste .

    APPEND gs_paste TO gt_paste .
    CLEAR: gs_paste ,  lv_expcom , lv_expdt.
      endif.

    ENDLOOP.


ENDFORM.
FORM prepare_section5a_paste_data .
  DATA : lv_index  TYPE sy-tabix,
         lv_col_no TYPE sy-tabix.
  CLEAR gs_paste.
  REFRESH gt_paste .
  DO gv_5a_cols TIMES .
    lv_col_no = sy-index .
    READ TABLE gt_dyn_fcat INTO gs_dyn_fcat INDEX lv_col_no .
    IF sy-subrc IS  INITIAL.
```

```
6299        gs_paste = gs_dyn_fcat-coltext .
6300      ENDIF.
6301      LOOP AT <gfs_sec5a_table> ASSIGNING <gfs_dyn_line>.
6302        lv_index = sy-tabix .
6303        ASSIGN COMPONENT lv_col_no OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
      ld> .
6304        CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab INTO g
      s_paste .
6305        CONCATENATE gs_paste <gfs_field> INTO gs_paste .
6306      ENDLOOP .
6307      APPEND gs_paste TO gt_paste .
6308    ENDDO.
6309  ENDFORM.
6310  FORM prepare_section6_paste_data .
6311    DATA lv_index TYPE sy-tabix .
6312    CLEAR gs_paste.
6313    REFRESH gt_paste .
6314    LOOP AT <gfs_sec6_table> ASSIGNING <gfs_dyn_line>.
6315      CLEAR gs_paste .
6316      DO .
6317        lv_index = sy-index .
6318        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_fiel
      d> .
6319        IF sy-subrc IS INITIAL.
6320          IF lv_index NE 1.
6321            CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab IN
      TO gs_paste .
6322          ENDIF.
6323          CONCATENATE gs_paste <gfs_field> INTO gs_paste .
6324        ELSE.
6325          EXIT .
6326        ENDIF.
6327      ENDDO.
6328      APPEND gs_paste TO gt_paste .
6329    ENDLOOP.
6330  ENDFORM.
6331  FORM prepare_section3d_paste_data .
6332    DATA lv_index TYPE sy-tabix .
6333    CLEAR gs_paste.
6334    REFRESH gt_paste .
6335    LOOP AT <gfs_sec3d_table> ASSIGNING <gfs_dyn_line>.
6336      CLEAR gs_paste .
6337      DO .
6338        lv_index = sy-index .
6339        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_fiel
      d> .
6340        IF sy-subrc IS INITIAL.
6341          IF lv_index NE 1.
6342            CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab IN
      TO gs_paste .
6343          ENDIF.
6344          CONCATENATE gs_paste <gfs_field> INTO gs_paste .
6345        ELSE.
6346          EXIT .
6347        ENDIF.
6348      ENDDO.
6349      APPEND gs_paste TO gt_paste .
6350    ENDLOOP.
6351  ENDFORM.
6352  FORM prepare_section3e_paste_data .
6353    DATA lv_index TYPE sy-tabix .
```

```
  CLEAR gs_paste.
  REFRESH gt_paste .
  LOOP AT <gfs_sec3e_table> ASSIGNING <gfs_dyn_line>.
    CLEAR gs_paste .
    DO .
      lv_index = sy-index .
      ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
      IF sy-subrc IS INITIAL.
        IF lv_index NE 1.
          CONCATENATE gs_paste cl_abap_char_utilities=>horizontal_tab INTO gs_paste .
        ENDIF.
        CONCATENATE gs_paste <gfs_field> INTO gs_paste .
      ELSE.
        EXIT .
      ENDIF.
    ENDDO.
    APPEND gs_paste TO gt_paste .
  ENDLOOP.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  FETCH_SECTION2A2_DATA
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM fetch_section2a2_data .
  DATA : lt_zpra_t_dly_prd         TYPE STANDARD TABLE OF zpra_t_dly_prd,
         lt_zpra_t_dly_prd_current TYPE STANDARD TABLE OF zpra_t_dly_prd.
  DATA : lv_last_year_date  TYPE sy-datum,
         lv_last_year_gjahr TYPE gjahr,
         lv_index           TYPE sy-tabix.
  lt_zpra_t_dly_prd_current = gt_zpra_t_dly_prd .
  SORT lt_zpra_t_dly_prd_current BY product asset block prd_vl_type .
  DELETE ADJACENT DUPLICATES FROM lt_zpra_t_dly_prd_current COMPARING product asset block prd_vl_type .

  lv_last_year_gjahr = p_date(4) - 1 .
  CONCATENATE lv_last_year_gjahr p_date+4(4) INTO lv_last_year_date .
  CALL FUNCTION 'HR_JP_MONTH_BEGIN_END_DATE'
    EXPORTING
      iv_date             = lv_last_year_date
    IMPORTING
      ev_month_begin_date = gv_year_back_begin_datum
      ev_month_end_date   = gv_year_back_end_datum.

  SELECT *
   FROM zpra_t_dly_prd
   INTO TABLE gt_zpra_t_dly_prd_mb
    FOR ALL ENTRIES IN gt_zpra_c_prd_prof
  WHERE product EQ gt_zpra_c_prd_prof-product
    AND asset   EQ gt_zpra_c_prd_prof-asset
    AND block   EQ gt_zpra_c_prd_prof-block
    AND production_date LE lv_last_year_date
    AND production_date GE gv_year_back_begin_datum
    AND prd_vl_type IN r_prd_vl_type[] .
```

```
6410
6411    PERFORM populate_no_data_entries TABLES gt_zpra_t_dly_prd_mb USING gv_
      year_back_begin_datum lv_last_year_date .
6412
6413    SORT  gt_zpra_t_dly_prd_mb BY production_date product asset .
6414  * LOOP AT gt_zpra_t_dly_prd_mb INTO gs_zpra_t_dly_prd_mb.
6415  *   READ TABLE gt_zpra_t_dly_prd TRANSPORTING NO FIELDS WITH KEY product
       = gs_zpra_t_dly_prd_mb-product .
6416  * ENDLOOP.
6417    lt_zpra_t_dly_prd = gt_zpra_t_dly_prd_mb .
6418    SORT lt_zpra_t_dly_prd BY asset block product prd_vl_type.
6419    DELETE ADJACENT DUPLICATES FROM lt_zpra_t_dly_prd COMPARING asset bloc
      k product prd_vl_type .
6420
6421    SELECT asset
6422           block
6423           vld_frm
6424           vld_to
6425           pi
6426           prod_start_date
6427     FROM zpra_t_prd_pi
6428     INTO TABLE gt_zpra_t_prd_pi_mb
6429      FOR ALL ENTRIES IN gt_zpra_t_dly_prd_mb
6430    WHERE asset EQ gt_zpra_t_dly_prd_mb-asset
6431      AND block EQ gt_zpra_t_dly_prd_mb-block
6432      AND vld_frm LE gt_zpra_t_dly_prd_mb-production_date
6433      AND vld_to  GE gt_zpra_t_dly_prd_mb-production_date .
6434
6435    SORT gt_zpra_t_prd_pi_mb BY asset block vld_frm vld_to .
6436
6437
6438  ENDFORM.
6439  *&———————————————————————————————————————————————————————————*
6440  *&      Form  FETCH_DATA_SECTION2B
6441  *&———————————————————————————————————————————————————————————*
6442  *       text
6443  *———————————————————————————————————————————————————————————*
6444  *  —>  p1        text
6445  *  <—  p2        text
6446  *———————————————————————————————————————————————————————————*
6447  FORM fetch_data_section2b .
6448  * DATA : lt_zpra_t_dly_prd  TYPE STANDARD TABLE OF zpra_t_dly_prd .
6449  *
6450  * lt_zpra_t_dly_prd = gt_zpra_t_dly_prd .
6451  * SORT lt_zpra_t_dly_prd BY asset block product prd_vl_type.
6452  * DELETE ADJACENT DUPLICATES FROM lt_zpra_t_dly_prd COMPARING asset bloc
      k product prd_vl_type .
6453
6454    REFRESH : gt_zpra_t_tar_pi ,
6455              gt_zpra_t_prd_tar .
6456    IF r_tar_code[] IS NOT INITIAL.
6457
6458      SELECT asset
6459             block
6460             tar_code
6461             vld_frm
6462             vld_to
6463             pi
6464        FROM zpra_t_tar_pi
6465        INTO TABLE gt_zpra_t_tar_pi
6466         FOR ALL ENTRIES IN gt_zpra_c_prd_prof
```

```
      WHERE asset EQ gt_zpra_c_prd_prof-asset
        AND block EQ gt_zpra_c_prd_prof-block
        AND vld_frm LE gv_year_end_date
        AND vld_to  GE gv_year_start_date
        AND tar_code IN r_tar_code[] .
     SORT gt_zpra_t_tar_pi BY asset block tar_code vld_frm .

     SELECT tar_code
            gjahr
            monat
            asset
            block
            product
            prod_vl_type_cd
            tar_qty
            uom
       FROM zpra_t_prd_tar
       INTO TABLE gt_zpra_t_prd_tar
        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
      WHERE gjahr EQ gv_current_gjahr
        AND asset EQ gt_zpra_c_prd_prof-asset
        AND block EQ gt_zpra_c_prd_prof-block
        AND product EQ gt_zpra_c_prd_prof-product
        AND prod_vl_type_cd IN r_prd_vl_type[]
        AND tar_code IN r_tar_code[] .
     SORT gt_zpra_t_prd_tar BY gjahr monat asset block product prod_vl_type_cd tar_code .

     gt_zpra_t_prd_tar_2c = gt_zpra_t_prd_tar.
     gt_zpra_t_prd_tar_3a = gt_zpra_t_prd_tar.
     gt_zpra_t_prd_tar_3b = gt_zpra_t_prd_tar.

     SELECT *
       FROM zpra_t_tar_cf
       INTO TABLE gt_zpra_t_tar_cf
        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
      WHERE gjahr EQ gv_current_gjahr
        AND asset EQ gt_zpra_c_prd_prof-asset
        AND block EQ gt_zpra_c_prd_prof-block
        AND product EQ gt_zpra_c_prd_prof-product .

     SORT gt_zpra_t_tar_cf BY gjahr asset block product .
   ENDIF.

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  PROCESS_SEC2B_DATA
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM process_sec2b_data .
  REFRESH <gfs_sec2_table> .

  PERFORM convert_sec2b_to_jvl.
  PERFORM fill_dynamic_table_sec2b    .
  gv_colour = gv_sec2b_colour .
  CONCATENATE 'Target :'  gv_current_gjahr '-' gv_next_gjahr+2(2) INTO gv_txt SEPARATED BY space .
```

```
  PERFORM display_section2a1 .
  <gfs_sec2b_table> = <gfs_sec2_table> .
  PERFORM show_progress USING '10' .

ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  PROCESS_SEC2C_DATA
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM process_sec2c_data .
  REFRESH <gfs_sec2_table> .
  PERFORM convert_sec2c_to_jvl.
  PERFORM fill_dynamic_table_sec2c    .
  gv_colour = gv_sec2c_colour .
  CONCATENATE 'YTD Target :'  gv_current_gjahr '-' gv_next_gjahr+2(2) IN
TO gv_txt SEPARATED BY space .
  PERFORM display_section2a1 .
  <gfs_sec2c_table> = <gfs_sec2_table> .
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  PROCESS_SEC3b_DATA
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM process_sec3b_data .
  FIELD-SYMBOLS : <lfs_zpra_t_prd_tar> TYPE ty_zpra_t_prd_tar .
  DATA:  lv_denominator TYPE                  zpra_t_dly_prd-prod_vl_qt
y1,
         lv_first_day   TYPE                  sy-datum,
         lv_last_day    TYPE                  sy-datum.

  DELETE gt_zpra_t_prd_tar_3b WHERE monat GT gv_current_monat .
  LOOP AT gt_zpra_t_prd_tar_3b ASSIGNING <lfs_zpra_t_prd_tar>.
    CLEAR lv_denominator .

    CALL FUNCTION 'PERIOD_DAY_DETERMINE'
      EXPORTING
        i_gjahr              = <lfs_zpra_t_prd_tar>-gjahr
        i_monat              = <lfs_zpra_t_prd_tar>-monat
        i_periv              = gv_periv
      IMPORTING
        e_fday               = lv_first_day
        e_lday               = lv_last_day
*       E_SPERIOD            =
      EXCEPTIONS
        error_period         = 1
        error_period_version = 2
        firstday_not_defined = 3
        period_not_defined   = 4
        year_invalid         = 5
        OTHERS               = 6.
    IF sy-subrc <> 0.
      MESSAGE 'Internal Date error' TYPE 'E' .
    ENDIF.
```

```
6585
6586      IF <lfs_zpra_t_prd_tar>-monat EQ gv_current_monat.
6587        <lfs_zpra_t_prd_tar>-tar_qty = <lfs_zpra_t_prd_tar>-tar_qty * ( p_
      date - lv_first_day + 1 ) / ( lv_last_day - lv_first_day + 1 ) .
6588        lv_last_day = p_date .
6589      ENDIF.
6590    ENDLOOP.
6591  ENDFORM.
6592  *&———————————————————————————————————————————————————————————*
6593  *&      Form  FETCH_DATA_SECTION2C
6594  *&———————————————————————————————————————————————————————————*
6595  *       text
6596  *———————————————————————————————————————————————————————————*
6597  *  —>  p1        text
6598  *  <—  p2        text
6599  *———————————————————————————————————————————————————————————*
6600  FORM fetch_data_section2c .
6601    DELETE gt_zpra_t_prd_tar    WHERE monat GT gv_current_monat .
6602    DELETE gt_zpra_t_prd_tar_2c WHERE monat GT gv_current_monat .
6603
6604    gt_zpra_t_prd_tar_5a = gt_zpra_t_prd_tar .
6605  ENDFORM.
6606  *&———————————————————————————————————————————————————————————*
6607  *&      Form  FETCH_DATA_SECTION2A3
6608  *&———————————————————————————————————————————————————————————*
6609  *       text
6610  *———————————————————————————————————————————————————————————*
6611  *  —>  p1        text
6612  *  <—  p2        text
6613  *———————————————————————————————————————————————————————————*
6614  FORM fetch_data_section2a3 .
6615  *  DATA : lt_zpra_t_dly_prd TYPE STANDARD TABLE OF zpra_t_dly_prd .
6616  *  lt_zpra_t_dly_prd = gt_zpra_t_dly_prd .
6617  *  SORT lt_zpra_t_dly_prd BY asset block product .
6618  *  DELETE ADJACENT DUPLICATES FROM lt_zpra_t_dly_prd COMPARING asset blo
      ck product .
6619    DATA : lv_gjahr TYPE zpra_t_mrec_prd-gjahr .
6620    SELECT *
6621      FROM zpra_t_dly_prd
6622      INTO TABLE gt_zpra_t_dly_prd_2a3
6623       FOR ALL ENTRIES IN gt_zpra_c_prd_prof
6624     WHERE asset EQ gt_zpra_c_prd_prof-asset
6625       AND block EQ gt_zpra_c_prd_prof-block
6626       AND product EQ gt_zpra_c_prd_prof-product
6627       AND prd_vl_type IN r_prd_vl_type[]
6628       AND production_date GE gv_month_back_begin_datum
6629       AND production_date LE gv_month_back_end_datum .
6630
6631    PERFORM populate_no_data_entries TABLES gt_zpra_t_dly_prd_2a3 USING gv
      _month_back_begin_datum gv_month_back_end_datum .
6632
6633    lv_gjahr = gv_month_back_gjahr - 2 .
6634    SELECT gjahr
6635           monat
6636           asset
6637           block
6638           product
6639           prd_vl_type
6640           prod_vl_qty1
6641           prod_vl_uom1
6642           prod_vl_qty2
```

```
          prod_vl_uom2
     FROM zpra_t_mrec_prd
     INTO TABLE gt_zpra_t_mrec_prd_2a3
      FOR ALL ENTRIES IN gt_zpra_c_prd_prof
    WHERE asset        EQ gt_zpra_c_prd_prof-asset
      AND block        EQ gt_zpra_c_prd_prof-block
      AND product      EQ gt_zpra_c_prd_prof-product
      AND prd_vl_type  IN r_prd_vl_type[]
      AND gjahr GE lv_gjahr
      AND gjahr LE gv_month_back_gjahr .
*      AND gjahr EQ gv_month_back_gjahr
*      AND monat EQ gv_month_back_monat .
  DELETE gt_zpra_t_mrec_prd_2a3 WHERE gjahr EQ gv_month_back_gjahr AND m
onat GT gv_month_back_monat .
  IF p_mb    IS NOT INITIAL AND
     p_c_olv IS NOT INITIAL.
     SELECT gjahr
            monat
            asset
            block
            product
            prd_vl_type
            app_vl_qty
            app_vl_uom
       FROM zpra_t_mrec_app
       INTO TABLE gt_zpra_t_mrec_app_2a3
        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
    WHERE asset        EQ gt_zpra_c_prd_prof-asset
      AND block        EQ gt_zpra_c_prd_prof-block
      AND product      EQ gt_zpra_c_prd_prof-product
      AND prd_vl_type  IN r_prd_vl_type[]
      AND gjahr EQ gv_month_back_gjahr
      AND monat EQ gv_month_back_monat .

     SORT gt_zpra_t_mrec_app_2a3 BY gjahr monat product asset block  prd_
vl_type .
  ENDIF.
  SELECT asset
         block
         vld_frm
         vld_to
         pi
         prod_start_date
    FROM zpra_t_prd_pi
    INTO TABLE gt_zpra_t_prd_pi_lm
     FOR ALL ENTRIES IN gt_zpra_c_prd_prof
  WHERE asset EQ gt_zpra_c_prd_prof-asset
    AND block EQ gt_zpra_c_prd_prof-block
    AND vld_frm LE gv_month_back_end_datum
    AND vld_to  GE gv_month_back_begin_datum .

  SORT gt_zpra_t_prd_pi_lm BY asset block vld_frm vld_to .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  FETCH_DATA_SECTION2D
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
```

```
*_____________________________________________________________________*
FORM fetch_data_section2d .
*  DATA : lt_zpra_t_dly_prd   TYPE STANDARD TABLE OF zpra_t_dly_prd .
*  DATA : ls_zpra_t_dly_prd   TYPE                   zpra_t_dly_prd .
  DATA : lv_monat            TYPE                   zpra_t_mrec_prd-monat,
         lv_gjahr            TYPE                   zpra_t_mrec_prd-gjahr,
         lv_date             TYPE                   sy-datum,
         lv_month_begin_date TYPE                   sy-datum,
         lv_month_end_date   TYPE                   sy-datum,
         lv_poper            TYPE                   t009b-poper,
         lv_count            TYPE                   sy-tabix.
*  lt_zpra_t_dly_prd = gt_zpra_t_dly_prd .
*  SORT lt_zpra_t_dly_prd BY product asset block  .
*  DELETE ADJACENT DUPLICATES FROM lt_zpra_t_dly_prd COMPARING product asset block  .
  IF p_mb    IS NOT INITIAL AND
     p_c_olv IS NOT INITIAL.

    SELECT gjahr
           monat
           asset
           block
           product
           prd_vl_type
           app_vl_qty
           app_vl_uom
       FROM zpra_t_mrec_app
       INTO TABLE gt_zpra_t_mrec_app_2d
        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
      WHERE asset        EQ gt_zpra_c_prd_prof-asset
        AND block        EQ gt_zpra_c_prd_prof-block
        AND product      EQ gt_zpra_c_prd_prof-product
        AND prd_vl_type  IN r_prd_vl_type[]
        AND ( gjahr EQ gv_last_gjahr
         OR ( gjahr EQ gv_current_gjahr  AND
              monat LE gv_current_monat ) ) .

    SORT gt_zpra_t_mrec_app_2d BY gjahr monat product asset block  prd_vl_type .
  ENDIF.
  lv_gjahr = gv_last_gjahr - 2 .

  SELECT gjahr
         monat
         asset
         block
         product
         prd_vl_type
         prod_vl_qty1
         prod_vl_uom1
         prod_vl_qty2
         prod_vl_uom2
    FROM zpra_t_mrec_prd
    INTO TABLE gt_zpra_t_mrec_prd_2d
     FOR ALL ENTRIES IN gt_zpra_c_prd_prof
   WHERE asset        EQ gt_zpra_c_prd_prof-asset
     AND block        EQ gt_zpra_c_prd_prof-block
     AND product      EQ gt_zpra_c_prd_prof-product
     AND prd_vl_type  IN r_prd_vl_type[]
```

```
      AND gjahr GE lv_gjahr
      AND gjahr LE gv_current_gjahr  .
   DELETE gt_zpra_t_mrec_prd_2d WHERE gjahr EQ gv_current_gjahr AND monat GT gv_current_monat .
*     AND ( gjahr EQ gv_last_gjahr
*      OR ( gjahr EQ gv_current_gjahr  AND
*           monat LE gv_current_monat ) ) .

*  SORT gt_zpra_t_mrec_prd_2d BY gjahr monat product asset block .
   SORT gt_zpra_t_mrec_prd_2d BY product ASCENDING asset ASCENDING block ASCENDING gjahr DESCENDING monat DESCENDING .

   REFRESH r_production_date[] .

   LOOP AT gt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof .
     lv_monat = '00' .
     DO .
       lv_monat = lv_monat + 1 .
       lv_gjahr = gv_last_gjahr .
       DO  2 TIMES .
         READ TABLE gt_zpra_t_mrec_app_2d TRANSPORTING NO FIELDS WITH KEY gjahr = lv_gjahr

       monat = lv_monat
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
       p
roduct = gs_zpra_c_prd_prof-product

       asset = gs_zpra_c_prd_prof-asset BINARY SEARCH .
         IF sy-subrc IS NOT INITIAL.
           lv_poper = lv_monat .
           CALL FUNCTION 'FIRST_DAY_IN_PERIOD_GET'
             EXPORTING
               i_gjahr        = lv_gjahr
*              I_MONMIT       = 00
               i_periv        = gv_periv
               i_poper        = lv_poper
             IMPORTING
               e_date         = lv_month_begin_date
             EXCEPTIONS
               input_false    = 1
               t009_notfound  = 2
               t009b_notfound = 3
               OTHERS         = 4.
           IF sy-subrc <> 0.
             MESSAGE 'Internal Error' TYPE 'E' .
           ENDIF.

           CALL FUNCTION 'LAST_DAY_IN_PERIOD_GET'
             EXPORTING
               i_gjahr        = lv_gjahr
*              I_MONMIT       = 00
               i_periv        = gv_periv
               i_poper        = lv_poper
             IMPORTING
               e_date         = lv_month_end_date
             EXCEPTIONS
               input_false    = 1
               t009_notfound  = 2
               t009b_notfound = 3
               OTHERS         = 4.
           IF sy-subrc <> 0.
```

```
6814              MESSAGE 'Internal Error' TYPE 'E' .
6815            ENDIF.
6816            r_production_date-sign   = 'I' .
6817            r_production_date-option = 'BT' .
6818            r_production_date-low    = lv_month_begin_date .
6819            r_production_date-high   = lv_month_end_date .
6820            APPEND r_production_date .
6821          ENDIF.
6822
6823          lv_gjahr = lv_gjahr + 1 .
6824        ENDDO.
6825        IF lv_monat GE gv_current_monat .
6826          EXIT .
6827        ENDIF.
6828      ENDDO .
6829    ENDLOOP.
6830    SORT r_production_date[] .
6831    DELETE ADJACENT DUPLICATES FROM r_production_date[] COMPARING ALL FIEL
      DS .
6832    IF r_production_date[] IS NOT INITIAL .
6833      SELECT *
6834        FROM zpra_t_dly_prd
6835        INTO TABLE gt_zpra_t_dly_prd_2d
6836         FOR ALL ENTRIES IN gt_zpra_c_prd_prof
6837       WHERE asset            EQ gt_zpra_c_prd_prof-asset
6838         AND block            EQ gt_zpra_c_prd_prof-block
6839         AND product          EQ gt_zpra_c_prd_prof-product
6840         AND prd_vl_type      IN r_prd_vl_type[]
6841         AND production_date  IN r_production_date[] .
6842      DESCRIBE TABLE r_production_date[] LINES lv_count .
6843      lv_count = lv_count / 2 .
6844      LOOP AT r_production_date.
6845        IF sy-tabix EQ lv_count .
6846          PERFORM show_progress USING '20' .
6847        ENDIF.
6848        PERFORM populate_no_data_entries TABLES gt_zpra_t_dly_prd_2d USING
       r_production_date-low r_production_date-high .
6849      ENDLOOP.
6850      SORT gt_zpra_t_dly_prd_2d BY product asset block production_date .
6851
6852      SELECT *
6853        FROM zpra_t_dly_rprd
6854        INTO TABLE gt_zpra_t_dly_rprd_2d
6855         FOR ALL ENTRIES IN gt_zpra_c_prd_prof
6856       WHERE asset EQ gt_zpra_c_prd_prof-asset
6857         AND block EQ gt_zpra_c_prd_prof-block
6858         AND product EQ gt_zpra_c_prd_prof-product
6859  *       AND prd_vl_type IN r_prd_vl_type[]
6860         AND prd_vl_type EQ 'NET_PROD'
6861         AND production_date IN r_production_date[] .
6862
6863      SORT gt_zpra_t_dly_rprd_2d BY product asset block production_date .
6864
6865    ENDIF.
6866    CONCATENATE gv_last_gjahr '0101' INTO lv_date .
6867    SELECT asset
6868          block
6869          vld_frm
6870          vld_to
6871          pi
6872          prod_start_date
```

```
    FROM zpra_t_prd_pi
    INTO TABLE gt_zpra_t_prd_pi_2d
     FOR ALL ENTRIES IN gt_zpra_c_prd_prof
   WHERE asset EQ gt_zpra_c_prd_prof-asset
     AND block EQ gt_zpra_c_prd_prof-block
     AND vld_frm LE p_date
     AND vld_to  GE lv_date .

   SORT gt_zpra_t_prd_pi_2d BY asset block vld_frm vld_to .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  FETCH_DATA_SECTION2F
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM fetch_data_section2f .
*  DATA : lt_zpra_t_dly_prd   TYPE STANDARD TABLE OF zpra_t_dly_prd .
*  DATA : ls_zpra_t_dly_prd   TYPE                   zpra_t_dly_prd .
   DATA : lv_monat            TYPE                   zpra_t_mrec_prd-monat,
          lv_gjahr            TYPE                   zpra_t_mrec_prd-gjahr,
          lv_date             TYPE                   sy-datum,
          lv_date2            TYPE                   sy-datum,
          lv_month_begin_date TYPE                   sy-datum,
          lv_month_end_date   TYPE                   sy-datum,
          lv_poper            TYPE                   t009b-poper.

*  lt_zpra_t_dly_prd = gt_zpra_t_dly_prd .
*  SORT lt_zpra_t_dly_prd BY product asset block  .
*  DELETE ADJACENT DUPLICATES FROM lt_zpra_t_dly_prd COMPARING product asset block  .
   lv_gjahr = gv_last_gjahr - 3 .
   SELECT gjahr
          monat
          asset
          block
          product
          prd_vl_type
          prod_vl_qty1
          prod_vl_uom1
          prod_vl_qty2
          prod_vl_uom2
     FROM zpra_t_mrec_prd
     INTO TABLE gt_zpra_t_mrec_prd_2f
      FOR ALL ENTRIES IN gt_zpra_c_prd_prof
    WHERE asset        EQ gt_zpra_c_prd_prof-asset
      AND block        EQ gt_zpra_c_prd_prof-block
      AND product      EQ gt_zpra_c_prd_prof-product
      AND prd_vl_type  IN r_prd_vl_type[]
      AND gjahr        LE gv_last_gjahr
      AND gjahr        GE lv_gjahr .

*  SORT gt_zpra_t_mrec_prd_2f BY gjahr monat product asset block .
   SORT gt_zpra_t_mrec_prd_2f  BY product ASCENDING asset ASCENDING block ASCENDING gjahr DESCENDING monat DESCENDING .

```

```
  IF p_mb    IS NOT INITIAL AND
     p_c_olv IS NOT INITIAL.
    SELECT gjahr
           monat
           asset
           block
           product
           prd_vl_type
           app_vl_qty
           app_vl_uom
      FROM zpra_t_mrec_app
      INTO TABLE gt_zpra_t_mrec_app_2f
       FOR ALL ENTRIES IN gt_zpra_c_prd_prof
     WHERE gjahr EQ gv_last_gjahr
       AND asset EQ gt_zpra_c_prd_prof-asset
       AND block EQ gt_zpra_c_prd_prof-block
       AND product EQ gt_zpra_c_prd_prof-product
       AND prd_vl_type IN r_prd_vl_type[] .

    SORT gt_zpra_t_mrec_app_2f BY gjahr monat product asset block  prd_vl_type .
  ENDIF.
  REFRESH r_production_date[] .

  LOOP AT gt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof .
    lv_monat = '00' .
    DO .
      lv_monat = lv_monat + 1 .
      lv_gjahr = gv_last_gjahr .
      DO  1 TIMES .
        READ TABLE gt_zpra_t_mrec_app_2f TRANSPORTING NO FIELDS WITH KEY gjahr = lv_gjahr

 monat = lv_monat
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
 p
roduct = gs_zpra_c_prd_prof-product

 asset = gs_zpra_c_prd_prof-asset BINARY SEARCH .
        IF sy-subrc IS NOT INITIAL.
          lv_poper = lv_monat .
          CALL FUNCTION 'FIRST_DAY_IN_PERIOD_GET'
            EXPORTING
              i_gjahr        = lv_gjahr
*             I_MONMIT       = 00
              i_periv        = gv_periv
              i_poper        = lv_poper
            IMPORTING
              e_date         = lv_month_begin_date
            EXCEPTIONS
              input_false    = 1
              t009_notfound  = 2
              t009b_notfound = 3
              OTHERS         = 4.
          IF sy-subrc <> 0.
            MESSAGE 'Internal Error' TYPE 'E' .
          ENDIF.

          CALL FUNCTION 'LAST_DAY_IN_PERIOD_GET'
            EXPORTING
              i_gjahr        = lv_gjahr
*             I_MONMIT       = 00
```

```
              i_periv        = gv_periv
              i_poper        = lv_poper
            IMPORTING
              e_date         = lv_month_end_date
            EXCEPTIONS
              input_false    = 1
              t009_notfound  = 2
              t009b_notfound = 3
              OTHERS         = 4.
          IF sy-subrc <> 0.
            MESSAGE 'Internal Error' TYPE 'E' .
          ENDIF.
          r_production_date-sign   = 'I' .
          r_production_date-option = 'BT' .
          r_production_date-low    = lv_month_begin_date .
          r_production_date-high   = lv_month_end_date .
          APPEND r_production_date .
        ENDIF.

        lv_gjahr = lv_gjahr + 1 .
      ENDDO.
      IF lv_monat GE 12 .
        EXIT .
      ENDIF.
    ENDDO .
  ENDLOOP.
  SORT r_production_date[] .
  DELETE ADJACENT DUPLICATES FROM r_production_date[] COMPARING ALL FIELDS .

  IF r_production_date[] IS NOT INITIAL .
    SELECT *
      FROM zpra_t_dly_prd
      INTO TABLE gt_zpra_t_dly_prd_2f
       FOR ALL ENTRIES IN gt_zpra_c_prd_prof
     WHERE asset EQ gt_zpra_c_prd_prof-asset
       AND block EQ gt_zpra_c_prd_prof-block
       AND product EQ gt_zpra_c_prd_prof-product
       AND prd_vl_type IN r_prd_vl_type[]
       AND production_date IN r_production_date[] .

    LOOP AT r_production_date.
      PERFORM populate_no_data_entries TABLES gt_zpra_t_dly_prd_2f USING r_production_date-low r_production_date-high .
    ENDLOOP.


    SORT gt_zpra_t_dly_prd_2f BY product asset block production_date .

    SELECT *
      FROM zpra_t_dly_rprd
      INTO TABLE gt_zpra_t_dly_rprd_2f
       FOR ALL ENTRIES IN gt_zpra_c_prd_prof
     WHERE asset EQ gt_zpra_c_prd_prof-asset
       AND block EQ gt_zpra_c_prd_prof-block
       AND product EQ gt_zpra_c_prd_prof-product
*       AND prd_vl_type IN r_prd_vl_type[]
       AND prd_vl_type EQ 'NET_PROD'
       AND production_date IN r_production_date[] .

    SORT gt_zpra_t_dly_rprd_2f BY product asset block production_date .
```

```
7045
7046
7047    ENDIF.
7048
7049    CONCATENATE gv_last_gjahr '0401' INTO lv_date .
7050    CONCATENATE gv_current_gjahr '0331' INTO lv_date2 .
7051
7052    SELECT asset
7053          block
7054          vld_frm
7055          vld_to
7056          pi
7057          prod_start_date
7058     FROM zpra_t_prd_pi
7059     INTO TABLE gt_zpra_t_prd_pi_2f
7060      FOR ALL ENTRIES IN gt_zpra_c_prd_prof
7061    WHERE asset EQ gt_zpra_c_prd_prof-asset
7062      AND block EQ gt_zpra_c_prd_prof-block
7063      AND vld_frm LE lv_date2
7064      AND vld_to  GE lv_date .
7065
7066    SORT gt_zpra_t_prd_pi_2f BY asset block vld_frm vld_to .
7067    PERFORM show_progress USING '30' .
7068
7069  ENDFORM.
7070  *&———————————————————————————————————————————————————————————*
7071  *&      Form  PROCESS_GAS_RECORDS_2A3
7072  *&———————————————————————————————————————————————————————————*
7073  *       text
7074  *———————————————————————————————————————————————————————————*
7075  *  —>  p1        text
7076  *  <—  p2        text
7077  *———————————————————————————————————————————————————————————*
7078  FORM process_gas_records_2a3 .
7079
7080    DATA : lt_zpra_t_dly_prd  TYPE STANDARD TABLE OF zpra_t_dly_prd,
7081           lt_zpra_t_mrec_prd TYPE STANDARD TABLE OF ty_zpra_t_mrec_prd.
7082
7083    DATA : ls_zpra_t_dly_prd  TYPE                  zpra_t_dly_prd,
7084           ls_zpra_t_mrec_prd TYPE                  ty_zpra_t_mrec_prd.
7085    FIELD-SYMBOLS : <fs_dly_prd> TYPE zpra_t_dly_prd,
7086                    <fs_mrec>    TYPE ty_zpra_t_mrec_prd.
7087
7088    DATA : lv_index TYPE sy-tabix,
7089           lv_days  TYPE sy-tabix.
7090  * lv_days = gv_month_back_end_datum - gv_month_back_begin_datum + 1 .
7091    lv_days = 1 .
7092    lt_zpra_t_dly_prd  = gt_zpra_t_dly_prd_2a3 .
7093    SORT  lt_zpra_t_dly_prd BY production_date product asset block prd_vl_
      type .
7094
7095    DELETE gt_zpra_t_dly_prd_2a3  WHERE product EQ c_prod_gas .
7096    DELETE lt_zpra_t_dly_prd      WHERE product NE c_prod_gas .
7097
7098    IF lt_zpra_t_dly_prd IS NOT INITIAL.
7099
7100      LOOP AT lt_zpra_t_dly_prd INTO ls_zpra_t_dly_prd .
7101        lv_index = sy-tabix .
7102        IF lv_index EQ 1 OR
7103          <fs_dly_prd>-production_date NE ls_zpra_t_dly_prd-production_dat
      e  OR
```

```
7104          <fs_dly_prd>-product         NE ls_zpra_t_dly_prd-product
        OR
7105          <fs_dly_prd>-asset           NE ls_zpra_t_dly_prd-asset
        OR
7106          <fs_dly_prd>-block           NE ls_zpra_t_dly_prd-block .
7107
7108          APPEND INITIAL LINE TO gt_zpra_t_dly_prd_2a3 ASSIGNING <fs_dly_p
      rd> .
7109          <fs_dly_prd>-mandt           = ls_zpra_t_dly_prd-mandt .
7110          <fs_dly_prd>-production_date = ls_zpra_t_dly_prd-production_date
       .
7111          <fs_dly_prd>-product         = ls_zpra_t_dly_prd-product .
7112          <fs_dly_prd>-asset           = ls_zpra_t_dly_prd-asset .
7113          <fs_dly_prd>-block           = ls_zpra_t_dly_prd-block .
7114          <fs_dly_prd>-prod_vl_uom1    = ls_zpra_t_dly_prd-prod_vl_uom1 .
7115          <fs_dly_prd>-prd_vl_type     = 'NET_PROD' .
7116        ENDIF.
7117        PERFORM convert_gas_units CHANGING ls_zpra_t_dly_prd.
7118        PERFORM convert_gas_per_day_units USING lv_days CHANGING ls_zpra_t
      _dly_prd .
7119        IF ls_zpra_t_dly_prd-prd_vl_type EQ 'GAS_INJ' .
7120          ls_zpra_t_dly_prd-prod_vl_qty1 = ls_zpra_t_dly_prd-prod_vl_qty1
      * -1 .
7121        ENDIF.
7122        <fs_dly_prd>-prod_vl_qty1    = <fs_dly_prd>-prod_vl_qty1 + ls_zpra
      _t_dly_prd-prod_vl_qty1 .
7123      ENDLOOP.
7124      SORT  gt_zpra_t_dly_prd_2a3 BY production_date product asset .
7125    ENDIF.
7126
7127
7128    lt_zpra_t_mrec_prd  = gt_zpra_t_mrec_prd_2a3 .
7129    SORT  lt_zpra_t_mrec_prd BY gjahr monat product asset block prd_vl_typ
      e .
7130
7131    DELETE gt_zpra_t_mrec_prd_2a3  WHERE product EQ c_prod_gas .
7132    DELETE lt_zpra_t_mrec_prd      WHERE product NE c_prod_gas .
7133    CHECK lt_zpra_t_mrec_prd IS NOT INITIAL.
7134
7135    LOOP AT lt_zpra_t_mrec_prd INTO ls_zpra_t_mrec_prd .
7136      lv_index = sy-tabix .
7137      IF lv_index EQ 1 OR
7138        <fs_mrec>-gjahr     NE ls_zpra_t_mrec_prd-gjahr   OR
7139        <fs_mrec>-monat     NE ls_zpra_t_mrec_prd-monat   OR
7140        <fs_mrec>-product   NE ls_zpra_t_mrec_prd-product OR
7141        <fs_mrec>-asset     NE ls_zpra_t_mrec_prd-asset   OR
7142        <fs_mrec>-block     NE ls_zpra_t_mrec_prd-block .
7143
7144        APPEND INITIAL LINE TO gt_zpra_t_mrec_prd_2a3 ASSIGNING <fs_mrec>
      .
7145        <fs_mrec>-gjahr         = ls_zpra_t_mrec_prd-gjahr .
7146        <fs_mrec>-monat         = ls_zpra_t_mrec_prd-monat .
7147        <fs_mrec>-product       = ls_zpra_t_mrec_prd-product .
7148        <fs_mrec>-asset         = ls_zpra_t_mrec_prd-asset .
7149        <fs_mrec>-block         = ls_zpra_t_mrec_prd-block .
7150        <fs_mrec>-prod_vl_uom1  = ls_zpra_t_mrec_prd-prod_vl_uom1 .
7151        <fs_mrec>-prd_vl_type   = 'NET_PROD' .
7152      ENDIF.
7153      PERFORM convert_mrec_gas_units CHANGING ls_zpra_t_mrec_prd.
7154      PERFORM convert_mrec_gas_per_day_units USING lv_days CHANGING ls_zpr
      a_t_mrec_prd .
```

```
7155      IF ls_zpra_t_mrec_prd-prd_vl_type EQ 'GAS_INJ' .
7156        ls_zpra_t_mrec_prd-prod_vl_qty1 = ls_zpra_t_mrec_prd-prod_vl_qty1
    * -1 .
7157      ENDIF.
7158      <fs_mrec>-prod_vl_qty1    = <fs_mrec>-prod_vl_qty1 + ls_zpra_t_mrec_
    prd-prod_vl_qty1 .
7159    ENDLOOP.
7160    * SORT  gt_zpra_t_mrec_prd_2a3 BY gjahr monat product asset block prd_vl
    _type . .
7161    SORT gt_zpra_t_mrec_prd_2a3 BY product ASCENDING asset ASCENDING block
        ASCENDING gjahr DESCENDING monat DESCENDING .
7162
7163  ENDFORM.
7164  *&---------------------------------------------------------------------*
7165  *&      Form  PROCESS_GAS_RECORDS_2D
7166  *&---------------------------------------------------------------------*
7167  *       text
7168  *---------------------------------------------------------------------*
7169  *  -->  p1        text
7170  *  <--  p2        text
7171  *---------------------------------------------------------------------*
7172  FORM process_gas_records_2d .
7173
7174    DATA : lt_zpra_t_dly_prd  TYPE STANDARD TABLE OF zpra_t_dly_prd,
7175           lt_zpra_t_mrec_prd TYPE STANDARD TABLE OF ty_zpra_t_mrec_prd.
7176
7177    DATA : ls_zpra_t_dly_prd  TYPE                  zpra_t_dly_prd,
7178           ls_zpra_t_mrec_prd TYPE                  ty_zpra_t_mrec_prd.
7179    FIELD-SYMBOLS : <fs_dly_prd> TYPE zpra_t_dly_prd,
7180                    <fs_mrec>    TYPE ty_zpra_t_mrec_prd.
7181
7182    DATA : lv_index            TYPE sy-tabix,
7183           lv_days             TYPE sy-tabix,
7184           lv_poper            TYPE t009b-poper,
7185           lv_month_begin_date TYPE sy-datum,
7186           lv_month_end_date   TYPE sy-datum.
7187
7188    lt_zpra_t_dly_prd  = gt_zpra_t_dly_prd_2d .
7189    SORT  lt_zpra_t_dly_prd BY production_date product asset block prd_vl_
    type .
7190
7191    DELETE gt_zpra_t_dly_prd_2d  WHERE product EQ c_prod_gas .
7192    DELETE lt_zpra_t_dly_prd     WHERE product NE c_prod_gas .
7193
7194    IF lt_zpra_t_dly_prd IS NOT INITIAL.
7195
7196      LOOP AT lt_zpra_t_dly_prd INTO ls_zpra_t_dly_prd .
7197        lv_index = sy-tabix .
7198        IF lv_index EQ 1 OR
7199          <fs_dly_prd>-production_date NE ls_zpra_t_dly_prd-production_dat
    e  OR
7200          <fs_dly_prd>-product         NE ls_zpra_t_dly_prd-product
      OR
7201          <fs_dly_prd>-asset           NE ls_zpra_t_dly_prd-asset
      OR
7202          <fs_dly_prd>-block           NE ls_zpra_t_dly_prd-block .
7203
7204          APPEND INITIAL LINE TO gt_zpra_t_dly_prd_2d ASSIGNING <fs_dly_pr
    d> .
7205          <fs_dly_prd>-mandt           = ls_zpra_t_dly_prd-mandt .
7206          <fs_dly_prd>-production_date = ls_zpra_t_dly_prd-production_date
```

```
.
        <fs_dly_prd>-product         = ls_zpra_t_dly_prd-product .
        <fs_dly_prd>-asset           = ls_zpra_t_dly_prd-asset .
        <fs_dly_prd>-block           = ls_zpra_t_dly_prd-block .
        <fs_dly_prd>-prod_vl_uom1    = ls_zpra_t_dly_prd-prod_vl_uom1 .
        <fs_dly_prd>-prd_vl_type     = 'NET_PROD' .
      ENDIF.
      PERFORM convert_gas_units CHANGING ls_zpra_t_dly_prd.

      CALL FUNCTION 'HR_JP_MONTH_BEGIN_END_DATE'
        EXPORTING
          iv_date             = ls_zpra_t_dly_prd-production_date
        IMPORTING
          ev_month_begin_date = lv_month_begin_date
          ev_month_end_date   = lv_month_end_date.
*   lv_days = lv_month_end_date - lv_month_begin_date + 1 .
*   PERFORM convert_gas_per_day_units USING lv_days CHANGING ls_zpra_t_dly_prd .

      IF ls_zpra_t_dly_prd-prd_vl_type EQ 'GAS_INJ' .
        ls_zpra_t_dly_prd-prod_vl_qty1 = ls_zpra_t_dly_prd-prod_vl_qty1 * -1 .
      ENDIF.
      <fs_dly_prd>-prod_vl_qty1    = <fs_dly_prd>-prod_vl_qty1 + ls_zpra_t_dly_prd-prod_vl_qty1 .
    ENDLOOP.
    SORT  gt_zpra_t_dly_prd_2d BY  product asset block production_date.
  ENDIF.


  lt_zpra_t_mrec_prd  = gt_zpra_t_mrec_prd_2d .
  SORT  lt_zpra_t_mrec_prd BY gjahr monat product asset block prd_vl_type .

  DELETE gt_zpra_t_mrec_prd_2d  WHERE product EQ c_prod_gas .
  DELETE lt_zpra_t_mrec_prd     WHERE product NE c_prod_gas .
  CHECK lt_zpra_t_mrec_prd IS NOT INITIAL.

  LOOP AT lt_zpra_t_mrec_prd INTO ls_zpra_t_mrec_prd .
    lv_index = sy-tabix .
    IF lv_index EQ 1 OR
      <fs_mrec>-gjahr     NE ls_zpra_t_mrec_prd-gjahr   OR
      <fs_mrec>-monat     NE ls_zpra_t_mrec_prd-monat   OR
      <fs_mrec>-product   NE ls_zpra_t_mrec_prd-product OR
      <fs_mrec>-asset     NE ls_zpra_t_mrec_prd-asset   OR
      <fs_mrec>-block     NE ls_zpra_t_mrec_prd-block .

      APPEND INITIAL LINE TO gt_zpra_t_mrec_prd_2d ASSIGNING <fs_mrec> .
      <fs_mrec>-gjahr         = ls_zpra_t_mrec_prd-gjahr .
      <fs_mrec>-monat         = ls_zpra_t_mrec_prd-monat .
      <fs_mrec>-product       = ls_zpra_t_mrec_prd-product .
      <fs_mrec>-asset         = ls_zpra_t_mrec_prd-asset .
      <fs_mrec>-block         = ls_zpra_t_mrec_prd-block .
      <fs_mrec>-prod_vl_uom1  = ls_zpra_t_mrec_prd-prod_vl_uom1 .
      <fs_mrec>-prd_vl_type   = 'NET_PROD' .
    ENDIF.
    PERFORM convert_mrec_gas_units CHANGING ls_zpra_t_mrec_prd.

    lv_poper = ls_zpra_t_mrec_prd-monat .
    CALL FUNCTION 'FIRST_DAY_IN_PERIOD_GET'
      EXPORTING
```

```
7263          i_gjahr        = ls_zpra_t_mrec_prd-gjahr
7264  *       I_MONMIT       = 00
7265          i_periv        = gv_periv
7266          i_poper        = lv_poper
7267        IMPORTING
7268          e_date         = lv_month_begin_date
7269        EXCEPTIONS
7270          input_false    = 1
7271          t009_notfound  = 2
7272          t009b_notfound = 3
7273          OTHERS         = 4.
7274      IF sy-subrc <> 0.
7275        MESSAGE 'Internal Error' TYPE 'E' .
7276      ENDIF.
7277
7278      CALL FUNCTION 'LAST_DAY_IN_PERIOD_GET'
7279        EXPORTING
7280          i_gjahr        = ls_zpra_t_mrec_prd-gjahr
7281  *       I_MONMIT       = 00
7282          i_periv        = gv_periv
7283          i_poper        = lv_poper
7284        IMPORTING
7285          e_date         = lv_month_end_date
7286        EXCEPTIONS
7287          input_false    = 1
7288          t009_notfound  = 2
7289          t009b_notfound = 3
7290          OTHERS         = 4.
7291      IF sy-subrc <> 0.
7292        MESSAGE 'Internal Error' TYPE 'E' .
7293      ENDIF.
7294
7295  *   lv_days = lv_month_end_date - lv_month_begin_date + 1 .
7296  *   PERFORM convert_mrec_gas_per_day_units USING lv_days CHANGING ls_zpr
      a_t_mrec_prd .
7297
7298      IF ls_zpra_t_mrec_prd-prd_vl_type EQ 'GAS_INJ' .
7299        ls_zpra_t_mrec_prd-prod_vl_qty1 = ls_zpra_t_mrec_prd-prod_vl_qty1
      * -1 .
7300      ENDIF.
7301      <fs_mrec>-prod_vl_qty1    = <fs_mrec>-prod_vl_qty1 + ls_zpra_t_mrec_
      prd-prod_vl_qty1 .
7302    ENDLOOP.
7303  * SORT  gt_zpra_t_mrec_prd_2d BY gjahr monat product asset block prd_vl_
      type . .
7304    SORT gt_zpra_t_mrec_prd_2d BY product ASCENDING asset ASCENDING block
       ASCENDING gjahr DESCENDING monat DESCENDING .
7305  ENDFORM.
7306  *&———————————————————————————————————————————————————————————————*
7307  *&      Form  PROCESS_GAS_RECORDS_3C
7308  *&———————————————————————————————————————————————————————————————*
7309  *       text
7310  *———————————————————————————————————————————————————————————————*
7311  *  —>  p1        text
7312  *  <—  p2        text
7313  *———————————————————————————————————————————————————————————————*
7314  FORM process_gas_records_3c .
7315
7316    DATA : lt_zpra_t_dly_prd  TYPE STANDARD TABLE OF zpra_t_dly_prd,
7317           lt_zpra_t_mrec_app TYPE STANDARD TABLE OF ty_zpra_t_mrec_app,
7318           lt_zpra_t_mrec_prd TYPE STANDARD TABLE OF ty_zpra_t_mrec_prd.
```

```
7319
7320    DATA : ls_zpra_t_dly_prd  TYPE                  zpra_t_dly_prd,
7321           ls_zpra_t_mrec_app TYPE                  ty_zpra_t_mrec_app,
7322           ls_zpra_t_mrec_prd TYPE                  ty_zpra_t_mrec_prd.
7323    FIELD-SYMBOLS : <fs_dly_prd>  TYPE zpra_t_dly_prd,
7324                    <fs_mrec_app> TYPE ty_zpra_t_mrec_app,
7325                    <fs_mrec_prd> TYPE ty_zpra_t_mrec_prd.
7326
7327    DATA : lv_index            TYPE sy-tabix,
7328           lv_days             TYPE sy-tabix,
7329           lv_poper            TYPE t009b-poper,
7330           lv_month_begin_date TYPE sy-datum,
7331           lv_month_end_date   TYPE sy-datum.
7332
7333    lt_zpra_t_dly_prd  = gt_zpra_t_dly_prd_3c .
7334    SORT  lt_zpra_t_dly_prd BY production_date product asset block prd_vl_
      type .
7335
7336    DELETE gt_zpra_t_dly_prd_3c  WHERE product EQ c_prod_gas .
7337    DELETE lt_zpra_t_dly_prd     WHERE product NE c_prod_gas .
7338
7339    IF lt_zpra_t_dly_prd IS NOT INITIAL.
7340
7341      LOOP AT lt_zpra_t_dly_prd INTO ls_zpra_t_dly_prd .
7342        lv_index = sy-tabix .
7343        IF lv_index EQ 1 OR
7344          <fs_dly_prd>-production_date NE ls_zpra_t_dly_prd-production_dat
      e  OR
7345          <fs_dly_prd>-product         NE ls_zpra_t_dly_prd-product
         OR
7346          <fs_dly_prd>-asset           NE ls_zpra_t_dly_prd-asset
         OR
7347          <fs_dly_prd>-block           NE ls_zpra_t_dly_prd-block .
7348
7349          APPEND INITIAL LINE TO gt_zpra_t_dly_prd_3c ASSIGNING <fs_dly_pr
      d> .
7350          <fs_dly_prd>-mandt           = ls_zpra_t_dly_prd-mandt .
7351          <fs_dly_prd>-production_date = ls_zpra_t_dly_prd-production_date
       .
7352          <fs_dly_prd>-product         = ls_zpra_t_dly_prd-product .
7353          <fs_dly_prd>-asset           = ls_zpra_t_dly_prd-asset .
7354          <fs_dly_prd>-block           = ls_zpra_t_dly_prd-block .
7355          <fs_dly_prd>-prod_vl_uom1    = ls_zpra_t_dly_prd-prod_vl_uom1 .
7356          <fs_dly_prd>-prd_vl_type     = 'NET_PROD' .
7357        ENDIF.
7358        PERFORM convert_gas_units_to_bcm CHANGING ls_zpra_t_dly_prd.
7359
7360        IF ls_zpra_t_dly_prd-prd_vl_type EQ 'GAS_INJ' .
7361          ls_zpra_t_dly_prd-prod_vl_qty1 = ls_zpra_t_dly_prd-prod_vl_qty1
      * -1 .
7362        ENDIF.
7363        <fs_dly_prd>-prod_vl_qty1    = <fs_dly_prd>-prod_vl_qty1 + ls_zpra
      _t_dly_prd-prod_vl_qty1 .
7364      ENDLOOP.
7365      SORT  gt_zpra_t_dly_prd_3c BY  product asset block production_date.
7366    ENDIF.
7367
7368
7369    lt_zpra_t_mrec_prd  = gt_zpra_t_mrec_prd_3c .
7370    SORT  lt_zpra_t_mrec_prd BY gjahr monat product asset block prd_vl_typ
      e .
```

```
7371
7372    DELETE gt_zpra_t_mrec_prd_3c  WHERE product EQ c_prod_gas .
7373    DELETE lt_zpra_t_mrec_prd  WHERE product NE c_prod_gas .
7374
7375    IF lt_zpra_t_mrec_prd IS NOT INITIAL.
7376
7377      LOOP AT lt_zpra_t_mrec_prd INTO ls_zpra_t_mrec_prd .
7378        lv_index = sy-tabix .
7379        IF lv_index EQ 1 OR
7380          <fs_mrec_prd>-gjahr     NE ls_zpra_t_mrec_prd-gjahr   OR
7381          <fs_mrec_prd>-monat     NE ls_zpra_t_mrec_prd-monat   OR
7382          <fs_mrec_prd>-product   NE ls_zpra_t_mrec_prd-product OR
7383          <fs_mrec_prd>-asset     NE ls_zpra_t_mrec_prd-asset   OR
7384          <fs_mrec_prd>-block     NE ls_zpra_t_mrec_prd-block .
7385
7386          APPEND INITIAL LINE TO gt_zpra_t_mrec_prd_3c ASSIGNING <fs_mrec_
      prd> .
7387          <fs_mrec_prd>-gjahr         = ls_zpra_t_mrec_prd-gjahr .
7388          <fs_mrec_prd>-monat         = ls_zpra_t_mrec_prd-monat .
7389          <fs_mrec_prd>-product       = ls_zpra_t_mrec_prd-product .
7390          <fs_mrec_prd>-asset         = ls_zpra_t_mrec_prd-asset .
7391          <fs_mrec_prd>-block         = ls_zpra_t_mrec_prd-block .
7392          <fs_mrec_prd>-prod_vl_uom1    = ls_zpra_t_mrec_prd-prod_vl_uom1
      .
7393          <fs_mrec_prd>-prd_vl_type   = 'NET_PROD' .
7394        ENDIF.
7395        PERFORM convert_mrec_to_bcm CHANGING ls_zpra_t_mrec_prd.
7396
7397        IF ls_zpra_t_mrec_prd-prd_vl_type EQ 'GAS_INJ' .
7398          ls_zpra_t_mrec_prd-prod_vl_qty1 = ls_zpra_t_mrec_prd-prod_vl_qty
      1 * -1 .
7399        ENDIF.
7400        <fs_mrec_prd>-prod_vl_qty1    = <fs_mrec_prd>-prod_vl_qty1 + ls_zp
      ra_t_mrec_prd-prod_vl_qty1 .
7401      ENDLOOP.
7402    ENDIF.
7403  * SORT  gt_zpra_t_mrec_app BY gjahr monat product asset block prd_vl_typ
      e .
7404    SORT gt_zpra_t_mrec_prd_3c BY product ASCENDING asset ASCENDING block
       ASCENDING gjahr DESCENDING monat DESCENDING .
7405
7406  ENDFORM.
7407  *&———————————————————————————————————————————————————————————*
7408  *&      Form  PROCESS_GAS_RECORDS_3C
7409  *&———————————————————————————————————————————————————————————*
7410  *       text
7411  *———————————————————————————————————————————————————————————*
7412  *  —>  p1        text
7413  *  <—  p2        text
7414  *———————————————————————————————————————————————————————————*
7415  FORM process_gas_records_3f .
7416
7417    DATA : lt_zpra_t_dly_prd  TYPE STANDARD TABLE OF zpra_t_dly_prd,
7418           lt_zpra_t_mrec_app TYPE STANDARD TABLE OF ty_zpra_t_mrec_app,
7419           lt_zpra_t_mrec_prd TYPE STANDARD TABLE OF ty_zpra_t_mrec_prd.
7420
7421    DATA : ls_zpra_t_dly_prd  TYPE                  zpra_t_dly_prd,
7422           ls_zpra_t_mrec_app TYPE                  ty_zpra_t_mrec_app,
7423           ls_zpra_t_mrec_prd TYPE                  ty_zpra_t_mrec_prd.
7424    FIELD-SYMBOLS : <fs_dly_prd>  TYPE zpra_t_dly_prd,
7425                    <fs_mrec_app> TYPE ty_zpra_t_mrec_app,
```

```
                    <fs_mrec_prd> TYPE ty_zpra_t_mrec_prd.

  DATA : lv_index            TYPE sy-tabix,
         lv_days             TYPE sy-tabix,
         lv_poper            TYPE t009b-poper,
         lv_month_begin_date TYPE sy-datum,
         lv_month_end_date   TYPE sy-datum.

  lt_zpra_t_dly_prd  = gt_zpra_t_dly_prd_3f .
  SORT  lt_zpra_t_dly_prd BY production_date product asset block prd_vl_type .

  DELETE gt_zpra_t_dly_prd_3f  WHERE product EQ c_prod_gas .
  DELETE lt_zpra_t_dly_prd     WHERE product NE c_prod_gas .

  IF lt_zpra_t_dly_prd IS NOT INITIAL.

    LOOP AT lt_zpra_t_dly_prd INTO ls_zpra_t_dly_prd .
      lv_index = sy-tabix .
      IF lv_index EQ 1 OR
        <fs_dly_prd>-production_date NE ls_zpra_t_dly_prd-production_date  OR
        <fs_dly_prd>-product         NE ls_zpra_t_dly_prd-product  OR
        <fs_dly_prd>-asset           NE ls_zpra_t_dly_prd-asset  OR
        <fs_dly_prd>-block           NE ls_zpra_t_dly_prd-block .

        APPEND INITIAL LINE TO gt_zpra_t_dly_prd_3f ASSIGNING <fs_dly_prd> .
        <fs_dly_prd>-mandt           = ls_zpra_t_dly_prd-mandt .
        <fs_dly_prd>-production_date = ls_zpra_t_dly_prd-production_date .
        <fs_dly_prd>-product         = ls_zpra_t_dly_prd-product .
        <fs_dly_prd>-asset           = ls_zpra_t_dly_prd-asset .
        <fs_dly_prd>-block           = ls_zpra_t_dly_prd-block .
        <fs_dly_prd>-prod_vl_uom1    = ls_zpra_t_dly_prd-prod_vl_uom1 .
        <fs_dly_prd>-prd_vl_type     = 'NET_PROD' .
      ENDIF.
      PERFORM convert_gas_units_to_bcm CHANGING ls_zpra_t_dly_prd.

      IF ls_zpra_t_dly_prd-prd_vl_type EQ 'GAS_INJ' .
        ls_zpra_t_dly_prd-prod_vl_qty1 = ls_zpra_t_dly_prd-prod_vl_qty1 * -1 .
      ENDIF.
      <fs_dly_prd>-prod_vl_qty1    = <fs_dly_prd>-prod_vl_qty1 + ls_zpra_t_dly_prd-prod_vl_qty1 .
    ENDLOOP.
    SORT  gt_zpra_t_dly_prd_3f BY  product asset block production_date.
  ENDIF.

  lt_zpra_t_mrec_prd  = gt_zpra_t_mrec_prd_3f .
  SORT  lt_zpra_t_mrec_prd BY gjahr monat product asset block prd_vl_type .

  DELETE gt_zpra_t_mrec_prd_3f  WHERE product EQ c_prod_gas .
  DELETE lt_zpra_t_mrec_prd  WHERE product NE c_prod_gas .

  IF lt_zpra_t_mrec_prd IS NOT INITIAL.

    LOOP AT lt_zpra_t_mrec_prd INTO ls_zpra_t_mrec_prd .
```

```
      lv_index = sy-tabix .
      IF lv_index EQ 1 OR
        <fs_mrec_prd>-gjahr     NE ls_zpra_t_mrec_prd-gjahr   OR
        <fs_mrec_prd>-monat     NE ls_zpra_t_mrec_prd-monat   OR
        <fs_mrec_prd>-product   NE ls_zpra_t_mrec_prd-product OR
        <fs_mrec_prd>-asset     NE ls_zpra_t_mrec_prd-asset   OR
        <fs_mrec_prd>-block     NE ls_zpra_t_mrec_prd-block .

        APPEND INITIAL LINE TO gt_zpra_t_mrec_prd_3f ASSIGNING <fs_mrec_prd> .
        <fs_mrec_prd>-gjahr         = ls_zpra_t_mrec_prd-gjahr .
        <fs_mrec_prd>-monat         = ls_zpra_t_mrec_prd-monat .
        <fs_mrec_prd>-product       = ls_zpra_t_mrec_prd-product .
        <fs_mrec_prd>-asset         = ls_zpra_t_mrec_prd-asset .
        <fs_mrec_prd>-block         = ls_zpra_t_mrec_prd-block .
        <fs_mrec_prd>-prod_vl_uom1    = ls_zpra_t_mrec_prd-prod_vl_uom1 .
        <fs_mrec_prd>-prd_vl_type   = 'NET_PROD' .
      ENDIF.
      PERFORM convert_mrec_to_bcm CHANGING ls_zpra_t_mrec_prd.

      IF ls_zpra_t_mrec_prd-prd_vl_type EQ 'GAS_INJ' .
        ls_zpra_t_mrec_prd-prod_vl_qty1 = ls_zpra_t_mrec_prd-prod_vl_qty1 * -1 .
      ENDIF.
      <fs_mrec_prd>-prod_vl_qty1    = <fs_mrec_prd>-prod_vl_qty1 + ls_zpra_t_mrec_prd-prod_vl_qty1 .
    ENDLOOP.
  ENDIF.
* SORT  gt_zpra_t_mrec_app BY gjahr monat product asset block prd_vl_type .
  SORT gt_zpra_t_mrec_prd_3f BY product ASCENDING asset ASCENDING block ASCENDING gjahr DESCENDING monat DESCENDING .

ENDFORM.

*&———————————————————————————————————————————————————————————————*
*&      Form  PROCESS_GAS_RECORDS_4A
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM process_gas_records_4a .

  DATA : lt_zpra_t_dly_prd  TYPE STANDARD TABLE OF zpra_t_dly_prd,
         lt_zpra_t_mrec_app TYPE STANDARD TABLE OF ty_zpra_t_mrec_app.

  DATA : ls_zpra_t_dly_prd  TYPE                  zpra_t_dly_prd,
         ls_zpra_t_mrec_app TYPE                  ty_zpra_t_mrec_app.
  FIELD-SYMBOLS : <fs_dly_prd>  TYPE zpra_t_dly_prd,
                  <fs_mrec_app> TYPE ty_zpra_t_mrec_app.

  DATA : lv_index            TYPE sy-tabix,
         lv_days             TYPE sy-tabix,
         lv_poper            TYPE t009b-poper,
         lv_month_begin_date TYPE sy-datum,
         lv_month_end_date   TYPE sy-datum.

  lt_zpra_t_dly_prd  = gt_zpra_t_dly_prd_4a .
```

```
  SORT  lt_zpra_t_dly_prd BY production_date product asset block prd_vl_type .

  DELETE gt_zpra_t_dly_prd_4a  WHERE product EQ '722000004' .
  DELETE lt_zpra_t_dly_prd     WHERE product NE '722000004' .

  IF lt_zpra_t_dly_prd IS NOT INITIAL.

    LOOP AT lt_zpra_t_dly_prd INTO ls_zpra_t_dly_prd .
      lv_index = sy-tabix .
      IF lv_index EQ 1 OR
        <fs_dly_prd>-production_date NE ls_zpra_t_dly_prd-production_date  OR
        <fs_dly_prd>-product         NE ls_zpra_t_dly_prd-product  OR
        <fs_dly_prd>-asset           NE ls_zpra_t_dly_prd-asset  OR
        <fs_dly_prd>-block           NE ls_zpra_t_dly_prd-block .

        APPEND INITIAL LINE TO gt_zpra_t_dly_prd_4a ASSIGNING <fs_dly_prd> .
        <fs_dly_prd>-mandt           = ls_zpra_t_dly_prd-mandt .
        <fs_dly_prd>-production_date = ls_zpra_t_dly_prd-production_date .
        <fs_dly_prd>-product         = ls_zpra_t_dly_prd-product .
        <fs_dly_prd>-asset           = ls_zpra_t_dly_prd-asset .
        <fs_dly_prd>-block           = ls_zpra_t_dly_prd-block .
        <fs_dly_prd>-prod_vl_uom1    = ls_zpra_t_dly_prd-prod_vl_uom1 .
        <fs_dly_prd>-prd_vl_type     = 'NET_PROD' .
      ENDIF.
      PERFORM convert_gas_units_to_mmscm CHANGING ls_zpra_t_dly_prd.

      IF ls_zpra_t_dly_prd-prd_vl_type EQ 'GAS_INJ' .
        ls_zpra_t_dly_prd-prod_vl_qty1 = ls_zpra_t_dly_prd-prod_vl_qty1 * -1 .
      ENDIF.
      <fs_dly_prd>-prod_vl_qty1    = <fs_dly_prd>-prod_vl_qty1 + ls_zpra_t_dly_prd-prod_vl_qty1 .
    ENDLOOP.
    SORT  gt_zpra_t_dly_prd_4a BY  product asset block production_date.
  ENDIF.


  lt_zpra_t_mrec_app  = gt_zpra_t_mrec_app_4a .
  SORT  lt_zpra_t_mrec_app BY gjahr monat product asset block prd_vl_type .

  DELETE gt_zpra_t_mrec_app_4a  WHERE product EQ '722000004' .
  DELETE lt_zpra_t_mrec_app     WHERE product NE '722000004' .
  CHECK lt_zpra_t_mrec_app IS NOT INITIAL.

  LOOP AT lt_zpra_t_mrec_app INTO ls_zpra_t_mrec_app .
    lv_index = sy-tabix .
    IF lv_index EQ 1 OR
      <fs_mrec_app>-gjahr     NE ls_zpra_t_mrec_app-gjahr   OR
      <fs_mrec_app>-monat     NE ls_zpra_t_mrec_app-monat   OR
      <fs_mrec_app>-product   NE ls_zpra_t_mrec_app-product OR
      <fs_mrec_app>-asset     NE ls_zpra_t_mrec_app-asset   OR
      <fs_mrec_app>-block     NE ls_zpra_t_mrec_app-block .

      APPEND INITIAL LINE TO gt_zpra_t_mrec_app_4a ASSIGNING <fs_mrec_ap
```

```
p> .
      <fs_mrec_app>-gjahr         = ls_zpra_t_mrec_app-gjahr .
      <fs_mrec_app>-monat         = ls_zpra_t_mrec_app-monat .
      <fs_mrec_app>-product       = ls_zpra_t_mrec_app-product .
      <fs_mrec_app>-asset         = ls_zpra_t_mrec_app-asset .
      <fs_mrec_app>-block         = ls_zpra_t_mrec_app-block .
      <fs_mrec_app>-app_vl_uom    = ls_zpra_t_mrec_app-app_vl_uom .
      <fs_mrec_app>-prd_vl_type   = 'NET_PROD' .
    ENDIF.
    PERFORM convert_mrec_gas_to_mmscm CHANGING ls_zpra_t_mrec_app.

    IF ls_zpra_t_mrec_app-prd_vl_type EQ 'GAS_INJ' .
      ls_zpra_t_mrec_app-app_vl_qty = ls_zpra_t_mrec_app-app_vl_qty * -1
 .
    ENDIF.
    <fs_mrec_app>-app_vl_qty    = <fs_mrec_app>-app_vl_qty + ls_zpra_t_m
rec_app-app_vl_qty .
  ENDLOOP.
  SORT  gt_zpra_t_mrec_app_4a BY gjahr monat product asset block prd_vl_
type . .

ENDFORM.

*&———————————————————————————————————————————————————————————————*
*&      Form  PROCESS_GAS_RECORDS_2F
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM process_gas_records_2f .

  DATA : lt_zpra_t_dly_prd  TYPE STANDARD TABLE OF zpra_t_dly_prd,
         lt_zpra_t_mrec_prd TYPE STANDARD TABLE OF ty_zpra_t_mrec_prd.

  DATA : ls_zpra_t_dly_prd  TYPE                  zpra_t_dly_prd,
         ls_zpra_t_mrec_prd TYPE                  ty_zpra_t_mrec_prd.
  FIELD-SYMBOLS : <fs_dly_prd> TYPE zpra_t_dly_prd,
                  <fs_mrec>    TYPE ty_zpra_t_mrec_prd.

  DATA : lv_index            TYPE sy-tabix,
         lv_days             TYPE sy-tabix,
         lv_poper            TYPE t009b-poper,
         lv_month_begin_date TYPE sy-datum,
         lv_month_end_date   TYPE sy-datum.

  lt_zpra_t_dly_prd  = gt_zpra_t_dly_prd_2f .
  SORT  lt_zpra_t_dly_prd BY production_date product asset block prd_vl_
type .

  DELETE gt_zpra_t_dly_prd_2f  WHERE product EQ c_prod_gas .
  DELETE lt_zpra_t_dly_prd     WHERE product NE c_prod_gas .

  IF lt_zpra_t_dly_prd IS NOT INITIAL.

    LOOP AT lt_zpra_t_dly_prd INTO ls_zpra_t_dly_prd .
      lv_index = sy-tabix .
      IF lv_index EQ 1 OR
        <fs_dly_prd>-production_date NE ls_zpra_t_dly_prd-production_dat
e  OR
```

```
7640          <fs_dly_prd>-product         NE ls_zpra_t_dly_prd-product
        OR
7641          <fs_dly_prd>-asset           NE ls_zpra_t_dly_prd-asset
        OR
7642          <fs_dly_prd>-block           NE ls_zpra_t_dly_prd-block .
7643
7644          APPEND INITIAL LINE TO gt_zpra_t_dly_prd_2f ASSIGNING <fs_dly_pr
      d> .
7645          <fs_dly_prd>-mandt           = ls_zpra_t_dly_prd-mandt .
7646          <fs_dly_prd>-production_date = ls_zpra_t_dly_prd-production_date
       .
7647          <fs_dly_prd>-product         = ls_zpra_t_dly_prd-product .
7648          <fs_dly_prd>-asset           = ls_zpra_t_dly_prd-asset .
7649          <fs_dly_prd>-block           = ls_zpra_t_dly_prd-block .
7650          <fs_dly_prd>-prod_vl_uom1    = ls_zpra_t_dly_prd-prod_vl_uom1 .
7651          <fs_dly_prd>-prd_vl_type     = 'NET_PROD' .
7652        ENDIF.
7653        PERFORM convert_gas_units CHANGING ls_zpra_t_dly_prd.
7654
7655        CALL FUNCTION 'HR_JP_MONTH_BEGIN_END_DATE'
7656          EXPORTING
7657            iv_date             = ls_zpra_t_dly_prd-production_date
7658          IMPORTING
7659            ev_month_begin_date = lv_month_begin_date
7660            ev_month_end_date   = lv_month_end_date.
7661        lv_days = lv_month_end_date - lv_month_begin_date + 1 .
7662        lv_days = 1 . "will be calculated at end .
7663        PERFORM convert_gas_per_day_units USING lv_days CHANGING ls_zpra_t
      _dly_prd .
7664        IF ls_zpra_t_dly_prd-prd_vl_type EQ 'GAS_INJ' .
7665          ls_zpra_t_dly_prd-prod_vl_qty1 = ls_zpra_t_dly_prd-prod_vl_qty1
      * -1 .
7666        ENDIF.
7667        <fs_dly_prd>-prod_vl_qty1    = <fs_dly_prd>-prod_vl_qty1 + ls_zpra
      _t_dly_prd-prod_vl_qty1 .
7668      ENDLOOP.
7669      SORT  gt_zpra_t_dly_prd_2f BY  product asset block production_date.
7670    ENDIF.
7671
7672
7673    lt_zpra_t_mrec_prd  = gt_zpra_t_mrec_prd_2f .
7674    SORT  lt_zpra_t_mrec_prd BY gjahr monat product asset block prd_vl_typ
      e .
7675
7676    DELETE gt_zpra_t_mrec_prd_2f  WHERE product EQ c_prod_gas .
7677    DELETE lt_zpra_t_mrec_prd     WHERE product NE c_prod_gas .
7678    CHECK lt_zpra_t_mrec_prd IS NOT INITIAL.
7679
7680    LOOP AT lt_zpra_t_mrec_prd INTO ls_zpra_t_mrec_prd .
7681      lv_index = sy-tabix .
7682      IF lv_index EQ 1 OR
7683        <fs_mrec>-gjahr     NE ls_zpra_t_mrec_prd-gjahr   OR
7684        <fs_mrec>-monat     NE ls_zpra_t_mrec_prd-monat   OR
7685        <fs_mrec>-product   NE ls_zpra_t_mrec_prd-product OR
7686        <fs_mrec>-asset     NE ls_zpra_t_mrec_prd-asset   OR
7687        <fs_mrec>-block     NE ls_zpra_t_mrec_prd-block .
7688
7689        APPEND INITIAL LINE TO gt_zpra_t_mrec_prd_2f ASSIGNING <fs_mrec> .
7690        <fs_mrec>-gjahr         = ls_zpra_t_mrec_prd-gjahr .
7691        <fs_mrec>-monat         = ls_zpra_t_mrec_prd-monat .
7692        <fs_mrec>-product       = ls_zpra_t_mrec_prd-product .
```

```
      <fs_mrec>-asset         = ls_zpra_t_mrec_prd-asset .
      <fs_mrec>-block         = ls_zpra_t_mrec_prd-block .
      <fs_mrec>-prod_vl_uom1  = ls_zpra_t_mrec_prd-prod_vl_uom1 .
      <fs_mrec>-prd_vl_type   = 'NET_PROD' .
    ENDIF.
    PERFORM convert_mrec_gas_units CHANGING ls_zpra_t_mrec_prd.

    lv_poper = ls_zpra_t_mrec_prd-monat .
    CALL FUNCTION 'FIRST_DAY_IN_PERIOD_GET'
      EXPORTING
        i_gjahr        = ls_zpra_t_mrec_prd-gjahr
*       I_MONMIT       = 00
        i_periv        = gv_periv
        i_poper        = lv_poper
      IMPORTING
        e_date         = lv_month_begin_date
      EXCEPTIONS
        input_false    = 1
        t009_notfound  = 2
        t009b_notfound = 3
        OTHERS         = 4.
    IF sy-subrc <> 0.
      MESSAGE 'Internal Error' TYPE 'E' .
    ENDIF.

    CALL FUNCTION 'LAST_DAY_IN_PERIOD_GET'
      EXPORTING
        i_gjahr        = ls_zpra_t_mrec_prd-gjahr
*       I_MONMIT       = 00
        i_periv        = gv_periv
        i_poper        = lv_poper
      IMPORTING
        e_date         = lv_month_end_date
      EXCEPTIONS
        input_false    = 1
        t009_notfound  = 2
        t009b_notfound = 3
        OTHERS         = 4.
    IF sy-subrc <> 0.
      MESSAGE 'Internal Error' TYPE 'E' .
    ENDIF.

    lv_days = lv_month_end_date - lv_month_begin_date + 1 .
    lv_days = 1 . "will be calculated at end
    PERFORM convert_mrec_gas_per_day_units USING lv_days CHANGING ls_zpra_t_mrec_prd .
    IF ls_zpra_t_mrec_prd-prd_vl_type EQ 'GAS_INJ' .
      ls_zpra_t_mrec_prd-prod_vl_qty1 = ls_zpra_t_mrec_prd-prod_vl_qty1 * -1 .
    ENDIF.
    <fs_mrec>-prod_vl_qty1    = <fs_mrec>-prod_vl_qty1 + ls_zpra_t_mrec_prd-prod_vl_qty1 .
  ENDLOOP.
* SORT  gt_zpra_t_mrec_prd_2f BY gjahr monat product asset block prd_vl_type . .
  SORT gt_zpra_t_mrec_prd_2f  BY product ASCENDING asset ASCENDING block ASCENDING gjahr DESCENDING monat DESCENDING .
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  CONVERT_GAS_PER_DAY_UNITS
*&---------------------------------------------------------------------*
```

```
*       text
*————————————————————————————————————————————————————————————————————*
*      ——>P_LV_DAYS  text
*      <——P_GS_ZPRA_T_DLY_PRD  text
*————————————————————————————————————————————————————————————————————*
FORM convert_gas_per_day_units  USING    p_days
                                CHANGING p_zpra_t_dly_prd TYPE zpra_t_dly_prd.
  CHECK p_zpra_t_dly_prd-product EQ c_prod_gas .
  IF p_bbd IS NOT INITIAL OR
     p_tmd IS NOT INITIAL OR
     p_bmd IS NOT INITIAL .
    p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / p_days .
  ENDIF.
ENDFORM.
*&————————————————————————————————————————————————————————————————————*
*&      Form  CONVERT_GAS_PER_DAY_UNITS
*&————————————————————————————————————————————————————————————————————*
*       text
*————————————————————————————————————————————————————————————————————*
*      ——>P_LV_DAYS  text
*      <——P_GS_ZPRA_T_DLY_PRD  text
*————————————————————————————————————————————————————————————————————*
FORM convert_mrec_gas_per_day_units  USING    p_days
                                CHANGING p_zpra_t_mrec_prd TYPE ty_zpra_t_mrec_prd.
  CHECK p_zpra_t_mrec_prd-product EQ '722000004' .
  IF p_bbd IS NOT INITIAL OR
     p_tmd IS NOT INITIAL OR
     p_bmd IS NOT INITIAL .
    p_zpra_t_mrec_prd-prod_vl_qty1 = p_zpra_t_mrec_prd-prod_vl_qty1 / p_days .
  ENDIF.
ENDFORM.
*&————————————————————————————————————————————————————————————————————*
*&      Form  PROTECT_WORKSHEET
*&————————————————————————————————————————————————————————————————————*
*       text
*————————————————————————————————————————————————————————————————————*
*  ——>  p1        text
*  <——  p2        text
*————————————————————————————————————————————————————————————————————*
FORM finalize_worksheet .
  DATA: lv_sheet_name TYPE char50,
        lv_uzeit_ext  TYPE char8,
        lv_datum_ext  TYPE char10.
  CONCATENATE sy-uzeit(2) '-' sy-uzeit+2(2) '-' sy-uzeit+4(2) INTO lv_uzeit_ext .
  CONCATENATE sy-datum+6(2) '-' sy-datum+4(2) '-' sy-datum(4) INTO lv_datum_ext .

  PERFORM free_clipboard .
  PERFORM lock_xls .

  CALL FUNCTION 'CONVERSION_EXIT_SDATE_OUTPUT'
    EXPORTING
      input  = gv_rep_date
    IMPORTING
      output = lv_sheet_name.
  IF sy-subrc <> 0.
```

```
      MESSAGE 'Unexpteced Internal Error' TYPE 'E' .
    ENDIF.

    REPLACE ALL OCCURRENCES OF '.' IN lv_sheet_name WITH '-' .
    CONCATENATE 'DPR (' lv_sheet_name ')' INTO lv_sheet_name SEPARATED BY space .
    SET PROPERTY OF go_worksheet 'Name' = lv_sheet_name.
    SET PROPERTY OF go_excel 'VISIBLE' = 1 .
    SET PROPERTY OF go_worksheet2 'Visible' = 0 .
    PERFORM select_range USING 1 1 1 1 .
    .
    CONCATENATE p_fname '\DPR -' gv_repdate_e '-' 'On -' lv_datum_ext '-' lv_uzeit_ext '.PDF' INTO p_fname .
  * CONCATENATE p_fname '\DPR -' gv_repdate_e '-' 'Extracted On -' lv_datum_ext '-' lv_uzeit_ext '.PDF' INTO p_fname .

    CALL METHOD OF go_workbook 'ExportAsFixedFormat'
      EXPORTING
        #1 = 0
        #2 = p_fname
        #3 = 0
        #4 = 1.

  *CALL METHOD OF go_workbook 'Close'
  *EXPORTING #1 = 0 .
  *CALL METHOD OF go_workbook 'Quit'.
  *  PERFORM delete_image   .
    EXPORT p_fname TO MEMORY ID 'DPR_FILE_NAME' .
    MESSAGE 'Report downloaded sucessfully' TYPE 'S' .
  ENDFORM.
  *&---------------------------------------------------------------------*
  *&      Form  LOCK_XLS
  *&---------------------------------------------------------------------*
  *       text
  *----------------------------------------------------------------------*
  *  -->  p1        text
  *  <--  p2        text
  *----------------------------------------------------------------------*
  FORM lock_xls .

    DATA : lv_uuid_16    TYPE sysuuid_x16 .

  * lv_uuid_16 = cl_system_uuid=>create_uuid_x16_static( ).
  *  CALL METHOD OF go_worksheet 'PROTECT'
  *    EXPORTING #1 = lv_uuid_16.

  ENDFORM.

  *&---------------------------------------------------------------------*
  *&      Form  FILL_DYNAMIC_TABLE_SEC2D
  *&---------------------------------------------------------------------*
  *       text
  *----------------------------------------------------------------------*
  *  -->  p1        text
  *  <--  p2        text
  *----------------------------------------------------------------------*
  FORM fill_dynamic_table_sec2d .

    DATA : lt_dly_prd_main   TYPE STANDARD TABLE OF zpra_t_dly_prd,
           lt_zpra_t_dly_prd TYPE STANDARD TABLE OF zpra_t_dly_prd.
    DATA : ls_dly_prd_main   TYPE                   zpra_t_dly_prd,
```

```
          ls_zpra_t_dly_prd TYPE                    zpra_t_dly_prd.
  DATA : lv_monat         TYPE                    zpra_t_mrec_prd-monat,
         lv_gjahr         TYPE                    zpra_t_mrec_prd-gjahr,
         lv_gjahr_next    TYPE                    zpra_t_mrec_prd-gjahr,
         lv_combine_field TYPE                    c,
         lv_days          TYPE                    sy-tabix,
         lv_days2         TYPE                    sy-tabix,
         lv_index         TYPE                    sy-tabix,
         lv_rprd_index    TYPE                    sy-tabix,
         lv_col_name      TYPE                    lvc_fname,
         lv_year_index    TYPE                    sy-tabix,
         lv_start_date    TYPE                    sy-datum,
         lv_mmscmd_mul    TYPE                    sy-tabix,
         lv_product       TYPE                    char100,
         lv_asset         TYPE                    char100.

  gt_zpra_t_mrec_prd =  gt_zpra_t_mrec_prd_2d .
  lt_dly_prd_main = gt_zpra_t_dly_prd_2d .
  SORT lt_dly_prd_main BY product asset block  .
  DELETE ADJACENT DUPLICATES FROM lt_dly_prd_main COMPARING product asset block  .
  lv_gjahr = gv_current_gjahr .
  DO 2 TIMES .
    lv_gjahr_next = lv_gjahr + 1 .
    APPEND INITIAL LINE TO <gfs_sec2d_table> ASSIGNING <gfs_dyn_line> .
    ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    CONCATENATE lv_gjahr '-' lv_gjahr_next INTO <gfs_field> SEPARATED BY space.

    LOOP AT gt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof.
      lv_monat = '00' .
      DO .
        lv_monat = lv_monat + 1 .
        CLEAR lv_combine_field .
        READ TABLE gt_zpra_t_mrec_app_2d INTO gs_zpra_t_mrec_app WITH KEY gjahr   = lv_gjahr

        monat   = lv_monat

        product = gs_zpra_c_prd_prof-product

        asset   = gs_zpra_c_prd_prof-asset

        block   = gs_zpra_c_prd_prof-block BINARY SEARCH .
        IF sy-subrc IS INITIAL. "if found in MREC APP
          gs_zpra_t_mrec_app-app_vl_qty = gs_zpra_t_mrec_app-app_vl_qty * 1000000 .
          IF gs_zpra_t_mrec_app-product = c_prod_gas.
            READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_t_mrec_app-asset BINARY SEARCH .
            IF  sy-subrc IS INITIAL.
              lv_combine_field = 'X' .
            ENDIF.
          ENDIF.
*     Individual Column..
          gv_len = strlen( gs_zpra_t_mrec_app-product ) .
          IF lv_combine_field IS INITIAL.
            CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' gs_zpra_t_mrec_app-asset INTO lv_col_name .
          ELSE.
```

```
7912              CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' 'COMBINE'
                     INTO lv_col_name .
7913            ENDIF.
7914            ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <g
      fs_field> .
7915            IF <gfs_field> IS ASSIGNED.
7916              <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty .
7917              UNASSIGN <gfs_field> .
7918            ENDIF.
7919  *     Product Total..
7920            CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' 'TOTAL'
                     INTO lv_col_name .
7921            ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <g
      fs_field> .
7922            IF <gfs_field> IS ASSIGNED.
7923              <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty .
7924              UNASSIGN <gfs_field> .
7925            ENDIF.
7926  *     Grand Total
7927            CONCATENATE 'GRAND' '-' 'TOTAL'
                     INTO lv_col_name .
7928            ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <g
      fs_field> .
7929            IF <gfs_field> IS ASSIGNED.
7930              <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty.
7931              UNASSIGN <gfs_field> .
7932            ENDIF.
7933  **        ELSE . "Now get from MREC
7934  **          READ TABLE gt_zpra_t_mrec_prd_2d INTO gs_zpra_t_mrec_prd WIT
      H KEY gjahr   = lv_gjahr
7935  **
            monat   = lv_monat
7936  **
            product = gs_zpra_c_prd_prof-product
7937  **
            asset   = gs_zpra_c_prd_prof-asset
7938  **
            block   = gs_zpra_c_prd_prof-block BINARY SEARCH .
7939  **          IF sy-subrc IS INITIAL. "if found in MREC
7940  **            gs_zpra_t_dly_prd = ls_dly_prd_main .
7941  **            PERFORM get_days_in_period USING gs_zpra_t_mrec_prd-gjahr
      gs_zpra_t_mrec_prd-monat CHANGING lv_days .
7942  **            IF lv_monat EQ gv_current_monat.
7943  **              gs_zpra_t_mrec_prd-prod_vl_qty2 = gs_zpra_t_mrec_prd-pro
      d_vl_qty2 * gv_month_days_till_today / lv_days.
7944  **              gs_zpra_t_mrec_prd-prod_vl_qty1 = gs_zpra_t_mrec_prd-pro
      d_vl_qty1 * gv_month_days_till_today / lv_days.
7945  **              CONCATENATE gv_search_end_datum(6) p_date+6(2) INTO gv_s
      earch_end_datum .
7946  **            ENDIF.
7947  ***   Dividing monthly quantity is giving slightly off values, due to pr
      ecision. Will sum the quantity for the whole period
7948  ***   and then divide by total days. to keep code structure intact, curr
      ently making days as 1
7949  **            lv_days = 1 .
7950  **            PERFORM convert_non_gas_mrec_units USING lv_days CHANGING
      gs_zpra_t_mrec_prd.
7951  **            IF gs_zpra_t_mrec_prd-product = c_prod_gas.
7952  **             READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine W
      ITH KEY asset = gs_zpra_t_mrec_prd-asset BINARY SEARCH .
7953  **             IF  sy-subrc IS INITIAL.
```

```
7954  **                lv_combine_field = 'X' .
7955  **              ENDIF.
7956  **             ENDIF.
7957  **             PERFORM get_constorium_multipliers_2a2 USING '5'.
7958  **
7959  ***       Individual Column..
7960  **           gv_len = strlen( gs_zpra_t_mrec_prd-product ) .
7961  **           IF lv_combine_field IS INITIAL.
7962  **             CONCATENATE gs_zpra_t_mrec_prd-product(gv_len) '-' gs_zpr
      a_t_mrec_prd-asset INTO lv_col_name .
7963  **           ELSE.
7964  **             CONCATENATE gs_zpra_t_mrec_prd-product(gv_len) '-' 'COMBI
      NE'                INTO lv_col_name .
7965  **           ENDIF.
7966  **           ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO
       <gfs_field> .
7967  **           IF <gfs_field> IS ASSIGNED.
7968  **             <gfs_field> = <gfs_field> + gs_zpra_t_mrec_prd-prod_vl_qt
      y1 * gv_individual_mul.
7969  **             UNASSIGN <gfs_field> .
7970  **           ENDIF.
7971  ***       Product Total..
7972  **           CONCATENATE gs_zpra_t_mrec_prd-product(gv_len) '-' 'TOTAL'
                       INTO lv_col_name .
7973  **           ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO
       <gfs_field> .
7974  **           IF <gfs_field> IS ASSIGNED.
7975  **             <gfs_field> = <gfs_field> + gs_zpra_t_mrec_prd-prod_vl_qt
      y1 * gv_total_mul .
7976  **             UNASSIGN <gfs_field> .
7977  **           ENDIF.
7978  ***       Grand Total
7979  **           CONCATENATE 'GRAND' '-' 'TOTAL'
                       INTO lv_col_name .
7980  **           ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO
       <gfs_field> .
7981  **           IF <gfs_field> IS ASSIGNED.
7982  **             <gfs_field> = <gfs_field> + gs_zpra_t_mrec_prd-prod_vl_qt
      y1 * gv_grand_total_mul.
7983  **             UNASSIGN <gfs_field> .
7984  **           ENDIF.
7985          ELSE.
7986            PERFORM get_days_in_period USING lv_gjahr lv_monat CHANGING lv
      _days .
7987            IF lv_monat EQ gv_current_monat.
7988              CONCATENATE gv_search_end_datum(4) p_date+4(4) INTO gv_searc
      h_end_datum .
7989            ENDIF.
7990            READ TABLE gt_zpra_t_dly_rprd_2d INTO gs_zpra_t_dly_rprd WITH
      KEY product = gs_zpra_c_prd_prof-product
7991
                  asset = gs_zpra_c_prd_prof-asset
7992
                  block = gs_zpra_c_prd_prof-block
7993                                                                    prod
      uction_date = gv_search_begin_datum BINARY SEARCH .
7994            IF sy-subrc IS NOT INITIAL.
7995              LOOP AT gt_zpra_t_dly_rprd_2d INTO gs_zpra_t_dly_rprd WHERE
      product         EQ gs_zpra_c_prd_prof-product
7996                                                                    AND
      asset           EQ gs_zpra_c_prd_prof-asset
```

```
                                                                AND
      block           EQ gs_zpra_c_prd_prof-block
                                                                AND
      production_date GE gv_search_begin_datum
                                                                AND
      production_date LE gv_search_end_datum .
                EXIT .
              ENDLOOP .
            ENDIF.
            IF sy-subrc IS INITIAL.
              lv_rprd_index = sy-tabix .
              LOOP AT gt_zpra_t_dly_rprd_2d INTO gs_zpra_t_dly_rprd FROM l
      v_rprd_index .
                IF gs_zpra_t_dly_rprd-product EQ gs_zpra_c_prd_prof-produc
      t      AND
                   gs_zpra_t_dly_rprd-asset   EQ gs_zpra_c_prd_prof-asset
             AND
                   gs_zpra_t_dly_rprd-block   EQ gs_zpra_c_prd_prof-block
             AND
                   gs_zpra_t_dly_rprd-production_date GE gv_search_begin_d
      atum   AND
                   gs_zpra_t_dly_rprd-production_date LE gv_search_end_dat
      um .
                ELSE.
                  EXIT .
                ENDIF.
                lv_index = sy-tabix .
                CLEAR lv_combine_field .

                PERFORM convert_rprd_units USING gs_zpra_t_dly_rprd .

                IF gs_zpra_t_dly_rprd-product = c_prod_gas .
                  READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine
      WITH KEY asset = gs_zpra_t_dly_rprd-asset BINARY SEARCH .
                  IF  sy-subrc IS INITIAL.
                    lv_combine_field = 'X' .
                  ENDIF.
                ENDIF.
*             Individual Column..
                gv_len = strlen( gs_zpra_t_dly_rprd-product ) .
                IF lv_combine_field IS INITIAL.
                  CONCATENATE gs_zpra_t_dly_rprd-product(gv_len) '-' gs_zp
      ra_t_dly_rprd-asset INTO lv_col_name .
                ELSE.
                  CONCATENATE gs_zpra_t_dly_rprd-product(gv_len) '-' 'COMB
      INE'                INTO lv_col_name .
                ENDIF.
                ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> T
      O <gfs_field> .
                IF <gfs_field> IS ASSIGNED.
                  <gfs_field> = <gfs_field> + gs_zpra_t_dly_rprd-jv_prd_vl
      _qty1.
                  UNASSIGN <gfs_field> .
                ENDIF.
*             Product Total..
                CONCATENATE gs_zpra_t_dly_rprd-product(gv_len) '-' 'TOTAL'
                     INTO lv_col_name .
                ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> T
      O <gfs_field> .
                IF <gfs_field> IS ASSIGNED.
                  <gfs_field> = <gfs_field> +  gs_zpra_t_dly_rprd-ovl_prd_
```

```
vl_qty1 .
                UNASSIGN <gfs_field> .
              ENDIF.
*           Grand Total
              CONCATENATE 'GRAND' '-' 'TOTAL'
                   INTO lv_col_name .
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
              IF <gfs_field> IS ASSIGNED.
                lv_mmscmd_mul = 1 .
                IF gs_zpra_t_dly_rprd-product EQ c_prod_gas AND p_bmd IS NOT INITIAL.
*                  lv_mmscmd_mul = 6290 .
                ENDIF.
                <gfs_field> = <gfs_field> + ( gs_zpra_t_dly_rprd-ovl_prd_vl_qty1 * lv_mmscmd_mul ).
                UNASSIGN <gfs_field> .
              ENDIF.
            ENDLOOP.
          ELSE. "Now work out from daily production
            PERFORM get_days_in_period USING lv_gjahr lv_monat CHANGING lv_days .
            lv_days = 1 .
            IF lv_monat EQ gv_current_monat.
              CONCATENATE gv_search_end_datum(4) p_date+4(4) INTO gv_search_end_datum .
            ENDIF.
            LOOP AT gt_zpra_t_dly_prd_2d INTO gs_zpra_t_dly_prd WHERE product EQ gs_zpra_c_prd_prof-product
                                                                      AND asset   EQ gs_zpra_c_prd_prof-asset
                                                                      AND block   EQ gs_zpra_c_prd_prof-block
                                                                      AND production_date GE gv_search_begin_datum
                                                                      AND production_date LE gv_search_end_datum .
              lv_index = sy-tabix .
              CLEAR lv_combine_field .

              PERFORM convert_non_gas_units2 USING lv_days CHANGING gs_zpra_t_dly_prd .

              IF gs_zpra_t_dly_prd-product = c_prod_gas.
                READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_t_dly_prd-asset BINARY SEARCH .
                IF  sy-subrc IS INITIAL.
                  lv_combine_field = 'X' .
                ENDIF.
              ENDIF.
              PERFORM get_constorium_multipliers_2a2 USING '1'.
*           Individual Column..
              gv_len = strlen( gs_zpra_t_dly_prd-product ) .
              IF lv_combine_field IS INITIAL.
                CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' gs_zpra_t_dly_prd-asset INTO lv_col_name .
              ELSE.
                CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'COMBINE'
                  INTO lv_col_name .
              ENDIF.
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> T
```

```
O <gfs_field> .
              IF <gfs_field> IS ASSIGNED.
                <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_individual_mul.
                UNASSIGN <gfs_field> .
              ENDIF.
*           Product Total..
              CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'TOTAL'
                  INTO lv_col_name .
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
              IF <gfs_field> IS ASSIGNED.
                <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_total_mul .
                UNASSIGN <gfs_field> .
              ENDIF.
*           Grand Total
              CONCATENATE 'GRAND' '-' 'TOTAL'
                   INTO lv_col_name .
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
              IF <gfs_field> IS ASSIGNED.
                lv_mmscmd_mul = 1 .
                IF gs_zpra_t_dly_rprd-product EQ c_prod_gas AND p_bmd IS NOT INITIAL.
*                  lv_mmscmd_mul = 6290 .
                ENDIF.
*                <gfs_field> = <gfs_field> + ( gs_zpra_t_dly_prd-prod_vl_qty1 * lv_mmscmd_mul ).
                <gfs_field> = <gfs_field> + ( gs_zpra_t_dly_prd-prod_vl_qty1 * gv_grand_total_mul ).
                UNASSIGN <gfs_field> .
              ENDIF.
            ENDLOOP.
          ENDIF.
        ENDIF.
        IF lv_monat GE gv_current_monat.
          EXIT .
        ENDIF.
      ENDDO.
    ENDLOOP.
    lv_gjahr = lv_gjahr - 1 .
  ENDDO .
* Now dividing by no of days in case of per day units
  IF p_bbd IS NOT INITIAL OR
     p_tmd IS NOT INITIAL OR
     p_bmd IS NOT INITIAL .
    LOOP AT <gfs_sec2d_table> ASSIGNING <gfs_dyn_line> .
      lv_year_index = sy-tabix .
      DO .
        lv_index = sy-index .
        IF lv_index GT 2.
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
          IF sy-subrc IS NOT INITIAL.
            EXIT .
          ENDIF.
          PERFORM get_asset_start_date USING lv_index CHANGING lv_start_date .
          IF lv_year_index = 1.
            IF lv_start_date GT gv_year_start_date .
```

```
              lv_days2 = p_date - lv_start_date + 1 .
            ELSE.
              lv_days2 = p_date - gv_year_start_date + 1 .
            ENDIF.
          ELSE.
            IF lv_start_date GT gv_last_year_s_date .
              lv_days2 = gv_last_year_t_date - lv_start_date + 1 .
            ELSE.
              lv_days2 = gv_last_year_t_date - gv_last_year_s_date + 1 .
            ENDIF.
          ENDIF.

          <gfs_field> = <gfs_field> / lv_days2 .
          UNASSIGN <gfs_field> .
        ENDIF.
      ENDDO.
    ENDLOOP .
  ENDIF.
  IF p_mb IS NOT INITIAL.
    LOOP AT <gfs_sec2d_table> ASSIGNING <gfs_dyn_line> .
      DO .
        lv_index = sy-index .
        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
        IF sy-subrc IS NOT INITIAL.
          EXIT .
        ELSE.
          IF lv_index GT 2.
            <gfs_field> = <gfs_field> / 1000000 .
          ENDIF.
        ENDIF.
      ENDDO.
    ENDLOOP.
  ENDIF.


  IF p_bmd IS NOT INITIAL.
    LOOP AT <gfs_sec2d_table> ASSIGNING <gfs_dyn_line> .
      DO .
        lv_index = sy-index .
        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
        IF sy-subrc IS NOT INITIAL.
          EXIT .
        ELSE.
          IF lv_index GT 2.
            READ TABLE gt_dyn_fcat INTO gs_dyn_fcat WITH KEY col_pos = lv_index BINARY SEARCH .
            IF sy-subrc IS INITIAL.
              SPLIT gs_dyn_fcat-fieldname AT '-' INTO lv_product lv_asset .
              IF lv_product EQ c_prod_gas.
                <gfs_field> = <gfs_field> / 6290 .
              ENDIF.
            ENDIF.
          ENDIF.
        ENDIF.
      ENDDO.
    ENDLOOP.
  ENDIF.

```

```
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  FILL_DYNAMIC_TABLE_SEC2D
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM fill_dynamic_table_sec2f .


  DATA : lt_dly_prd_main   TYPE STANDARD TABLE OF zpra_t_dly_prd,
         lt_zpra_t_dly_prd TYPE STANDARD TABLE OF zpra_t_dly_prd.
  DATA : ls_dly_prd_main   TYPE                   zpra_t_dly_prd,
         ls_zpra_t_dly_prd TYPE                   zpra_t_dly_prd.
  DATA : lv_monat         TYPE                   zpra_t_mrec_prd-monat,
         lv_gjahr         TYPE                   zpra_t_mrec_prd-gjahr,
         lv_gjahr_next    TYPE                   zpra_t_mrec_prd-gjahr,
         lv_combine_field TYPE                   c,
         lv_days          TYPE                   sy-tabix,
         lv_index         TYPE                   sy-tabix,
         lv_rprd_index    TYPE                   sy-tabix,
         lv_col_name      TYPE                   lvc_fname,
         lv_days2         TYPE                   sy-tabix,
         lv_start_date    TYPE                   sy-datum,
         lv_mmscmd_mul    TYPE                   sy-tabix,
         lv_product       TYPE                   char100,
         lv_asset         TYPE                   char100.

  gt_zpra_t_mrec_prd =  gt_zpra_t_mrec_prd_2f .
  lt_dly_prd_main = gt_zpra_t_dly_prd .
  SORT lt_dly_prd_main BY product asset block  .
  DELETE ADJACENT DUPLICATES FROM lt_dly_prd_main COMPARING product asset block  .

  lv_gjahr = gv_last_gjahr .
  lv_gjahr_next = lv_gjahr + 1 .
  APPEND INITIAL LINE TO <gfs_sec2f_table> ASSIGNING <gfs_dyn_line> .
  ASSIGN COMPONENT 'COL01' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
  CONCATENATE 'Actual Prod. :' lv_gjahr '-' lv_gjahr_next+2(2) INTO <gfs_field> SEPARATED BY space.

  ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
*     <gfs_field> = 'Last Year' .
  LOOP AT gt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof.
    lv_monat = '00' .
    DO .
      lv_monat = lv_monat + 1 .
      CLEAR lv_combine_field .
      READ TABLE gt_zpra_t_mrec_app_2f INTO gs_zpra_t_mrec_app WITH KEY gjahr   = lv_gjahr

      monat   = lv_monat

      product = gs_zpra_c_prd_prof-product

      asset   = gs_zpra_c_prd_prof-asset

      block   = gs_zpra_c_prd_prof-block BINARY SEARCH .
      IF sy-subrc IS INITIAL. "if found in MREC APP
```

```
8247          gs_zpra_t_mrec_app-app_vl_qty = gs_zpra_t_mrec_app-app_vl_qty *
      1000000 .
8248          IF gs_zpra_t_mrec_app-product = c_prod_gas.
8249            READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH K
      EY asset = gs_zpra_t_mrec_app-asset BINARY SEARCH .
8250            IF  sy-subrc IS INITIAL.
8251              lv_combine_field = 'X' .
8252            ENDIF.
8253          ENDIF.
8254  *     Individual Column..
8255          gv_len = strlen( gs_zpra_t_mrec_app-product ) .
8256          IF lv_combine_field IS INITIAL.
8257            CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' gs_zpra_t_m
      rec_app-asset INTO lv_col_name .
8258          ELSE.
8259            CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' 'COMBINE'
                 INTO lv_col_name .
8260          ENDIF.
8261          ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs
      _field> .
8262          IF <gfs_field> IS ASSIGNED.
8263            <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty .
8264            UNASSIGN <gfs_field> .
8265          ENDIF.
8266  *     Product Total..
8267          CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' 'TOTAL'
                 INTO lv_col_name .
8268          ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs
      _field> .
8269          IF <gfs_field> IS ASSIGNED.
8270            <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty .
8271            UNASSIGN <gfs_field> .
8272          ENDIF.
8273  *     Grand Total
8274          CONCATENATE 'GRAND' '-' 'TOTAL'
                 INTO lv_col_name .
8275          ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs
      _field> .
8276          IF <gfs_field> IS ASSIGNED.
8277            <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty.
8278            UNASSIGN <gfs_field> .
8279          ENDIF.
8280  **        ELSE. "Now work out from MREC
8281  **          READ TABLE gt_zpra_t_mrec_prd_2f INTO gs_zpra_t_mrec_prd WIT
      H KEY gjahr   = lv_gjahr
8282  **
            monat   = lv_monat
8283  **
            product = gs_zpra_c_prd_prof-product
8284  **
            asset   = gs_zpra_c_prd_prof-asset
8285  **
            block   = gs_zpra_c_prd_prof-block BINARY SEARCH .
8286  **          IF sy-subrc IS INITIAL. "if found in MREC
8287  **
8288  **            gs_zpra_t_dly_prd = ls_dly_prd_main .
8289  **            PERFORM get_days_in_period USING gs_zpra_t_mrec_prd-gjahr
      gs_zpra_t_mrec_prd-monat CHANGING lv_days .
8290  ***            lv_days = lv_days * 12 .
8291  **            lv_days = 1 .
8292  **            PERFORM convert_non_gas_mrec_units USING lv_days CHANGING
```

```
gs_zpra_t_mrec_prd.
**            IF gs_zpra_c_prd_prof-product = c_prod_gas.
**             READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_t_mrec_prd-asset BINARY SEARCH .
**             IF  sy-subrc IS INITIAL.
**               lv_combine_field = 'X' .
**             ENDIF.
**            ENDIF.
**            PERFORM get_constorium_multipliers_2a2 USING '6'.
**
***       Individual Column..
**           gv_len = strlen( gs_zpra_t_mrec_prd-product ) .
**           IF lv_combine_field IS INITIAL.
**             CONCATENATE gs_zpra_t_mrec_prd-product(gv_len) '-' gs_zpra_t_mrec_prd-asset INTO lv_col_name .
**           ELSE.
**             CONCATENATE gs_zpra_t_mrec_prd-product(gv_len) '-' 'COMBINE'
                 INTO lv_col_name .
**           ENDIF.
**           ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
**           IF <gfs_field> IS ASSIGNED.
**             <gfs_field> = <gfs_field> + gs_zpra_t_mrec_prd-prod_vl_qty1 * gv_individual_mul.
**             UNASSIGN <gfs_field> .
**           ENDIF.
***       Product Total..
**           CONCATENATE gs_zpra_t_mrec_prd-product(gv_len) '-' 'TOTAL'
                 INTO lv_col_name .
**           ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
**           IF <gfs_field> IS ASSIGNED.
**             <gfs_field> = <gfs_field> + gs_zpra_t_mrec_prd-prod_vl_qty1 * gv_total_mul .
**             UNASSIGN <gfs_field> .
**           ENDIF.
***       Grand Total
**           CONCATENATE 'GRAND' '-' 'TOTAL'
                 INTO lv_col_name .
**           ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
**           IF <gfs_field> IS ASSIGNED.
**             <gfs_field> = <gfs_field> + gs_zpra_t_mrec_prd-prod_vl_qty1 * gv_grand_total_mul.
**             UNASSIGN <gfs_field> .
**           ENDIF.
      ELSE.
        PERFORM get_days_in_period USING lv_gjahr lv_monat CHANGING lv_days .
        READ TABLE gt_zpra_t_dly_rprd_2f INTO gs_zpra_t_dly_rprd WITH KEY product = gs_zpra_c_prd_prof-product

    asset = gs_zpra_c_prd_prof-asset

    block = gs_zpra_c_prd_prof-block
                                                                production_date = gv_search_begin_datum BINARY SEARCH .
        IF sy-subrc IS NOT INITIAL.
          LOOP AT gt_zpra_t_dly_rprd_2f INTO gs_zpra_t_dly_rprd WHERE product         EQ gs_zpra_c_prd_prof-product
                                                                AND as
```

```
set           EQ gs_zpra_c_prd_prof-asset
                                                                AND block           EQ gs_zpra_c_prd_prof-block
                                                                AND production_date GE gv_search_begin_datum
                                                                AND production_date LE gv_search_end_datum .
             EXIT .
           ENDLOOP .
         ENDIF.
         IF sy-subrc IS INITIAL.
           lv_rprd_index = sy-tabix .
           LOOP AT gt_zpra_t_dly_rprd_2f INTO gs_zpra_t_dly_rprd FROM lv_rprd_index .
             IF gs_zpra_t_dly_rprd-product EQ gs_zpra_c_prd_prof-product AND
                gs_zpra_t_dly_rprd-asset   EQ gs_zpra_c_prd_prof-asset AND
                gs_zpra_t_dly_rprd-block   EQ gs_zpra_c_prd_prof-block AND
                gs_zpra_t_dly_rprd-production_date GE gv_search_begin_datum   AND
                gs_zpra_t_dly_rprd-production_date LE gv_search_end_datum .
             ELSE.
               EXIT .
             ENDIF.
             lv_index = sy-tabix .
             CLEAR lv_combine_field .

             PERFORM convert_rprd_units USING gs_zpra_t_dly_rprd .

             IF gs_zpra_t_dly_rprd-product = c_prod_gas .
               READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_t_dly_rprd-asset BINARY SEARCH .
               IF  sy-subrc IS INITIAL.
                 lv_combine_field = 'X' .
               ENDIF.
             ENDIF.
*            Individual Column..
             gv_len = strlen( gs_zpra_t_dly_rprd-product ) .
             IF lv_combine_field IS INITIAL.
               CONCATENATE gs_zpra_t_dly_rprd-product(gv_len) '-' gs_zpra_t_dly_rprd-asset INTO lv_col_name .
             ELSE.
               CONCATENATE gs_zpra_t_dly_rprd-product(gv_len) '-' 'COMBINE'
                   INTO lv_col_name .
             ENDIF.
             ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
             IF <gfs_field> IS ASSIGNED.
               <gfs_field> = <gfs_field> + gs_zpra_t_dly_rprd-jv_prd_vl_qty1.
               UNASSIGN <gfs_field> .
             ENDIF.
*            Product Total..
             CONCATENATE gs_zpra_t_dly_rprd-product(gv_len) '-' 'TOTAL'
                   INTO lv_col_name .
             ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
             IF <gfs_field> IS ASSIGNED.
```

```
8380                <gfs_field> = <gfs_field> + gs_zpra_t_dly_rprd-ovl_prd_vl_
      qty1.
8381                UNASSIGN <gfs_field> .
8382              ENDIF.
8383  *           Grand Total
8384              CONCATENATE 'GRAND' '-' 'TOTAL'
                       INTO lv_col_name .
8385              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO
      <gfs_field> .
8386              IF <gfs_field> IS ASSIGNED.
8387                lv_mmscmd_mul = 1 .
8388                IF gs_zpra_t_dly_rprd-product EQ c_prod_gas AND p_bmd IS N
      OT INITIAL.
8389  *                 lv_mmscmd_mul = 6290 .
8390                ENDIF.
8391                <gfs_field> = <gfs_field> + ( gs_zpra_t_dly_rprd-ovl_prd_v
      l_qty1 * lv_mmscmd_mul ).
8392                UNASSIGN <gfs_field> .
8393              ENDIF.
8394            ENDLOOP.
8395          ELSE. "Now work out from daily production
8396            PERFORM get_days_in_period USING lv_gjahr lv_monat CHANGING lv
      _days .
8397  *            lv_days = lv_days * 12 .
8398            lv_days = 1 .
8399            LOOP AT gt_zpra_t_dly_prd_2f INTO gs_zpra_t_dly_prd WHERE prod
      uct EQ gs_zpra_c_prd_prof-product
8400                                                                  AND asse
      t   EQ gs_zpra_c_prd_prof-asset
8401                                                                  AND bloc
      k   EQ gs_zpra_c_prd_prof-block
8402                                                                  AND prod
      uction_date GE gv_search_begin_datum
8403                                                                  AND prod
      uction_date LE gv_search_end_datum .
8404              lv_index = sy-tabix .
8405              CLEAR lv_combine_field .
8406
8407              PERFORM convert_non_gas_units2 USING lv_days CHANGING gs_zpr
      a_t_dly_prd .
8408
8409              IF gs_zpra_t_dly_prd-product = c_prod_gas.
8410                READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WI
      TH KEY asset = gs_zpra_t_dly_prd-asset BINARY SEARCH .
8411                IF  sy-subrc IS INITIAL.
8412                  lv_combine_field = 'X' .
8413                ENDIF.
8414              ENDIF.
8415              PERFORM get_constorium_multipliers_2a2 USING '1'.
8416  *           Individual Column..
8417              gv_len = strlen( gs_zpra_t_dly_prd-product ) .
8418              IF lv_combine_field IS INITIAL.
8419                CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' gs_zpra_
      t_dly_prd-asset INTO lv_col_name .
8420              ELSE.
8421                CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'COMBINE
      '                 INTO lv_col_name .
8422              ENDIF.
8423              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO
      <gfs_field> .
8424              IF <gfs_field> IS ASSIGNED.
```

```
8425                <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1
      * gv_individual_mul.
8426                UNASSIGN <gfs_field> .
8427              ENDIF.
8428  *           Product Total..
8429              CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'TOTAL'
                      INTO lv_col_name .
8430              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO
      <gfs_field> .
8431              IF <gfs_field> IS ASSIGNED.
8432                <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1
      * gv_total_mul .
8433                UNASSIGN <gfs_field> .
8434              ENDIF.
8435  *           Grand Total
8436              CONCATENATE 'GRAND' '-' 'TOTAL'
                       INTO lv_col_name .
8437              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO
      <gfs_field> .
8438              IF <gfs_field> IS ASSIGNED.
8439                <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1
      * gv_grand_total_mul.
8440                UNASSIGN <gfs_field> .
8441              ENDIF.
8442            ENDLOOP.
8443          ENDIF.
8444        ENDIF.
8445        IF lv_monat GE 12.
8446          EXIT .
8447        ENDIF.
8448      ENDDO.
8449    ENDLOOP.
8450  * Now dividing by no of days in case of per day units
8451    IF p_bbd IS NOT INITIAL OR
8452       p_tmd IS NOT INITIAL OR
8453       p_bmd IS NOT INITIAL .
8454
8455      LOOP AT <gfs_sec2f_table> ASSIGNING <gfs_dyn_line> .
8456        DO .
8457          lv_index = sy-index .
8458          lv_days2 = gv_last_gjahr_days .
8459          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_fi
      eld> .
8460          IF sy-subrc IS INITIAL.
8461            IF lv_index GT 2.
8462              PERFORM get_asset_start_date USING lv_index CHANGING lv_star
      t_date .
8463              IF lv_start_date GT gv_last_year_s_date .
8464                lv_days2 = lv_days2 + gv_last_year_s_date - lv_start_date
      .
8465              ENDIF.
8466              <gfs_field> = <gfs_field> / lv_days2 .
8467            ENDIF.
8468          ELSE.
8469            EXIT .
8470          ENDIF.
8471        ENDDO.
8472      ENDLOOP.
8473    ENDIF.
8474    IF p_mb IS NOT INITIAL.
8475      LOOP AT <gfs_sec2f_table> ASSIGNING <gfs_dyn_line> .
```

```
      DO .
        lv_index = sy-index .
        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
        IF sy-subrc IS NOT INITIAL.
          EXIT .
        ELSE.
          IF lv_index GT 2.
            <gfs_field> = <gfs_field> / 1000000 .
          ENDIF.
        ENDIF.
      ENDDO.
    ENDLOOP.
  ENDIF.

  IF p_bmd IS NOT INITIAL.
    LOOP AT <gfs_sec2f_table> ASSIGNING <gfs_dyn_line> .
      DO .
        lv_index = sy-index .
        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
        IF sy-subrc IS NOT INITIAL.
          EXIT .
        ELSE.
          IF lv_index GT 2.
            READ TABLE gt_dyn_fcat INTO gs_dyn_fcat WITH KEY col_pos = lv_index BINARY SEARCH .
            IF sy-subrc IS INITIAL.
              SPLIT gs_dyn_fcat-fieldname AT '-' INTO lv_product lv_asset .
              IF lv_product EQ c_prod_gas.
                <gfs_field> = <gfs_field> / 6290 .
              ENDIF.
            ENDIF.
          ENDIF.
        ENDIF.
      ENDDO.
    ENDLOOP.
  ENDIF.


ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  FILL_DYNAMIC_TABLE_SEC2E
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM fill_dynamic_table_sec2e .
  DATA   lv_index TYPE sy-index .
  DATA : lv_amt         TYPE p LENGTH 12 DECIMALS 9,
         lv_amt2        TYPE p LENGTH 12 DECIMALS 9,
         lv_amt3        TYPE p LENGTH 12 DECIMALS 9,
         lv_target_name TYPE char25,
         lv_days        TYPE sy-tabix,
         lv_start_date  TYPE sy-tabix,
         lv_tar_code    TYPE zpra_t_prd_tar-tar_code.

  REFRESH <gfs_sec2e_table> .
```

```
  READ TABLE <gfs_sec2d_table> ASSIGNING <gfs_dyn_line> INDEX 1 .
  IF  sy-subrc IS INITIAL .
    LOOP AT <gfs_sec2b_table> ASSIGNING <gfs_dyn_line2>.
      APPEND INITIAL LINE TO <gfs_sec2e_table> ASSIGNING <gfs_dyn_line3> .
      DO gv_table_columns TIMES.
        lv_index = sy-index .
        IF lv_index EQ 1.
        ELSEIF lv_index EQ 2 .
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line2> TO <gfs_field2> .
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line3> TO <gfs_field3> .
          <gfs_field3> = <gfs_field2> .
          lv_target_name = <gfs_field2> .
          PERFORM get_target_from_name USING lv_target_name CHANGING lv_tar_code.
        ELSE .
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line>  TO <gfs_field> .
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line2> TO <gfs_field2> .
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line3> TO <gfs_field3> .
          IF <gfs_field>  IS ASSIGNED AND
             <gfs_field2> IS ASSIGNED AND
             <gfs_field3> IS ASSIGNED .
            IF p_bbd IS NOT INITIAL OR
               p_tmd IS NOT INITIAL OR
               p_bmd IS NOT INITIAL .
              IF gv_days_left_in_year IS NOT INITIAL .
                IF <gfs_field2> IS INITIAL.
                  <gfs_field3> = '-' .
                ELSE.
                  lv_days = gv_year_end_date - gv_year_start_date + 1 .
                  PERFORM get_target_start_date USING lv_index lv_tar_code CHANGING lv_start_date .
                  IF lv_start_date GT gv_year_start_date .
                    lv_days =  gv_year_end_date - lv_start_date + 1 .
                  ENDIF.
                  lv_amt  = <gfs_field2> * lv_days .

                  lv_days = p_date - gv_year_start_date + 1 .
                  PERFORM get_asset_start_date USING lv_index CHANGING lv_start_date .
                  IF lv_start_date GT gv_year_start_date .
                    lv_days =  p_date - lv_start_date + 1 .
                  ENDIF.
                  lv_amt2 = <gfs_field> * lv_days .

                  lv_amt3 = ( lv_amt - lv_amt2 ) / gv_days_left_in_year .
                  <gfs_field3> = lv_amt3 .
                  IF lv_amt3 LT 0.
                    <gfs_field3> = <gfs_field3> * -1 .
                    SHIFT <gfs_field3> LEFT DELETING LEADING space .
                    CONCATENATE '-' <gfs_field3> INTO <gfs_field3> .
                  ENDIF.
                ENDIF.
              ELSE.
                <gfs_field3> = '-' .
```

```
              ENDIF.
            ELSE.
              <gfs_field3> = '-' .
            ENDIF.
          ENDIF.
          UNASSIGN : <gfs_field> , <gfs_field2> , <gfs_field3>.
        ENDIF.
      ENDDO.
    ENDLOOP.

  ENDIF.
ENDFORM.

*&———————————————————————————————————————————————————————————————*
*&      Form  FILL_DYNAMIC_TABLE_SEC3C
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM fill_dynamic_table_sec3c .


  DATA : lt_zpra_t_dly_prd     TYPE STANDARD TABLE OF zpra_t_dly_prd .
  DATA : ls_zpra_t_dly_prd     TYPE                   zpra_t_dly_prd .

  DATA : lv_monat         TYPE                  zpra_t_mrec_prd-monat,
         lv_gjahr         TYPE                  zpra_t_mrec_prd-gjahr,
         lv_gjahr_next    TYPE                  zpra_t_mrec_prd-gjahr,
         lv_combine_field TYPE                  c,
         lv_days          TYPE                  sy-tabix,
         lv_index         TYPE                  sy-tabix,
         lv_rprd_index    TYPE                  sy-tabix,
         lv_col_name      TYPE                  lvc_fname,
         lv_found         TYPE                  c.
  FIELD-SYMBOLS : <lfs_dyn_line> ,
                  <lfs_field>    .
* If directly working with MMT, then due to rounding off, value mismatches..
* First doing in MMT * 1000, at the end will divide by 1000 .
  lv_gjahr = gv_current_gjahr .
  lv_gjahr_next = lv_gjahr + 1 .
  APPEND INITIAL LINE TO <gfs_sec3c_table> ASSIGNING <gfs_dyn_line> .
  ASSIGN COMPONENT 'COL01' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
  CONCATENATE 'YTD Actual Prod.:' lv_gjahr '-' lv_gjahr_next+2(2) INTO <gfs_field> SEPARATED BY space.
  LOOP AT gt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof.
    lv_monat = '00' .
    DO .
      lv_monat = lv_monat + 1 .
      CLEAR lv_combine_field .
      READ TABLE gt_zpra_t_mrec_app INTO gs_zpra_t_mrec_app WITH KEY gjahr   = lv_gjahr
                                                                     monat   = lv_monat
                                                                     product = gs_zpra_c_prd_prof-product
                                                                     asset   = gs_zpra_c_prd_prof-asset
                                                                     blo
```

```
ck   = gs_zpra_c_prd_prof-block BINARY SEARCH .
      IF sy-subrc IS INITIAL. "if found in MREC APP
        gs_zpra_t_mrec_app-app_vl_qty = gs_zpra_t_mrec_app-app_vl_qty * 1000000 .
        IF gs_zpra_t_mrec_app-monat EQ gv_current_monat.
          PERFORM get_days_in_period USING gs_zpra_t_mrec_app-gjahr gs_zpra_t_mrec_app-monat CHANGING lv_days .
          gs_zpra_t_mrec_app-app_vl_qty = gs_zpra_t_mrec_app-app_vl_qty * ( p_date - gv_search_begin_datum + 1 ) / lv_days .
        ENDIF.
        IF gs_zpra_t_mrec_app-product = c_prod_gas.
          READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_t_mrec_app-asset BINARY SEARCH .
          IF  sy-subrc IS INITIAL.
            lv_combine_field = 'X' .
          ENDIF.
        ENDIF.
*     Individual Column..
        gv_len = strlen( gs_zpra_t_mrec_app-product ) .
        IF lv_combine_field IS INITIAL.
          CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' gs_zpra_t_mrec_app-asset INTO lv_col_name .
        ELSE.
          CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' 'COMBINE'
             INTO lv_col_name .
        ENDIF.
        ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
        IF <gfs_field> IS ASSIGNED.
          <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty .
          UNASSIGN <gfs_field> .
        ENDIF.
*     Product Total..
        CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' 'TOTAL'
             INTO lv_col_name .
        ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
        IF <gfs_field> IS ASSIGNED.
          <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty .
          UNASSIGN <gfs_field> .
        ENDIF.
*     Grand Total
        CONCATENATE 'GRAND' '-' 'TOTAL'
             INTO lv_col_name .
        ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
        IF <gfs_field> IS ASSIGNED.
          <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty.
          UNASSIGN <gfs_field> .
        ENDIF.
      ELSE. "Now work out from daily reconciled production
        PERFORM get_days_in_period USING lv_gjahr lv_monat CHANGING lv_days .
        IF lv_monat EQ gv_current_monat.
          gv_search_end_datum = p_date .
        ENDIF.
        CLEAR lv_found .
        READ TABLE gt_zpra_t_dly_rprd_3c INTO gs_zpra_t_dly_rprd WITH KEY product = gs_zpra_c_prd_prof-product

    asset = gs_zpra_c_prd_prof-asset
```

```
8685
          block = gs_zpra_c_prd_prof-block
8686                                                                produc
      tion_date = gv_search_begin_datum BINARY SEARCH .
8687          IF sy-subrc IS NOT INITIAL.
8688            LOOP AT gt_zpra_t_dly_rprd_3c INTO gs_zpra_t_dly_rprd WHERE pr
      oduct         EQ gs_zpra_c_prd_prof-product
8689                                                                AND as
      set           EQ gs_zpra_c_prd_prof-asset
8690                                                                AND bl
      ock           EQ gs_zpra_c_prd_prof-block
8691                                                                AND pr
      oduction_date GE gv_search_begin_datum
8692                                                                AND pr
      oduction_date LE gv_search_end_datum .
8693              EXIT .
8694            ENDLOOP .
8695          ENDIF.
8696          IF sy-subrc IS INITIAL.
8697            lv_rprd_index = sy-tabix .
8698            lv_found      = 'X' .
8699            LOOP AT gt_zpra_t_dly_rprd_3c INTO gs_zpra_t_dly_rprd FROM lv_
      rprd_index .
8700              IF gs_zpra_t_dly_rprd-product EQ gs_zpra_c_prd_prof-product
          AND
8701                 gs_zpra_t_dly_rprd-asset   EQ gs_zpra_c_prd_prof-asset
          AND
8702                 gs_zpra_t_dly_rprd-block   EQ gs_zpra_c_prd_prof-block
          AND
8703                 gs_zpra_t_dly_rprd-production_date GE gv_search_begin_dat
      um   AND
8704                 gs_zpra_t_dly_rprd-production_date LE gv_search_end_datum
       .
8705              ELSE.
8706                EXIT .
8707              ENDIF.
8708              lv_index = sy-tabix .
8709              CLEAR lv_combine_field .
8710
8711              PERFORM convert_rprd_units_mb USING gs_zpra_t_dly_rprd .
8712
8713              IF gs_zpra_t_dly_rprd-product = c_prod_gas .
8714                READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WI
      TH KEY asset = gs_zpra_t_dly_rprd-asset BINARY SEARCH .
8715                IF  sy-subrc IS INITIAL.
8716                  lv_combine_field = 'X' .
8717                ENDIF.
8718              ENDIF.
8719  *           Individual Column..
8720              gv_len = strlen( gs_zpra_t_dly_rprd-product ) .
8721              IF lv_combine_field IS INITIAL.
8722                CONCATENATE gs_zpra_t_dly_rprd-product(gv_len) '-' gs_zpra
      _t_dly_rprd-asset INTO lv_col_name .
8723              ELSE.
8724                CONCATENATE gs_zpra_t_dly_rprd-product(gv_len) '-' 'COMBIN
      E'                 INTO lv_col_name .
8725              ENDIF.
8726              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO
      <gfs_field> .
8727              IF <gfs_field> IS ASSIGNED.
8728                <gfs_field> = <gfs_field> + gs_zpra_t_dly_rprd-jv_prd_vl_q
```

```
ty1.
                UNASSIGN <gfs_field> .
              ENDIF.
*             Product Total..
              CONCATENATE gs_zpra_t_dly_rprd-product(gv_len) '-' 'TOTAL'
                   INTO lv_col_name .
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO
<gfs_field> .
              IF <gfs_field> IS ASSIGNED.
                <gfs_field> = <gfs_field> + gs_zpra_t_dly_rprd-ovl_prd_vl_
qty1.
                UNASSIGN <gfs_field> .
              ENDIF.
*             Grand Total
              CONCATENATE 'GRAND' '-' 'TOTAL'
                   INTO lv_col_name .
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO
<gfs_field> .
              IF <gfs_field> IS ASSIGNED.
                <gfs_field> = <gfs_field> + gs_zpra_t_dly_rprd-ovl_prd_vl_
qty1.
                UNASSIGN <gfs_field> .
              ENDIF.
            ENDLOOP.
          ENDIF.
          IF lv_found IS INITIAL ."Now work out from daily production
            LOOP AT gt_zpra_t_dly_prd_3c INTO gs_zpra_t_dly_prd WHERE prod
uct EQ gs_zpra_c_prd_prof-product
                                                                  AND asse
t   EQ gs_zpra_c_prd_prof-asset
                                                                  AND bloc
k   EQ gs_zpra_c_prd_prof-block
                                                                  AND prod
uction_date GE gv_search_begin_datum
                                                                  AND prod
uction_date LE gv_search_end_datum .
              lv_index = sy-tabix .
              CLEAR lv_combine_field .

              PERFORM convert_non_gas_units3c USING lv_monat CHANGING gs_z
pra_t_dly_prd .

              IF gs_zpra_t_dly_prd-product = c_prod_gas .
                READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WI
TH KEY asset = gs_zpra_t_dly_prd-asset BINARY SEARCH .
                IF  sy-subrc IS INITIAL.
                  lv_combine_field = 'X' .
                ENDIF.
              ENDIF.
              PERFORM get_constorium_multipliers_2a2 USING '7'.
              gv_individual_mul  = gv_total_mul . "section 3 is always OVL
 Level
              gv_grand_total_mul = gv_total_mul . "section 3 is always OVL
 Level
*             Individual Column..
              gv_len = strlen( gs_zpra_t_dly_prd-product ) .
              IF lv_combine_field IS INITIAL.
                CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' gs_zpra_
t_dly_prd-asset INTO lv_col_name .
              ELSE.
                CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'COMBINE
```

```
'                INTO lv_col_name .
              ENDIF.
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
              IF <gfs_field> IS ASSIGNED.
                <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_individual_mul.
                UNASSIGN <gfs_field> .
              ENDIF.
*             Product Total..
              CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'TOTAL'
                  INTO lv_col_name .
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
              IF <gfs_field> IS ASSIGNED.
                <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_total_mul .
                UNASSIGN <gfs_field> .
              ENDIF.
*             Grand Total
              CONCATENATE 'GRAND' '-' 'TOTAL'
                   INTO lv_col_name .
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
              IF <gfs_field> IS ASSIGNED.
                <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_total_mul.
                UNASSIGN <gfs_field> .
              ENDIF.
            ENDLOOP.
          ENDIF.
        ENDIF.
        IF lv_monat GE gv_current_monat.
          EXIT .
        ENDIF.
      ENDDO.
    ENDLOOP.

    LOOP AT <gfs_sec3c_table> ASSIGNING <gfs_dyn_line> .
      DO .
        lv_index = sy-index .
        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
        IF sy-subrc IS NOT INITIAL.
          EXIT .
        ENDIF.
        IF lv_index GT 2.
          <gfs_field> = <gfs_field> / 1000000 .
        ENDIF.
      ENDDO.
    ENDLOOP.
    IF p_mb IS NOT INITIAL AND p_c_jv IS NOT INITIAL.
      LOOP AT gt_product_col INTO gs_product_col.
        CONCATENATE gs_product_col-product 'TOTAL' INTO lv_col_name SEPARATED BY '-' .
        UNASSIGN <gfs_field>.
        ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
        IF <gfs_field> IS ASSIGNED.
          gv_s_row = gv_sec2d_start_row .
          gv_e_row = gv_sec2d_start_row .
```

```
8822          gv_s_col = gs_product_col-e_col .
8823          gv_e_col = gs_product_col-e_col .
8824          PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col
      .
8825          PERFORM set_range USING <gfs_field> 0 .
8826        ENDIF.
8827        READ TABLE <gfs_sec2d_table> ASSIGNING <lfs_dyn_line> INDEX 1 .
8828        IF sy-subrc IS INITIAL.
8829          ASSIGN COMPONENT lv_col_name OF STRUCTURE <lfs_dyn_line> TO <lfs
      _field> .
8830          IF <lfs_field> IS ASSIGNED.
8831            <lfs_field> = <gfs_field>.
8832          ENDIF.
8833        ENDIF.
8834      ENDLOOP.
8835      lv_col_name = 'GRAND-TOTAL' .
8836      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
      ld> .
8837      IF <gfs_field> IS ASSIGNED.
8838        gv_s_row = gv_sec2d_start_row .
8839        gv_e_row = gv_sec2d_start_row .
8840        gv_s_col = gv_table_columns   .
8841        gv_e_col = gv_table_columns   .
8842        PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
8843        PERFORM set_range USING <gfs_field> 0 .
8844        READ TABLE <gfs_sec2d_table> ASSIGNING <lfs_dyn_line> INDEX 1 .
8845        IF sy-subrc IS INITIAL.
8846          ASSIGN COMPONENT lv_col_name OF STRUCTURE <lfs_dyn_line> TO <lfs
      _field> .
8847          IF <lfs_field> IS ASSIGNED.
8848            <lfs_field> = <gfs_field>.
8849          ENDIF.
8850        ENDIF.
8851      ENDIF.
8852    ENDIF.
8853
8854  ENDFORM.
8855  *&———————————————————————————————————————————————————————————*
8856  *&      Form  FILL_DYNAMIC_TABLE_SEC3F
8857  *&———————————————————————————————————————————————————————————*
8858  *       text
8859  *———————————————————————————————————————————————————————————*
8860  *  —>  p1        text
8861  *  <—  p2        text
8862  *———————————————————————————————————————————————————————————*
8863  FORM fill_dynamic_table_sec3f .
8864
8865    DATA : lt_zpra_t_dly_prd     TYPE STANDARD TABLE OF zpra_t_dly_prd .
8866    DATA : ls_zpra_t_dly_prd     TYPE                   zpra_t_dly_prd .
8867
8868    DATA : lv_monat         TYPE                  zpra_t_mrec_prd-monat,
8869           lv_gjahr         TYPE                  zpra_t_mrec_prd-gjahr,
8870           lv_gjahr_next    TYPE                  zpra_t_mrec_prd-gjahr,
8871           lv_combine_field TYPE                  c,
8872           lv_days          TYPE                  sy-tabix,
8873           lv_index         TYPE                  sy-tabix,
8874           lv_rprd_index    TYPE                  sy-tabix,
8875           lv_col_name      TYPE                  lvc_fname,
8876           lv_found         TYPE                  c.
8877    FIELD-SYMBOLS : <lfs_dyn_line> ,
8878                    <lfs_field>    .
```

```
8879
8880    lv_gjahr = gv_5_back_gjahr .
8881    DO 5 TIMES .
8882      IF sy-index EQ 2.
8883        PERFORM show_progress USING '60' .
8884      ENDIF.
8885      IF sy-index EQ 3.
8886        PERFORM show_progress USING '70' .
8887      ENDIF.
8888      IF sy-index EQ 5.
8889        PERFORM show_progress USING '80' .
8890      ENDIF.
8891      lv_gjahr_next = lv_gjahr + 1 .
8892
8893      APPEND INITIAL LINE TO <gfs_sec3f_table> ASSIGNING <gfs_dyn_line> .
8894      ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <gfs_field>
     .
8895      CONCATENATE lv_gjahr '-' lv_gjahr_next+2(2) INTO <gfs_field> SEPARAT
     ED BY space.
8896      LOOP AT gt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof.
8897        lv_monat = '00' .
8898        DO .
8899          lv_monat = lv_monat + 1 .
8900          CLEAR lv_combine_field .
8901          READ TABLE gt_zpra_t_mrec_app_3f INTO gs_zpra_t_mrec_app WITH KE
     Y gjahr   = lv_gjahr
8902
        monat   = lv_monat
8903
        product = gs_zpra_c_prd_prof-product
8904
        asset   = gs_zpra_c_prd_prof-asset
8905
        block   = gs_zpra_c_prd_prof-block BINARY SEARCH .
8906          IF sy-subrc IS INITIAL. "if found in MREC APP
8907            gs_zpra_t_mrec_app-app_vl_qty = gs_zpra_t_mrec_app-app_vl_qty
     * 1000000 .
8908            IF gs_zpra_t_mrec_app-product = c_prod_gas.
8909              READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH
      KEY asset = gs_zpra_t_mrec_app-asset BINARY SEARCH .
8910              IF  sy-subrc IS INITIAL.
8911                lv_combine_field = 'X' .
8912              ENDIF.
8913            ENDIF.
8914  *     Individual Column..
8915            gv_len = strlen( gs_zpra_t_mrec_app-product ) .
8916            IF lv_combine_field IS INITIAL.
8917              CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' gs_zpra_t
     _mrec_app-asset INTO lv_col_name .
8918            ELSE.
8919              CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' 'COMBINE'
                  INTO lv_col_name .
8920            ENDIF.
8921            ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <g
     fs_field> .
8922            IF <gfs_field> IS ASSIGNED.
8923              <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty .
8924              UNASSIGN <gfs_field> .
8925            ENDIF.
8926  *     Product Total..
8927            CONCATENATE gs_zpra_t_mrec_app-product(gv_len) '-' 'TOTAL'
```

```
                    INTO lv_col_name .
          ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
          IF <gfs_field> IS ASSIGNED.
            <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty .
            UNASSIGN <gfs_field> .
          ENDIF.
*     Grand Total
          CONCATENATE 'GRAND' '-' 'TOTAL'
                    INTO lv_col_name .
          ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
          IF <gfs_field> IS ASSIGNED.
            <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty.
            UNASSIGN <gfs_field> .
          ENDIF.
        ELSE. "Now work out from daily reconciled production
          PERFORM get_days_in_period USING lv_gjahr lv_monat CHANGING lv_days .
          CLEAR lv_found .
          READ TABLE gt_zpra_t_dly_rprd_3f INTO gs_zpra_t_dly_rprd WITH KEY product = gs_zpra_c_prd_prof-product

      asset = gs_zpra_c_prd_prof-asset

      block = gs_zpra_c_prd_prof-block
                                                                    production_date = gv_search_begin_datum BINARY SEARCH.
          IF sy-subrc IS NOT INITIAL.
            LOOP AT gt_zpra_t_dly_rprd_3f INTO gs_zpra_t_dly_rprd WHERE product EQ gs_zpra_c_prd_prof-product
                                                                  AND asset   EQ gs_zpra_c_prd_prof-asset
                                                                  AND block   EQ gs_zpra_c_prd_prof-block
                                                                  AND production_date GE gv_search_begin_datum
                                                                  AND production_date LE gv_search_end_datum .
              EXIT.
            ENDLOOP.
          ENDIF.
          IF sy-subrc IS INITIAL.
            lv_rprd_index = sy-tabix .
            lv_found      = 'X' .
            LOOP AT gt_zpra_t_dly_rprd_3f INTO gs_zpra_t_dly_rprd FROM lv_rprd_index .
              IF gs_zpra_t_dly_rprd-product EQ gs_zpra_c_prd_prof-product      AND
                 gs_zpra_t_dly_rprd-asset   EQ gs_zpra_c_prd_prof-asset        AND
                 gs_zpra_t_dly_rprd-block   EQ gs_zpra_c_prd_prof-block        AND
                 gs_zpra_t_dly_rprd-production_date GE gv_search_begin_datum   AND
                 gs_zpra_t_dly_rprd-production_date LE gv_search_end_datum .
              ELSE.
                EXIT .
              ENDIF.
              lv_index = sy-tabix .
```

```
8969                CLEAR lv_combine_field .
8970
8971                PERFORM convert_rprd_units_mb USING gs_zpra_t_dly_rprd .
8972
8973                IF gs_zpra_t_dly_rprd-product = c_prod_gas .
8974                  READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine
      WITH KEY asset = gs_zpra_t_dly_rprd-asset BINARY SEARCH .
8975                  IF  sy-subrc IS INITIAL.
8976                    lv_combine_field = 'X' .
8977                  ENDIF.
8978                ENDIF.
8979  *           Individual Column..
8980                gv_len = strlen( gs_zpra_t_dly_rprd-product ) .
8981                IF lv_combine_field IS INITIAL.
8982                  CONCATENATE gs_zpra_t_dly_rprd-product(gv_len) '-' gs_zp
      ra_t_dly_rprd-asset INTO lv_col_name .
8983                ELSE.
8984                  CONCATENATE gs_zpra_t_dly_rprd-product(gv_len) '-' 'COMB
      INE'                INTO lv_col_name .
8985                ENDIF.
8986                ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> T
      O <gfs_field> .
8987                IF <gfs_field> IS ASSIGNED.
8988                  <gfs_field> = <gfs_field> + gs_zpra_t_dly_rprd-jv_prd_vl
      _qty1.
8989                  UNASSIGN <gfs_field> .
8990                ENDIF.
8991  *           Product Total..
8992                CONCATENATE gs_zpra_t_dly_rprd-product(gv_len) '-' 'TOTAL'
                        INTO lv_col_name .
8993                ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> T
      O <gfs_field> .
8994                IF <gfs_field> IS ASSIGNED.
8995                  <gfs_field> = <gfs_field> + gs_zpra_t_dly_rprd-ovl_prd_v
      l_qty1.
8996                  UNASSIGN <gfs_field> .
8997                ENDIF.
8998  *           Grand Total
8999                CONCATENATE 'GRAND' '-' 'TOTAL'
                        INTO lv_col_name .
9000                ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> T
      O <gfs_field> .
9001                IF <gfs_field> IS ASSIGNED.
9002                  <gfs_field> = <gfs_field> + gs_zpra_t_dly_rprd-ovl_prd_v
      l_qty1.
9003                  UNASSIGN <gfs_field> .
9004                ENDIF.
9005              ENDLOOP.
9006            ENDIF.
9007            IF lv_found IS INITIAL ."Now work out from daily production
9008              LOOP AT gt_zpra_t_dly_prd_3f INTO gs_zpra_t_dly_prd WHERE pr
      oduct EQ gs_zpra_c_prd_prof-product
9009                                                                  AND as
      set   EQ gs_zpra_c_prd_prof-asset
9010                                                                  AND bl
      ock   EQ gs_zpra_c_prd_prof-block
9011                                                                  AND pr
      oduction_date GE gv_search_begin_datum
9012                                                                  AND pr
      oduction_date LE gv_search_end_datum .
9013                lv_index = sy-tabix .
```

```
              CLEAR lv_combine_field .

              PERFORM convert_non_gas_units3f USING lv_days CHANGING gs_zpra_t_dly_prd .

              IF gs_zpra_t_dly_prd-product EQ c_prod_gas.
                READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY asset = gs_zpra_t_dly_prd-asset BINARY SEARCH .
                IF  sy-subrc IS INITIAL.
                  lv_combine_field = 'X' .
                ENDIF.
              ENDIF.
              PERFORM get_constorium_multipliers_2a2 USING '8'.
              gv_individual_mul = gv_total_mul . "section 3 is always OVL Level
              gv_grand_total_mul = gv_total_mul . "section 3 is always OVL Level
*           Individual Column..
              gv_len = strlen( gs_zpra_t_dly_prd-product ) .
              IF lv_combine_field IS INITIAL.
                CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' gs_zpra_t_dly_prd-asset INTO lv_col_name .
              ELSE.
                CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'COMBINE'
                  INTO lv_col_name .
              ENDIF.
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
              IF <gfs_field> IS ASSIGNED.
                <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_individual_mul.
                UNASSIGN <gfs_field> .
              ENDIF.
*           Product Total..
              CONCATENATE gs_zpra_t_dly_prd-product(gv_len) '-' 'TOTAL'
                  INTO lv_col_name .
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
              IF <gfs_field> IS ASSIGNED.
                <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_total_mul .
                UNASSIGN <gfs_field> .
              ENDIF.
*           Grand Total
              CONCATENATE 'GRAND' '-' 'TOTAL'
                   INTO lv_col_name .
              ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
              IF <gfs_field> IS ASSIGNED.
                <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 * gv_total_mul.
                UNASSIGN <gfs_field> .
              ENDIF.
            ENDLOOP.
          ENDIF.
        ENDIF.
        IF lv_monat GE 12.
          EXIT .
        ENDIF.
      ENDDO.
    ENDLOOP.
```

```
9061      lv_gjahr = lv_gjahr + 1 .
9062    ENDDO .
9063
9064    LOOP AT <gfs_sec3f_table> ASSIGNING <gfs_dyn_line> .
9065      DO .
9066        lv_index = sy-index .
9067        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line> TO <gfs_fiel
      d> .
9068        IF sy-subrc IS NOT INITIAL.
9069          EXIT .
9070        ENDIF.
9071        IF lv_index GT 2.
9072          <gfs_field> = <gfs_field> / 1000000 .
9073        ENDIF.
9074      ENDDO.
9075    ENDLOOP.
9076    IF p_mb IS NOT INITIAL AND p_c_jv IS NOT INITIAL.
9077      READ TABLE <gfs_sec3f_table> ASSIGNING <gfs_dyn_line> INDEX 5 .
9078      IF sy-subrc IS INITIAL.
9079        LOOP AT gt_product_col INTO gs_product_col.
9080          CONCATENATE gs_product_col-product 'TOTAL' INTO lv_col_name SEPA
      RATED BY '-' .
9081          UNASSIGN <gfs_field>.
9082          ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs
      _field> .
9083          IF <gfs_field> IS ASSIGNED.
9084            gv_s_row = gv_sec2f_start_row .
9085            gv_e_row = gv_sec2f_start_row .
9086            gv_s_col = gs_product_col-e_col .
9087            gv_e_col = gs_product_col-e_col .
9088            PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col
      .
9089            PERFORM set_range USING <gfs_field> 0 .
9090          ENDIF.
9091          READ TABLE <gfs_sec2f_table> ASSIGNING <lfs_dyn_line> INDEX 1 .
9092          IF sy-subrc IS INITIAL.
9093            ASSIGN COMPONENT lv_col_name OF STRUCTURE <lfs_dyn_line> TO <l
      fs_field> .
9094            IF <lfs_field> IS ASSIGNED.
9095              <lfs_field> = <gfs_field>.
9096            ENDIF.
9097          ENDIF.
9098        ENDLOOP.
9099        lv_col_name = 'GRAND-TOTAL' .
9100        ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_f
      ield> .
9101        IF <gfs_field> IS ASSIGNED.
9102          gv_s_row = gv_sec2f_start_row .
9103          gv_e_row = gv_sec2f_start_row .
9104          gv_s_col = gv_table_columns   .
9105          gv_e_col = gv_table_columns   .
9106          PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col
      .
9107          PERFORM set_range USING <gfs_field> 0 .
9108          READ TABLE <gfs_sec2f_table> ASSIGNING <lfs_dyn_line> INDEX 1 .
9109          IF sy-subrc IS INITIAL.
9110            ASSIGN COMPONENT lv_col_name OF STRUCTURE <lfs_dyn_line> TO <l
      fs_field> .
9111            IF <lfs_field> IS ASSIGNED.
9112              <lfs_field> = <gfs_field>.
9113            ENDIF.
```

```
        ENDIF.
      ENDIF.
    ENDIF.
  ENDIF.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  FILL_DYNAMIC_TABLE_SEC4A
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM fill_dynamic_table_sec4a .

  DATA : lt_zpra_t_dly_prd     TYPE STANDARD TABLE OF zpra_t_dly_prd .
  DATA : ls_zpra_t_dly_prd     TYPE                   zpra_t_dly_prd .

  DATA : lv_monat      TYPE                  zpra_t_mrec_prd-monat,
         lv_gjahr      TYPE                  zpra_t_mrec_prd-gjahr,
         lv_gjahr_next TYPE                  zpra_t_mrec_prd-gjahr,
         lv_last_asset TYPE                  zpra_t_mrec_prd-asset,
         lv_days       TYPE                  sy-tabix,
         lv_index      TYPE                  sy-tabix,
         lv_col_name   TYPE                  lvc_fname,
         lv_found      TYPE                  c.

  lv_gjahr = gv_current_gjahr .
  LOOP AT gt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof WHERE product EQ '722000004' .
    READ TABLE gt_zdpr_gas_combine TRANSPORTING NO FIELDS WITH KEY asset = gs_zpra_c_prd_prof-asset BINARY SEARCH .
    IF sy-subrc IS NOT INITIAL.
      CONTINUE .
    ENDIF.
    IF lv_last_asset IS INITIAL OR
       lv_last_asset NE gs_zpra_c_prd_prof-asset .
      APPEND INITIAL LINE TO <gfs_sec4a_table> ASSIGNING <gfs_dyn_line> .
      ASSIGN COMPONENT 'COL01' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .

      READ TABLE gt_asset_desc INTO gs_asset_desc WITH KEY asset = gs_zpra_c_prd_prof-asset BINARY SEARCH .
      IF sy-subrc IS INITIAL.
        <gfs_field> = gs_asset_desc-desc .
      ENDIF.
    ENDIF.
    lv_monat = '00' .
    DO .
      lv_monat = lv_monat + 1 .
      READ TABLE gt_zpra_t_mrec_app_4a INTO gs_zpra_t_mrec_app WITH KEY gjahr   = lv_gjahr
monat   = lv_monat
product = gs_zpra_c_prd_prof-product
asset   = gs_zpra_c_prd_prof-asset
block   = gs_zpra_c_prd_prof-block BINARY SEARCH .
```

```
9165        IF sy-subrc IS INITIAL. "if found in MREC APP
9166          ASSIGN COMPONENT 2 OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
9167          IF <gfs_field> IS ASSIGNED.
9168            <gfs_field> = <gfs_field> + gs_zpra_t_mrec_app-app_vl_qty .
9169            UNASSIGN <gfs_field> .
9170          ENDIF.
9171        ELSE. "Now work out from RPRD
9172  **        READ TABLE gt_zpra_t_mrec_prd_4a INTO gs_zpra_t_mrec_prd WITH K
      EY gjahr   = lv_gjahr
9173  **
         monat   = lv_monat
9174  **
         product = gs_zpra_c_prd_prof-product
9175  **
         asset   = gs_zpra_c_prd_prof-asset
9176  **
         block   = gs_zpra_c_prd_prof-block BINARY SEARCH .
9177  **        IF sy-subrc IS INITIAL. "if found in MREC
9178  **          ASSIGN COMPONENT 2 OF STRUCTURE <gfs_dyn_line> TO <gfs_field>
       .
9179  **          IF <gfs_field> IS ASSIGNED.
9180  **            <gfs_field> = <gfs_field> + gs_zpra_t_mrec_prd-prod_vl_qty1
       .
9181  **            UNASSIGN <gfs_field> .
9182  **          ENDIF.
9183          PERFORM get_days_in_period USING lv_gjahr lv_monat CHANGING lv_d
      ays .
9184          IF lv_monat EQ gv_current_monat.
9185            gv_search_end_datum = p_date .
9186            lv_days = gv_search_end_datum - gv_search_begin_datum + 1 .
9187          ENDIF.
9188          CLEAR lv_found .
9189          LOOP AT gt_zpra_t_dly_rprd_4a INTO gs_zpra_t_dly_rprd WHERE prod
      uct EQ gs_zpra_c_prd_prof-product
9190                                                                  AND asse
      t   EQ gs_zpra_c_prd_prof-asset
9191                                                                  AND bloc
      k   EQ gs_zpra_c_prd_prof-block
9192                                                                  AND prod
      uction_date GE gv_search_begin_datum
9193                                                                  AND prod
      uction_date LE gv_search_end_datum .
9194            lv_found = 'X' .
9195            lv_index = sy-tabix .
9196  *            PERFORM convert_gas_rprd_to_boe CHANGING gs_zpra_t_dly_rprd
       .
9197            gv_individual_mul = gv_total_mul . "section 4 is always OVL Le
      vel
9198            ASSIGN COMPONENT 2 OF STRUCTURE <gfs_dyn_line> TO <gfs_field>
       .
9199            IF <gfs_field> IS ASSIGNED.
9200              IF p_c_jv IS NOT INITIAL .
9201                <gfs_field> = <gfs_field> + gs_zpra_t_dly_rprd-jv_rcn_vl_q
      ty3 * 1000.
9202              ELSE.
9203                <gfs_field> = <gfs_field> + gs_zpra_t_dly_rprd-ovl_rcn_vl_
      qty3 * 1000.
9204              ENDIF.
9205              UNASSIGN <gfs_field> .
9206            ENDIF.
9207          ENDLOOP.
```

```
9208          IF lv_found IS INITIAL.
9209            PERFORM get_days_in_period USING lv_gjahr lv_monat CHANGING lv
      _days .
9210            IF lv_monat EQ gv_current_monat.
9211              gv_search_end_datum = p_date .
9212              lv_days = gv_search_end_datum - gv_search_begin_datum + 1 .
9213            ENDIF.
9214            LOOP AT gt_zpra_t_dly_prd_4a INTO gs_zpra_t_dly_prd WHERE prod
      uct EQ gs_zpra_c_prd_prof-product
9215                                                                  AND asse
      t   EQ gs_zpra_c_prd_prof-asset
9216                                                                  AND bloc
      k   EQ gs_zpra_c_prd_prof-block
9217                                                                  AND prod
      uction_date GE gv_search_begin_datum
9218                                                                  AND prod
      uction_date LE gv_search_end_datum .
9219              lv_index = sy-tabix .
9220              PERFORM get_constorium_multipliers_2a2 USING '9'.
9221  *            gv_individual_mul = gv_total_mul . "section 4 is always OVL
       Level
9222              ASSIGN COMPONENT 2 OF STRUCTURE <gfs_dyn_line> TO <gfs_field
      > .
9223              IF <gfs_field> IS ASSIGNED.
9224                <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1
       * gv_individual_mul.
9225                UNASSIGN <gfs_field> .
9226              ENDIF.
9227            ENDLOOP.
9228          ENDIF.
9229        ENDIF.
9230        IF lv_monat GE gv_current_monat.
9231          EXIT .
9232        ENDIF.
9233      ENDDO.
9234      lv_last_asset = gs_zpra_c_prd_prof-asset .
9235    ENDLOOP.
9236
9237    LOOP AT <gfs_sec4a_table> ASSIGNING <gfs_dyn_line>.
9238      ASSIGN COMPONENT 2 OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
9239      IF <gfs_field> IS ASSIGNED.
9240        <gfs_field> = <gfs_field> / 1000 .
9241      ENDIF.
9242    ENDLOOP.
9243  ENDFORM.
9244  *&———————————————————————————————————————————————————————————*
9245  *&      Form  FILL_DYNAMIC_TABLE_SEC3D
9246  *&———————————————————————————————————————————————————————————*
9247  *       text
9248  *———————————————————————————————————————————————————————————*
9249  *  —>  p1        text
9250  *  <—  p2        text
9251  *———————————————————————————————————————————————————————————*
9252  FORM fill_dynamic_table_sec3d .
9253    DATA lv_index TYPE sy-index .
9254    DATA lv_amt TYPE p LENGTH 12 DECIMALS 9 .
9255    REFRESH <gfs_sec3d_table> .
9256    READ TABLE <gfs_sec3c_table> ASSIGNING <gfs_dyn_line> INDEX 1 .
9257    IF  sy-subrc IS INITIAL .
9258      LOOP AT <gfs_sec3b_table> ASSIGNING <gfs_dyn_line2>.
9259        APPEND INITIAL LINE TO <gfs_sec3d_table> ASSIGNING <gfs_dyn_line3>
```

```
      .
        DO gv_table_columns TIMES.
          lv_index = sy-index .
          IF lv_index EQ 1.
          ELSEIF lv_index EQ 2 .
            ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line2> TO <gfs
      _field2> .
            ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line3> TO <gfs
      _field3> .
            <gfs_field3> = <gfs_field2> .
          ELSE .
            ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line>  TO <gfs
      _field> .
            ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line2> TO <gfs
      _field2> .
            ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line3> TO <gfs
      _field3> .
            IF <gfs_field>  IS ASSIGNED AND
               <gfs_field2> IS ASSIGNED AND
               <gfs_field3> IS ASSIGNED .
              IF <gfs_field2> IS NOT INITIAL .
                SHIFT <gfs_field2> LEFT DELETING LEADING space .
                IF <gfs_field2> NE '0'.
                  <gfs_field3>  =  <gfs_field>  / <gfs_field2> * 100.
                ENDIF.
                lv_amt = <gfs_field3> .
                IF lv_amt LT 0.
                  <gfs_field3> = <gfs_field3> * -1 .
                  SHIFT <gfs_field3> LEFT DELETING LEADING space .
                  CONCATENATE '-' <gfs_field3> INTO <gfs_field3> .
                ENDIF.
              ENDIF.
            ENDIF.
            UNASSIGN : <gfs_field> , <gfs_field2> , <gfs_field3>.
          ENDIF.
        ENDDO.
      ENDLOOP.
    ENDIF.
  ENDFORM.
  *&———————————————————————————————————————————————————————————*
  *&      Form  FILL_DYNAMIC_TABLE_SEC3E
  *&———————————————————————————————————————————————————————————*
  *       text
  *———————————————————————————————————————————————————————————*
  *  —>  p1        text
  *  <—  p2        text
  *———————————————————————————————————————————————————————————*
  FORM fill_dynamic_table_sec3e .
    DATA lv_index TYPE sy-index .
    DATA lv_amt TYPE p LENGTH 12 DECIMALS 9 .

    REFRESH <gfs_sec3e_table> .
    READ TABLE <gfs_sec3c_table> ASSIGNING <gfs_dyn_line> INDEX 1 .
    IF  sy-subrc IS INITIAL .
      LOOP AT <gfs_sec3a_table> ASSIGNING <gfs_dyn_line2>.
        APPEND INITIAL LINE TO <gfs_sec3e_table> ASSIGNING <gfs_dyn_line3>
      .
        DO gv_table_columns TIMES.
          lv_index = sy-index .
          IF lv_index EQ 1.
          ELSEIF lv_index EQ 2 .
```

```
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line2> TO <gfs_field2> .
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line3> TO <gfs_field3> .
          <gfs_field3> = <gfs_field2> .
        ELSE .
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line>  TO <gfs_field> .
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line2> TO <gfs_field2> .
          ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line3> TO <gfs_field3> .
          IF <gfs_field>  IS ASSIGNED AND
             <gfs_field2> IS ASSIGNED AND
             <gfs_field3> IS ASSIGNED .
            IF <gfs_field2> IS NOT INITIAL .
              SHIFT <gfs_field2> LEFT DELETING LEADING space .
              IF <gfs_field2> NE '0'.
                <gfs_field3>  = <gfs_field>  / <gfs_field2> * 100.
              ENDIF.
              lv_amt = <gfs_field3>.
              IF lv_amt LT 0.
                <gfs_field3> = <gfs_field3> * -1 .
                SHIFT <gfs_field3> LEFT DELETING LEADING space .
                CONCATENATE '-' <gfs_field3> INTO <gfs_field3> .
              ENDIF.
            ENDIF.
          ENDIF.
          UNASSIGN : <gfs_field> , <gfs_field2> , <gfs_field3>.
        ENDIF.
      ENDDO.
    ENDLOOP.
  ENDIF.
ENDFORM.
*&———————————————————————————————————————————————————————————————————*
*&      Form  GET_DAYS_IN_PERIOD
*&———————————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————————*
*      ——>P_GS_ZPRA_T_MREC_PRD_GJAHR  text
*      ——>P_GS_ZPRA_T_MREC_PRD_MONAT  text
*      <——P_LV_DAYS  text
*———————————————————————————————————————————————————————————————————*
FORM get_days_in_period  USING    p_gjahr
                                  p_monat
                         CHANGING p_days.

  DATA : lv_poper TYPE t009b-poper .
  lv_poper = p_monat .

  CALL FUNCTION 'FIRST_DAY_IN_PERIOD_GET'
    EXPORTING
      i_gjahr        = p_gjahr
*     I_MONMIT       = 00
      i_periv        = gv_periv
      i_poper        = lv_poper
    IMPORTING
      e_date         = gv_search_begin_datum
    EXCEPTIONS
      input_false    = 1
      t009_notfound  = 2
```

```
      t009b_notfound = 3
      OTHERS         = 4.
  IF sy-subrc <> 0.
    MESSAGE 'Internal Error' TYPE 'E' .
  ENDIF.

  CALL FUNCTION 'LAST_DAY_IN_PERIOD_GET'
    EXPORTING
      i_gjahr        = p_gjahr
*     I_MONMIT       = 00
      i_periv        = gv_periv
      i_poper        = lv_poper
    IMPORTING
      e_date         = gv_search_end_datum
    EXCEPTIONS
      input_false    = 1
      t009_notfound  = 2
      t009b_notfound = 3
      OTHERS         = 4.
  IF sy-subrc <> 0.
    MESSAGE 'Internal Error' TYPE 'E' .
  ENDIF.

  p_days = gv_search_end_datum - gv_search_begin_datum + 1 .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  DISPLAY_SECTION3_HEADER
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM display_section3_header .
  PERFORM display_section3_header_1 .
  PERFORM display_section3_header_2 .
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  DISPLAY_SECTION3_HEADER_1
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM display_section3_header_1 .

  gv_row = gv_row + 1 .
  gv_sec3_h_start_row = gv_row .

  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row .
  gv_e_col = 2 .

  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .

  CALL METHOD OF go_range 'Merge' .
  PERFORM set_range USING 'Consortium Level' 0.

```

```
  gv_s_row = gv_row .
  gv_s_col = 3 .
  gv_e_row = gv_s_row .
  gv_e_col = gv_table_columns .

  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .

  CALL METHOD OF go_range 'Merge' .
  PERFORM set_range USING 'ONGC Videsh' 0.

  PERFORM select_range USING gv_s_row 1 gv_e_row gv_e_col  .

  GET PROPERTY OF go_range 'interior' = go_interior .
  SET PROPERTY OF go_interior 'Color' = gv_sec3h_colour .

  GET PROPERTY OF go_range 'FONT' = go_font .
  SET PROPERTY OF go_font  'BOLD' = 1 .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  DISPLAY_SECTION3_HEADER_2
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM display_section3_header_2 .
  DATA gv_txt TYPE char50 .

  gv_row = gv_row + 1 .
  gv_sec3_h_start_row = gv_row .

  gv_s_row = gv_row .
  gv_s_col = 1 .
  gv_e_row = gv_s_row .
  gv_e_col = 2 .

  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .

  CALL METHOD OF go_range 'Merge' .
  PERFORM set_range USING 'Unit' 0.

  LOOP AT gt_product_col INTO gs_product_col.
    gv_s_col = gs_product_col-s_col .
    gv_e_col = gs_product_col-e_col .
    PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
    CALL METHOD OF go_range 'Merge' .
    IF gs_product_col-product EQ c_prod_gas.
      gv_txt = 'BCM' .
    ELSE.
      gv_txt = 'MMT' .
    ENDIF.
    PERFORM set_range USING gv_txt 0.
  ENDLOOP.

  gv_s_col = gv_table_columns .
  gv_e_col = gv_table_columns .

  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM set_range USING 'MMTOE' 0.
```

```
9492
9493    PERFORM select_range USING gv_sec3_h_start_row 1 gv_e_row gv_e_col  .
9494
9495    GET PROPERTY OF go_range 'interior' = go_interior .
9496    SET PROPERTY OF go_interior 'Color' = gv_sec3h_colour .
9497
9498    GET PROPERTY OF go_range 'FONT' = go_font .
9499    SET PROPERTY OF go_font  'BOLD' = 1 .
9500
9501  ENDFORM.
9502
9503  *&———————————————————————————————————————————————————————————*
9504  *&      Form  FILL_DYNAMIC_TABLE_SE3A
9505  *&———————————————————————————————————————————————————————————*
9506  *       text
9507  *———————————————————————————————————————————————————————————*
9508  *  ——>  p1        text
9509  *  <——  p2        text
9510  *———————————————————————————————————————————————————————————*
9511  FORM fill_dynamic_table_sec3a .
9512    DATA : lv_col_name      TYPE lvc_fname,
9513           lv_combine_field TYPE c,
9514           lv_index         TYPE sy-tabix,
9515           lv_target_name   TYPE char25.
9516    SORT gt_zpra_t_prd_tar_3a BY tar_code .
9517    LOOP AT gt_zpra_t_prd_tar_3a INTO gs_zpra_t_prd_tar .
9518      CLEAR lv_combine_field .
9519      AT NEW tar_code .
9520        APPEND INITIAL LINE TO <gfs_sec3a_table> ASSIGNING <gfs_dyn_line>
      .
9521        ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <gfs_field
      > .
9522        PERFORM get_target_type_name USING gs_zpra_t_prd_tar-tar_code CHAN
      GING lv_target_name.
9523        <gfs_field> = lv_target_name.
9524      ENDAT .
9525  *    PERFORM convert_sec2a_units CHANGING gs_zpra_t_prd_tar.
9526
9527      IF gs_zpra_t_prd_tar-product = '722000004'.
9528        READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY a
      sset = gs_zpra_t_prd_tar-asset BINARY SEARCH .
9529        IF  sy-subrc IS INITIAL.
9530          lv_combine_field = 'X' .
9531        ENDIF.
9532      ENDIF.
9533  * Individual Column..
9534      gv_len = strlen( gs_zpra_t_prd_tar-product ) .
9535      IF lv_combine_field IS INITIAL.
9536        CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' gs_zpra_t_prd_ta
      r-asset INTO lv_col_name .
9537      ELSE.
9538        CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' 'COMBINE'
            INTO lv_col_name .
9539      ENDIF.
9540      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
      ld> .
9541      IF <gfs_field> IS ASSIGNED.
9542        <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty .
9543        UNASSIGN <gfs_field> .
9544      ENDIF.
9545  * Product Total..
```

```
9546      CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' 'TOTAL'
              INTO lv_col_name .
9547      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
      ld> .
9548      IF <gfs_field> IS ASSIGNED.
9549        <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty .
9550        UNASSIGN <gfs_field> .
9551      ENDIF.
9552  * Grand Total
9553      CONCATENATE 'GRAND' '-' 'TOTAL'
               INTO lv_col_name .
9554      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
      ld> .
9555      IF <gfs_field> IS ASSIGNED.
9556        <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty .
9557        UNASSIGN <gfs_field> .
9558      ENDIF.
9559    ENDLOOP.
9560  ENDFORM.
9561  *&---------------------------------------------------------------------*
9562  *&      Form  FILL_DYNAMIC_TABLE_SE3B
9563  *&---------------------------------------------------------------------*
9564  *       text
9565  *---------------------------------------------------------------------*
9566  *  -->  p1        text
9567  *  <--  p2        text
9568  *---------------------------------------------------------------------*
9569  FORM fill_dynamic_table_sec3b .
9570    DATA : lv_col_name      TYPE lvc_fname,
9571           lv_combine_field TYPE c,
9572           lv_index         TYPE sy-tabix,
9573           lv_target_name   TYPE char25.
9574    SORT gt_zpra_t_prd_tar_3b BY tar_code .
9575    LOOP AT gt_zpra_t_prd_tar_3b INTO gs_zpra_t_prd_tar .
9576      CLEAR lv_combine_field .
9577      AT NEW tar_code .
9578        APPEND INITIAL LINE TO <gfs_sec3b_table> ASSIGNING <gfs_dyn_line>
      .
9579        ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <gfs_field
      > .
9580        PERFORM get_target_type_name USING gs_zpra_t_prd_tar-tar_code CHAN
      GING lv_target_name.
9581        <gfs_field> = lv_target_name.
9582      ENDAT .
9583  *    PERFORM convert_sec2a_units CHANGING gs_zpra_t_prd_tar.
9584
9585      IF gs_zpra_t_prd_tar-product = '722000004'.
9586        READ TABLE gt_zdpr_gas_combine INTO gs_zdpr_gas_combine WITH KEY a
      sset = gs_zpra_t_prd_tar-asset BINARY SEARCH .
9587        IF  sy-subrc IS INITIAL.
9588          lv_combine_field = 'X' .
9589        ENDIF.
9590      ENDIF.
9591  * Individual Column..
9592      gv_len = strlen( gs_zpra_t_prd_tar-product ) .
9593      IF lv_combine_field IS INITIAL.
9594        CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' gs_zpra_t_prd_ta
      r-asset INTO lv_col_name .
9595      ELSE.
9596        CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' 'COMBINE'
              INTO lv_col_name .
```

```
9597      ENDIF.
9598      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
      ld> .
9599      IF <gfs_field> IS ASSIGNED.
9600        <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty .
9601        UNASSIGN <gfs_field> .
9602      ENDIF.
9603  * Product Total..
9604      CONCATENATE gs_zpra_t_prd_tar-product(gv_len) '-' 'TOTAL'
              INTO lv_col_name .
9605      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
      ld> .
9606      IF <gfs_field> IS ASSIGNED.
9607        <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty .
9608        UNASSIGN <gfs_field> .
9609      ENDIF.
9610  * Grand Total
9611      CONCATENATE 'GRAND' '-' 'TOTAL'
               INTO lv_col_name .
9612      ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
      ld> .
9613      IF <gfs_field> IS ASSIGNED.
9614        <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty .
9615        UNASSIGN <gfs_field> .
9616      ENDIF.
9617    ENDLOOP.
9618  ENDFORM.
9619  *&———————————————————————————————————————————————————————————*
9620  *&      Form  FETCH_DATA_SECTION3C
9621  *&———————————————————————————————————————————————————————————*
9622  *       text
9623  *———————————————————————————————————————————————————————————*
9624  *  —>  p1        text
9625  *  <—  p2        text
9626  *———————————————————————————————————————————————————————————*
9627  FORM fetch_data_section3c .
9628
9629    DATA : lv_monat            TYPE                    zpra_t_mrec_prd-mona
      t,
9630           lv_gjahr            TYPE                    zpra_t_mrec_prd-gjah
      r,
9631           lv_date             TYPE                    sy-datum,
9632           lv_month_begin_date TYPE                    sy-datum,
9633           lv_month_end_date   TYPE                    sy-datum,
9634           lv_poper            TYPE                    t009b-poper.
9635    lv_gjahr = gv_last_gjahr - 1 .
9636    SELECT gjahr
9637           monat
9638           asset
9639           block
9640           product
9641           prd_vl_type
9642           prod_vl_qty1
9643           prod_vl_uom1
9644           prod_vl_qty2
9645           prod_vl_uom2
9646      FROM zpra_t_mrec_prd
9647      INTO TABLE gt_zpra_t_mrec_prd_3c
9648       FOR ALL ENTRIES IN gt_zpra_c_prd_prof
9649     WHERE asset        EQ gt_zpra_c_prd_prof-asset
9650       AND block        EQ gt_zpra_c_prd_prof-block
```

```
9651        AND product      EQ gt_zpra_c_prd_prof-product
9652        AND prd_vl_type  IN r_prd_vl_type[]
9653  *      AND ( gjahr EQ gv_last_gjahr OR gjahr EQ gv_current_gjahr   ) .
9654       AND gjahr GE lv_gjahr
9655       AND gjahr LE gv_current_gjahr .
9656     SORT gt_zpra_t_mrec_prd_3c BY product ASCENDING asset ASCENDING block
       ASCENDING gjahr DESCENDING monat DESCENDING .
9657     DELETE gt_zpra_t_mrec_prd_3c WHERE gjahr EQ gv_current_gjahr AND monat
       GT gv_current_monat .
9658     SELECT gjahr
9659            monat
9660            asset
9661            block
9662            product
9663            prd_vl_type
9664            app_vl_qty
9665            app_vl_uom
9666       FROM zpra_t_mrec_app
9667       INTO TABLE gt_zpra_t_mrec_app
9668        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
9669      WHERE gjahr EQ gv_current_gjahr
9670        AND monat LE gv_current_monat
9671        AND asset EQ gt_zpra_c_prd_prof-asset
9672        AND block EQ gt_zpra_c_prd_prof-block
9673        AND product EQ gt_zpra_c_prd_prof-product
9674        AND prd_vl_type IN r_prd_vl_type[] .
9675
9676     SORT gt_zpra_t_mrec_app BY gjahr monat product asset block  prd_vl_typ
      e .
9677
9678     SELECT *
9679       FROM zpra_t_tar_cf
9680       INTO TABLE gt_zpra_t_tar_cf_3c
9681        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
9682      WHERE gjahr        EQ gv_current_gjahr
9683        AND asset        EQ gt_zpra_c_prd_prof-asset
9684        AND block        EQ gt_zpra_c_prd_prof-block
9685        AND product      EQ gt_zpra_c_prd_prof-product .
9686
9687     SORT gt_zpra_t_tar_cf_3c BY gjahr asset block product.
9688
9689     REFRESH r_production_date[] .
9690
9691     LOOP AT gt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof .
9692       lv_monat = '00' .
9693       DO .
9694         lv_monat = lv_monat + 1 .
9695         READ TABLE gt_zpra_t_mrec_app TRANSPORTING NO FIELDS WITH KEY gjah
      r = gv_current_gjahr
9696                                                                    mona
      t = lv_monat
9697                                                                  produc
      t = gs_zpra_c_prd_prof-product
9698                                                                    asse
      t = gs_zpra_c_prd_prof-asset BINARY SEARCH .
9699         IF sy-subrc IS NOT INITIAL.
9700           lv_poper = lv_monat .
9701           CALL FUNCTION 'FIRST_DAY_IN_PERIOD_GET'
9702             EXPORTING
9703               i_gjahr        = gv_current_gjahr
9704  *            I_MONMIT       = 00
```

```
            i_periv        = gv_periv
            i_poper        = lv_poper
          IMPORTING
            e_date         = lv_month_begin_date
          EXCEPTIONS
            input_false    = 1
            t009_notfound  = 2
            t009b_notfound = 3
            OTHERS         = 4.
        IF sy-subrc <> 0.
          MESSAGE 'Internal Error' TYPE 'E' .
        ENDIF.

        CALL FUNCTION 'LAST_DAY_IN_PERIOD_GET'
          EXPORTING
            i_gjahr        = gv_current_gjahr
*           I_MONMIT       = 00
            i_periv        = gv_periv
            i_poper        = lv_poper
          IMPORTING
            e_date         = lv_month_end_date
          EXCEPTIONS
            input_false    = 1
            t009_notfound  = 2
            t009b_notfound = 3
            OTHERS         = 4.
        IF sy-subrc <> 0.
          MESSAGE 'Internal Error' TYPE 'E' .
        ENDIF.
        r_production_date-sign   = 'I' .
        r_production_date-option = 'BT' .
        r_production_date-low    = lv_month_begin_date .
        r_production_date-high   = lv_month_end_date .
        APPEND r_production_date .
      ENDIF.
      IF lv_monat GE gv_current_monat .
        EXIT .
      ENDIF.
    ENDDO .
  ENDLOOP.
  SORT r_production_date[] .
  DELETE ADJACENT DUPLICATES FROM r_production_date[] COMPARING ALL FIELDS .

  IF r_production_date[] IS NOT INITIAL .
    SELECT *
      FROM zpra_t_dly_prd
      INTO TABLE gt_zpra_t_dly_prd_3c
       FOR ALL ENTRIES IN gt_zpra_c_prd_prof
     WHERE asset EQ gt_zpra_c_prd_prof-asset
       AND block EQ gt_zpra_c_prd_prof-block
       AND product EQ gt_zpra_c_prd_prof-product
       AND prd_vl_type IN r_prd_vl_type[]
       AND production_date IN r_production_date[] .
    LOOP AT r_production_date.
      PERFORM populate_no_data_entries TABLES gt_zpra_t_dly_prd_3c USING r_production_date-low r_production_date-high .
    ENDLOOP.

    SORT gt_zpra_t_dly_prd_3c BY product asset block production_date .

```

```
     SELECT *
       FROM zpra_t_dly_rprd
       INTO TABLE gt_zpra_t_dly_rprd_3c
        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
      WHERE asset EQ gt_zpra_c_prd_prof-asset
        AND block EQ gt_zpra_c_prd_prof-block
        AND product EQ gt_zpra_c_prd_prof-product
*        AND prd_vl_type IN r_prd_vl_type[]
        AND prd_vl_type EQ 'NET_PROD'
        AND production_date IN r_production_date[] .

     SORT gt_zpra_t_dly_rprd_3c BY product asset block production_date .

     CONCATENATE gv_current_gjahr '0101' INTO lv_date .
     SELECT asset
           block
           vld_frm
           vld_to
           pi
           prod_start_date
      FROM zpra_t_prd_pi
      INTO TABLE gt_zpra_t_prd_pi_3c
       FOR ALL ENTRIES IN gt_zpra_c_prd_prof
     WHERE asset EQ gt_zpra_c_prd_prof-asset
       AND block EQ gt_zpra_c_prd_prof-block
       AND vld_frm LE p_date
       AND vld_to  GE lv_date .

     SORT gt_zpra_t_prd_pi_3c BY asset block vld_frm vld_to .


   ENDIF.

ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  FETCH_DATA_SECTION3F
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM fetch_data_section3f .

  DATA : lv_monat            TYPE                    zpra_t_mrec_prd-monat,
         lv_gjahr            TYPE                    zpra_t_mrec_prd-gjahr,
         lv_date             TYPE                    sy-datum,
         lv_date2            TYPE                    sy-datum,
         lv_month_begin_date TYPE                    sy-datum,
         lv_month_end_date   TYPE                    sy-datum,
         lv_poper            TYPE                    t009b-poper,
         lv_count            TYPE                    sy-tabix,
         lv_flagnd.

  SELECT gjahr
         monat
         asset
         block
         product
```

```
          prd_vl_type
          prod_vl_qty1
          prod_vl_uom1
          prod_vl_qty2
          prod_vl_uom2
      FROM zpra_t_mrec_prd
      INTO TABLE gt_zpra_t_mrec_prd_3f
       FOR ALL ENTRIES IN gt_zpra_c_prd_prof
     WHERE asset        EQ gt_zpra_c_prd_prof-asset
       AND block        EQ gt_zpra_c_prd_prof-block
       AND product      EQ gt_zpra_c_prd_prof-product
       AND prd_vl_type  IN r_prd_vl_type[]
       AND gjahr GE gv_5_back_gjahr
       AND gjahr LE gv_last_gjahr.
    SORT gt_zpra_t_mrec_prd_3f BY product ASCENDING asset ASCENDING block
     ASCENDING gjahr DESCENDING monat DESCENDING .

    SELECT gjahr
           monat
           asset
           block
           product
           prd_vl_type
           app_vl_qty
           app_vl_uom
      FROM zpra_t_mrec_app
      INTO TABLE gt_zpra_t_mrec_app_3f
       FOR ALL ENTRIES IN gt_zpra_c_prd_prof
     WHERE gjahr GE gv_5_back_gjahr
       AND gjahr LE gv_last_gjahr
       AND asset EQ gt_zpra_c_prd_prof-asset
       AND block EQ gt_zpra_c_prd_prof-block
       AND product EQ gt_zpra_c_prd_prof-product
       AND prd_vl_type IN r_prd_vl_type[] .

    SORT gt_zpra_t_mrec_app_3f BY gjahr monat product asset block  prd_vl_
    type .

      CONCATENATE gv_5_back_gjahr '0401' INTO lv_date  .
      CONCATENATE gv_current_gjahr   '0331' INTO lv_date2 .
      SELECT asset
             block
             vld_frm
             vld_to
             pi
             prod_start_date
       FROM zpra_t_prd_pi
       INTO TABLE gt_zpra_t_prd_pi_3f
        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
      WHERE asset EQ gt_zpra_c_prd_prof-asset
        AND block EQ gt_zpra_c_prd_prof-block
        AND vld_frm LE lv_date2
        AND vld_to  GE lv_date .

      SORT gt_zpra_t_prd_pi_3f BY asset block vld_frm vld_to .

         SELECT *
        FROM zpra_t_dly_prd
        INTO TABLE gt_zpra_t_dly_prd_3f
         FOR ALL ENTRIES IN gt_zpra_c_prd_prof
       WHERE asset EQ gt_zpra_c_prd_prof-asset
```

```
        AND block EQ gt_zpra_c_prd_prof-block
        AND product EQ gt_zpra_c_prd_prof-product
        AND prd_vl_type IN r_prd_vl_type[]
        AND  production_date BETWEEN  lv_date  AND LV_DATE2 .
*        AND production_date IN r_production_date[] .

        SELECT *
       FROM zpra_t_dly_rprd
       INTO TABLE gt_zpra_t_dly_rprd_3f
        FOR ALL ENTRIES IN gt_zpra_c_prd_prof
      WHERE asset EQ gt_zpra_c_prd_prof-asset
        AND block EQ gt_zpra_c_prd_prof-block
        AND product EQ gt_zpra_c_prd_prof-product
*        AND prd_vl_type IN r_prd_vl_type[]
        AND prd_vl_type EQ 'NET_PROD'
         AND  production_date BETWEEN  lv_date  AND LV_DATE2 .
*        AND production_date IN r_production_date[] .
     SORT gt_zpra_t_dly_rprd_3f BY product asset block production_date .


   REFRESH r_production_date[] .
   lv_gjahr = gv_5_back_gjahr - 1.
   DO 5 TIMES .
     lv_gjahr = lv_gjahr + 1 .
     LOOP AT gt_zpra_c_prd_prof INTO gs_zpra_c_prd_prof .
       lv_monat = '00' .
       DO .
         lv_monat = lv_monat + 1 .

         READ TABLE gt_zpra_t_mrec_prd_3f TRANSPORTING NO FIELDS WITH KEY
 gjahr = lv_gjahr

 monat = lv_monat
                                                                        p
roduct = gs_zpra_c_prd_prof-product

 asset = gs_zpra_c_prd_prof-asset.
         IF sy-subrc IS NOT INITIAL.
           lv_poper = lv_monat .
           CALL FUNCTION 'FIRST_DAY_IN_PERIOD_GET'
             EXPORTING
               i_gjahr        = lv_gjahr
*              I_MONMIT       = 00
               i_periv        = gv_periv
               i_poper        = lv_poper
             IMPORTING
               e_date         = lv_month_begin_date
             EXCEPTIONS
               input_false    = 1
               t009_notfound  = 2
               t009b_notfound = 3
               OTHERS         = 4.
           IF sy-subrc <> 0.
             MESSAGE 'Internal Error' TYPE 'E' .
           ENDIF.
    clear GS_ZPRA_T_PRD_PI.
    READ TABLE gt_zpra_t_prd_pi_3f INTO GS_ZPRA_T_PRD_PI WITH KEY  asset
= GS_zpra_c_prd_prof-asset
                                                                  block
= GS_zpra_c_prd_prof-block.
     IF SY-SUBRC eq 0 and  GS_ZPRA_T_PRD_PI-PROD_START_DATE LE  lv_month_
```

```
      begin_date.
9937
9938
9939
9940
9941
9942            CALL FUNCTION 'LAST_DAY_IN_PERIOD_GET'
9943              EXPORTING
9944                i_gjahr        = lv_gjahr
9945  *             I_MONMIT       = 00
9946                i_periv        = gv_periv
9947                i_poper        = lv_poper
9948              IMPORTING
9949                e_date         = lv_month_end_date
9950              EXCEPTIONS
9951                input_false    = 1
9952                t009_notfound  = 2
9953                t009b_notfound = 3
9954                OTHERS         = 4.
9955            IF sy-subrc <> 0.
9956              MESSAGE 'Internal Error' TYPE 'E' .
9957            ENDIF.
9958            r_production_date-sign   = 'I' .
9959            r_production_date-option = 'BT' .
9960            r_production_date-low    = lv_month_begin_date .
9961            r_production_date-high   = lv_month_end_date .
9962            APPEND r_production_date .
9963          ENDIF.
9964            ENDIF.
9965          IF lv_monat GE 12 .
9966            EXIT .
9967          ENDIF.
9968        ENDDO .
9969      ENDLOOP.
9970    ENDDO.
9971
9972    SORT r_production_date[] .
9973    DELETE ADJACENT DUPLICATES FROM r_production_date[] COMPARING ALL FIEL
      DS .
9974
9975    IF r_production_date[] IS NOT INITIAL .
9976
9977        SELECT *
9978        FROM zpra_dly_prd_nd
9979        INTO TABLE gt_zpra_t_dly_prd_nd
9980         FOR ALL ENTRIES IN gt_zpra_c_prd_prof
9981       WHERE asset EQ gt_zpra_c_prd_prof-asset
9982         AND block EQ gt_zpra_c_prd_prof-block
9983         AND product EQ gt_zpra_c_prd_prof-product
9984         AND prd_vl_type IN r_prd_vl_type[]
9985         AND  production_date BETWEEN  lv_date  AND LV_DATE2 .
9986
9987  *     LOOP AT r_production_date.
9988  *     loop at gt_zpra_t_dly_prd_nd INTO gs_zpra_t_dly_prd_nd where PRODUC
      TION_DATE BETWEEN  r_production_date-low  AND r_production_date-high .
9989  *     lv_flagnd = 'X'.
9990  *     exit.
9991  *     endloop.
9992  *     if lv_flagnd <> 'X'.
9993  *       PERFORM populate_no_data_entries_3f TABLES gt_zpra_t_dly_prd_3f U
      SING r_production_date-low r_production_date-high .
```

```
 9994  *      endif.
 9995  *    clear lv_flagnd.
 9996  *    ENDLOOP.
 9997
 9998        if gt_zpra_t_dly_prd_nd[] is INITIAL.
 9999        LOOP AT r_production_date.
10000        PERFORM populate_no_data_entries_3f TABLES gt_zpra_t_dly_prd_3f US
       ING r_production_date-low r_production_date-high .
10001        ENDLOOP.
10002        else.
10003        APPEND LINES OF gt_zpra_t_dly_prd_nd to gt_zpra_t_dly_prd_3f.
10004        endif.
10005
10006
10007
10008
10009      SORT gt_zpra_t_dly_prd_3f BY product asset block production_date .
10010
10011
10012      PERFORM show_progress USING '50' .
10013
10014
10015    ENDIF.
10016  clear GS_ZPRA_T_PRD_PI.
10017  ENDFORM.
10018  *&———————————————————————————————————————————————————————*
10019  *&      Form  FETCH_DATA_SECTION3F
10020  *&———————————————————————————————————————————————————————*
10021  *       text
10022  *———————————————————————————————————————————————————————*
10023  *  —>  p1        text
10024  *  <—  p2        text
10025  *———————————————————————————————————————————————————————*
10026  FORM fetch_data_section4a .
10027    DATA : lv_monat            TYPE                    zpra_t_mrec_prd-mona
       t,
10028           lv_gjahr            TYPE                    zpra_t_mrec_prd-gjah
       r,
10029           lv_date             TYPE                    sy-datum,
10030           lv_date2            TYPE                    sy-datum,
10031           lv_month_begin_date TYPE                    sy-datum,
10032           lv_month_end_date   TYPE                    sy-datum,
10033           lv_poper            TYPE                    t009b-poper.
10034
10035    SELECT gjahr
10036           monat
10037           asset
10038           block
10039           product
10040           prd_vl_type
10041           prod_vl_qty1
10042           prod_vl_uom1
10043           prod_vl_qty2
10044           prod_vl_uom2
10045      FROM zpra_t_mrec_prd
10046      INTO TABLE gt_zpra_t_mrec_prd_4a
10047       FOR ALL ENTRIES IN gt_zdpr_gas_combine
10048     WHERE asset        EQ gt_zdpr_gas_combine-asset
10049       AND product      EQ c_prod_gas
10050       AND prd_vl_type  IN r_prd_vl_type[]
10051       AND gjahr GE gv_last_gjahr
```

```
10052       AND monat LE gv_current_monat .
10053
10054    SORT gt_zpra_t_mrec_prd_4a BY product ASCENDING asset ASCENDING block
         ASCENDING gjahr DESCENDING monat DESCENDING .
10055   ***  SELECT gjahr
10056   ***         monat
10057   ***         asset
10058   ***         block
10059   ***         product
10060   ***         prd_vl_type
10061   ***         app_vl_qty
10062   ***         app_vl_uom
10063   ***    FROM zpra_t_mrec_app
10064   ***    INTO TABLE gt_zpra_t_mrec_app_4a
10065   ***     FOR ALL ENTRIES IN gt_zdpr_gas_combine
10066   ***   WHERE gjahr EQ gv_current_gjahr
10067   ***     AND monat LE gv_current_monat
10068   ***     AND asset EQ gt_zdpr_gas_combine-asset
10069   ***     AND product EQ c_prod_gas
10070   ***     AND prd_vl_type IN r_prd_vl_type[] .
10071   ***
10072   ***  SORT gt_zpra_t_mrec_app_4a BY gjahr monat product asset block  prd_
        vl_type .
10073   ***
10074
10075    REFRESH r_production_date[] .
10076    lv_gjahr = gv_current_gjahr .
10077    LOOP AT gt_zdpr_gas_combine INTO gs_zdpr_gas_combine .
10078      lv_monat = '00' .
10079      DO .
10080        lv_monat = lv_monat + 1 .
10081        READ TABLE gt_zpra_t_mrec_app_4a TRANSPORTING NO FIELDS WITH KEY g
        jahr = lv_gjahr
10082                                                                        m
        onat = lv_monat
10083                                                                      pro
        duct = c_prod_gas
10084                                                                        a
        sset = gs_zdpr_gas_combine-asset BINARY SEARCH .
10085        IF sy-subrc IS NOT INITIAL.
10086          lv_poper = lv_monat .
10087          CALL FUNCTION 'FIRST_DAY_IN_PERIOD_GET'
10088            EXPORTING
10089              i_gjahr        = lv_gjahr
10090   *          I_MONMIT       = 00
10091              i_periv        = gv_periv
10092              i_poper        = lv_poper
10093            IMPORTING
10094              e_date         = lv_month_begin_date
10095            EXCEPTIONS
10096              input_false    = 1
10097              t009_notfound  = 2
10098              t009b_notfound = 3
10099              OTHERS         = 4.
10100          IF sy-subrc <> 0.
10101            MESSAGE 'Internal Error' TYPE 'E' .
10102          ENDIF.
10103
10104          CALL FUNCTION 'LAST_DAY_IN_PERIOD_GET'
10105            EXPORTING
10106              i_gjahr        = lv_gjahr
```

```
10107  *           I_MONMIT       = 00
10108              i_periv        = gv_periv
10109              i_poper        = lv_poper
10110            IMPORTING
10111              e_date         = lv_month_end_date
10112            EXCEPTIONS
10113              input_false    = 1
10114              t009_notfound  = 2
10115              t009b_notfound = 3
10116              OTHERS         = 4.
10117          IF sy-subrc <> 0.
10118            MESSAGE 'Internal Error' TYPE 'E' .
10119          ENDIF.
10120          r_production_date-sign   = 'I' .
10121          r_production_date-option = 'BT' .
10122          r_production_date-low    = lv_month_begin_date .
10123          r_production_date-high   = lv_month_end_date .
10124          APPEND r_production_date .
10125        ENDIF.
10126        IF lv_monat GE gv_current_monat .
10127          EXIT .
10128        ENDIF.
10129      ENDDO .
10130    ENDLOOP.
10131
10132    SORT r_production_date[] .
10133    DELETE ADJACENT DUPLICATES FROM r_production_date[] COMPARING ALL FIEL
       DS .
10134
10135    IF r_production_date[] IS NOT INITIAL .
10136      SELECT *
10137        FROM zpra_t_dly_rprd
10138        INTO TABLE gt_zpra_t_dly_rprd_4a
10139         FOR ALL ENTRIES IN gt_zdpr_gas_combine
10140       WHERE asset EQ gt_zdpr_gas_combine-asset
10141         AND product EQ c_prod_gas
10142         AND prd_vl_type EQ 'NET_PROD'
10143         AND production_date IN r_production_date[].
10144      SORT gt_zpra_t_dly_rprd_4a BY product asset block production_date .
10145
10146      SELECT *
10147        FROM zpra_t_dly_prd
10148        INTO TABLE gt_zpra_t_dly_prd_4a
10149         FOR ALL ENTRIES IN gt_zdpr_gas_combine
10150       WHERE asset EQ gt_zdpr_gas_combine-asset
10151         AND product EQ c_prod_gas
10152         AND prd_vl_type IN r_prd_vl_type[]
10153         AND production_date IN r_production_date[] .
10154      SORT gt_zpra_t_dly_prd_4a BY product asset block production_date .
10155
10156      LOOP AT r_production_date.
10157        PERFORM populate_no_data_entries TABLES gt_zpra_t_dly_prd_4a USING
        r_production_date-low r_production_date-high .
10158      ENDLOOP.
10159
10160      CONCATENATE gv_last_gjahr '0101' INTO lv_date  .
10161      SELECT asset
10162            block
10163            vld_frm
10164            vld_to
10165            pi
```

```
10166           prod_start_date
10167      FROM zpra_t_prd_pi
10168      INTO TABLE gt_zpra_t_prd_pi_4a
10169       FOR ALL ENTRIES IN gt_zdpr_gas_combine
10170     WHERE asset EQ gt_zdpr_gas_combine-asset
10171       AND vld_frm LE p_date
10172       AND vld_to  GE lv_date .
10173
10174     SORT gt_zpra_t_prd_pi_4a BY asset block vld_frm vld_to .
10175   ENDIF.
10176 ENDFORM.
10177 *&———————————————————————————————————————————————————————————————*
10178 *&      Form  PREPARE_DYNAMIC_TABLE_SEC5A
10179 *&———————————————————————————————————————————————————————————————*
10180 *       text
10181 *———————————————————————————————————————————————————————————————*
10182 *  —>  p1        text
10183 *  <—  p2        text
10184 *———————————————————————————————————————————————————————————————*
10185 FORM prepare_dynamic_table_sec5a .
10186   REFRESH gt_dyn_fcat .
10187
10188   PERFORM add_dyn_field USING 'PRODUCTION_DATE' 'Production Date'  15 .
10189
10190   PERFORM add_dyn_field USING 'ACT_PRODUCTION' 'Actual Production'  15 .
10191
10192   IF p_t_be IS NOT INITIAL.
10193     PERFORM add_dyn_field USING 'TAR_BE' 'BE Target' 15 .
10194   ENDIF.
10195   IF p_t_in IS NOT INITIAL.
10196     PERFORM add_dyn_field USING 'TAR_IN' 'Internal Target' 15 .
10197   ENDIF.
10198   IF p_t_ex IS NOT INITIAL.
10199     PERFORM add_dyn_field USING 'TAR_EX' 'MOU Excellent Target' 15 .
10200   ENDIF.
10201   IF p_t_vg IS NOT INITIAL.
10202     PERFORM add_dyn_field USING 'TAR_VG' 'MOU Very Good Target' 15 .
10203   ENDIF.
10204   IF p_t_pc IS NOT INITIAL.
10205     PERFORM add_dyn_field USING 'TAR_PC' 'PC Target' 15 .
10206   ENDIF.
10207   IF p_t_re IS NOT INITIAL.
10208     PERFORM add_dyn_field USING 'TAR_RE' 'RE Target' 15 .
10209   ENDIF.
10210
10211   DESCRIBE TABLE gt_dyn_fcat LINES gv_5a_cols .
10212
10213   CALL METHOD cl_alv_table_create=>create_dynamic_table
10214     EXPORTING
10215 *     i_style_table             = #X#
10216       it_fieldcatalog           = gt_dyn_fcat
10217     IMPORTING
10218       ep_table                  = gt_sec5a_table
10219     EXCEPTIONS
10220       generate_subpool_dir_full = 1
10221       OTHERS                    = 2.
10222   IF sy-subrc NE 0.
10223     MESSAGE 'Unexpected Internal Error' TYPE 'E' .
10224   ELSE.
10225 * Assign the new table to field symbol
10226     ASSIGN gt_sec5a_table->* TO <gfs_sec5a_table>.
```

```
  ENDIF.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  PREPARE_DYNAMIC_TABLE_SEC6
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM prepare_dynamic_table_sec6 .
  REFRESH gt_dyn_fcat .

  PERFORM add_dyn_field USING 'COL1' 'COL1'  50 .
  PERFORM add_dyn_field USING 'OIL_ANNUAL' 'OIL_ANNUAL'  35 .
  PERFORM add_dyn_field USING 'OIL_YTD' 'OIL_YTD'  35 .
  PERFORM add_dyn_field USING 'GAS_ANNUAL' 'GAS_ANNUAL'  35 .
  PERFORM add_dyn_field USING 'GAS_YTD' 'GAS_YTD'  35 .
  PERFORM add_dyn_field USING 'TOTAL_ANNUAL' 'TOTAL_ANNUAL'  35 .
  PERFORM add_dyn_field USING 'TOTAL_YTD' 'TOTAL_YTD'  35 .

  CALL METHOD cl_alv_table_create=>create_dynamic_table
    EXPORTING
*     i_style_table             = #X#
      it_fieldcatalog           = gt_dyn_fcat
    IMPORTING
      ep_table                  = gt_sec6_table
    EXCEPTIONS
      generate_subpool_dir_full = 1
      OTHERS                    = 2.
  IF sy-subrc NE 0.
    MESSAGE 'Unexpected Internal Error' TYPE 'E' .
  ELSE.
* Assign the new table to field symbol
    ASSIGN gt_sec6_table->* TO <gfs_sec6_table>.
  ENDIF.
ENDFORM.
*&———————————————————————————————————————————————————————————————*
*&      Form  FILL_DYNAMIC_TABLE_SEC5A
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM fill_dynamic_table_sec5a .
  DATA : lv_date         TYPE sy-datum,
         lv_monat_3digit TYPE t009b-poper,
         lv_monat_2digit TYPE bkpf-monat,
         lv_gjahr        TYPE bkpf-gjahr,
         lv_first_day    TYPE sy-datum,
         lv_last_day     TYPE sy-datum,
         lv_days         TYPE sy-tabix,
         lv_col_name     TYPE lvc_fname.

  lv_date =  gv_year_start_date.
  DO .
    IF lv_date GT p_date.
      EXIT .
    ENDIF.
    APPEND INITIAL LINE TO <gfs_sec5a_table> ASSIGNING <gfs_dyn_line> .
```

```
10288      ASSIGN COMPONENT 'PRODUCTION_DATE' OF STRUCTURE <gfs_dyn_line> TO <g
       fs_field> .
10289      CALL FUNCTION 'CONVERSION_EXIT_SDATE_OUTPUT'
10290        EXPORTING
10291          input  = lv_date
10292        IMPORTING
10293          output = <gfs_field>.
10294      IF sy-subrc <> 0.
10295        MESSAGE 'Unexpteced Internal Error' TYPE 'E' .
10296      ENDIF.
10297      IF lv_date+6(2) EQ '01'.
10298        CALL FUNCTION 'DATE_TO_PERIOD_CONVERT'
10299          EXPORTING
10300            i_date         = lv_date
10301          " I_MONMIT       = 00
10302            i_periv        = gv_periv
10303          IMPORTING
10304            e_buper        = lv_monat_3digit
10305            e_gjahr        = lv_gjahr
10306          EXCEPTIONS
10307            input_false    = 01
10308            t009_notfound  = 02
10309            t009b_notfound = 03.
10310
10311        IF sy-subrc <> 0.
10312          MESSAGE 'Error getting Period' TYPE 'E' .
10313        ENDIF.
10314        lv_monat_2digit = lv_monat_3digit+1(2) .
10315        CALL FUNCTION 'PERIOD_DAY_DETERMINE'
10316          EXPORTING
10317            i_gjahr              = lv_gjahr
10318            i_monat              = lv_monat_2digit
10319            i_periv              = gv_periv
10320          IMPORTING
10321            e_fday               = lv_first_day
10322            e_lday               = lv_last_day
10323  *         E_SPERIOD            =
10324          EXCEPTIONS
10325            error_period         = 1
10326            error_period_version = 2
10327            firstday_not_defined = 3
10328            period_not_defined   = 4
10329            year_invalid         = 5
10330            OTHERS               = 6.
10331        IF sy-subrc <> 0.
10332          MESSAGE 'Internal Date error' TYPE 'E' .
10333        ENDIF.
10334        lv_days = lv_last_day - lv_first_day + 1 .
10335      ENDIF.
10336      ASSIGN COMPONENT 2 OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
10337      IF <gfs_field> IS ASSIGNED.
10338        LOOP AT gt_zpra_t_dly_prd_5a INTO gs_zpra_t_dly_prd WHERE producti
       on_date EQ lv_date.
10339          PERFORM convert_non_gas_units_5a CHANGING gs_zpra_t_dly_prd.
10340          IF p_c_jv IS INITIAL.
10341            LOOP AT gt_zpra_t_prd_pi_5a INTO gs_zpra_t_prd_pi WHERE asset
       EQ gs_zpra_t_dly_prd-asset
10342                                                          AND block
       EQ gs_zpra_t_dly_prd-block
10343                                                          AND vld_fr
       m LE lv_date
```

```
                                                          AND vld_to
  GE lv_date.
              EXIT .
            ENDLOOP.
            gs_zpra_t_dly_prd-prod_vl_qty1 = gs_zpra_t_dly_prd-prod_vl_qty
1 * gs_zpra_t_prd_pi-pi / 100 .
          ENDIF.
          <gfs_field> = <gfs_field> + gs_zpra_t_dly_prd-prod_vl_qty1 .
        ENDLOOP.
        UNASSIGN <gfs_field> .
      ENDIF.
      LOOP AT gt_zpra_t_prd_tar_5a INTO gs_zpra_t_prd_tar WHERE gjahr EQ l
v_gjahr
                                                            AND monat EQ l
v_monat_2digit.
***     IF gs_zpra_t_prd_tar-product EQ c_prod_gas .
***      gs_zpra_t_prd_tar-tar_qty2 = gs_zpra_t_prd_tar-tar_qty2 * 1000
* 6290.
***     ELSE.
***      CLEAR gs_zpra_t_tar_cf .
***      READ TABLE gt_zpra_t_tar_cf_5a INTO gs_zpra_t_tar_cf WITH KEY g
jahr   = lv_gjahr
***                                                                     p
roduct = gs_zpra_t_prd_tar-product
***                                                                     a
sset   = gs_zpra_t_prd_tar-asset
***                                                                     b
lock   = gs_zpra_t_prd_tar-block BINARY SEARCH .
***      gs_zpra_t_prd_tar-tar_qty2 = gs_zpra_t_prd_tar-tar_qty2 *   gs_
zpra_t_tar_cf-conv_factor .
***      gs_zpra_t_prd_tar-tar_qty2 = gs_zpra_t_prd_tar-tar_qty2 * 10000
00 .
***     ENDIF.
        PERFORM convert_sec5a_units CHANGING gs_zpra_t_prd_tar.

        lv_col_name = gs_zpra_t_prd_tar-tar_code .
        ASSIGN COMPONENT lv_col_name OF STRUCTURE <gfs_dyn_line> TO <gfs_f
ield> .
*       <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty2 / lv_days .
        <gfs_field> = <gfs_field> + gs_zpra_t_prd_tar-tar_qty / lv_days .
        UNASSIGN <gfs_field> .

      ENDLOOP.

      lv_date = lv_date + 1 .
    ENDDO.
    IF p_mb IS NOT INITIAL.
      LOOP AT <gfs_sec5a_table> ASSIGNING <gfs_dyn_line> .
        ASSIGN COMPONENT 'ACT_PRODUCTION' OF STRUCTURE <gfs_dyn_line> TO <
gfs_field> .
        <gfs_field> = <gfs_field> / 1000000 .
      ENDLOOP.
    ENDIF.
ENDFORM.
*&———————————————————————————————————————————————————————————*
*&      Form  FILL_DYNAMIC_TABLE_SEC6
*&———————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
```

```
*———————————————————————————————————————————————————————————————————*
FORM fill_dynamic_table_sec6 .
  PERFORM fill_dynamic_table_sec6a .
  PERFORM fill_dynamic_table_sec6b .
ENDFORM .
*&———————————————————————————————————————————————————————————————————*
*&      Form  FILL_DYNAMIC_TABLE_SEC6a
*&———————————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————————*
FORM fill_dynamic_table_sec6a .
  DATA lv_qty TYPE char50 .

  READ TABLE <gfs_sec2d_table> ASSIGNING <gfs_dyn_line> INDEX 1 .
  IF sy-subrc IS INITIAL.
    UNASSIGN <gfs_field>.
    APPEND INITIAL LINE TO <gfs_sec6_table> ASSIGNING <gfs_dyn_line2> .
    ASSIGN COMPONENT 'COL1' OF STRUCTURE <gfs_dyn_line2> TO <gfs_field2> .
    <gfs_field2> = 'Actual' .
    UNASSIGN <gfs_field2> .

    ASSIGN COMPONENT '722000001-TOTAL' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED .
      lv_qty = lv_qty + <gfs_field> .
      UNASSIGN <gfs_field> .
    ENDIF.

    ASSIGN COMPONENT '722000003-TOTAL' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED .
      lv_qty = lv_qty + <gfs_field> .
      UNASSIGN <gfs_field> .
    ENDIF.

    ASSIGN COMPONENT '722000005-TOTAL' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED .
      lv_qty = lv_qty + <gfs_field> .
      UNASSIGN <gfs_field> .
    ENDIF.
    SHIFT lv_qty LEFT DELETING LEADING space .
    ASSIGN COMPONENT 'OIL_ANNUAL' OF STRUCTURE  <gfs_dyn_line2> TO <gfs_field2> .
    IF <gfs_field2> IS ASSIGNED.
      <gfs_field2> = lv_qty .
      UNASSIGN <gfs_field2> .
    ENDIF.
    ASSIGN COMPONENT 'OIL_YTD' OF STRUCTURE  <gfs_dyn_line2> TO <gfs_field2> .
    IF <gfs_field2> IS ASSIGNED.
      <gfs_field2> = lv_qty .
      UNASSIGN <gfs_field2> .
    ENDIF.

    CLEAR lv_qty .
    ASSIGN COMPONENT '722000004-TOTAL' OF STRUCTURE <gfs_dyn_line> TO <g
```

```
fs_field> .
    IF <gfs_field> IS ASSIGNED .
      lv_qty =  <gfs_field> .
      UNASSIGN <gfs_field> .
    ENDIF.
    SHIFT lv_qty LEFT DELETING LEADING space .
    ASSIGN COMPONENT 'GAS_ANNUAL' OF STRUCTURE  <gfs_dyn_line2> TO <gfs_field2> .
    IF <gfs_field2> IS ASSIGNED.
      <gfs_field2> = lv_qty .
      UNASSIGN <gfs_field2> .
    ENDIF.
    ASSIGN COMPONENT 'GAS_YTD' OF STRUCTURE  <gfs_dyn_line2> TO <gfs_field2> .
    IF <gfs_field2> IS ASSIGNED.
      <gfs_field2> = lv_qty .
      UNASSIGN <gfs_field2> .
    ENDIF.

    CLEAR lv_qty .
    ASSIGN COMPONENT 'GRAND-TOTAL' OF STRUCTURE <gfs_dyn_line> TO <gfs_field> .
    IF <gfs_field> IS ASSIGNED .
      lv_qty =  <gfs_field> .
      UNASSIGN <gfs_field> .
    ENDIF.
    SHIFT lv_qty LEFT DELETING LEADING space .
    ASSIGN COMPONENT 'TOTAL_ANNUAL' OF STRUCTURE  <gfs_dyn_line2> TO <gfs_field2> .
    IF <gfs_field2> IS ASSIGNED.
      <gfs_field2> = lv_qty .
      UNASSIGN <gfs_field2> .
    ENDIF.
    ASSIGN COMPONENT 'TOTAL_YTD' OF STRUCTURE  <gfs_dyn_line2> TO <gfs_field2> .
    IF <gfs_field2> IS ASSIGNED.
      <gfs_field2> = lv_qty .
      UNASSIGN <gfs_field2> .
    ENDIF.

  ENDIF.
ENDFORM .
*&———————————————————————————————————————————————————————————————*
*&      Form  FILL_DYNAMIC_TABLE_SEC6B
*&———————————————————————————————————————————————————————————————*
*       text
*———————————————————————————————————————————————————————————————*
*  —>  p1        text
*  <—  p2        text
*———————————————————————————————————————————————————————————————*
FORM fill_dynamic_table_sec6b .
  FIELD-SYMBOLS : <lfs_field1> ,
                  <lfs_field2>  .
  READ TABLE <gfs_sec6_table> ASSIGNING <gfs_dyn_line1> INDEX 1 .
  LOOP AT <gfs_sec2b_table> ASSIGNING <gfs_dyn_line>.

    APPEND INITIAL LINE TO <gfs_sec6_table> ASSIGNING <gfs_dyn_line2> .
    ASSIGN COMPONENT 1 OF STRUCTURE <gfs_dyn_line2> TO <lfs_field2> .
    ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <lfs_field1> .

```

```
10501      CONCATENATE <lfs_field1> 'Target' INTO <lfs_field2> SEPARATED BY spa
       ce .
10502
10503      PERFORM get_annual_targets_6b .
10504      PERFORM get_ytd_targets_6b .
10505      PERFORM get_achievement_6c .
10506    ENDLOOP.
10507  ENDFORM .
10508  FORM get_annual_targets_6b .
10509    DATA lv_qty TYPE char35 .
10510    ASSIGN COMPONENT '722000001-TOTAL' OF STRUCTURE <gfs_dyn_line> TO <gfs
       _field> .
10511    IF <gfs_field> IS ASSIGNED .
10512      lv_qty = lv_qty + <gfs_field> .
10513      UNASSIGN <gfs_field> .
10514    ENDIF.
10515
10516    ASSIGN COMPONENT '722000003-TOTAL' OF STRUCTURE <gfs_dyn_line> TO <gfs
       _field> .
10517    IF <gfs_field> IS ASSIGNED .
10518      lv_qty = lv_qty + <gfs_field> .
10519      UNASSIGN <gfs_field> .
10520    ENDIF.
10521
10522    ASSIGN COMPONENT '722000005-TOTAL' OF STRUCTURE <gfs_dyn_line> TO <gfs
       _field> .
10523    IF <gfs_field> IS ASSIGNED .
10524      lv_qty = lv_qty + <gfs_field> .
10525      UNASSIGN <gfs_field> .
10526    ENDIF.
10527
10528    ASSIGN COMPONENT 'OIL_ANNUAL' OF STRUCTURE  <gfs_dyn_line2> TO <gfs_fi
       eld2> .
10529    IF <gfs_field2> IS ASSIGNED.
10530      <gfs_field2> = lv_qty .
10531      UNASSIGN <gfs_field2> .
10532    ENDIF.
10533
10534    CLEAR lv_qty .
10535    ASSIGN COMPONENT '722000004-TOTAL' OF STRUCTURE <gfs_dyn_line> TO <gfs
       _field> .
10536    IF <gfs_field> IS ASSIGNED .
10537      lv_qty =  <gfs_field> .
10538      UNASSIGN <gfs_field> .
10539    ENDIF.
10540
10541    ASSIGN COMPONENT 'GAS_ANNUAL' OF STRUCTURE  <gfs_dyn_line2> TO <gfs_fi
       eld2> .
10542    IF <gfs_field2> IS ASSIGNED.
10543      <gfs_field2> = lv_qty .
10544      UNASSIGN <gfs_field2> .
10545    ENDIF.
10546
10547    CLEAR lv_qty .
10548    ASSIGN COMPONENT 'GRAND-TOTAL' OF STRUCTURE <gfs_dyn_line> TO <gfs_fie
       ld> .
10549    IF <gfs_field> IS ASSIGNED .
10550      lv_qty =  <gfs_field> .
10551      UNASSIGN <gfs_field> .
10552    ENDIF.
10553
```

```
10554     ASSIGN COMPONENT 'TOTAL_ANNUAL' OF STRUCTURE  <gfs_dyn_line2> TO <gfs_
      field2> .
10555     IF <gfs_field2> IS ASSIGNED.
10556       <gfs_field2> = lv_qty .
10557       UNASSIGN <gfs_field2> .
10558     ENDIF.
10559   ENDFORM .
10560   FORM get_ytd_targets_6b .
10561     DATA lv_qty TYPE char35 .
10562     FIELD-SYMBOLS : <lfs_field1> ,
10563                     <lfs_field2> .
10564     LOOP AT <gfs_sec2c_table> ASSIGNING <gfs_dyn_line3> .
10565       ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <lfs_field1>
      .
10566       ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line3> TO <lfs_field2
      >.
10567       IF <lfs_field1> NE <lfs_field2>.
10568         CONTINUE .
10569       ENDIF.
10570       ASSIGN COMPONENT '722000001-TOTAL' OF STRUCTURE <gfs_dyn_line3> TO <
      gfs_field> .
10571       IF <gfs_field> IS ASSIGNED .
10572         lv_qty = lv_qty + <gfs_field> .
10573         UNASSIGN <gfs_field> .
10574       ENDIF.
10575
10576       ASSIGN COMPONENT '722000003-TOTAL' OF STRUCTURE <gfs_dyn_line3> TO <
      gfs_field> .
10577       IF <gfs_field> IS ASSIGNED .
10578         lv_qty = lv_qty + <gfs_field> .
10579         UNASSIGN <gfs_field> .
10580       ENDIF.
10581
10582       ASSIGN COMPONENT '722000005-TOTAL' OF STRUCTURE <gfs_dyn_line3> TO <
      gfs_field> .
10583       IF <gfs_field> IS ASSIGNED .
10584         lv_qty = lv_qty + <gfs_field> .
10585         UNASSIGN <gfs_field> .
10586       ENDIF.
10587
10588       ASSIGN COMPONENT 'OIL_YTD' OF STRUCTURE  <gfs_dyn_line2> TO <gfs_fie
      ld2> .
10589       IF <gfs_field2> IS ASSIGNED.
10590         <gfs_field2> = lv_qty .
10591         UNASSIGN <gfs_field2> .
10592       ENDIF.
10593
10594       CLEAR lv_qty .
10595       ASSIGN COMPONENT '722000004-TOTAL' OF STRUCTURE <gfs_dyn_line3> TO <
      gfs_field> .
10596       IF <gfs_field> IS ASSIGNED .
10597         lv_qty =  <gfs_field> .
10598         UNASSIGN <gfs_field> .
10599       ENDIF.
10600
10601       ASSIGN COMPONENT 'GAS_YTD' OF STRUCTURE  <gfs_dyn_line2> TO <gfs_fie
      ld2> .
10602       IF <gfs_field2> IS ASSIGNED.
10603         <gfs_field2> = lv_qty .
10604         UNASSIGN <gfs_field2> .
10605       ENDIF.
```

```
10606
10607      CLEAR lv_qty .
10608      ASSIGN COMPONENT 'GRAND-TOTAL' OF STRUCTURE <gfs_dyn_line3> TO <gfs_
       field> .
10609      IF <gfs_field> IS ASSIGNED .
10610        lv_qty =  <gfs_field> .
10611        UNASSIGN <gfs_field> .
10612      ENDIF.
10613
10614      ASSIGN COMPONENT 'TOTAL_YTD' OF STRUCTURE  <gfs_dyn_line2> TO <gfs_f
       ield2> .
10615      IF <gfs_field2> IS ASSIGNED.
10616        <gfs_field2> = lv_qty .
10617        UNASSIGN <gfs_field2> .
10618      ENDIF.
10619    ENDLOOP.
10620  ENDFORM .
10621  FORM get_achievement_6c .
10622    DATA lv_index TYPE sy-tabix .
10623    DATA: lv_num TYPE p length 16 DECIMALS 10.
10624    FIELD-SYMBOLS : <lfs_field1> ,
10625                    <lfs_field2> ,
10626                    <lfs_field3> .
10627    APPEND INITIAL LINE TO <gfs_sec6_table> ASSIGNING <gfs_dyn_line3> .
10628
10629    ASSIGN COMPONENT 1 OF STRUCTURE <gfs_dyn_line3> TO <lfs_field3> .
10630    ASSIGN COMPONENT 'COL02' OF STRUCTURE <gfs_dyn_line> TO <lfs_field1> .
10631
10632    CONCATENATE '% Achv w.r.t.' <lfs_field1> 'Target' INTO <lfs_field3> SE
       PARATED BY space .
10633
10634    DO 7 TIMES.
10635      lv_index = sy-index .
10636      ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line3> TO <lfs_field
       3> .
10637      IF lv_index = 1.
10638      ELSE.
10639        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line1> TO <lfs_fie
       ld1> .
10640  * Begin of changes by Arnav on 24.03.2026
10641        ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line2> TO <lfs_fie
       ld2>.
10642        IF <lfs_field2> IS ASSIGNED.
10643          lv_num = <lfs_field2>.  " implicit conversion
10644        IF lv_num <> 0.
10645          <lfs_field3> = <lfs_field1> / lv_num * 100.
10646        ENDIF.
10647        ENDIF.
10648
10649  *      ASSIGN COMPONENT lv_index OF STRUCTURE <gfs_dyn_line2> TO <lfs_fi
       eld2> .
10650  *      CONDENSE: <lfs_field2>.
10651  *      IF <lfs_field2> IS NOT INITIAL and <lfs_field2> <> '0'.
10652  *        <lfs_field3> = <lfs_field1> / <lfs_field2> * 100 .
10653  *      ENDIF.
10654
10655  * End of changes by Arnav on 24.03.2026
10656
10657      ENDIF.
10658    ENDDO.
10659  ENDFORM .
```

```
*&---------------------------------------------------------------------*
*&      Form  FETCH_DATA_SECTION5A
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM fetch_data_section5a .
  DATA : lv_gjahr TYPE zpra_t_mrec_prd-gjahr .
  SELECT *
    FROM zpra_t_dly_prd
    INTO TABLE gt_zpra_t_dly_prd_5a
     FOR ALL ENTRIES IN gt_zpra_c_prd_prof
   WHERE asset EQ gt_zpra_c_prd_prof-asset
     AND block EQ gt_zpra_c_prd_prof-block
     AND product EQ gt_zpra_c_prd_prof-product
     AND prd_vl_type IN r_prd_vl_type[]
     AND production_date GE gv_year_start_date
     AND production_date LE p_date .

  PERFORM populate_no_data_entries TABLES gt_zpra_t_dly_prd_5a USING gv_year_start_date p_date .

  SORT gt_zpra_t_dly_prd_5a BY production_date product asset block  .

  lv_gjahr = gv_current_gjahr - 3 .
  SELECT gjahr
         monat
         asset
         block
         product
         prd_vl_type
         prod_vl_qty1
         prod_vl_uom1
         prod_vl_qty2
         prod_vl_uom2
    FROM zpra_t_mrec_prd
    INTO TABLE gt_zpra_t_mrec_prd_5a
     FOR ALL ENTRIES IN gt_zpra_c_prd_prof
   WHERE asset        EQ gt_zpra_c_prd_prof-asset
     AND block        EQ gt_zpra_c_prd_prof-block
     AND product      EQ gt_zpra_c_prd_prof-product
     AND prd_vl_type  IN r_prd_vl_type[]
     AND gjahr        GE lv_gjahr
     AND gjahr        LE gv_current_gjahr .

  SORT gt_zpra_t_mrec_prd_5a BY product ASCENDING asset ASCENDING block ASCENDING prd_vl_type ASCENDING gjahr DESCENDING monat DESCENDING .


  SELECT asset
         block
         vld_frm
         vld_to
         pi
         prod_start_date
   FROM zpra_t_prd_pi
   INTO TABLE gt_zpra_t_prd_pi_5a
    FOR ALL ENTRIES IN gt_zpra_c_prd_prof
  WHERE asset EQ gt_zpra_c_prd_prof-asset
```

```
10719      AND block EQ gt_zpra_c_prd_prof-block
10720      AND vld_frm LE p_date
10721      AND vld_to  GE gv_year_start_date .
10722
10723    SELECT *
10724      FROM zpra_t_tar_cf
10725      INTO TABLE gt_zpra_t_tar_cf_5a
10726       FOR ALL ENTRIES IN gt_zpra_c_prd_prof
10727     WHERE gjahr EQ gv_current_gjahr
10728       AND asset EQ gt_zpra_c_prd_prof-asset
10729       AND block EQ gt_zpra_c_prd_prof-block
10730       AND product EQ gt_zpra_c_prd_prof-product .
10731    SORT gt_zpra_t_tar_cf_5a BY gjahr product asset block .
10732  ENDFORM.
10733  *&---------------------------------------------------------------------*
10734  *&      Form  PROCESS_GAS_RECORDS_5A
10735  *&---------------------------------------------------------------------*
10736  *       text
10737  *---------------------------------------------------------------------*
10738  *  -->  p1        text
10739  *  <--  p2        text
10740  *---------------------------------------------------------------------*
10741  FORM process_gas_records_5a .
10742
10743    DATA : lt_zpra_t_dly_prd  TYPE STANDARD TABLE OF zpra_t_dly_prd  .
10744
10745    DATA : ls_zpra_t_dly_prd   TYPE                  zpra_t_dly_prd  .
10746    FIELD-SYMBOLS : <fs_dly_prd> TYPE zpra_t_dly_prd  .
10747
10748    DATA : lv_index TYPE sy-tabix .
10749    lt_zpra_t_dly_prd  = gt_zpra_t_dly_prd_5a .
10750    SORT  lt_zpra_t_dly_prd BY production_date product asset block prd_vl_
     type .
10751
10752    DELETE gt_zpra_t_dly_prd_5a  WHERE product EQ c_prod_gas .
10753    DELETE lt_zpra_t_dly_prd     WHERE product NE c_prod_gas .
10754    CHECK lt_zpra_t_dly_prd IS NOT INITIAL.
10755
10756    LOOP AT lt_zpra_t_dly_prd INTO ls_zpra_t_dly_prd .
10757      lv_index = sy-tabix .
10758      IF lv_index EQ 1 OR
10759        <fs_dly_prd>-production_date NE ls_zpra_t_dly_prd-production_date
     OR
10760        <fs_dly_prd>-product         NE ls_zpra_t_dly_prd-product
     OR
10761        <fs_dly_prd>-asset           NE ls_zpra_t_dly_prd-asset
     OR
10762        <fs_dly_prd>-block           NE ls_zpra_t_dly_prd-block .
10763
10764        APPEND INITIAL LINE TO gt_zpra_t_dly_prd_5a ASSIGNING <fs_dly_prd>
     .
10765        <fs_dly_prd>-mandt           = ls_zpra_t_dly_prd-mandt .
10766        <fs_dly_prd>-production_date = ls_zpra_t_dly_prd-production_date .
10767        <fs_dly_prd>-product         = ls_zpra_t_dly_prd-product .
10768        <fs_dly_prd>-asset           = ls_zpra_t_dly_prd-asset .
10769        <fs_dly_prd>-block           = ls_zpra_t_dly_prd-block .
10770        <fs_dly_prd>-prod_vl_uom1    = ls_zpra_t_dly_prd-prod_vl_uom1 .
10771        <fs_dly_prd>-prd_vl_type     = 'NET_PROD' .
10772      ENDIF.
10773  *   PERFORM convert_gas_units_to_bopd CHANGING ls_zpra_t_dly_prd.
10774      PERFORM convert_gas_units CHANGING ls_zpra_t_dly_prd.
```

```
    IF p_bmd IS NOT INITIAL.
      ls_zpra_t_dly_prd-prod_vl_qty1 = ls_zpra_t_dly_prd-prod_vl_qty1 / 6290 .
    ENDIF.
    IF ls_zpra_t_dly_prd-prd_vl_type EQ 'GAS_INJ' .
      ls_zpra_t_dly_prd-prod_vl_qty1 = ls_zpra_t_dly_prd-prod_vl_qty1 * -1 .
    ENDIF.
    <fs_dly_prd>-prod_vl_qty1    = <fs_dly_prd>-prod_vl_qty1 + ls_zpra_t_dly_prd-prod_vl_qty1 .
  ENDLOOP.
  SORT  gt_zpra_t_dly_prd_5a BY production_date product asset .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  BORDER_CELLS
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM border_cells .

  gv_s_row  = gv_start_row .
  gv_s_col  = 1 .
  gv_e_row  = gv_sec3_end_row .
  gv_e_col  = gv_table_columns .

  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM set_all_borders_range .


  gv_s_row  = gv_sec4_start_row .
  gv_s_col  = 1 .
  gv_e_row  = gv_sec4_end_row .
  gv_e_col  = 2 .

  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  PERFORM set_all_borders_range .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  SET_CELL_FORMATS
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM set_cell_formats .
  DATA : lv_number_format TYPE char10,
         lv_row           TYPE sy-tabix,
         lv_row2          TYPE sy-tabix.
  PERFORM get_full_number_format CHANGING lv_number_format .
  PERFORM select_range USING gv_sec1_data_start_row 3 gv_sec2_end_row gv_table_columns  .
  PERFORM set_numberformat USING lv_number_format .
  IF p_bmd IS NOT INITIAL.
    lv_number_format = '0.000' .
    READ TABLE gt_product_col INTO gs_product_col WITH KEY product = c_p
```

```
rod_gas .
    IF sy-subrc IS INITIAL.
      PERFORM select_range USING gv_sec1_data_start_row gs_product_col-s
_col gv_sec2_end_row gs_product_col-e_col  .
      PERFORM set_numberformat USING lv_number_format .
    ENDIF.
  ENDIF.
*  lv_row
  PERFORM select_range USING gv_sec3a_start_row 3 gv_sec3_end_row gv_tab
le_columns  .
  PERFORM set_numberformat USING '0.000' .

  PERFORM select_range USING gv_sec1_h_start_row 1 gv_sec3_end_row gv_ta
ble_columns  .
  PERFORM set_range_formatting USING  1 'C' 'C' .

  PERFORM select_range USING gv_sec2_start_row 1 gv_sec3_end_row 1  .
  PERFORM set_range_formatting USING  1 'L' 'C' .

  lv_row = gv_sec4_start_row + 1 .
  PERFORM select_range USING gv_sec4_start_row 1 lv_row 2  .
  PERFORM set_range_formatting USING  1 'C' 'C' .
  lv_row = gv_sec1_h_start_row + 4 .
  PERFORM row_height USING lv_row 15 .

*  lv_row  = gv_sec2_end_row + 1 .
*  lv_row2 = gv_sec3a_start_row - 1 .
*  PERFORM select_range USING lv_row 3 lv_row2 gv_table_columns  .
*  PERFORM set_range_formatting USING  1 'C' 'C' .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  SET_BORDER_RANGE
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      -->P_1      text
*      -->P_1      text
*      -->P_1      text
*      -->P_1      text
*----------------------------------------------------------------------*
FORM set_border_range  USING    p_left
                                p_right
                                p_top
                                p_bottom .
  IF p_left EQ 1 .
    CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '7' .
    SET PROPERTY OF go_border 'LineStyle' = '1'  .
  ENDIF.
  IF p_right EQ 1 .
    CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '10'.
    SET PROPERTY OF go_border 'LineStyle' = '1' .
  ENDIF.
  IF p_top EQ 1 .
    CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '8'.
    SET PROPERTY OF go_border 'LineStyle' = '1' .
  ENDIF.
  IF p_bottom EQ 1 .
    CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '9'.
    SET PROPERTY OF go_border 'LineStyle' = '1' .
  ENDIF.
```

```
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  SET_ALL_BORDERS_RANGE
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      -->P_1      text
*      -->P_1      text
*      -->P_1      text
*      -->P_1      text
*----------------------------------------------------------------------*
FORM set_all_borders_range  .
  CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '7' .
  SET PROPERTY OF go_border 'LineStyle' = '1'  .

  CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '8'.
  SET PROPERTY OF go_border 'LineStyle' = '1' .

  CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '9'.
  SET PROPERTY OF go_border 'LineStyle' = '1' .

  CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '10'.
  SET PROPERTY OF go_border 'LineStyle' = '1' .

  CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '11'.
  SET PROPERTY OF go_border 'LineStyle' = '1' .

  CALL METHOD OF go_range 'Borders' = go_border EXPORTING #1 = '12'.
  SET PROPERTY OF go_border 'LineStyle' = '1' .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  CLEAR_VARIABLES
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM clear_variables .

  REFRESH : gt_zpra_t_dly_prd           ,
            gt_zpra_t_dly_prd2          ,
            gt_zpra_t_dly_prd_mb        ,
            gt_zpra_t_dly_prd_2a3       ,
            gt_zpra_t_dly_prd_2d        ,
            gt_zpra_t_dly_prd_2f        ,
            gt_zpra_t_dly_prd_3c        ,
            gt_zpra_t_dly_prd_3f        ,
            gt_zpra_t_dly_prd_4a        ,
            gt_zpra_t_dly_prd_5a        ,
            gt_zdpr_gas_combine         ,
            gt_zpra_t_prd_pi            ,
            gt_zpra_t_prd_pi_mb         ,
            gt_zpra_t_prd_pi_lm         ,
            gt_zpra_t_prd_pi_2d         ,
            gt_zpra_t_prd_pi_2f         ,
            gt_zpra_t_prd_pi_3c         ,
            gt_zpra_t_prd_pi_3f         ,
            gt_zpra_t_prd_pi_4a         ,
            gt_zpra_t_prd_pi_5a         ,
```

```
                gt_zpra_t_mrec_prd          ,
                gt_zpra_t_mrec_prd_2a3      ,
                gt_zpra_t_mrec_prd_2d       ,
                gt_zpra_t_mrec_prd_2f       ,
                gt_zpra_t_mrec_prd_3c       ,
                gt_zpra_t_mrec_prd_3f       ,
                gt_zpra_t_mrec_prd_4a       ,
                gt_wtd_pi                   ,
                gt_wtd_cf                   ,
                gt_asset_desc               ,
                gt_paste                    ,
                gt_copied_cells             ,
                gt_product_col              ,
                gt_zpra_t_tar_pi            ,
                gt_zpra_t_prd_tar           ,
                gt_zpra_t_prd_tar_2c        ,
                gt_zpra_t_prd_tar_3a        ,
                gt_zpra_t_prd_tar_3b        ,
                gt_zpra_t_prd_tar_5a        ,
                gt_zpra_t_tar_cf            ,
                gt_zpra_t_tar_cf_5a         ,
                gt_zpra_c_prd_prof          ,
                gt_zpra_c_dpr_prof          ,
                gt_zpra_t_mrec_app          ,
                gt_zpra_t_mrec_app_3f       ,
                gt_zpra_t_mrec_app_4a       .
    CLEAR    : gs_zpra_t_dly_prd            ,
               gs_zdpr_gas_combine          ,
               gs_zpra_t_prd_pi             ,
               gs_zpra_t_mrec_prd           ,
               gs_wtd_pi                    ,
               gs_wtd_cf                    ,
               gs_asset_desc                ,
               gs_paste                     ,
               gs_copied_cells              ,
               gs_product_col               ,
               gs_zpra_t_tar_pi             ,
               gs_zpra_t_prd_tar            ,
               gs_zpra_t_tar_cf             ,
               gs_zpra_c_prd_prof           ,
               gs_zpra_c_dpr_prof           ,
               gs_zpra_t_mrec_app           .

    REFRESH  : r_product[]                  ,
               r_prd_vl_type[]              ,
               r_combine_asset[]            ,
               r_tar_code[]                 ,
               r_production_date[]          .

    CLEAR    : r_product                    ,
               r_prd_vl_type                ,
               r_combine_asset              ,
               r_tar_code                   ,
               r_production_date            .

    FREE     : gt_dyn_table                 ,
               gt_sec2a1_table              ,
               gt_sec2a2_table              ,
               gt_sec2a3_table              ,
               gt_sec2b_table               ,
               gt_sec2d_table               ,
```

```
              gt_sec2e_table                ,
              gt_sec2f_table                ,
              gt_sec3a_table                ,
              gt_sec3b_table                ,
              gt_sec3c_table                ,
              gt_sec3d_table                ,
              gt_sec3e_table                ,
              gt_sec3f_table                ,
              gt_sec4a_table                ,
              gt_sec5a_table                ,
              gs_line                       ,
              gs_line1                      ,
              gs_dyn_fcat                   ,
              gt_dyn_fcat                   .

    CLEAR   : gv_pos                        ,
              gv_len                        ,
              gv_table_columns              ,
              gv_year_start_date            ,
              gv_year_end_date              ,
              gv_current_gjahr              ,
              gv_current_calendar_gjahr     ,
              gv_next_gjahr                 ,
              gv_last_gjahr                 ,
              gv_5_back_gjahr               ,
              gv_current_monat              ,
              gv_month_name                 ,
              gv_last_month_name            ,
              gv_current_month_days         ,
              gv_month_back_datum           ,
              gv_month_back_begin_datum     ,
              gv_search_begin_datum         ,
              gv_month_back_end_datum       ,
              gv_search_end_datum           ,
              gv_month_back_gjahr           ,
              gv_month_back_calendar_gjahr  ,
              gv_month_back_monat           ,
              gv_month_begin_datum          ,
              gv_month_end_datum            ,
              gv_year_back_begin_datum      ,
              gv_year_back_end_datum        ,
              gv_total_days_in_year         ,
              gv_days_gone_in_year          ,
              gv_days_left_in_year          ,
              gv_mrec_monat_start           ,
              gv_mrec_monat_end             ,
              gv_mrec_gjahr_start           ,
              gv_mrec_gjahr_end             ,
              gv_individual_mul             ,
              gv_total_mul                  ,
              gv_periv                      ,
              gv_grand_total_desc           ,
              gv_5a_cols                    ,
              gv_5a_rows                    ,
              gv_image_name                 .

    UNASSIGN: <gfs_dyn_table>               ,
              <gfs_sec2_table>              ,
              <gfs_sec2a2_table>            ,
              <gfs_sec2a3_table>            ,
              <gfs_sec2b_table>             ,
```

```
                <gfs_sec2d_table>            ,
                <gfs_sec2e_table>            ,
                <gfs_sec2f_table>            ,
                <gfs_sec3a_table>            ,
                <gfs_sec3b_table>            ,
                <gfs_sec3c_table>            ,
                <gfs_sec3d_table>            ,
                <gfs_sec3e_table>            ,
                <gfs_sec3f_table>            ,
                <gfs_sec4a_table>            ,
                <gfs_sec5a_table>            ,
                <gfs_dyn_line>               ,
                <gfs_dyn_line2>              ,
                <gfs_dyn_line3>              ,
                <gfs_field>                  ,
                <gfs_field2>                 ,
                <gfs_field3>                 .
    CLEAR :     gv_row                       ,
                gv_col                       ,
                gv_s_row                     ,
                gv_s_col                     ,
                gv_e_row                     ,
                gv_e_col                     ,
                gv_sec1_data_start_row       ,
                gv_sec1_h_start_row          ,
                gv_sec3_h_start_row          ,
                gv_sec3a_start_row           ,
                gv_sec3_end_row              ,
                gv_sec2a_tgt_start_row       ,
                gv_sec2b_start_row           ,
                gv_sec1_lines                ,
                gv_rc                        ,
                gv_txt                       ,
                gv_start_row                 ,
                gv_sheet1_name               ,
                gv_sheet2_name               ,
                gv_sheet3_name               .

    FREE OBJECT:  go_excel                   ,
                  go_workbooks               ,
                  go_workbook                ,
                  go_workbook2               ,
                  go_sheets                  ,
                  go_worksheet               ,
                  go_worksheet2              ,
                  go_worksheet3              ,
                  go_application             ,
                  go_pagesetup               ,
                  go_cell                    ,
                  go_font                    ,
                  go_border                  ,
                  go_range0                  ,
                  go_range                   ,
                  go_column                  ,
                  go_row                     ,
                  go_cell_from               ,
                  go_cell_to                 ,
                  go_interior                ,
                  go_charts                  ,
                  go_chart                   ,
                  go_chartobjects            ,
```

```
                go_title                ,
                go_titlechar            ,
                go_axes                 ,
                go_axestitle            ,
                go_legend               .


ENDFORM.
*&————————————————————————————————————————————————————————————————————*
*&      Form  SHOW_PROGRESS
*&————————————————————————————————————————————————————————————————————*
*       text
*————————————————————————————————————————————————————————————————————*
*      ——>P_1004   text
*————————————————————————————————————————————————————————————————————*
FORM show_progress  USING   p_percent .
  DATA lv_text TYPE char50 .
  CONCATENATE p_percent '% Complete' INTO lv_text .
  CALL FUNCTION 'SAPGUI_PROGRESS_INDICATOR'
    EXPORTING
*     PERCENTAGE       = 0
      text = lv_text.

ENDFORM.
*&————————————————————————————————————————————————————————————————————*
*&      Form  GET_FULL_NUMBER_FORMAT
*&————————————————————————————————————————————————————————————————————*
*       text
*————————————————————————————————————————————————————————————————————*
*      <——P_NUMBER_FORMAT  text
*————————————————————————————————————————————————————————————————————*
FORM get_full_number_format  CHANGING p_number_format.

  p_number_format = '0.000' .

  IF p_bb  IS NOT INITIAL OR
     p_bbd IS NOT INITIAL OR
     p_bmd IS NOT INITIAL .
    p_number_format = '0' .
  ENDIF.
ENDFORM.
*&————————————————————————————————————————————————————————————————————*
*&      Form  ADD_DUMMY_DATA
*&————————————————————————————————————————————————————————————————————*
*       text
*————————————————————————————————————————————————————————————————————*
*      ——>P_LT_ZPRA_T_DLY_PRD  text
*————————————————————————————————————————————————————————————————————*
FORM add_dummy_data  TABLES   pt_zpra_t_dly_prd STRUCTURE zpra_t_dly_prd
.

  DATA : ls_zpra_t_dly_prd TYPE zpra_t_dly_prd .
  DATA : lv_date           TYPE sy-datum       .

  lv_date = gv_month_back_datum .
  APPEND LINES OF gt_zpra_t_dly_prd TO pt_zpra_t_dly_prd .
  DO .
    READ TABLE pt_zpra_t_dly_prd TRANSPORTING NO FIELDS WITH KEY product
ion_date = lv_date .
    IF sy-subrc IS NOT INITIAL.
      ls_zpra_t_dly_prd-mandt           = sy-mandt .
```

```
      ls_zpra_t_dly_prd-production_date = lv_date .
      ls_zpra_t_dly_prd-comments        = 'DUMMYDPRDATA' .
      APPEND ls_zpra_t_dly_prd TO pt_zpra_t_dly_prd .
    ENDIF.
    lv_date = lv_date + 1 .
    IF lv_date GT p_date.
      EXIT .
    ENDIF.
  ENDDO.
  SORT pt_zpra_t_dly_prd BY production_date .
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  GET_ASSET_START_DATE
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      -->P_LV_INDEX  text
*      <--P_LV_START_DATE  text
*----------------------------------------------------------------------*
FORM get_asset_start_date  USING    p_index
                           CHANGING p_start_date.
  DATA : lt_zpra_t_prd_pi TYPE STANDARD TABLE OF ty_zpra_t_prd_pi .
  DATA : ls_zpra_t_prd_pi TYPE ty_zpra_t_prd_pi .

  DATA :  lv_product TYPE zpra_t_dly_prd-product,
          lv_asset   TYPE zpra_t_prd_pi-asset.

  CLEAR p_start_date .

  READ TABLE gt_dyn_fcat INTO gs_dyn_fcat INDEX p_index .
  IF sy-subrc IS INITIAL.
    SPLIT gs_dyn_fcat-fieldname AT '-' INTO lv_product lv_asset .
    IF sy-subrc IS INITIAL.
      IF lv_asset NE 'TOTAL' AND lv_asset NE 'COMBINE' .
        lt_zpra_t_prd_pi = gt_zpra_t_prd_pi .
        DELETE lt_zpra_t_prd_pi WHERE asset NE lv_asset .
        SORT lt_zpra_t_prd_pi BY prod_start_date .
        READ TABLE lt_zpra_t_prd_pi INTO ls_zpra_t_prd_pi WITH KEY asset = lv_asset .
        IF sy-subrc IS INITIAL.
          p_start_date = ls_zpra_t_prd_pi-prod_start_date .
        ENDIF.
      ENDIF.
    ENDIF.
  ENDIF.

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  GET_TARGET_START_DATE
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      -->P_LV_INDEX  text
*      <--P_LV_START_DATE  text
*----------------------------------------------------------------------*
FORM get_target_start_date  USING    p_index
                                     p_tar_code
                            CHANGING p_start_date.

  DATA :  lv_product TYPE zpra_t_dly_prd-product,
          lv_asset   TYPE zpra_t_prd_pi-asset.
```

```
11252
11253    CLEAR p_start_date .
11254
11255    READ TABLE gt_dyn_fcat INTO gs_dyn_fcat INDEX p_index .
11256    IF sy-subrc IS INITIAL.
11257      SPLIT gs_dyn_fcat-fieldname AT '-' INTO lv_product lv_asset .
11258      IF sy-subrc IS INITIAL.
11259        IF lv_asset NE 'TOTAL' AND lv_asset NE 'COMBINE' .
11260
11261          READ TABLE gt_tar_start_dates INTO gs_tar_start_dates WITH KEY a
       sset = lv_asset
11262                                                                  tar_
       code = p_tar_code BINARY SEARCH.
11263          IF sy-subrc IS INITIAL.
11264            p_start_date = gs_tar_start_dates-vld_frm .
11265          ENDIF.
11266        ENDIF.
11267      ENDIF.
11268    ENDIF.
11269
11270  ENDFORM.
11271  *&———————————————————————————————————————————————————————————*
11272  *&      Form  FREE_CLIPBOARD
11273  *&———————————————————————————————————————————————————————————*
11274  *       text
11275  *———————————————————————————————————————————————————————————*
11276  *  ——>  p1        text
11277  *  <——  p2        text
11278  *———————————————————————————————————————————————————————————*
11279  FORM free_clipboard .
11280    REFRESH gt_paste .
11281    CALL METHOD cl_gui_frontend_services=>clipboard_export
11282      IMPORTING
11283        data         = gt_paste
11284      CHANGING
11285        rc           = gv_rc
11286      EXCEPTIONS
11287        cntl_error   = 1
11288        error_no_gui = 2
11289        OTHERS       = 4.
11290
11291  ENDFORM.
11292  *&———————————————————————————————————————————————————————————*
11293  *&      Form  POPULATE_NO_DATA_ENTRIES
11294  *&———————————————————————————————————————————————————————————*
11295  *       text
11296  *———————————————————————————————————————————————————————————*
11297  *      ——>P_GT_ZPRA_T_DLY_PRD  text
11298  *———————————————————————————————————————————————————————————*
11299  FORM populate_no_data_entries  TABLES p_zpra_t_dly_prd STRUCTURE zpra_t_
       dly_prd
11300                                 USING  p_from_date p_to_date.
11301
11302    DATA : lt_zpra_c_prd_prof   TYPE STANDARD TABLE OF ty_zpra_c_prd_prof
       .
11303    DATA : ls_zpra_c_prd_prof TYPE                    ty_zpra_c_prd_prof,
11304           ls_zpra_t_dly_prd  TYPE                    zpra_t_dly_prd,
11305           ls_zpra_t_dly_prd2 TYPE                    zpra_t_dly_prd.
11306    DATA : lv_date  TYPE                   sy-datum,
11307           lv_date2 TYPE                   sy-datum,
11308           lv_index TYPE                   sy-tabix.
```

```
11309
11310    lt_zpra_c_prd_prof = gt_zpra_c_prd_prof .
11311    SORT lt_zpra_c_prd_prof BY product asset block .
11312    DELETE ADJACENT DUPLICATES FROM  lt_zpra_c_prd_prof COMPARING product
       asset .
11313
11314    SORT p_zpra_t_dly_prd BY product asset production_date block prd_vl_ty
       pe DESCENDING.
11315
11316    LOOP AT lt_zpra_c_prd_prof INTO ls_zpra_c_prd_prof.
11317      lv_date = p_from_date .
11318      DO .
11319        IF lv_date GT p_to_date.
11320          EXIT .
11321        ENDIF.
11322        READ TABLE p_zpra_t_dly_prd INTO ls_zpra_t_dly_prd WITH KEY produc
       t          = ls_zpra_c_prd_prof-product
11323                                                                  asset
                  = ls_zpra_c_prd_prof-asset
11324                                                                  produc
       tion_date = lv_date BINARY SEARCH .
11325        IF sy-subrc IS NOT INITIAL.
11326          lv_date2 = lv_date - 1.
11327          READ TABLE p_zpra_t_dly_prd INTO ls_zpra_t_dly_prd WITH KEY prod
       uct         = ls_zpra_c_prd_prof-product
11328                                                                    asse
       t           = ls_zpra_c_prd_prof-asset
11329                                                                    prod
       uction_date = lv_date2 BINARY SEARCH .
11330          IF sy-subrc IS INITIAL .
11331            READ TABLE p_zpra_t_dly_prd INTO ls_zpra_t_dly_prd WITH KEY pr
       oduct         = ls_zpra_c_prd_prof-product
11332                                                                      as
       set           = ls_zpra_c_prd_prof-asset
11333                                                                      pr
       oduction_date = lv_date2
11334                                                                      bl
       ock           = ls_zpra_c_prd_prof-block .
11335            IF sy-subrc IS INITIAL.
11336              lv_index = sy-tabix .
11337              LOOP AT p_zpra_t_dly_prd INTO ls_zpra_t_dly_prd FROM lv_inde
       x.
11338                IF ls_zpra_t_dly_prd-product          NE ls_zpra_c_prd_pro
       f-product OR
11339                   ls_zpra_t_dly_prd-asset            NE ls_zpra_c_prd_pro
       f-asset   OR
11340                   ls_zpra_t_dly_prd-production_date  NE lv_date2
            .
11341                  EXIT .
11342                ENDIF.
11343                ls_zpra_t_dly_prd-production_date = lv_date .
11344                APPEND ls_zpra_t_dly_prd TO p_zpra_t_dly_prd .
11345              ENDLOOP.
11346              SORT p_zpra_t_dly_prd BY product asset production_date block
        prd_vl_type DESCENDING.
11347            ENDIF.
11348          ENDIF.
11349        ENDIF.
11350        lv_date = lv_date + 1 .
11351      ENDDO.
11352    ENDLOOP.
```

```
ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  DISPLAY_RUN_DATE_TIME
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM display_run_date_time .
  DATA : lv_date_ext TYPE char20,
         lv_time_ext TYPE char8.

  CALL FUNCTION 'CONVERSION_EXIT_SDATE_OUTPUT'
    EXPORTING
      input  = sy-datum
    IMPORTING
      output = lv_date_ext.
  CONCATENATE sy-uzeit(2) ':' sy-uzeit+2(2) ':' sy-uzeit+4(2) INTO lv_time_ext .
  CONCATENATE 'Report Generated On:' lv_date_ext
              'At'                   lv_time_ext
         INTO gv_txt SEPARATED BY space .

  gv_s_row = gv_row + 3.
  gv_s_col = 1 .
  gv_e_row = gv_s_row .
  gv_e_col = 5.

  PERFORM select_range USING gv_s_row gv_s_col gv_e_row gv_e_col  .
  CALL METHOD OF go_range 'Merge' .

  PERFORM set_range USING gv_txt 0.
  PERFORM set_range_formatting USING  0 'L' 'C' .
  PERFORM set_range_font  USING 9 0 .

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  REMOVE_EXPIRED_BLOCKS
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      -->P_GT_ZPRA_C_PRD_PROF  text
*      -->P_SY_DATUM  text
*      -->P_SY_DATUM  text
*----------------------------------------------------------------------*
FORM remove_expired_blocks  TABLES   p_zpra_c_prd_prof STRUCTURE gs_zpra_c_prd_prof
                            USING    p_from_date p_to_date.

  DATA : ls_zpra_c_prd_prof TYPE ty_zpra_c_prd_prof .
  DATA : lv_index TYPE sy-tabix .
  REFRESH gt_prod_start_end_dates .
  SELECT asset
         block
         vld_frm
         vld_to
         prod_start_date
         liscense_exp_dt
    FROM zpra_t_prd_pi
    INTO TABLE gt_prod_start_end_dates
```

```
11412       FOR ALL ENTRIES IN p_zpra_c_prd_prof
11413     WHERE asset EQ p_zpra_c_prd_prof-asset
11414       AND block EQ p_zpra_c_prd_prof-block .
11415
11416    SORT gt_prod_start_end_dates BY asset ASCENDING block ASCENDING vld_to
         DESCENDING .
11417    DELETE ADJACENT DUPLICATES FROM gt_prod_start_end_dates COMPARING asse
         t block .
11418    LOOP AT p_zpra_c_prd_prof INTO ls_zpra_c_prd_prof.
11419      lv_index = sy-tabix .
11420      READ TABLE gt_prod_start_end_dates INTO gs_prod_start_end_dates WITH
         KEY asset = ls_zpra_c_prd_prof-asset
11421
             block = ls_zpra_c_prd_prof-block BINARY SEARCH .
11422      IF sy-subrc IS INITIAL.
11423        IF gs_prod_start_end_dates-prod_start_date IS INITIAL.
11424          gs_prod_start_end_dates-prod_start_date = '00000000' .
11425        ENDIF.
11426        IF gs_prod_start_end_dates-liscense_exp_dt IS INITIAL.
11427          gs_prod_start_end_dates-liscense_exp_dt = '99991231' .
11428        ENDIF.
11429        IF gs_prod_start_end_dates-liscense_exp_dt    LT p_from_date OR
11430           gs_prod_start_end_dates-prod_start_date GT p_to_date .
11431          DELETE p_zpra_c_prd_prof INDEX lv_index .
11432        ENDIF.
11433      ENDIF.
11434    ENDLOOP.
11435  ENDFORM.
11436  *&---------------------------------------------------------------------*
11437  *&      Form  CONVERT_GAS_RPRD_TO_BOE
11438  *&---------------------------------------------------------------------*
11439  *       text
11440  *---------------------------------------------------------------------*
11441  *      <--P_GS_ZPRA_T_DLY_RPRD  text
11442  *---------------------------------------------------------------------*
11443  FORM convert_gas_rprd_to_boe  CHANGING p_zpra_t_dly_rprd TYPE zpra_t_dly
       _rprd.
11444
11445    p_zpra_t_dly_rprd-jv_rcn_vl_qty3   = p_zpra_t_dly_rprd-jv_rcn_vl_qty3
         * 6290 .
11446    p_zpra_t_dly_rprd-ovl_rcn_vl_qty3  = p_zpra_t_dly_rprd-ovl_rcn_vl_qty3
         * 6290 .
11447
11448  ENDFORM.
11449  *&---------------------------------------------------------------------*
11450  *&      Form  GET_CF_FROM_DATE
11451  *&---------------------------------------------------------------------*
11452  *       text
11453  *---------------------------------------------------------------------*
11454  *      -->P_P_ZPRA_T_DLY_PRD_PRODUCTION_DA  text
11455  *      <--P_LV_CF  text
11456  *---------------------------------------------------------------------*
11457  FORM get_cf_from_date  USING    p_rec_date
11458                                  p_product
11459                                  p_asset
11460                                  p_block
11461                         CHANGING p_cf.
11462
11463    DATA : lv_monat TYPE t009b-poper,
11464           lv_gjahr TYPE t009b-bdatj.
11465
```

```
11466    CALL FUNCTION 'DATE_TO_PERIOD_CONVERT'
11467      EXPORTING
11468        i_date         = p_rec_date
11469      " I_MONMIT       = 00
11470        i_periv        = gv_periv
11471      IMPORTING
11472        e_buper        = lv_monat
11473        e_gjahr        = lv_gjahr
11474      EXCEPTIONS
11475        input_false    = 01
11476        t009_notfound  = 02
11477        t009b_notfound = 03.
11478
11479    IF sy-subrc IS INITIAL.
11480      READ TABLE gt_cf INTO gs_cf WITH KEY gjahr = lv_gjahr
11481                                           product = p_product
11482                                             asset = p_asset
11483                                             block = p_block BINARY SEARCH .
11484      IF sy-subrc IS INITIAL.
11485        p_cf = gs_cf-conv_factor .
11486
11487      ENDIF.
11488    ENDIF.
11489  ENDFORM.
11490  FORM convert_non_gas_units_5a  CHANGING p_zpra_t_dly_prd TYPE zpra_t_dly
       _prd.
11491
11492    DATA : lv_cf    TYPE zpra_t_tar_cf-conv_factor,
11493           lv_poper TYPE t009b-poper,
11494           lv_monat TYPE zpra_t_mrec_prd-monat,
11495           lv_gjahr TYPE zpra_t_mrec_prd-gjahr.
11496
11497    CHECK p_zpra_t_dly_prd-product NE c_prod_gas .
11498
11499    CALL FUNCTION 'DATE_TO_PERIOD_CONVERT'
11500      EXPORTING
11501        i_date         = p_zpra_t_dly_prd-production_date
11502      " I_MONMIT       = 00
11503        i_periv        = gv_periv
11504      IMPORTING
11505        e_buper        = lv_poper
11506        e_gjahr        = lv_gjahr
11507      EXCEPTIONS
11508        input_false    = 01
11509        t009_notfound  = 02
11510        t009b_notfound = 03.
11511
11512    IF sy-subrc <> 0.
11513      MESSAGE 'Error getting Period' TYPE 'E' .
11514    ENDIF.
11515    lv_monat = lv_poper+1(2) .
11516    CLEAR gs_zpra_t_mrec_prd .
11517    LOOP AT gt_zpra_t_mrec_prd_5a INTO gs_zpra_t_mrec_prd WHERE product
         EQ p_zpra_t_dly_prd-product
11518                                                          AND asset
         EQ p_zpra_t_dly_prd-asset
11519                                                          AND block
         EQ p_zpra_t_dly_prd-block
11520                                                          AND prd_vl_typ
       e EQ p_zpra_t_dly_prd-prd_vl_type .
11521      IF gs_zpra_t_mrec_prd-gjahr LT lv_gjahr.
```

```
      EXIT .
    ELSEIF gs_zpra_t_mrec_prd-gjahr EQ lv_gjahr AND
           gs_zpra_t_mrec_prd-monat LE lv_monat .
      EXIT.
    ENDIF.
  ENDLOOP.
  CASE abap_true.
    WHEN p_bb.
    WHEN p_bbd.
    WHEN p_tm.
      IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL.
        lv_cf = gs_zpra_t_mrec_prd-prod_vl_qty1 /  gs_zpra_t_mrec_prd-prod_vl_qty2 .
      ELSE.
        PERFORM get_cf_from_date USING  p_zpra_t_dly_prd-production_date
                                        p_zpra_t_dly_prd-product
                                        p_zpra_t_dly_prd-asset
                                        p_zpra_t_dly_prd-block
                                CHANGING lv_cf.
      ENDIF.
      IF lv_cf IS NOT INITIAL.
        p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / lv_cf .
      ENDIF.
    WHEN p_tmd.
      IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL.
        lv_cf = gs_zpra_t_mrec_prd-prod_vl_qty1 /  gs_zpra_t_mrec_prd-prod_vl_qty2 .
      ELSE.
        PERFORM get_cf_from_date USING  p_zpra_t_dly_prd-production_date
                                        p_zpra_t_dly_prd-product
                                        p_zpra_t_dly_prd-asset
                                        p_zpra_t_dly_prd-block
                                CHANGING lv_cf.
      ENDIF.
      IF lv_cf IS NOT INITIAL.
        p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / lv_cf .
      ENDIF.
    WHEN p_mb .
      IF gs_zpra_t_mrec_prd-prod_vl_qty2 IS NOT INITIAL.
        lv_cf = gs_zpra_t_mrec_prd-prod_vl_qty1 /  gs_zpra_t_mrec_prd-prod_vl_qty2 .
      ELSE.
        PERFORM get_cf_from_date USING  p_zpra_t_dly_prd-production_date
                                        p_zpra_t_dly_prd-product
                                        p_zpra_t_dly_prd-asset
                                        p_zpra_t_dly_prd-block
                                CHANGING lv_cf.
      ENDIF.
      IF lv_cf IS NOT INITIAL.
        p_zpra_t_dly_prd-prod_vl_qty1 = p_zpra_t_dly_prd-prod_vl_qty1 / lv_cf .
      ENDIF.
    WHEN p_bmd.
    WHEN OTHERS.
  ENDCASE.

ENDFORM.
*&---------------------------------------------------------------------*
*&      Form  POPULATE_NO_DATA_ENTRIES_3F
```

```
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
*      ---->P_GT_ZPRA_T_DLY_PRD_3F  text
*      ---->P_R_PRODUCTION_DATE_LOW  text
*      ---->P_R_PRODUCTION_DATE_HIGH  text
*---------------------------------------------------------------------*
FORM POPULATE_NO_DATA_ENTRIES_3F  TABLES p_zpra_t_dly_prd STRUCTURE zpra_t_dly_prd
                                  USING  p_from_date p_to_date.

   DATA : lt_zpra_c_prd_prof   TYPE STANDARD TABLE OF ty_zpra_c_prd_prof .
  DATA : ls_zpra_c_prd_prof TYPE                    ty_zpra_c_prd_prof,
         ls_zpra_t_dly_prd  TYPE                    zpra_t_dly_prd,
         ls_zpra_t_dly_prd2 TYPE                    zpra_t_dly_prd.
  DATA : lv_date  TYPE                    sy-datum,
         lv_date2 TYPE                    sy-datum,
         lv_index TYPE                    sy-tabix.

  lt_zpra_c_prd_prof = gt_zpra_c_prd_prof .
  SORT lt_zpra_c_prd_prof BY product asset block .
  DELETE ADJACENT DUPLICATES FROM  lt_zpra_c_prd_prof COMPARING product asset .

  SORT p_zpra_t_dly_prd BY product asset production_date block prd_vl_type DESCENDING.

  LOOP AT lt_zpra_c_prd_prof INTO ls_zpra_c_prd_prof.
    clear GS_ZPRA_T_PRD_PI.
    READ TABLE GT_ZPRA_T_PRD_PI_3F INTO GS_ZPRA_T_PRD_PI WITH KEY  asset = ls_zpra_c_prd_prof-asset
                                                                   block = ls_zpra_c_prd_prof-block .
    IF SY-SUBRC = 0.
     IF GS_ZPRA_T_PRD_PI-PROD_START_DATE gt p_from_date .
      CONTINUE.
     ENDIF.
    ENDIF.
    lv_date = p_from_date .
    DO .
      IF lv_date GT p_to_date.
        EXIT .
      ENDIF.
      READ TABLE p_zpra_t_dly_prd INTO ls_zpra_t_dly_prd WITH KEY product          = ls_zpra_c_prd_prof-product
                                                                  asset            = ls_zpra_c_prd_prof-asset
                                                                  production_date = lv_date BINARY SEARCH .
      IF sy-subrc IS NOT INITIAL.
        lv_date2 = lv_date - 1.
        READ TABLE p_zpra_t_dly_prd INTO ls_zpra_t_dly_prd WITH KEY product          = ls_zpra_c_prd_prof-product
                                                                    asset            = ls_zpra_c_prd_prof-asset
                                                                    production_date = lv_date2 BINARY SEARCH .
        IF sy-subrc IS INITIAL .
          READ TABLE p_zpra_t_dly_prd INTO ls_zpra_t_dly_prd WITH KEY product          = ls_zpra_c_prd_prof-product
```

```
                                                                    asset           = ls_zpra_c_prd_prof-asset
                                                                    production_date = lv_date2
                                                                    block           = ls_zpra_c_prd_prof-block .
          IF sy-subrc IS INITIAL.
            lv_index = sy-tabix .
            LOOP AT p_zpra_t_dly_prd INTO ls_zpra_t_dly_prd FROM lv_index.
              IF ls_zpra_t_dly_prd-product          NE ls_zpra_c_prd_prof-product OR
                 ls_zpra_t_dly_prd-asset            NE ls_zpra_c_prd_prof-asset   OR
                 ls_zpra_t_dly_prd-production_date  NE lv_date2
          .
                EXIT .
              ENDIF.
              ls_zpra_t_dly_prd-production_date = lv_date .
              APPEND ls_zpra_t_dly_prd TO p_zpra_t_dly_prd .
              insert ZPRA_DLY_PRD_ND FROM ls_zpra_t_dly_prd.
            ENDLOOP.
            SORT p_zpra_t_dly_prd BY product asset production_date block prd_vl_type DESCENDING.
          ENDIF.
        ENDIF.
      ENDIF.
      lv_date = lv_date + 1 .
    ENDDO.
  ENDLOOP.

ENDFORM.
```

```
*&---------------------------------------------------------------------*
*&  Include           ZPRA_DPR_REPORT_TOP
*&---------------------------------------------------------------------*
CONSTANTS : c_prod_oil TYPE zpra_t_dly_prd-product VALUE  '722000001',
            c_prod_con TYPE zpra_t_dly_prd-product VALUE  '722000003',
            c_prod_gas TYPE zpra_t_dly_prd-product VALUE  '722000004',
            c_prod_lng TYPE zpra_t_dly_prd-product VALUE  '722000005'.

CONSTANTS : c_oil_desc TYPE char20                 VALUE  'Oil',
            c_con_desc TYPE char20                 VALUE  'Condensate',
            c_gas_desc TYPE char20                 VALUE  'Gas',
            c_lng_desc TYPE char20                 VALUE  'LNG'.


TYPES : BEGIN OF ty_zpra_t_prd_pi  ,
         asset          TYPE zpra_t_prd_pi-asset            ,
         block          TYPE zpra_t_prd_pi-block            ,
         vld_frm        TYPE zpra_t_prd_pi-vld_frm          ,
         vld_to         TYPE zpra_t_prd_pi-vld_to           ,
         pi             TYPE zpra_t_prd_pi-pi               ,
         prod_start_date TYPE zpra_t_prd_pi-prod_start_date ,
        END OF ty_zpra_t_prd_pi ,
        BEGIN OF ty_zpra_t_mrec_prd ,
         gjahr          TYPE zpra_t_mrec_prd-gjahr          ,
         monat          TYPE zpra_t_mrec_prd-monat          ,
         asset          TYPE zpra_t_mrec_prd-asset          ,
         block          TYPE zpra_t_mrec_prd-block          ,
         product        TYPE zpra_t_mrec_prd-product        ,
         prd_vl_type    TYPE zpra_t_mrec_prd-prd_vl_type    ,
         prod_vl_qty1   TYPE zpra_t_mrec_prd-prod_vl_qty1   ,
         prod_vl_uom1   TYPE zpra_t_mrec_prd-prod_vl_uom1   ,
         prod_vl_qty2   TYPE zpra_t_mrec_prd-prod_vl_qty2   ,
         prod_vl_uom2   TYPE zpra_t_mrec_prd-prod_vl_uom2   ,
        END OF ty_zpra_t_mrec_prd ,
        BEGIN OF ty_wtd_pi ,
         production_date TYPE zpra_t_dly_prd-production_date,
         product        TYPE zpra_t_dly_prd-product         ,
         asset          TYPE zpra_t_prd_pi-asset            ,
         pi             TYPE p LENGTH 12  DECIMALS 9        ,
         numerator      TYPE zpra_t_dly_prd-prod_vl_qty1    ,
         denominator    TYPE zpra_t_dly_prd-prod_vl_qty1    ,
        END OF   ty_wtd_pi ,
        BEGIN OF ty_wtd_cf ,
         product        TYPE zpra_t_mrec_prd-product         ,
         asset          TYPE zpra_t_mrec_prd-asset           ,
         cf          TYPE zpra_t_mrec_prd-prod_vl_qty1    ,
        END OF   ty_wtd_cf ,
        BEGIN OF ty_asset_desc ,
          asset         TYPE zoiu_pr_dn-dn_no ,
          desc          TYPE zoiu_pr_dn-dn_de ,
        END OF   ty_asset_desc ,
        BEGIN OF ty_paste ,
         lv_data(7000) TYPE c,
        END OF ty_paste ,
        BEGIN OF ty_copied_cells,
          row TYPE sy-tabix ,
          col TYPE sy-tabix ,
        END OF   ty_copied_cells ,
        BEGIN OF ty_product_col ,
           product TYPE char50 ,
           s_col   TYPE sy-tabix,
```

```
            e_col   TYPE sy-tabix,
            desc    TYPE char50  ,
         END OF   ty_product_col ,
         BEGIN OF ty_zpra_t_tar_pi ,
           asset       TYPE zpra_t_tar_pi-asset ,
           block       TYPE zpra_t_tar_pi-block ,
           tar_code    TYPE zpra_t_tar_pi-tar_code ,
           vld_frm     TYPE zpra_t_tar_pi-vld_frm ,
           vld_to      TYPE zpra_t_tar_pi-vld_to ,
           pi          TYPE zpra_t_tar_pi-pi ,
         END OF   ty_zpra_t_tar_pi ,
         BEGIN OF ty_zpra_t_prd_tar,
           tar_code TYPE zpra_t_prd_tar-tar_code ,
           gjahr TYPE zpra_t_prd_tar-gjahr ,
           monat TYPE zpra_t_prd_tar-monat ,
           asset TYPE zpra_t_prd_tar-asset ,
           block TYPE zpra_t_prd_tar-block ,
           product TYPE zpra_t_prd_tar-product ,
           prod_vl_type_cd TYPE zpra_t_prd_tar-prod_vl_type_cd ,
*           tar_qty TYPE zpra_t_prd_tar-tar_qty ,
           tar_qty TYPE p LENGTH 16  DECIMALS 9 ,
           uom     TYPE zpra_t_prd_tar-uom    ,
           tar_qty2 TYPE p LENGTH 16 DECIMALS 9  ,
         END   OF ty_zpra_t_prd_tar,
         BEGIN OF ty_zpra_c_prd_prof ,
           product TYPE zpra_c_prd_prof-product ,
           asset   TYPE zpra_c_prd_prof-asset ,
           block   TYPE zpra_c_prd_prof-block ,
         END   OF ty_zpra_c_prd_prof ,
         BEGIN OF ty_zpra_t_mrec_app ,
           gjahr                     TYPE zpra_t_mrec_app-gjahr ,
           monat                     TYPE zpra_t_mrec_app-monat ,
           asset                     TYPE zpra_t_mrec_app-asset ,
           block                     TYPE zpra_t_mrec_app-block ,
           product                   TYPE zpra_t_mrec_app-product ,
           prd_vl_type               TYPE zpra_t_mrec_app-prd_vl_type ,
           app_vl_qty                TYPE zpra_t_mrec_app-app_vl_qty ,
           app_vl_uom                TYPE zpra_t_mrec_app-app_vl_uom ,
         END   OF ty_zpra_t_mrec_app ,
         BEGIN OF ty_gas_combine     ,
           asset TYPE zpra_t_mrec_app-asset ,
         END OF   ty_gas_combine     ,
         BEGIN OF ty_prod_start_end_dates ,
           asset TYPE zpra_t_prd_pi-asset ,
           block TYPE zpra_t_prd_pi-block ,
           vld_frm TYPE zpra_t_prd_pi-vld_frm ,
           vld_to TYPE zpra_t_prd_pi-vld_to ,
           prod_start_date TYPE zpra_t_prd_pi-prod_start_date ,
           liscense_exp_dt TYPE zpra_t_prd_pi-liscense_exp_dt ,
         END   OF ty_prod_start_end_dates ,
         BEGIN OF ty_tar_start_dates ,
           asset TYPE zpra_t_tar_pi-asset ,
           tar_code TYPE zpra_t_tar_pi-tar_code ,
           vld_frm  TYPE zpra_t_tar_pi-vld_frm ,
         END   OF ty_tar_start_dates .

DATA : gt_zpra_t_dly_prd            TYPE STANDARD TABLE OF zpra_t_dly_prd ,
       gt_zpra_t_dly_prd2           TYPE STANDARD TABLE OF zpra_t_dly_prd ,
       gt_zpra_t_dly_prd_mb         TYPE STANDARD TABLE OF zpra_t_dly_pr
```

```
d ,
121         gt_zpra_t_dly_prd_2a3        TYPE STANDARD TABLE OF zpra_t_dly_pr
d ,
122         gt_zpra_t_dly_prd_2d         TYPE STANDARD TABLE OF zpra_t_dly_pr
d ,
123         gt_zpra_t_dly_prd_2f         TYPE STANDARD TABLE OF zpra_t_dly_pr
d ,
124         gt_zpra_t_dly_prd_3c         TYPE STANDARD TABLE OF zpra_t_dly_pr
d ,
125         gt_zpra_t_dly_rprd_2d        TYPE STANDARD TABLE OF zpra_t_dly_rp
rd,
126         gt_zpra_t_dly_rprd_2f        TYPE STANDARD TABLE OF zpra_t_dly_rp
rd,
127         gt_zpra_t_dly_rprd_3c        TYPE STANDARD TABLE OF zpra_t_dly_rp
rd,
128         gt_zpra_t_dly_prd_3f         TYPE STANDARD TABLE OF zpra_t_dly_pr
d ,
129         gt_zpra_t_dly_prd_nd         TYPE STANDARD TABLE OF zpra_t_dly_pr
d ,
130         gt_zpra_t_dly_rprd_3f        TYPE STANDARD TABLE OF zpra_t_dly_rp
rd,
131         gt_zpra_t_dly_prd_4a         TYPE STANDARD TABLE OF zpra_t_dly_pr
d ,
132         gt_zpra_t_dly_rprd_4a        TYPE STANDARD TABLE OF zpra_t_dly_rp
rd ,
133         gt_zpra_t_dly_prd_5a         TYPE STANDARD TABLE OF zpra_t_dly_pr
d ,
134         gt_zdpr_gas_combine          TYPE STANDARD TABLE OF ty_gas_combin
e ,
135         gt_zpra_t_prd_pi             TYPE STANDARD TABLE OF ty_zpra_t_prd
_pi   ,
136         gt_zpra_t_prd_pi_mb          TYPE STANDARD TABLE OF ty_zpra_t_prd
_pi   ,
137         gt_zpra_t_prd_pi_lm          TYPE STANDARD TABLE OF ty_zpra_t_prd
_pi   ,
138         gt_zpra_t_prd_pi_2d          TYPE STANDARD TABLE OF ty_zpra_t_prd
_pi   ,
139         gt_zpra_t_prd_pi_2f          TYPE STANDARD TABLE OF ty_zpra_t_prd
_pi   ,
140         gt_zpra_t_prd_pi_3c          TYPE STANDARD TABLE OF ty_zpra_t_prd
_pi   ,
141         gt_zpra_t_prd_pi_3f          TYPE STANDARD TABLE OF ty_zpra_t_prd
_pi   ,
142         gt_zpra_t_prd_pi_4a          TYPE STANDARD TABLE OF ty_zpra_t_prd
_pi   ,
143         gt_zpra_t_prd_pi_5a          TYPE STANDARD TABLE OF ty_zpra_t_prd
_pi   ,
144         gt_zpra_t_mrec_prd           TYPE STANDARD TABLE OF ty_zpra_t_mre
c_prd ,
145         gt_zpra_t_mrec_prd_2a3       TYPE STANDARD TABLE OF ty_zpra_t_mre
c_prd ,
146         gt_zpra_t_mrec_prd_2d        TYPE STANDARD TABLE OF ty_zpra_t_mre
c_prd ,
147         gt_zpra_t_mrec_prd_2f        TYPE STANDARD TABLE OF ty_zpra_t_mre
c_prd ,
148         gt_zpra_t_mrec_prd_3c        TYPE STANDARD TABLE OF ty_zpra_t_mre
c_prd ,
149         gt_zpra_t_tar_cf_3c          TYPE STANDARD TABLE OF zpra_t_tar_cf
,
150         gt_zpra_t_mrec_prd_3f        TYPE STANDARD TABLE OF ty_zpra_t_mre
c_prd ,
```

```
151         gt_zpra_t_mrec_prd_4a        TYPE STANDARD TABLE OF ty_zpra_t_mre
    c_prd ,
152         gt_zpra_t_mrec_prd_5a        TYPE STANDARD TABLE OF ty_zpra_t_mre
    c_prd ,
153         gt_wtd_pi                    TYPE STANDARD TABLE OF ty_wtd_pi
    ,
154         gt_wtd_cf                    TYPE STANDARD TABLE OF ty_wtd_cf
    ,
155         gt_asset_desc                TYPE STANDARD TABLE OF ty_asset_desc
     ,
156         gt_paste                     TYPE STANDARD TABLE OF ty_paste
      ,
157         gt_copied_cells              TYPE STANDARD TABLE OF ty_copied_cel
    ls   ,
158         gt_product_col               TYPE STANDARD TABLE OF ty_product_co
    l ,
159         gt_zpra_t_tar_pi             TYPE STANDARD TABLE OF ty_zpra_t_tar
    _pi ,
160         gt_zpra_t_prd_tar            TYPE STANDARD TABLE OF ty_zpra_t_prd
    _tar  ,
161         gt_zpra_t_prd_tar_2c         TYPE STANDARD TABLE OF ty_zpra_t_prd
    _tar  ,
162         gt_zpra_t_prd_tar_3a         TYPE STANDARD TABLE OF ty_zpra_t_prd
    _tar  ,
163         gt_zpra_t_prd_tar_3b         TYPE STANDARD TABLE OF ty_zpra_t_prd
    _tar  ,
164         gt_zpra_t_prd_tar_5a         TYPE STANDARD TABLE OF ty_zpra_t_prd
    _tar  ,
165         gt_zpra_t_tar_cf             TYPE STANDARD TABLE OF zpra_t_tar_cf
        ,
166         gt_cf                        TYPE STANDARD TABLE OF zpra_t_tar_cf
        ,
167         gt_zpra_t_tar_cf_5a          TYPE STANDARD TABLE OF zpra_t_tar_cf
        ,
168         gt_zpra_c_prd_prof           TYPE STANDARD TABLE OF ty_zpra_c_prd
    _prof ,
169         gt_zpra_c_dpr_prof           TYPE STANDARD TABLE OF zpra_c_dpr_pr
    of ,
170         gt_zpra_t_mrec_app           TYPE STANDARD TABLE OF ty_zpra_t_mre
    c_app ,
171         gt_zpra_t_mrec_app_2a3       TYPE STANDARD TABLE OF ty_zpra_t_mre
    c_app ,
172         gt_zpra_t_mrec_app_2d        TYPE STANDARD TABLE OF ty_zpra_t_mre
    c_app ,
173         gt_zpra_t_mrec_app_2f        TYPE STANDARD TABLE OF ty_zpra_t_mre
    c_app ,
174         gt_zpra_t_mrec_app_3f        TYPE STANDARD TABLE OF ty_zpra_t_mre
    c_app ,
175         gt_zpra_t_mrec_app_4a        TYPE STANDARD TABLE OF ty_zpra_t_mre
    c_app ,
176         gt_prod_start_end_dates      TYPE STANDARD TABLE OF ty_prod_start
    _end_dates ,
177         gt_tar_start_dates           TYPE STANDARD TABLE OF ty_tar_start_
    dates .
178
179 DATA : gs_zpra_t_dly_prd             TYPE                    zpra_t_dly_pr
    d ,
180         gs_zpra_t_dly_prd_nd         TYPE                    zpra_t_dly_pr
    d ,
181         gs_zpra_t_dly_rprd           TYPE                    zpra_t_dly_rp
    rd,
```

```
182         gs_zdpr_gas_combine          TYPE                    ty_gas_combin
    e ,
183         gs_zpra_t_prd_pi             TYPE                    ty_zpra_t_prd
    _pi ,
184         gs_zpra_t_mrec_prd           TYPE                    ty_zpra_t_mre
    c_prd ,
185         gs_wtd_pi                    TYPE                    ty_wtd_pi
        ,
186         gs_wtd_cf                    TYPE                    ty_wtd_cf
     ,
187         gs_asset_desc                TYPE                    ty_asset_desc
      ,
188         gs_paste                     TYPE                    ty_paste
        ,
189         gs_copied_cells              TYPE                    ty_copied_cel
    ls  ,
190         gs_product_col               TYPE                    ty_product_co
    l ,
191         gs_zpra_t_tar_pi             TYPE                    ty_zpra_t_tar
    _pi ,
192         gs_zpra_t_prd_tar            TYPE                    ty_zpra_t_prd
    _tar ,
193         gs_zpra_t_tar_cf             TYPE                    zpra_t_tar_cf
     ,
194         gs_cf                        TYPE                    zpra_t_tar_cf
     ,
195         gs_zpra_c_prd_prof           TYPE                    ty_zpra_c_prd
    _prof ,
196         gs_zpra_c_dpr_prof           TYPE                    zpra_c_dpr_pr
    of ,
197         gs_zpra_t_mrec_app           TYPE                    ty_zpra_t_mre
    c_app ,
198         gs_zpra_t_tar_cf_3c          TYPE                    zpra_t_tar_cf
          ,
199         gs_prod_start_end_dates      TYPE                    ty_prod_start
    _end_dates ,
200         gs_tar_start_dates           TYPE                    ty_tar_start_
    dates .
201
202  RANGES : r_product                  FOR gs_zpra_t_dly_prd-product      ,
203           r_prd_vl_type              FOR gs_zpra_t_dly_prd-prd_vl_type ,
204           r_combine_asset            FOR gs_zdpr_gas_combine-asset      ,
205           r_tar_code                 FOR gs_zpra_t_tar_pi-tar_code      ,
206           r_production_date          FOR gs_zpra_t_dly_prd-production_dat
    e .
207
208
209  * Dynamic Table Declarations
210
211
212  DATA : gt_dyn_table                 TYPE REF TO data,
213         gt_sec2a1_table              TYPE REF TO data,
214         gt_sec2a2_table              TYPE REF TO data,
215         gt_sec2a3_table              TYPE REF TO data,
216         gt_sec2b_table               TYPE REF TO data,
217         gt_sec2c_table               TYPE REF TO data,
218         gt_sec2d_table               TYPE REF TO data,
219         gt_sec2e_table               TYPE REF TO data,
220         gt_sec2f_table               TYPE REF TO data,
221         gt_sec3a_table               TYPE REF TO data,
222         gt_sec3b_table               TYPE REF TO data,
```

```
223         gt_sec3c_table                TYPE REF TO data,
224         gt_sec3d_table                TYPE REF TO data,
225         gt_sec3e_table                TYPE REF TO data,
226         gt_sec3f_table                TYPE REF TO data,
227         gt_sec4a_table                TYPE REF TO data,
228         gt_sec5a_table                TYPE REF TO data,
229         gt_sec6_table                 TYPE REF TO data,
230         gs_line                       TYPE REF TO data,
231         gs_line1                      TYPE REF TO data,
232         gs_dyn_fcat                   TYPE lvc_s_fcat,
233         gt_dyn_fcat                   TYPE lvc_t_fcat.
234
235  DATA : gv_pos                        TYPE i ,
236         gv_len                        TYPE i ,
237         gv_table_columns              TYPE sy-tabix ,
238         gv_year_start_date            TYPE sy-datum ,
239         gv_year_end_date              TYPE sy-datum ,
240         gv_current_gjahr_days         TYPE sy-tabix ,
241         gv_month_days_till_today      TYPE sy-tabix ,
242         gv_current_gjahr              TYPE zpra_t_mrec_prd-gjahr ,
243         gv_current_calendar_gjahr     TYPE zpra_t_mrec_prd-gjahr ,
244         gv_next_gjahr                 TYPE zpra_t_mrec_prd-gjahr ,
245         gv_last_gjahr                 TYPE zpra_t_mrec_prd-gjahr ,
246         gv_5_back_gjahr               TYPE zpra_t_mrec_prd-gjahr ,
247         gv_current_monat              TYPE zpra_t_mrec_prd-monat ,
248         gv_last_gjahr_days            TYPE sy-tabix ,
249         gv_month_name                 TYPE t247-ltx ,
250         gv_last_month_name            TYPE t247-ltx ,
251         gv_current_month_days         TYPE sy-tabix ,
252         gv_month_back_datum           TYPE sy-datum ,
253         gv_month_back_begin_datum     TYPE sy-datum ,
254         gv_search_begin_datum         TYPE sy-datum ,
255         gv_month_back_end_datum       TYPE sy-datum ,
256         gv_search_end_datum           TYPE sy-datum ,
257         gv_month_back_gjahr           TYPE gjahr ,
258         gv_month_back_calendar_gjahr TYPE gjahr ,
259         gv_month_back_monat           TYPE zpra_t_mrec_prd-monat ,
260         gv_month_begin_datum          TYPE sy-datum ,
261         gv_month_end_datum            TYPE sy-datum ,
262         gv_year_back_begin_datum      TYPE sy-datum ,
263         gv_year_back_end_datum        TYPE sy-datum ,
264         gv_total_days_in_year         TYPE sy-tabix  ,
265         gv_days_gone_in_year          TYPE sy-tabix ,
266         gv_last_year_s_date           TYPE sy-datum ,
267         gv_last_year_t_date           TYPE sy-datum ,
268         gv_days_gone_in_last_year     TYPE sy-tabix ,
269         gv_days_left_in_year          TYPE sy-tabix  ,
270         gv_mrec_monat_start           TYPE zpra_t_mrec_prd-monat ,
271         gv_mrec_monat_end             TYPE zpra_t_mrec_prd-monat ,
272         gv_mrec_gjahr_start           TYPE zpra_t_mrec_prd-gjahr ,
273         gv_mrec_gjahr_end             TYPE zpra_t_mrec_prd-gjahr ,
274         gv_individual_mul             TYPE char10 ,
275         gv_total_mul                  TYPE char10 ,
276         gv_grand_total_mul            TYPE char10 ,
277         gv_periv                      TYPE t001-periv ,
278         gv_grand_total_desc           TYPE char100 ,
279         gv_5a_cols                    TYPE sy-tabix ,
280         gv_5a_rows                    TYPE sy-tabix ,
281         gv_image_name                 TYPE rlgrap-filename .
282
283  DATA : gv_header_colour              TYPE sy-tabix VALUE 10092543 ,
```

```
        gv_dates_colour              TYPE sy-tabix VALUE 10079487 ,
        gv_header_oil_colour         TYPE sy-tabix VALUE 8838836  ,
        gv_header_cond_colour        TYPE sy-tabix VALUE 16772019 ,
        gv_header_lng_colour         TYPE sy-tabix VALUE 5296274  ,
        gv_header_gas_colour         TYPE sy-tabix VALUE 7199999  ,
        gv_header_gt_colour          TYPE sy-tabix VALUE 14851781 ,
        gv_sec2a_tgt_colour          TYPE sy-tabix VALUE  6724095 ,
        gv_sec2b_colour              TYPE sy-tabix VALUE 12171181 ,
        gv_sec2c_colour              TYPE sy-tabix VALUE 16751001 ,
        gv_sec2a2_colour             TYPE sy-tabix VALUE  3381759 ,
        gv_sec2a3_colour             TYPE sy-tabix VALUE 10079487 ,
        gv_sec2d_colour              TYPE sy-tabix VALUE 10092441 ,
        gv_sec2e_colour              TYPE sy-tabix VALUE 16764057 ,
        gv_sec2f_colour              TYPE sy-tabix VALUE 11711154 ,
        gv_sec3h_colour              TYPE sy-tabix VALUE 13434879 ,
        gv_sec3a_colour              TYPE sy-tabix VALUE 13421823 ,
        gv_sec3b_colour              TYPE sy-tabix VALUE 16772300 ,
        gv_sec3c_colour              TYPE sy-tabix VALUE 9293736  ,
        gv_sec3d_colour              TYPE sy-tabix VALUE 16772300 ,
        gv_sec3e_colour              TYPE sy-tabix VALUE 11711154 ,
        gv_sec3f_colour              TYPE sy-tabix VALUE 16701082 ,
        gv_colour                    TYPE sy-tabix                ,
        gv_rep_date                  TYPE sy-datum                ,
        gv_repdate_e                 TYPE char20                  .

FIELD-SYMBOLS: <gfs_dyn_table>      TYPE STANDARD TABLE,
               <gfs_sec2_table>     TYPE STANDARD TABLE,
               <gfs_sec2a2_table>   TYPE STANDARD TABLE,
               <gfs_sec2a3_table>   TYPE STANDARD TABLE,
               <gfs_sec2b_table>    TYPE STANDARD TABLE,
               <gfs_sec2c_table>    TYPE STANDARD TABLE,
               <gfs_sec2d_table>    TYPE STANDARD TABLE,
               <gfs_sec2e_table>    TYPE STANDARD TABLE,
               <gfs_sec2f_table>    TYPE STANDARD TABLE,
               <gfs_sec3a_table>    TYPE STANDARD TABLE,
               <gfs_sec3b_table>    TYPE STANDARD TABLE,
               <gfs_sec3c_table>    TYPE STANDARD TABLE,
               <gfs_sec3d_table>    TYPE STANDARD TABLE,
               <gfs_sec3e_table>    TYPE STANDARD TABLE,
               <gfs_sec3f_table>    TYPE STANDARD TABLE,
               <gfs_sec4a_table>    TYPE STANDARD TABLE,
               <gfs_sec5a_table>    TYPE STANDARD TABLE,
               <gfs_sec6_table>     TYPE STANDARD TABLE,
               <gfs_dyn_line>  ,
               <gfs_dyn_line1> ,
               <gfs_dyn_line2>  ,
               <gfs_dyn_line3>  ,
               <gfs_field>      ,
               <gfs_field2>     ,
               <gfs_field3>     .

DATA: go_excel                      TYPE ole2_object,
      go_workbooks                  TYPE ole2_object,
      go_workbook                   TYPE ole2_object,
      go_workbook2                  TYPE ole2_object,
      go_sheets                     TYPE ole2_object,
      go_worksheet                  TYPE ole2_object,
      go_worksheet2                 TYPE ole2_object,
      go_worksheet3                 TYPE ole2_object,
      go_application                TYPE ole2_object ,
      go_pagesetup                  TYPE ole2_object ,
```

```
345         go_cell                        TYPE ole2_object,
346         go_font                        TYPE ole2_object,
347         go_border                      TYPE ole2_object,
348         go_range0                      TYPE ole2_object,
349         go_range                       TYPE ole2_object,
350         go_column                      TYPE ole2_object,
351         go_row                         TYPE ole2_object,
352         go_cell_from                   TYPE ole2_object,
353         go_cell_to                     TYPE ole2_object,
354         go_interior                    TYPE ole2_object,
355         go_charts                      TYPE ole2_object,
356         go_chart                       TYPE ole2_object,
357         go_chartobjects                TYPE ole2_object ,
358         go_title                       TYPE ole2_object ,
359         go_titlechar                   TYPE ole2_object ,
360         go_axes                        TYPE ole2_object ,
361         go_axestitle                   TYPE ole2_object ,
362         go_legend                      TYPE ole2_object .
363
364   DATA: gv_row                         TYPE sy-tabix   ,
365         gv_col                         TYPE sy-tabix   ,
366         gv_s_row                       TYPE sy-tabix   ,
367         gv_s_col                       TYPE sy-tabix   ,
368         gv_e_row                       TYPE sy-tabix   ,
369         gv_e_col                       TYPE sy-tabix   ,
370         gv_sec1_data_start_row         TYPE sy-tabix ,
371         gv_sec1_h_start_row            TYPE sy-tabix ,
372         gv_sec3_h_start_row            TYPE sy-tabix ,
373         gv_sec3a_start_row             TYPE sy-tabix ,
374         gv_sec4_start_row              TYPE sy-tabix ,
375         gv_sec4_end_row                TYPE sy-tabix ,
376         gv_sec2_start_row              TYPE sy-tabix ,
377         gv_sec2d_start_row             TYPE sy-tabix ,
378         gv_sec2f_start_row             TYPE sy-tabix ,
379         gv_sec2_end_row                TYPE sy-tabix ,
380         gv_sec3_end_row                TYPE sy-tabix ,
381         gv_sec2a_tgt_start_row         TYPE sy-tabix ,
382         gv_sec2b_start_row             TYPE sy-tabix ,
383         gv_sec1_lines                  TYPE sy-tabix  ,
384         gv_rc                          TYPE sy-subrc   ,
385         gv_txt                         TYPE char50     ,
386         gv_start_row                   TYPE sy-tabix  ,
387         gv_sheet1_name                 TYPE char50  ,
388         gv_sheet2_name                 TYPE char50  ,
389         gv_sheet3_name                 TYPE char50  .
390   DATA gs_temp TYPE zoiu_pr_dn .
391   SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE text-001 .
392     PARAMETERS : p_date TYPE sy-datum OBLIGATORY.
393   SELECTION-SCREEN END   OF BLOCK b1 .
394
395   SELECTION-SCREEN BEGIN OF BLOCK b2 WITH FRAME TITLE text-002 .
396     PARAMETERS : p_bb  TYPE char1 RADIOBUTTON GROUP rb1 ,                "B
      BL / BOE
397                  p_bbd TYPE char1 RADIOBUTTON GROUP rb1 ,                "B
      OPD / BOEPD
398                  p_tm  TYPE char1 RADIOBUTTON GROUP rb1 ,                "T
      ons / MSCM
399                  p_tmd TYPE char1 RADIOBUTTON GROUP rb1 ,                "T
      PD / MSCMD
400                  p_mb  TYPE char1 RADIOBUTTON GROUP rb1 ,                "M
      MT / BCM
```

```
401                 p_bmd TYPE char1 RADIOBUTTON GROUP rb1 DEFAULT 'X'.    "B
    OPD / MMSCMD
402  SELECTION-SCREEN END   OF BLOCK b2 .
403
404  SELECTION-SCREEN BEGIN OF BLOCK b3 WITH FRAME TITLE text-003 .
405    PARAMETERS : p_c_jv  TYPE char1 RADIOBUTTON GROUP rb2 DEFAULT 'X', "JV
406                 p_c_olv TYPE char1 RADIOBUTTON GROUP rb2            . "OV
    L
407  SELECTION-SCREEN END OF BLOCK   b3 .
408  SELECTION-SCREEN BEGIN OF BLOCK b4 WITH FRAME TITLE text-004 .
409    PARAMETERS : p_t_be TYPE char1 AS CHECKBOX DEFAULT 'X',
410                 p_t_in TYPE char1 AS CHECKBOX ,
411                 p_t_ex TYPE char1 AS CHECKBOX ,
412                 p_t_vg TYPE char1 AS CHECKBOX ,
413                 p_t_pc TYPE char1 AS CHECKBOX ,
414                 p_t_re TYPE char1 AS CHECKBOX .
415  SELECTION-SCREEN END OF BLOCK   b4 .
416  SELECTION-SCREEN BEGIN OF BLOCK b5 WITH FRAME TITLE text-005 .
417    PARAMETERS : p_fname TYPE string OBLIGATORY .
418  SELECTION-SCREEN END OF BLOCK   b5 .
419
420  AT SELECTION-SCREEN .
421    IF p_t_be IS INITIAL AND
422       p_t_in IS INITIAL AND
423       p_t_ex IS INITIAL AND
424       p_t_vg IS INITIAL AND
425       p_t_pc IS INITIAL AND
426       p_t_re IS INITIAL .
427      MESSAGE 'Please select atleast one target' TYPE 'E' .
428    ENDIF.
429    IF p_date GT sy-datum.
430      MESSAGE 'DPR cannot be run for future dates' TYPE 'E' .
431    ENDIF.
432    AT SELECTION-SCREEN ON VALUE-REQUEST FOR p_fname.
433  *
434  *CALL FUNCTION 'F4_FILENAME'
435  *
436  *EXPORTING
437  *
438  *program_name = syst-cprog
439  *
440  *dynpro_number = syst-dynnr
441  *
442  *field_name = ' '
443  *
444  *IMPORTING
445  *
446  *file_name = p_fname.
447
448  CALL METHOD cl_gui_frontend_services=>directory_browse
449    EXPORTING
450      window_title         = 'Please Select Directory to download File'
451  *    initial_folder       =
452    CHANGING
453      selected_folder      = p_fname
454    EXCEPTIONS
455      cntl_error           = 1
456      error_no_gui         = 2
457      not_supported_by_gui = 3
458      others               = 4
459          .
```

```
IF sy-subrc IS NOT INITIAL.
  MESSAGE 'Error getting Directory Name' TYPE 'E' .
ENDIF.

```

Title:
DPR Report

Text elements:

001 Date Selection
002 UoM Selection
003 Consortium
004 Targets Selection
005 Directory Path

Selection texts:

P_BB
BBL / BOE
P_BBD
BOPD / BOEPD
P_BMD
BOPD / MMSCMD
P_C_JV
JV
P_C_OLV
OVL
P_DATE
Date
P_FNAME
Directory Path
P_MB
MMT / BCM
P_TM
Tons / MSCM
P_TMD
TPD / MSCMD
P_T_BE
BE
P_T_EX
MOU Excellent
P_T_IN
Internal
P_T_PC
PC
P_T_RE
RE
P_T_VG
MOU Very Good

Subroutines:		Line	Page
ADD_DUMMY_DATA.....	ZPRA_DPR_REPORT_F01	771	15
	ZPRA_DPR_REPORT_F01	11181	207
ADD_DYN_FIELD.....	ZPRA_DPR_REPORT_F01	658	14
	ZPRA_DPR_REPORT_F01	659	14
	ZPRA_DPR_REPORT_F01	677	14
	ZPRA_DPR_REPORT_F01	681	14
	ZPRA_DPR_REPORT_F01	694	14
	ZPRA_DPR_REPORT_F01	702	14
	ZPRA_DPR_REPORT_F01	707	14
	ZPRA_DPR_REPORT_F01	710	14
	ZPRA_DPR_REPORT_F01	740	15
	ZPRA_DPR_REPORT_F01	10188	190
	ZPRA_DPR_REPORT_F01	10190	190
	ZPRA_DPR_REPORT_F01	10193	190
	ZPRA_DPR_REPORT_F01	10196	190
	ZPRA_DPR_REPORT_F01	10199	190
	ZPRA_DPR_REPORT_F01	10202	190
	ZPRA_DPR_REPORT_F01	10205	190
	ZPRA_DPR_REPORT_F01	10208	190
	ZPRA_DPR_REPORT_F01	10240	191
	ZPRA_DPR_REPORT_F01	10241	191
	ZPRA_DPR_REPORT_F01	10242	191
	ZPRA_DPR_REPORT_F01	10243	191
	ZPRA_DPR_REPORT_F01	10244	191
	ZPRA_DPR_REPORT_F01	10245	191
	ZPRA_DPR_REPORT_F01	10246	191
ADD_PRODUCT_RANGE.....	ZPRA_DPR_REPORT_F01	36	3
	ZPRA_DPR_REPORT_F01	36	3
	ZPRA_DPR_REPORT_F01	36	3
	ZPRA_DPR_REPORT_F01	36	3
	ZPRA_DPR_REPORT_F01	1886	35
ADD_TAR_CODE_RANGE.....	ZPRA_DPR_REPORT_F01	1916	35
	ZPRA_DPR_REPORT_F01	1919	35
	ZPRA_DPR_REPORT_F01	1922	35
	ZPRA_DPR_REPORT_F01	1925	35
	ZPRA_DPR_REPORT_F01	1928	35
	ZPRA_DPR_REPORT_F01	1931	35
	ZPRA_DPR_REPORT_F01	1941	35
ADD_VL_TYPE_RANGE.....	ZPRA_DPR_REPORT_F01	40	3
	ZPRA_DPR_REPORT_F01	40	3
	ZPRA_DPR_REPORT_F01	40	3
	ZPRA_DPR_REPORT_F01	1903	35
BORDER_CELLS.....	ZPRA_DPR_REPORT_F01	497	11
	ZPRA_DPR_REPORT_F01	10794	201
BUIILD_TAR_CODE_RANGE.....	ZPRA_DPR_REPORT_F01	43	3
	ZPRA_DPR_REPORT_F01	1913	35
CALCULATED_WTD_CF.....	ZPRA_DPR_REPORT_F01	514	11
	ZPRA_DPR_REPORT_F01	2199	40
CALCULATED_WTD_PI.....	ZPRA_DPR_REPORT_F01	513	11
	ZPRA_DPR_REPORT_F01	1959	36
CLEAR_VARIABLES.....	ZPRA_DPR_REPORT	12	2
	ZPRA_DPR_REPORT_F01	10928	203
COLOUR_ALTERNATE_ROWS.....	ZPRA_DPR_REPORT_F01	1555	29
	ZPRA_DPR_REPORT_F01	3332	60
COLOUR_DATES.....	ZPRA_DPR_REPORT_F01	1557	29
	ZPRA_DPR_REPORT_F01	4630	86
COLOUR_SEC1_DATA_TOTALS.....	ZPRA_DPR_REPORT_F01	1558	29
	ZPRA_DPR_REPORT_F01	4677	87
COLOUR_YELLOW_CELLS.....	ZPRA_DPR_REPORT_F01	1556	29

```
                                ZPRA_DPR_REPORT_F01          2599    48
COL_WIDTH...................... ZPRA_DPR_REPORT_F01          1388    26
                                ZPRA_DPR_REPORT_F01          1389    26
                                ZPRA_DPR_REPORT_F01          4600    86
                                ZPRA_DPR_REPORT_F01          4601    86
                                ZPRA_DPR_REPORT_F01          4602    86
                                ZPRA_DPR_REPORT_F01          4604    86
                                ZPRA_DPR_REPORT_F01          4606    86
                                ZPRA_DPR_REPORT_F01          4609    86
CONVERT_GAS_PER_DAY_UNITS..... ZPRA_DPR_REPORT_F01          5228    97
                                ZPRA_DPR_REPORT_F01          5274    98
                                ZPRA_DPR_REPORT_F01          7118   131
                                ZPRA_DPR_REPORT_F01          7663   141
                                ZPRA_DPR_REPORT_F01          7754   143
CONVERT_GAS_RPRD_TO_BOE....... ZPRA_DPR_REPORT_F01          11443  212
CONVERT_GAS_UNITS............. ZPRA_DPR_REPORT_F01          2320    43
                                ZPRA_DPR_REPORT_F01          2383    44
                                ZPRA_DPR_REPORT_F01          3401    61
                                ZPRA_DPR_REPORT_F01          7117   131
                                ZPRA_DPR_REPORT_F01          7213   133
                                ZPRA_DPR_REPORT_F01          7653   141
                                ZPRA_DPR_REPORT_F01          10774  200
CONVERT_GAS_UNITS2............ ZPRA_DPR_REPORT_F01          3439    62
CONVERT_GAS_UNITS_TO_BCM...... ZPRA_DPR_REPORT_F01          3501    63
                                ZPRA_DPR_REPORT_F01          7358   135
                                ZPRA_DPR_REPORT_F01          7459   137
CONVERT_GAS_UNITS_TO_BOPD..... ZPRA_DPR_REPORT_F01          3477    63
CONVERT_GAS_UNITS_TO_MMSCM.... ZPRA_DPR_REPORT_F01          3527    64
                                ZPRA_DPR_REPORT_F01          7557   139
CONVERT_MREC_GAS_PER_DAY_UNITS ZPRA_DPR_REPORT_F01          7154   131
                                ZPRA_DPR_REPORT_F01          7737   142
                                ZPRA_DPR_REPORT_F01          7771   143
CONVERT_MREC_GAS_TO_MMSCM..... ZPRA_DPR_REPORT_F01          3614    65
                                ZPRA_DPR_REPORT_F01          7593   140
CONVERT_MREC_GAS_UNITS........ ZPRA_DPR_REPORT_F01          3553    64
                                ZPRA_DPR_REPORT_F01          7153   131
                                ZPRA_DPR_REPORT_F01          7258   133
                                ZPRA_DPR_REPORT_F01          7698   142
CONVERT_MREC_TO_BCM........... ZPRA_DPR_REPORT_F01          3591    65
                                ZPRA_DPR_REPORT_F01          7395   136
                                ZPRA_DPR_REPORT_F01          7495   138
CONVERT_NON_GAS_MREC_UNITS.... ZPRA_DPR_REPORT_F01          4041    74
                                ZPRA_DPR_REPORT_F01          5463   102
CONVERT_NON_GAS_UNITS......... ZPRA_DPR_REPORT_F01          3633    65
CONVERT_NON_GAS_UNITS2........ ZPRA_DPR_REPORT_F01          3667    66
                                ZPRA_DPR_REPORT_F01          5506   103
                                ZPRA_DPR_REPORT_F01          8070   149
                                ZPRA_DPR_REPORT_F01          8407   156
CONVERT_NON_GAS_UNITS3C....... ZPRA_DPR_REPORT_F01          3765    68
                                ZPRA_DPR_REPORT_F01          8756   163
CONVERT_NON_GAS_UNITS3F....... ZPRA_DPR_REPORT_F01          3981    72
                                ZPRA_DPR_REPORT_F01          9016   169
CONVERT_NON_GAS_UNITS_2A2..... ZPRA_DPR_REPORT_F01           796    16
                                ZPRA_DPR_REPORT_F01          1989    36
                                ZPRA_DPR_REPORT_F01          4061    74
                                ZPRA_DPR_REPORT_F01          5227    97
                                ZPRA_DPR_REPORT_F01          5273    98
CONVERT_NON_GAS_UNITS_5A...... ZPRA_DPR_REPORT_F01          10339  192
                                ZPRA_DPR_REPORT_F01          11490  213
CONVERT_RPRD_UNITS............ ZPRA_DPR_REPORT_F01          3904    71
                                ZPRA_DPR_REPORT_F01          8017   148
```

```
                                  ZPRA_DPR_REPORT_F01            8356   155
CONVERT_RPRD_UNITS_MB......... ZPRA_DPR_REPORT_F01            3828    69
                                  ZPRA_DPR_REPORT_F01            8711   162
                                  ZPRA_DPR_REPORT_F01            8971   168
CONVERT_SEC2A_TO_JVL.......... ZPRA_DPR_REPORT_F01            4718    88
                                  ZPRA_DPR_REPORT_F01            4745    88
CONVERT_SEC2A_UNITS........... ZPRA_DPR_REPORT_F01            4123    75
                                  ZPRA_DPR_REPORT_F01            4918    91
CONVERT_SEC2B_TO_JVL.......... ZPRA_DPR_REPORT_F01            4777    89
                                  ZPRA_DPR_REPORT_F01            6522   120
CONVERT_SEC2B_UNITS........... ZPRA_DPR_REPORT_F01            4171    76
                                  ZPRA_DPR_REPORT_F01            5018    93
CONVERT_SEC2C_TO_JVL.......... ZPRA_DPR_REPORT_F01            4835    90
                                  ZPRA_DPR_REPORT_F01            6541   121
CONVERT_SEC2C_UNITS........... ZPRA_DPR_REPORT_F01            4219    78
                                  ZPRA_DPR_REPORT_F01            5119    95
CONVERT_SEC5A_UNITS........... ZPRA_DPR_REPORT_F01            4267    79
                                  ZPRA_DPR_REPORT_F01           10366   193
CREATE_CHART.................. ZPRA_DPR_REPORT_F01            1196    23
                                  ZPRA_DPR_REPORT_F01            1199    23
DELETE_IMAGE.................. ZPRA_DPR_REPORT_F01            2626    48
                                  ZPRA_DPR_REPORT_F01            2734    50
DISPLAY_ASSET_NAMES........... ZPRA_DPR_REPORT_F01            1525    28
                                  ZPRA_DPR_REPORT_F01            2854    52
DISPLAY_CF.................... ZPRA_DPR_REPORT_F01            1527    28
                                  ZPRA_DPR_REPORT_F01            3020    55
DISPLAY_CONSORTIUM_LEVEL...... ZPRA_DPR_REPORT_F01            1528    28
                                  ZPRA_DPR_REPORT_F01            3070    56
DISPLAY_IMAGE................. ZPRA_DPR_REPORT_F01            2625    48
                                  ZPRA_DPR_REPORT_F01            2708    50
DISPLAY_LOGO.................. ZPRA_DPR_REPORT_F01            1522    28
                                  ZPRA_DPR_REPORT_F01            2623    48
DISPLAY_PI.................... ZPRA_DPR_REPORT_F01            1526    28
                                  ZPRA_DPR_REPORT_F01            2956    54
DISPLAY_PRODUCT_NAMES......... ZPRA_DPR_REPORT_F01            1524    28
                                  ZPRA_DPR_REPORT_F01            2795    51
DISPLAY_REPORT_DATE........... ZPRA_DPR_REPORT_F01            1523    28
                                  ZPRA_DPR_REPORT_F01            2763    51
DISPLAY_RUN_DATE_TIME......... ZPRA_DPR_REPORT_F01             621    13
                                  ZPRA_DPR_REPORT_F01           11362   211
DISPLAY_SEC2A_TARGETS......... ZPRA_DPR_REPORT_F01             880    18
                                  ZPRA_DPR_REPORT_F01            6038   112
DISPLAY_SECTION1.............. ZPRA_DPR_REPORT_F01             479    11
                                  ZPRA_DPR_REPORT_F01             874    17
DISPLAY_SECTION1_DATA......... ZPRA_DPR_REPORT_F01             876    17
                                  ZPRA_DPR_REPORT_F01            1542    29
DISPLAY_SECTION1_HEADER....... ZPRA_DPR_REPORT_F01             875    17
                                  ZPRA_DPR_REPORT_F01            1521    28
DISPLAY_SECTION2A1............ ZPRA_DPR_REPORT_F01             879    17
                                  ZPRA_DPR_REPORT_F01            4723    88
                                  ZPRA_DPR_REPORT_F01            6526   121
                                  ZPRA_DPR_REPORT_F01            6545   121
DISPLAY_SECTION2A2............ ZPRA_DPR_REPORT_F01             882    18
                                  ZPRA_DPR_REPORT_F01            4728    88
DISPLAY_SECTION2A3............ ZPRA_DPR_REPORT_F01             906    18
                                  ZPRA_DPR_REPORT_F01            4734    88
DISPLAY_SECTION2D............. ZPRA_DPR_REPORT_F01             545    12
                                  ZPRA_DPR_REPORT_F01             929    18
DISPLAY_SECTION2E............. ZPRA_DPR_REPORT_F01             552    12
                                  ZPRA_DPR_REPORT_F01            1039    20
DISPLAY_SECTION2F............. ZPRA_DPR_REPORT_F01             558    12
```

```
                                ZPRA_DPR_REPORT_F01          953    19
DISPLAY_SECTION3A............. ZPRA_DPR_REPORT_F01          575    12
                                ZPRA_DPR_REPORT_F01          973    19
DISPLAY_SECTION3B............. ZPRA_DPR_REPORT_F01          580    12
                                ZPRA_DPR_REPORT_F01         1007    20
DISPLAY_SECTION3C............. ZPRA_DPR_REPORT_F01          586    12
                                ZPRA_DPR_REPORT_F01         1075    21
DISPLAY_SECTION3D............. ZPRA_DPR_REPORT_F01          590    12
                                ZPRA_DPR_REPORT_F01         1418    27
DISPLAY_SECTION3E............. ZPRA_DPR_REPORT_F01          594    12
                                ZPRA_DPR_REPORT_F01         1441    27
DISPLAY_SECTION3F............. ZPRA_DPR_REPORT_F01          600    13
                                ZPRA_DPR_REPORT_F01         1394    26
DISPLAY_SECTION3_HEADER....... ZPRA_DPR_REPORT_F01          571    12
                                ZPRA_DPR_REPORT_F01         9404   176
DISPLAY_SECTION3_HEADER_1..... ZPRA_DPR_REPORT_F01         9405   176
                                ZPRA_DPR_REPORT_F01         9416   176
DISPLAY_SECTION3_HEADER_2..... ZPRA_DPR_REPORT_F01         9406   176
                                ZPRA_DPR_REPORT_F01         9458   177
DISPLAY_SECTION4A............. ZPRA_DPR_REPORT_F01          618    13
                                ZPRA_DPR_REPORT_F01         1093    21
DISPLAY_SECTION4B............. ZPRA_DPR_REPORT_F01          619    13
                                ZPRA_DPR_REPORT_F01         1128    22
DISPLAY_SECTION5A............. ZPRA_DPR_REPORT_F01          632    13
                                ZPRA_DPR_REPORT_F01         1178    23
DISPLAY_SECTION6............... ZPRA_DPR_REPORT_F01          638    13
                                ZPRA_DPR_REPORT_F01         1256    24
DISPLAY_UNITS................... ZPRA_DPR_REPORT_F01         1529    28
                                ZPRA_DPR_REPORT_F01         3117    57
DOWNLOAD_IMAGE................. ZPRA_DPR_REPORT_F01         2624    48
                                ZPRA_DPR_REPORT_F01         2638    48
FETCH_DATA..................... ZPRA_DPR_REPORT               13     2
                                ZPRA_DPR_REPORT_F01           13     3
FETCH_DATA_SECTION2A3......... ZPRA_DPR_REPORT_F01         4730    88
                                ZPRA_DPR_REPORT_F01         6614   122
FETCH_DATA_SECTION2B........... ZPRA_DPR_REPORT_F01          532    11
                                ZPRA_DPR_REPORT_F01         6447   119
FETCH_DATA_SECTION2C........... ZPRA_DPR_REPORT_F01          537    12
                                ZPRA_DPR_REPORT_F01         6600   122
FETCH_DATA_SECTION2D........... ZPRA_DPR_REPORT_F01          541    12
                                ZPRA_DPR_REPORT_F01         6703   124
FETCH_DATA_SECTION2F........... ZPRA_DPR_REPORT_F01          554    12
                                ZPRA_DPR_REPORT_F01         6892   127
FETCH_DATA_SECTION3C........... ZPRA_DPR_REPORT_F01          582    12
                                ZPRA_DPR_REPORT_F01         9627   180
FETCH_DATA_SECTION3F........... ZPRA_DPR_REPORT_F01          596    12
                                ZPRA_DPR_REPORT_F01         9806   183
FETCH_DATA_SECTION4A........... ZPRA_DPR_REPORT_F01          613    13
                                ZPRA_DPR_REPORT_F01        10026   187
FETCH_DATA_SECTION5A........... ZPRA_DPR_REPORT_F01          629    13
                                ZPRA_DPR_REPORT_F01        10668   199
FETCH_SECTION2A2_DATA......... ZPRA_DPR_REPORT_F01          445    10
                                ZPRA_DPR_REPORT_F01         6381   118
FILL_DYNAMIC_TABLE_SEC1....... ZPRA_DPR_REPORT_F01          516    11
                                ZPRA_DPR_REPORT_F01          762    15
FILL_DYNAMIC_TABLE_SEC2A1..... ZPRA_DPR_REPORT_F01         4720    88
                                ZPRA_DPR_REPORT_F01         4899    91
FILL_DYNAMIC_TABLE_SEC2A2..... ZPRA_DPR_REPORT_F01         4727    88
                                ZPRA_DPR_REPORT_F01         5199    97
FILL_DYNAMIC_TABLE_SEC2A3..... ZPRA_DPR_REPORT_F01         4733    88
                                ZPRA_DPR_REPORT_F01         5388   100
```

```
FILL_DYNAMIC_TABLE_SEC2B...... ZPRA_DPR_REPORT_F01          4999    93
                               ZPRA_DPR_REPORT_F01          6523   120
FILL_DYNAMIC_TABLE_SEC2C...... ZPRA_DPR_REPORT_F01          5100    95
                               ZPRA_DPR_REPORT_F01          6542   121
FILL_DYNAMIC_TABLE_SEC2D...... ZPRA_DPR_REPORT_F01           544    12
                               ZPRA_DPR_REPORT_F01          7857   144
FILL_DYNAMIC_TABLE_SEC2E...... ZPRA_DPR_REPORT_F01           551    12
                               ZPRA_DPR_REPORT_F01          8522   158
FILL_DYNAMIC_TABLE_SEC2F...... ZPRA_DPR_REPORT_F01           557    12
                               ZPRA_DPR_REPORT_F01          8202   152
FILL_DYNAMIC_TABLE_SEC3A...... ZPRA_DPR_REPORT_F01           574    12
                               ZPRA_DPR_REPORT_F01          9511   178
FILL_DYNAMIC_TABLE_SEC3B...... ZPRA_DPR_REPORT_F01           579    12
                               ZPRA_DPR_REPORT_F01          9569   179
FILL_DYNAMIC_TABLE_SEC3C...... ZPRA_DPR_REPORT_F01           585    12
                               ZPRA_DPR_REPORT_F01          8605   160
FILL_DYNAMIC_TABLE_SEC3D...... ZPRA_DPR_REPORT_F01           589    12
                               ZPRA_DPR_REPORT_F01          9252   173
FILL_DYNAMIC_TABLE_SEC3E...... ZPRA_DPR_REPORT_F01           593    12
                               ZPRA_DPR_REPORT_F01          9301   174
FILL_DYNAMIC_TABLE_SEC3F...... ZPRA_DPR_REPORT_F01           599    13
                               ZPRA_DPR_REPORT_F01          8863   165
FILL_DYNAMIC_TABLE_SEC4A...... ZPRA_DPR_REPORT_F01           616    13
                               ZPRA_DPR_REPORT_F01          9127   171
FILL_DYNAMIC_TABLE_SEC5A...... ZPRA_DPR_REPORT_F01           631    13
                               ZPRA_DPR_REPORT_F01         10272   191
FILL_DYNAMIC_TABLE_SEC6....... ZPRA_DPR_REPORT_F01           637    13
                               ZPRA_DPR_REPORT_F01         10393   194
FILL_DYNAMIC_TABLE_SEC6A...... ZPRA_DPR_REPORT_F01         10394   194
                               ZPRA_DPR_REPORT_F01         10405   194
FILL_DYNAMIC_TABLE_SEC6B...... ZPRA_DPR_REPORT_F01         10395   194
                               ZPRA_DPR_REPORT_F01         10491   195
FILL_NULL_VALUES_WITH_PREVIOUS ZPRA_DPR_REPORT_F01           778    16
                               ZPRA_DPR_REPORT_F01           834    17
                               ZPRA_DPR_REPORT_F01          2401    44
FINALIZE_WORKSHEET............ ZPRA_DPR_REPORT_F01           499    11
                               ZPRA_DPR_REPORT_F01          7788   143
FORMATTING_SECTION1........... ZPRA_DPR_REPORT_F01           877    17
                               ZPRA_DPR_REPORT_F01          1566    29
FORMAT_SEC1_DATA_COL_1_2...... ZPRA_DPR_REPORT_F01          1569    29
                               ZPRA_DPR_REPORT_F01          1741    32
FORMAT_SEC1_HEADER............ ZPRA_DPR_REPORT_F01          1568    29
                               ZPRA_DPR_REPORT_F01          1733    32
FREE_CLIPBOARD................ ZPRA_DPR_REPORT_F01          7795   143
                               ZPRA_DPR_REPORT_F01         11279   209
GET_ACHIEVEMENT_6C............ ZPRA_DPR_REPORT_F01         10505   196
                               ZPRA_DPR_REPORT_F01         10621   198
GET_ANNUAL_TARGETS_6B......... ZPRA_DPR_REPORT_F01         10503   196
                               ZPRA_DPR_REPORT_F01         10508   196
GET_ASSET_START_DATE.......... ZPRA_DPR_REPORT_F01          5324    99
                               ZPRA_DPR_REPORT_F01          5559   104
                               ZPRA_DPR_REPORT_F01          8133   150
                               ZPRA_DPR_REPORT_F01          8462   157
                               ZPRA_DPR_REPORT_F01          8568   159
                               ZPRA_DPR_REPORT_F01         11211   208
GET_CF_FROM_DATE.............. ZPRA_DPR_REPORT_F01          3714    67
                               ZPRA_DPR_REPORT_F01          3727    67
                               ZPRA_DPR_REPORT_F01          3742    67
                               ZPRA_DPR_REPORT_F01          3808    69
                               ZPRA_DPR_REPORT_F01          4023    73
                               ZPRA_DPR_REPORT_F01          4078    74
```

	ZPRA_DPR_REPORT_F01	4091	75
	ZPRA_DPR_REPORT_F01	4106	75
	ZPRA_DPR_REPORT_F01	11457	212
	ZPRA_DPR_REPORT_F01	11535	214
	ZPRA_DPR_REPORT_F01	11548	214
	ZPRA_DPR_REPORT_F01	11561	214
GET_CONSTORIUM_MULTIPLIERS....	ZPRA_DPR_REPORT_F01	804	16
	ZPRA_DPR_REPORT_F01	4365	81
GET_CONSTORIUM_MULTIPLIERS_2A2	ZPRA_DPR_REPORT_F01	805	16
	ZPRA_DPR_REPORT_F01	4427	82
	ZPRA_DPR_REPORT_F01	5235	97
	ZPRA_DPR_REPORT_F01	5281	98
	ZPRA_DPR_REPORT_F01	5470	102
	ZPRA_DPR_REPORT_F01	5514	103
	ZPRA_DPR_REPORT_F01	8078	149
	ZPRA_DPR_REPORT_F01	8415	156
	ZPRA_DPR_REPORT_F01	8764	163
	ZPRA_DPR_REPORT_F01	9024	169
	ZPRA_DPR_REPORT_F01	9220	173
GET_DAYS_IN_PERIOD.....	ZPRA_DPR_REPORT_F01	7986	147
	ZPRA_DPR_REPORT_F01	8057	149
	ZPRA_DPR_REPORT_F01	8328	154
	ZPRA_DPR_REPORT_F01	8396	156
	ZPRA_DPR_REPORT_F01	8642	161
	ZPRA_DPR_REPORT_F01	8678	161
	ZPRA_DPR_REPORT_F01	8941	167
	ZPRA_DPR_REPORT_F01	9183	172
	ZPRA_DPR_REPORT_F01	9209	173
	ZPRA_DPR_REPORT_F01	9352	175
GET_FULL_NUMBER_FORMAT.....	ZPRA_DPR_REPORT_F01	10826	201
	ZPRA_DPR_REPORT_F01	11164	207
GET_PI_FOR_NULL_VALUES.....	ZPRA_DPR_REPORT_F01	2447	45
	ZPRA_DPR_REPORT_F01	4323	80
GET_TARGET_FROM_NAME.....	ZPRA_DPR_REPORT_F01	4969	92
	ZPRA_DPR_REPORT_F01	5068	94
	ZPRA_DPR_REPORT_F01	5169	96
	ZPRA_DPR_REPORT_F01	6012	111
	ZPRA_DPR_REPORT_F01	8545	159
GET_TARGET_START_DATE.....	ZPRA_DPR_REPORT_F01	4979	92
	ZPRA_DPR_REPORT_F01	5078	94
	ZPRA_DPR_REPORT_F01	5178	96
	ZPRA_DPR_REPORT_F01	8561	159
	ZPRA_DPR_REPORT_F01	11246	208
GET_TARGET_TYPE_NAME.....	ZPRA_DPR_REPORT_F01	4915	91
	ZPRA_DPR_REPORT_F01	5015	93
	ZPRA_DPR_REPORT_F01	5116	95
	ZPRA_DPR_REPORT_F01	5986	111
	ZPRA_DPR_REPORT_F01	9522	178
	ZPRA_DPR_REPORT_F01	9580	179
GET_UNIT_DESC.....	ZPRA_DPR_REPORT_F01	1285	24
	ZPRA_DPR_REPORT_F01	1295	25
	ZPRA_DPR_REPORT_F01	3139	57
	ZPRA_DPR_REPORT_F01	3185	58
GET_YTD_TARGETS_6B.....	ZPRA_DPR_REPORT_F01	10504	196
	ZPRA_DPR_REPORT_F01	10560	197
JOIN_HEADER1_COLUMN_1_2.....	ZPRA_DPR_REPORT_F01	1531	29
	ZPRA_DPR_REPORT_F01	4664	87
JOIN_HEADER_TOTAL_CELLS.....	ZPRA_DPR_REPORT_F01	1530	29
	ZPRA_DPR_REPORT_F01	4651	86
LOCK_XLS.....	ZPRA_DPR_REPORT_F01	7796	143
	ZPRA_DPR_REPORT_F01	7839	144

MERGE_COL_1_2_SECTION1.....	ZPRA_DPR_REPORT_F01	1567	29
	ZPRA_DPR_REPORT_F01	1718	32
PASTE_DATA.....	ZPRA_DPR_REPORT_F01	892	18
	ZPRA_DPR_REPORT_F01	916	18
	ZPRA_DPR_REPORT_F01	940	18
	ZPRA_DPR_REPORT_F01	964	19
	ZPRA_DPR_REPORT_F01	992	19
	ZPRA_DPR_REPORT_F01	1025	20
	ZPRA_DPR_REPORT_F01	1058	21
	ZPRA_DPR_REPORT_F01	1086	21
	ZPRA_DPR_REPORT_F01	1119	22
	ZPRA_DPR_REPORT_F01	1162	22
	ZPRA_DPR_REPORT_F01	1405	26
	ZPRA_DPR_REPORT_F01	1429	27
	ZPRA_DPR_REPORT_F01	1452	27
	ZPRA_DPR_REPORT_F01	1554	29
	ZPRA_DPR_REPORT_F01	2508	46
	ZPRA_DPR_REPORT_F01	2945	54
	ZPRA_DPR_REPORT_F01	3005	55
	ZPRA_DPR_REPORT_F01	3062	56
	ZPRA_DPR_REPORT_F01	6059	112
PASTE_DATA_SHEET2.....	ZPRA_DPR_REPORT_F01	1194	23
	ZPRA_DPR_REPORT_F01	2569	47
PASTE_DATA_SHEET3.....	ZPRA_DPR_REPORT_F01	1374	26
	ZPRA_DPR_REPORT_F01	2538	47
POPULATE_NO_DATA_ENTRIES.....	ZPRA_DPR_REPORT_F01	1983	36
	ZPRA_DPR_REPORT_F01	2352	43
	ZPRA_DPR_REPORT_F01	4726	88
	ZPRA_DPR_REPORT_F01	6411	119
	ZPRA_DPR_REPORT_F01	6631	122
	ZPRA_DPR_REPORT_F01	6848	126
	ZPRA_DPR_REPORT_F01	7027	129
	ZPRA_DPR_REPORT_F01	9759	182
	ZPRA_DPR_REPORT_F01	10157	189
	ZPRA_DPR_REPORT_F01	10681	199
	ZPRA_DPR_REPORT_F01	11299	209
POPULATE_NO_DATA_ENTRIES_3F...	ZPRA_DPR_REPORT_F01	10000	187
	ZPRA_DPR_REPORT_F01	11584	215
PREPARE_DYNAMIC_TABLE_SEC1....	ZPRA_DPR_REPORT_F01	515	11
	ZPRA_DPR_REPORT_F01	649	13
PREPARE_DYNAMIC_TABLE_SEC2A1..	ZPRA_DPR_REPORT_F01	4719	88
	ZPRA_DPR_REPORT_F01	5618	105
PREPARE_DYNAMIC_TABLE_SEC2A2..	ZPRA_DPR_REPORT_F01	4725	88
	ZPRA_DPR_REPORT_F01	5643	105
PREPARE_DYNAMIC_TABLE_SEC2A3..	ZPRA_DPR_REPORT_F01	4732	88
	ZPRA_DPR_REPORT_F01	5710	106
PREPARE_DYNAMIC_TABLE_SEC2B...	ZPRA_DPR_REPORT_F01	533	11
	ZPRA_DPR_REPORT_F01	5668	106
PREPARE_DYNAMIC_TABLE_SEC2C...	ZPRA_DPR_REPORT_F01	538	12
	ZPRA_DPR_REPORT_F01	5685	106
PREPARE_DYNAMIC_TABLE_SEC2D...	ZPRA_DPR_REPORT_F01	543	12
	ZPRA_DPR_REPORT_F01	5735	107
PREPARE_DYNAMIC_TABLE_SEC2E...	ZPRA_DPR_REPORT_F01	550	12
	ZPRA_DPR_REPORT_F01	5961	111
PREPARE_DYNAMIC_TABLE_SEC2F...	ZPRA_DPR_REPORT_F01	556	12
	ZPRA_DPR_REPORT_F01	5760	107
PREPARE_DYNAMIC_TABLE_SEC3A...	ZPRA_DPR_REPORT_F01	573	12
	ZPRA_DPR_REPORT_F01	5785	108
PREPARE_DYNAMIC_TABLE_SEC3B...	ZPRA_DPR_REPORT_F01	577	12
	ZPRA_DPR_REPORT_F01	5810	108
PREPARE_DYNAMIC_TABLE_SEC3C...	ZPRA_DPR_REPORT_F01	584	12

```
                                ZPRA_DPR_REPORT_F01          5835   108
PREPARE_DYNAMIC_TABLE_SEC3D... ZPRA_DPR_REPORT_F01           588    12
                                ZPRA_DPR_REPORT_F01          5860   109
PREPARE_DYNAMIC_TABLE_SEC3E... ZPRA_DPR_REPORT_F01           592    12
                                ZPRA_DPR_REPORT_F01          5885   109
PREPARE_DYNAMIC_TABLE_SEC3F... ZPRA_DPR_REPORT_F01           598    13
                                ZPRA_DPR_REPORT_F01          5910   110
PREPARE_DYNAMIC_TABLE_SEC4A... ZPRA_DPR_REPORT_F01           615    13
                                ZPRA_DPR_REPORT_F01          5935   110
PREPARE_DYNAMIC_TABLE_SEC5A... ZPRA_DPR_REPORT_F01           628    13
                                ZPRA_DPR_REPORT_F01          10185  190
PREPARE_DYNAMIC_TABLE_SEC6.... ZPRA_DPR_REPORT_F01           636    13
                                ZPRA_DPR_REPORT_F01          10237  191
PREPARE_SECTION1_PASTE_DATA... ZPRA_DPR_REPORT_F01           1552    29
                                ZPRA_DPR_REPORT_F01          1753    32
PREPARE_SECTION2A2_PASTE_DATA. ZPRA_DPR_REPORT_F01           890    18
                                ZPRA_DPR_REPORT_F01          1774    33
PREPARE_SECTION2A3_PASTE_DATA. ZPRA_DPR_REPORT_F01           914    18
                                ZPRA_DPR_REPORT_F01          1795    33
PREPARE_SECTION2A_PASTE_DATA.. ZPRA_DPR_REPORT_F01           6055   112
                                ZPRA_DPR_REPORT_F01          6073   112
PREPARE_SECTION2D_PASTE_DATA.. ZPRA_DPR_REPORT_F01           938    18
                                ZPRA_DPR_REPORT_F01          1816    33
PREPARE_SECTION2E_PASTE_DATA.. ZPRA_DPR_REPORT_F01           1054    21
                                ZPRA_DPR_REPORT_F01          1858    34
PREPARE_SECTION2F_PASTE_DATA.. ZPRA_DPR_REPORT_F01           962    19
                                ZPRA_DPR_REPORT_F01          1837    34
PREPARE_SECTION3A_PASTE_DATA.. ZPRA_DPR_REPORT_F01           988    19
                                ZPRA_DPR_REPORT_F01          6094   113
PREPARE_SECTION3B_PASTE_DATA.. ZPRA_DPR_REPORT_F01           1021    20
                                ZPRA_DPR_REPORT_F01          6115   113
PREPARE_SECTION3C_PASTE_DATA.. ZPRA_DPR_REPORT_F01           1084    21
                                ZPRA_DPR_REPORT_F01          6136   114
PREPARE_SECTION3D_PASTE_DATA.. ZPRA_DPR_REPORT_F01           1427    27
                                ZPRA_DPR_REPORT_F01          6331   117
PREPARE_SECTION3E_PASTE_DATA.. ZPRA_DPR_REPORT_F01           1450    27
                                ZPRA_DPR_REPORT_F01          6352   117
PREPARE_SECTION3F_PASTE_DATA.. ZPRA_DPR_REPORT_F01           1403    26
                                ZPRA_DPR_REPORT_F01          6157   114
PREPARE_SECTION4A_PASTE_DATA.. ZPRA_DPR_REPORT_F01           1117    22
                                ZPRA_DPR_REPORT_F01          6178   114
PREPARE_SECTION4B_PASTE_DATA.. ZPRA_DPR_REPORT_F01           1136    22
                                ZPRA_DPR_REPORT_F01          6199   115
PREPARE_SECTION5A_PASTE_DATA.. ZPRA_DPR_REPORT_F01           1192    23
                                ZPRA_DPR_REPORT_F01          6290   116
PREPARE_SECTION6_PASTE_DATA... ZPRA_DPR_REPORT_F01           1265    24
                                ZPRA_DPR_REPORT_F01          6310   117
PROCESS_DATA................... ZPRA_DPR_REPORT               15     2
                                ZPRA_DPR_REPORT_F01           475    10
PROCESS_GAS_RECORDS........... ZPRA_DPR_REPORT               14     2
                                ZPRA_DPR_REPORT_F01          2289    42
PROCESS_GAS_RECORDS_2A3....... ZPRA_DPR_REPORT_F01           4731    88
                                ZPRA_DPR_REPORT_F01          7078   130
PROCESS_GAS_RECORDS_2D........ ZPRA_DPR_REPORT_F01           542    12
                                ZPRA_DPR_REPORT_F01          7172   132
PROCESS_GAS_RECORDS_2F........ ZPRA_DPR_REPORT_F01           555    12
                                ZPRA_DPR_REPORT_F01          7612   140
PROCESS_GAS_RECORDS_3C........ ZPRA_DPR_REPORT_F01           583    12
                                ZPRA_DPR_REPORT_F01          7314   134
PROCESS_GAS_RECORDS_3F........ ZPRA_DPR_REPORT_F01           597    13
                                ZPRA_DPR_REPORT_F01          7415   136
```

PROCESS_GAS_RECORDS_4A.....	ZPRA_DPR_REPORT_F01	614	13
	ZPRA_DPR_REPORT_F01	7516	138
PROCESS_GAS_RECORDS_5A.....	ZPRA_DPR_REPORT_F01	630	13
	ZPRA_DPR_REPORT_F01	10741	200
PROCESS_SEC1_DATA.....	ZPRA_DPR_REPORT_F01	478	11
	ZPRA_DPR_REPORT_F01	509	11
PROCESS_SEC2A_DATA.....	ZPRA_DPR_REPORT_F01	529	11
	ZPRA_DPR_REPORT_F01	4715	88
PROCESS_SEC2B_DATA.....	ZPRA_DPR_REPORT_F01	534	11
	ZPRA_DPR_REPORT_F01	6519	120
PROCESS_SEC2C_DATA.....	ZPRA_DPR_REPORT_F01	539	12
	ZPRA_DPR_REPORT_F01	6539	121
PROCESS_SEC2_DATA.....	ZPRA_DPR_REPORT_F01	481	11
	ZPRA_DPR_REPORT_F01	527	11
PROCESS_SEC3B_DATA.....	ZPRA_DPR_REPORT_F01	578	12
	ZPRA_DPR_REPORT_F01	6556	121
PROCESS_SEC3_DATA.....	ZPRA_DPR_REPORT_F01	483	11
	ZPRA_DPR_REPORT_F01	569	12
PROCESS_SEC4_DATA.....	ZPRA_DPR_REPORT_F01	485	11
	ZPRA_DPR_REPORT_F01	612	13
PROCESS_SEC5A_DATA.....	ZPRA_DPR_REPORT_F01	625	13
	ZPRA_DPR_REPORT_F01	627	13
PROCESS_SEC5_DATA.....	ZPRA_DPR_REPORT_F01	489	11
	ZPRA_DPR_REPORT_F01	624	13
PROCESS_SEC6_DATA.....	ZPRA_DPR_REPORT_F01	491	11
	ZPRA_DPR_REPORT_F01	635	13
REMOVE_EXPIRED_BLOCKS.....	ZPRA_DPR_REPORT_F01	112	4
	ZPRA_DPR_REPORT_F01	11398	211
ROW_HEIGHT.....	ZPRA_DPR_REPORT_F01	1709	31
	ZPRA_DPR_REPORT_F01	1738	32
	ZPRA_DPR_REPORT_F01	1739	32
	ZPRA_DPR_REPORT_F01	10851	202
SELECT_CELL.....	ZPRA_DPR_REPORT_F01	1579	29
	ZPRA_DPR_REPORT_F01	2808	51
	ZPRA_DPR_REPORT_F01	3087	56
	ZPRA_DPR_REPORT_F01	3128	57
SELECT_RANGE.....	ZPRA_DPR_REPORT_F01	891	18
	ZPRA_DPR_REPORT_F01	898	18
	ZPRA_DPR_REPORT_F01	901	18
	ZPRA_DPR_REPORT_F01	915	18
	ZPRA_DPR_REPORT_F01	922	18
	ZPRA_DPR_REPORT_F01	939	18
	ZPRA_DPR_REPORT_F01	949	19
	ZPRA_DPR_REPORT_F01	963	19
	ZPRA_DPR_REPORT_F01	989	19
	ZPRA_DPR_REPORT_F01	1000	20
	ZPRA_DPR_REPORT_F01	1022	20
	ZPRA_DPR_REPORT_F01	1033	20
	ZPRA_DPR_REPORT_F01	1055	21
	ZPRA_DPR_REPORT_F01	1069	21
	ZPRA_DPR_REPORT_F01	1085	21
	ZPRA_DPR_REPORT_F01	1099	21
	ZPRA_DPR_REPORT_F01	1106	21
	ZPRA_DPR_REPORT_F01	1118	22
	ZPRA_DPR_REPORT_F01	1121	22
	ZPRA_DPR_REPORT_F01	1132	22
	ZPRA_DPR_REPORT_F01	1141	22
	ZPRA_DPR_REPORT_F01	1144	22
	ZPRA_DPR_REPORT_F01	1147	22
	ZPRA_DPR_REPORT_F01	1151	22
	ZPRA_DPR_REPORT_F01	1161	22

ZPRA_DPR_REPORT_F01	1164	22
ZPRA_DPR_REPORT_F01	1167	22
ZPRA_DPR_REPORT_F01	1193	23
ZPRA_DPR_REPORT_F01	1277	24
ZPRA_DPR_REPORT_F01	1287	24
ZPRA_DPR_REPORT_F01	1297	25
ZPRA_DPR_REPORT_F01	1322	25
ZPRA_DPR_REPORT_F01	1330	25
ZPRA_DPR_REPORT_F01	1335	25
ZPRA_DPR_REPORT_F01	1340	25
ZPRA_DPR_REPORT_F01	1345	25
ZPRA_DPR_REPORT_F01	1350	26
ZPRA_DPR_REPORT_F01	1355	26
ZPRA_DPR_REPORT_F01	1358	26
ZPRA_DPR_REPORT_F01	1373	26
ZPRA_DPR_REPORT_F01	1384	26
ZPRA_DPR_REPORT_F01	1391	26
ZPRA_DPR_REPORT_F01	1404	26
ZPRA_DPR_REPORT_F01	1411	27
ZPRA_DPR_REPORT_F01	1428	27
ZPRA_DPR_REPORT_F01	1435	27
ZPRA_DPR_REPORT_F01	1451	27
ZPRA_DPR_REPORT_F01	1458	27
ZPRA_DPR_REPORT_F01	1553	29
ZPRA_DPR_REPORT_F01	1563	29
ZPRA_DPR_REPORT_F01	1607	30
ZPRA_DPR_REPORT_F01	1720	32
ZPRA_DPR_REPORT_F01	1724	32
ZPRA_DPR_REPORT_F01	1729	32
ZPRA_DPR_REPORT_F01	1736	32
ZPRA_DPR_REPORT_F01	1742	32
ZPRA_DPR_REPORT_F01	2777	51
ZPRA_DPR_REPORT_F01	2944	54
ZPRA_DPR_REPORT_F01	3004	55
ZPRA_DPR_REPORT_F01	3008	55
ZPRA_DPR_REPORT_F01	3061	56
ZPRA_DPR_REPORT_F01	3066	56
ZPRA_DPR_REPORT_F01	3095	56
ZPRA_DPR_REPORT_F01	3103	56
ZPRA_DPR_REPORT_F01	3112	57
ZPRA_DPR_REPORT_F01	3136	57
ZPRA_DPR_REPORT_F01	3146	57
ZPRA_DPR_REPORT_F01	3171	58
ZPRA_DPR_REPORT_F01	3310	60
ZPRA_DPR_REPORT_F01	3342	60
ZPRA_DPR_REPORT_F01	3373	61
ZPRA_DPR_REPORT_F01	3377	61
ZPRA_DPR_REPORT_F01	3391	61
ZPRA_DPR_REPORT_F01	4616	86
ZPRA_DPR_REPORT_F01	4637	86
ZPRA_DPR_REPORT_F01	4655	86
ZPRA_DPR_REPORT_F01	4660	87
ZPRA_DPR_REPORT_F01	4665	87
ZPRA_DPR_REPORT_F01	4688	87
ZPRA_DPR_REPORT_F01	4703	87
ZPRA_DPR_REPORT_F01	6056	112
ZPRA_DPR_REPORT_F01	6067	112
ZPRA_DPR_REPORT_F01	7812	144
ZPRA_DPR_REPORT_F01	8824	165
ZPRA_DPR_REPORT_F01	8842	165
ZPRA_DPR_REPORT_F01	9088	170

	ZPRA_DPR_REPORT_F01	9106	170
	ZPRA_DPR_REPORT_F01	9426	176
	ZPRA_DPR_REPORT_F01	9436	177
	ZPRA_DPR_REPORT_F01	9441	177
	ZPRA_DPR_REPORT_F01	9469	177
	ZPRA_DPR_REPORT_F01	9477	177
	ZPRA_DPR_REPORT_F01	9490	177
	ZPRA_DPR_REPORT_F01	9493	178
	ZPRA_DPR_REPORT_F01	10801	201
	ZPRA_DPR_REPORT_F01	10810	201
	ZPRA_DPR_REPORT_F01	10827	201
	ZPRA_DPR_REPORT_F01	10833	202
	ZPRA_DPR_REPORT_F01	10838	202
	ZPRA_DPR_REPORT_F01	10841	202
	ZPRA_DPR_REPORT_F01	10844	202
	ZPRA_DPR_REPORT_F01	10848	202
	ZPRA_DPR_REPORT_F01	11381	211
SET_ALL_BORDERS_RANGE.....	ZPRA_DPR_REPORT_F01	1153	22
	ZPRA_DPR_REPORT_F01	1165	22
	ZPRA_DPR_REPORT_F01	1365	26
	ZPRA_DPR_REPORT_F01	1375	26
	ZPRA_DPR_REPORT_F01	10802	201
	ZPRA_DPR_REPORT_F01	10811	201
	ZPRA_DPR_REPORT_F01	10900	203
SET_BORDER_RANGE.....	ZPRA_DPR_REPORT_F01	10869	202
SET_CELL.....	ZPRA_DPR_REPORT_F01	1591	30
	ZPRA_DPR_REPORT_F01	2809	51
	ZPRA_DPR_REPORT_F01	3088	56
	ZPRA_DPR_REPORT_F01	3129	57
SET_CELL_FORMATS.....	ZPRA_DPR_REPORT_F01	498	11
	ZPRA_DPR_REPORT_F01	10822	201
SET_COLUMNS_WIDTH.....	ZPRA_DPR_REPORT_F01	495	11
	ZPRA_DPR_REPORT_F01	4597	85
SET_NUMBERFORMAT.....	ZPRA_DPR_REPORT_F01	899	18
	ZPRA_DPR_REPORT_F01	923	18
	ZPRA_DPR_REPORT_F01	1122	22
	ZPRA_DPR_REPORT_F01	1386	26
	ZPRA_DPR_REPORT_F01	1564	29
	ZPRA_DPR_REPORT_F01	1730	32
	ZPRA_DPR_REPORT_F01	3009	55
	ZPRA_DPR_REPORT_F01	3067	56
	ZPRA_DPR_REPORT_F01	3359	61
	ZPRA_DPR_REPORT_F01	10828	201
	ZPRA_DPR_REPORT_F01	10834	202
	ZPRA_DPR_REPORT_F01	10839	202
SET_RANGE.....	ZPRA_DPR_REPORT_F01	903	18
	ZPRA_DPR_REPORT_F01	951	19
	ZPRA_DPR_REPORT_F01	1003	20
	ZPRA_DPR_REPORT_F01	1036	20
	ZPRA_DPR_REPORT_F01	1072	21
	ZPRA_DPR_REPORT_F01	1101	21
	ZPRA_DPR_REPORT_F01	1108	21
	ZPRA_DPR_REPORT_F01	1133	22
	ZPRA_DPR_REPORT_F01	1142	22
	ZPRA_DPR_REPORT_F01	1145	22
	ZPRA_DPR_REPORT_F01	1149	22
	ZPRA_DPR_REPORT_F01	1279	24
	ZPRA_DPR_REPORT_F01	1289	25
	ZPRA_DPR_REPORT_F01	1299	25
	ZPRA_DPR_REPORT_F01	1324	25
	ZPRA_DPR_REPORT_F01	1331	25

```
                                  ZPRA_DPR_REPORT_F01            1336    25
                                  ZPRA_DPR_REPORT_F01            1341    25
                                  ZPRA_DPR_REPORT_F01            1346    25
                                  ZPRA_DPR_REPORT_F01            1351    26
                                  ZPRA_DPR_REPORT_F01            1356    26
                                  ZPRA_DPR_REPORT_F01            1414    27
                                  ZPRA_DPR_REPORT_F01            1438    27
                                  ZPRA_DPR_REPORT_F01            1461    27
                                  ZPRA_DPR_REPORT_F01            1630    30
                                  ZPRA_DPR_REPORT_F01            2779    51
                                  ZPRA_DPR_REPORT_F01            3097    56
                                  ZPRA_DPR_REPORT_F01            3104    56
                                  ZPRA_DPR_REPORT_F01            3113    57
                                  ZPRA_DPR_REPORT_F01            3141    57
                                  ZPRA_DPR_REPORT_F01            3147    57
                                  ZPRA_DPR_REPORT_F01            3172    58
                                  ZPRA_DPR_REPORT_F01            3313    60
                                  ZPRA_DPR_REPORT_F01            6069   112
                                  ZPRA_DPR_REPORT_F01            8825   165
                                  ZPRA_DPR_REPORT_F01            8843   165
                                  ZPRA_DPR_REPORT_F01            9089   170
                                  ZPRA_DPR_REPORT_F01            9107   170
                                  ZPRA_DPR_REPORT_F01            9429   176
                                  ZPRA_DPR_REPORT_F01            9439   177
                                  ZPRA_DPR_REPORT_F01            9472   177
                                  ZPRA_DPR_REPORT_F01            9484   177
                                  ZPRA_DPR_REPORT_F01            9491   177
                                  ZPRA_DPR_REPORT_F01            11384  211
SET_RANGE_FONT................... ZPRA_DPR_REPORT_F01            1134    22
                                  ZPRA_DPR_REPORT_F01            1152    22
                                  ZPRA_DPR_REPORT_F01            1359    26
                                  ZPRA_DPR_REPORT_F01            1642    30
                                  ZPRA_DPR_REPORT_F01            11386  211
SET_RANGE_FORMATTING.......... ZPRA_DPR_REPORT_F01               1364    26
                                  ZPRA_DPR_REPORT_F01            1385    26
                                  ZPRA_DPR_REPORT_F01            1649    30
                                  ZPRA_DPR_REPORT_F01            1737    32
                                  ZPRA_DPR_REPORT_F01            1743    32
                                  ZPRA_DPR_REPORT_F01            2781    51
                                  ZPRA_DPR_REPORT_F01            3098    56
                                  ZPRA_DPR_REPORT_F01            3105    56
                                  ZPRA_DPR_REPORT_F01            3114    57
                                  ZPRA_DPR_REPORT_F01            3142    57
                                  ZPRA_DPR_REPORT_F01            3148    57
                                  ZPRA_DPR_REPORT_F01            3173    58
                                  ZPRA_DPR_REPORT_F01            10842  202
                                  ZPRA_DPR_REPORT_F01            10845  202
                                  ZPRA_DPR_REPORT_F01            10849  202
                                  ZPRA_DPR_REPORT_F01            11385  211
SET_RANGE_INTERIOR............ ZPRA_DPR_REPORT_F01               1103    21
                                  ZPRA_DPR_REPORT_F01            1110    22
                                  ZPRA_DPR_REPORT_F01            2782    51
                                  ZPRA_DPR_REPORT_F01            3343    60
                                  ZPRA_DPR_REPORT_F01            3355    61
                                  ZPRA_DPR_REPORT_F01            3374    61
                                  ZPRA_DPR_REPORT_F01            3389    61
                                  ZPRA_DPR_REPORT_F01            3392    61
                                  ZPRA_DPR_REPORT_F01            4700    87
                                  ZPRA_DPR_REPORT_F01            4704    87
SET_SECTION1_HEADER_COLORS.... ZPRA_DPR_REPORT_F01               1532    29
                                  ZPRA_DPR_REPORT_F01            3371    61
```

SET_THIN_BORDER.....	ZPRA_DPR_REPORT_F01	1168	22
	ZPRA_DPR_REPORT_F01	1679	31
SHOW_PROGRESS.....	ZPRA_DPR_REPORT_F01	633	13
	ZPRA_DPR_REPORT_F01	1463	27
	ZPRA_DPR_REPORT_F01	2628	48
	ZPRA_DPR_REPORT_F01	6528	121
	ZPRA_DPR_REPORT_F01	6846	126
	ZPRA_DPR_REPORT_F01	7067	130
	ZPRA_DPR_REPORT_F01	8883	166
	ZPRA_DPR_REPORT_F01	8886	166
	ZPRA_DPR_REPORT_F01	8889	166
	ZPRA_DPR_REPORT_F01	10012	187
	ZPRA_DPR_REPORT_F01	11148	207
START_EXCEL.....	ZPRA_DPR_REPORT_F01	476	10
	ZPRA_DPR_REPORT_F01	1474	28
WRITE_PRODUCT_NAME.....	ZPRA_DPR_REPORT_F01	2825	52
	ZPRA_DPR_REPORT_F01	3288	60

Messages:

		Line	Page
?.....	ZPRA_DPR_REPORT_F01	51	3
?.....	ZPRA_DPR_REPORT_F01	62	4
?.....	ZPRA_DPR_REPORT_F01	76	4
?.....	ZPRA_DPR_REPORT_F01	93	4
?.....	ZPRA_DPR_REPORT_F01	109	4
?.....	ZPRA_DPR_REPORT_F01	180	6
?.....	ZPRA_DPR_REPORT_F01	218	6
?.....	ZPRA_DPR_REPORT_F01	234	6
?.....	ZPRA_DPR_REPORT_F01	254	7
?.....	ZPRA_DPR_REPORT_F01	284	7
?.....	ZPRA_DPR_REPORT_F01	302	8
?.....	ZPRA_DPR_REPORT_F01	320	8
?.....	ZPRA_DPR_REPORT_F01	724	15
?.....	ZPRA_DPR_REPORT_F01	788	16
?.....	ZPRA_DPR_REPORT_F01	2696	49
?.....	ZPRA_DPR_REPORT_F01	4803	89
?.....	ZPRA_DPR_REPORT_F01	4860	90
?.....	ZPRA_DPR_REPORT_F01	5629	105
?.....	ZPRA_DPR_REPORT_F01	5654	106
?.....	ZPRA_DPR_REPORT_F01	5679	106
?.....	ZPRA_DPR_REPORT_F01	5696	106
?.....	ZPRA_DPR_REPORT_F01	5721	107
?.....	ZPRA_DPR_REPORT_F01	5746	107
?.....	ZPRA_DPR_REPORT_F01	5771	107
?.....	ZPRA_DPR_REPORT_F01	5796	108
?.....	ZPRA_DPR_REPORT_F01	5821	108
?.....	ZPRA_DPR_REPORT_F01	5846	109
?.....	ZPRA_DPR_REPORT_F01	5871	109
?.....	ZPRA_DPR_REPORT_F01	5896	109
?.....	ZPRA_DPR_REPORT_F01	5921	110
?.....	ZPRA_DPR_REPORT_F01	5946	110
?.....	ZPRA_DPR_REPORT_F01	5972	111
?.....	ZPRA_DPR_REPORT_F01	6583	121
?.....	ZPRA_DPR_REPORT_F01	6797	125
?.....	ZPRA_DPR_REPORT_F01	6814	126
?.....	ZPRA_DPR_REPORT_F01	6979	128
?.....	ZPRA_DPR_REPORT_F01	6996	129
?.....	ZPRA_DPR_REPORT_F01	7275	134

```
?............................ ZPRA_DPR_REPORT_F01          7292   134
?............................ ZPRA_DPR_REPORT_F01          7715   142
?............................ ZPRA_DPR_REPORT_F01          7732   142
?............................ ZPRA_DPR_REPORT_F01          7804   144
?............................ ZPRA_DPR_REPORT_F01          7829   144
?............................ ZPRA_DPR_REPORT_F01          9373   176
?............................ ZPRA_DPR_REPORT_F01          9390   176
?............................ ZPRA_DPR_REPORT_F01          9715   182
?............................ ZPRA_DPR_REPORT_F01          9732   182
?............................ ZPRA_DPR_REPORT_F01          9931   185
?............................ ZPRA_DPR_REPORT_F01          9956   186
?............................ ZPRA_DPR_REPORT_F01          10101  188
?............................ ZPRA_DPR_REPORT_F01          10118  189
?............................ ZPRA_DPR_REPORT_F01          10223  190
?............................ ZPRA_DPR_REPORT_F01          10258  191
?............................ ZPRA_DPR_REPORT_F01          10295  192
?............................ ZPRA_DPR_REPORT_F01          10312  192
?............................ ZPRA_DPR_REPORT_F01          10332  192
?............................ ZPRA_DPR_REPORT_F01          11513  213
?............................ ZPRA_DPR_REPORT_TOP           427   225
?............................ ZPRA_DPR_REPORT_TOP           430   225
?............................ ZPRA_DPR_REPORT_TOP           461   226
```

External calls

Function modules:		Line	Page
CCM_GO_BACK_MONTHS.....	ZPRA_DPR_REPORT_F01	114	4
	ZPRA_DPR_REPORT_F01	206	6
CONVERSION_EXIT_SDATE_OUTPUT..	ZPRA_DPR_REPORT_F01	30	3
	ZPRA_DPR_REPORT_F01	782	16
	ZPRA_DPR_REPORT_F01	2767	51
	ZPRA_DPR_REPORT_F01	6208	115
	ZPRA_DPR_REPORT_F01	6256	116
	ZPRA_DPR_REPORT_F01	6264	116
	ZPRA_DPR_REPORT_F01	7798	143
	ZPRA_DPR_REPORT_F01	10289	192
	ZPRA_DPR_REPORT_F01	11366	211
DATE_TO_PERIOD_CONVERT.....	ZPRA_DPR_REPORT_F01	79	4
	ZPRA_DPR_REPORT_F01	220	6
	ZPRA_DPR_REPORT_F01	288	7
	ZPRA_DPR_REPORT_F01	306	8
	ZPRA_DPR_REPORT_F01	3676	66
	ZPRA_DPR_REPORT_F01	3774	68
	ZPRA_DPR_REPORT_F01	3991	73
	ZPRA_DPR_REPORT_F01	10298	192
	ZPRA_DPR_REPORT_F01	11466	213
	ZPRA_DPR_REPORT_F01	11499	213
FIRST_AND_LAST_DAY_IN_YEAR_GET	ZPRA_DPR_REPORT_F01	96	4
	ZPRA_DPR_REPORT_F01	241	7
	ZPRA_DPR_REPORT_F01	271	7
FIRST_DAY_IN_PERIOD_GET.....	ZPRA_DPR_REPORT_F01	6783	125
	ZPRA_DPR_REPORT_F01	6965	128
	ZPRA_DPR_REPORT_F01	7261	133
	ZPRA_DPR_REPORT_F01	7701	142
	ZPRA_DPR_REPORT_F01	9359	175
	ZPRA_DPR_REPORT_F01	9701	181
	ZPRA_DPR_REPORT_F01	9917	185
	ZPRA_DPR_REPORT_F01	10087	188
HR_JP_MONTH_BEGIN_END_DATE....	ZPRA_DPR_REPORT_F01	121	5
	ZPRA_DPR_REPORT_F01	129	5
	ZPRA_DPR_REPORT_F01	6393	118
	ZPRA_DPR_REPORT_F01	7215	133
	ZPRA_DPR_REPORT_F01	7655	141
LAST_DAY_IN_PERIOD_GET.....	ZPRA_DPR_REPORT_F01	6800	125
	ZPRA_DPR_REPORT_F01	6982	128
	ZPRA_DPR_REPORT_F01	7278	134
	ZPRA_DPR_REPORT_F01	7718	142
	ZPRA_DPR_REPORT_F01	9376	176
	ZPRA_DPR_REPORT_F01	9718	182
	ZPRA_DPR_REPORT_F01	9942	186
	ZPRA_DPR_REPORT_F01	10104	188
PERIOD_DAY_DETERMINE.....	ZPRA_DPR_REPORT_F01	4786	89
	ZPRA_DPR_REPORT_F01	4843	90
	ZPRA_DPR_REPORT_F01	6566	121
	ZPRA_DPR_REPORT_F01	10315	192
SAPGUI_PROGRESS_INDICATOR.....	ZPRA_DPR_REPORT_F01	11151	207
SAPSCRIPT_CONVERT_BITMAP.....	ZPRA_DPR_REPORT_F01	2667	49
SAPSCRIPT_GET_GRAPHIC_BDS.....	ZPRA_DPR_REPORT_F01	2651	49
WS_DOWNLOAD.....	ZPRA_DPR_REPORT_F01	2680	49

DATABASE OPERATIONS

SELECT:		Line	Page
T001.....	ZPRA_DPR_REPORT_F01	72	4
	ZPRA_DPR_REPORT_F01	214	6
T247.....	ZPRA_DPR_REPORT_F01	138	5
	ZPRA_DPR_REPORT_F01	144	5
ZOIU_PR_DN.....	ZPRA_DPR_REPORT_F01	377	9
ZPRA_C_DPR_PROF.....	ZPRA_DPR_REPORT_F01	46	3
ZPRA_C_PRD_PROF.....	ZPRA_DPR_REPORT_F01	56	3
ZPRA_DLY_PRD_ND.....	ZPRA_DPR_REPORT_F01	9978	186
ZPRA_T_AST_PI.....	ZPRA_DPR_REPORT_F01	2121	39
	ZPRA_DPR_REPORT_F01	2136	39
ZPRA_T_DLY_PRD.....	ZPRA_DPR_REPORT_F01	169	5
	ZPRA_DPR_REPORT_F01	6401	118
	ZPRA_DPR_REPORT_F01	6621	122
	ZPRA_DPR_REPORT_F01	6834	126
	ZPRA_DPR_REPORT_F01	7017	129
	ZPRA_DPR_REPORT_F01	9750	182
	ZPRA_DPR_REPORT_F01	9878	184
	ZPRA_DPR_REPORT_F01	10147	189
	ZPRA_DPR_REPORT_F01	10671	199
ZPRA_T_DLY_RPRD.....	ZPRA_DPR_REPORT_F01	6853	126
	ZPRA_DPR_REPORT_F01	7034	129
	ZPRA_DPR_REPORT_F01	9765	183
	ZPRA_DPR_REPORT_F01	9889	185
	ZPRA_DPR_REPORT_F01	10137	189
ZPRA_T_MREC_APP.....	ZPRA_DPR_REPORT_F01	6666	123
	ZPRA_DPR_REPORT_F01	6727	124
	ZPRA_DPR_REPORT_F01	6940	128
	ZPRA_DPR_REPORT_F01	9666	181
	ZPRA_DPR_REPORT_F01	9847	184
ZPRA_T_MREC_PRD.....	ZPRA_DPR_REPORT_F01	337	8
	ZPRA_DPR_REPORT_F01	358	8
	ZPRA_DPR_REPORT_F01	6644	123
	ZPRA_DPR_REPORT_F01	6752	124
	ZPRA_DPR_REPORT_F01	6917	127
	ZPRA_DPR_REPORT_F01	9646	180
	ZPRA_DPR_REPORT_F01	9828	184
	ZPRA_DPR_REPORT_F01	10045	187
	ZPRA_DPR_REPORT_F01	10696	199
ZPRA_T_PRD_PI.....	ZPRA_DPR_REPORT_F01	196	6
	ZPRA_DPR_REPORT_F01	2896	53
	ZPRA_DPR_REPORT_F01	6229	115
	ZPRA_DPR_REPORT_F01	6232	115
	ZPRA_DPR_REPORT_F01	6427	119
	ZPRA_DPR_REPORT_F01	6684	123
	ZPRA_DPR_REPORT_F01	6873	127
	ZPRA_DPR_REPORT_F01	7058	130
	ZPRA_DPR_REPORT_F01	9784	183
	ZPRA_DPR_REPORT_F01	9867	184
	ZPRA_DPR_REPORT_F01	10167	190
	ZPRA_DPR_REPORT_F01	10715	199
	ZPRA_DPR_REPORT_F01	11410	211
ZPRA_T_PRD_TAR.....	ZPRA_DPR_REPORT_F01	411	9
	ZPRA_DPR_REPORT_F01	6483	120
ZPRA_T_TAR_CF.....	ZPRA_DPR_REPORT_F01	424	10
	ZPRA_DPR_REPORT_F01	436	10
	ZPRA_DPR_REPORT_F01	6499	120

	ZPRA_DPR_REPORT_F01	9679	181
	ZPRA_DPR_REPORT_F01	10724	200
ZPRA_T_TAR_PI.....	ZPRA_DPR_REPORT_F01	393	9
	ZPRA_DPR_REPORT_F01	450	10
	ZPRA_DPR_REPORT_F01	6464	119

MODIFY:		Line	Page
GT_WTD_PI.....	ZPRA_DPR_REPORT_F01	2143	39

INTERNAL TABLE OPERATIONS

READ TABLE:		Line	Page
<GFS_DYN_TABLE>.....	ZPRA_DPR_REPORT_F01	2419	45
<GFS_SEC2D_TABLE>.....	ZPRA_DPR_REPORT_F01	8533	159
	ZPRA_DPR_REPORT_F01	8827	165
	ZPRA_DPR_REPORT_F01	8844	165
	ZPRA_DPR_REPORT_F01	10408	194
<GFS_SEC2F_TABLE>.....	ZPRA_DPR_REPORT_F01	9091	170
	ZPRA_DPR_REPORT_F01	9108	170
<GFS_SEC3C_TABLE>.....	ZPRA_DPR_REPORT_F01	9256	173
	ZPRA_DPR_REPORT_F01	9306	174
<GFS_SEC3F_TABLE>.....	ZPRA_DPR_REPORT_F01	9077	170
<GFS_SEC6_TABLE>.....	ZPRA_DPR_REPORT_F01	10494	195
GT_ASSET_DESC.....	ZPRA_DPR_REPORT_F01	2887	53
	ZPRA_DPR_REPORT_F01	2917	53
	ZPRA_DPR_REPORT_F01	6213	115
	ZPRA_DPR_REPORT_F01	6271	116
	ZPRA_DPR_REPORT_F01	9152	171
GT_CF.....	ZPRA_DPR_REPORT_F01	11480	213
GT_DYN_FCAT.....	ZPRA_DPR_REPORT_F01	848	17
	ZPRA_DPR_REPORT_F01	2427	45
	ZPRA_DPR_REPORT_F01	5366	100
	ZPRA_DPR_REPORT_F01	5596	105
	ZPRA_DPR_REPORT_F01	6297	116
	ZPRA_DPR_REPORT_F01	8180	151
	ZPRA_DPR_REPORT_F01	8499	158
	ZPRA_DPR_REPORT_F01	11221	208
	ZPRA_DPR_REPORT_F01	11255	209
GT_PRODUCT_COL.....	ZPRA_DPR_REPORT_F01	10831	202
GT_PROD_START_END_DATES.....	ZPRA_DPR_REPORT_F01	11420	212
GT_TAR_START_DATES.....	ZPRA_DPR_REPORT_F01	11261	209
GT_WTD_CF.....	ZPRA_DPR_REPORT_F01	2259	42
	ZPRA_DPR_REPORT_F01	3039	55
	ZPRA_DPR_REPORT_F01	3636	65
GT_WTD_PI.....	ZPRA_DPR_REPORT_F01	2072	38
	ZPRA_DPR_REPORT_F01	2077	38
	ZPRA_DPR_REPORT_F01	2096	39
	ZPRA_DPR_REPORT_F01	2975	54
	ZPRA_DPR_REPORT_F01	2982	54
	ZPRA_DPR_REPORT_F01	4327	80
	ZPRA_DPR_REPORT_F01	4332	80
GT_ZDPR_GAS_COMBINE.....	ZPRA_DPR_REPORT_F01	685	14
	ZPRA_DPR_REPORT_F01	799	16
	ZPRA_DPR_REPORT_F01	2336	43
	ZPRA_DPR_REPORT_F01	2345	43
	ZPRA_DPR_REPORT_F01	2439	45
	ZPRA_DPR_REPORT_F01	4921	91
	ZPRA_DPR_REPORT_F01	5021	93
	ZPRA_DPR_REPORT_F01	5122	95
	ZPRA_DPR_REPORT_F01	5230	97
	ZPRA_DPR_REPORT_F01	5276	98
	ZPRA_DPR_REPORT_F01	5422	101
	ZPRA_DPR_REPORT_F01	5465	102
	ZPRA_DPR_REPORT_F01	5509	103
	ZPRA_DPR_REPORT_F01	7902	145
	ZPRA_DPR_REPORT_F01	8020	148
	ZPRA_DPR_REPORT_F01	8073	149
	ZPRA_DPR_REPORT_F01	8249	153

```
                                  ZPRA_DPR_REPORT_F01            8359   155
                                  ZPRA_DPR_REPORT_F01            8410   156
                                  ZPRA_DPR_REPORT_F01            8646   161
                                  ZPRA_DPR_REPORT_F01            8714   162
                                  ZPRA_DPR_REPORT_F01            8759   163
                                  ZPRA_DPR_REPORT_F01            8909   166
                                  ZPRA_DPR_REPORT_F01            8974   168
                                  ZPRA_DPR_REPORT_F01            9019   169
                                  ZPRA_DPR_REPORT_F01            9143   171
                                  ZPRA_DPR_REPORT_F01            9528   178
                                  ZPRA_DPR_REPORT_F01            9586   179
GT_ZPRA_T_DLY_RPRD_2D......... ZPRA_DPR_REPORT_F01            7990   147
GT_ZPRA_T_DLY_RPRD_2F......... ZPRA_DPR_REPORT_F01            8329   154
GT_ZPRA_T_DLY_RPRD_3C......... ZPRA_DPR_REPORT_F01            8683   161
GT_ZPRA_T_DLY_RPRD_3F......... ZPRA_DPR_REPORT_F01            8943   167
GT_ZPRA_T_MREC_APP............ ZPRA_DPR_REPORT_F01            8634   160
                                  ZPRA_DPR_REPORT_F01            9695   181
GT_ZPRA_T_MREC_APP_2A3........ ZPRA_DPR_REPORT_F01            5414   101
GT_ZPRA_T_MREC_APP_2D......... ZPRA_DPR_REPORT_F01            6777   125
                                  ZPRA_DPR_REPORT_F01            7894   145
GT_ZPRA_T_MREC_APP_2F......... ZPRA_DPR_REPORT_F01            6959   128
                                  ZPRA_DPR_REPORT_F01            8241   152
GT_ZPRA_T_MREC_APP_3F......... ZPRA_DPR_REPORT_F01            8901   166
GT_ZPRA_T_MREC_APP_4A......... ZPRA_DPR_REPORT_F01            9160   171
                                  ZPRA_DPR_REPORT_F01            10081  188
GT_ZPRA_T_MREC_PRD............ ZPRA_DPR_REPORT_F01            4066    74
GT_ZPRA_T_MREC_PRD_2A3........ ZPRA_DPR_REPORT_F01            5456   102
GT_ZPRA_T_MREC_PRD_3F......... ZPRA_DPR_REPORT_F01            9911   185
GT_ZPRA_T_PRD_PI_3F........... ZPRA_DPR_REPORT_F01            9934   185
                                  ZPRA_DPR_REPORT_F01            11603  215
GT_ZPRA_T_TAR_CF.............. ZPRA_DPR_REPORT_F01            4125    75
                                  ZPRA_DPR_REPORT_F01            4173    76
                                  ZPRA_DPR_REPORT_F01            4221    78
                                  ZPRA_DPR_REPORT_F01            4269    79
LT_ZPRA_C_PRD_PROF............ ZPRA_DPR_REPORT_F01             461    10
LT_ZPRA_T_PRD_PI.............. ZPRA_DPR_REPORT_F01            11229  208
PT_ZPRA_T_DLY_PRD............. ZPRA_DPR_REPORT_F01            11189  207
P_ZPRA_T_DLY_PRD.............. ZPRA_DPR_REPORT_F01            11322  210
                                  ZPRA_DPR_REPORT_F01            11327  210
                                  ZPRA_DPR_REPORT_F01            11331  210
                                  ZPRA_DPR_REPORT_F01            11615  215
                                  ZPRA_DPR_REPORT_F01            11620  215
                                  ZPRA_DPR_REPORT_F01            11624  215
```

```
DELETE:                                                          Line   Page

GT_PROD_START_END_DATES....... ZPRA_DPR_REPORT_F01            11417  212
GT_TAR_START_DATES............ ZPRA_DPR_REPORT_F01             463    10
GT_WTD_PI..................... ZPRA_DPR_REPORT_F01            2093    39
                                  ZPRA_DPR_REPORT_F01            2152    40
GT_ZPRA_C_PRD_PROF............ ZPRA_DPR_REPORT_F01              65     4
GT_ZPRA_T_DLY_PRD............. ZPRA_DPR_REPORT_F01            2299    42
                                  ZPRA_DPR_REPORT_F01            2347    43
GT_ZPRA_T_DLY_PRD2............ ZPRA_DPR_REPORT_F01             512    11
GT_ZPRA_T_DLY_PRD_2A3......... ZPRA_DPR_REPORT_F01            7095   130
GT_ZPRA_T_DLY_PRD_2D.......... ZPRA_DPR_REPORT_F01            7191   132
GT_ZPRA_T_DLY_PRD_2F.......... ZPRA_DPR_REPORT_F01            7631   140
GT_ZPRA_T_DLY_PRD_3C.......... ZPRA_DPR_REPORT_F01            7336   135
GT_ZPRA_T_DLY_PRD_3F.......... ZPRA_DPR_REPORT_F01            7437   137
```

```
GT_ZPRA_T_DLY_PRD_4A.......... ZPRA_DPR_REPORT_F01          7535   139
GT_ZPRA_T_DLY_PRD_5A.......... ZPRA_DPR_REPORT_F01          10752  200
GT_ZPRA_T_DLY_PRD_MB.......... ZPRA_DPR_REPORT_F01          2362    44
GT_ZPRA_T_MREC_APP_4A......... ZPRA_DPR_REPORT_F01          7571   139
GT_ZPRA_T_MREC_PRD............ ZPRA_DPR_REPORT_F01           371     9
                               ZPRA_DPR_REPORT_F01           372     9
                               ZPRA_DPR_REPORT_F01           384     9
GT_ZPRA_T_MREC_PRD_2A3........ ZPRA_DPR_REPORT_F01          6655   123
                               ZPRA_DPR_REPORT_F01          7131   131
GT_ZPRA_T_MREC_PRD_2D......... ZPRA_DPR_REPORT_F01          6761   125
                               ZPRA_DPR_REPORT_F01          7236   133
GT_ZPRA_T_MREC_PRD_2F......... ZPRA_DPR_REPORT_F01          7676   141
GT_ZPRA_T_MREC_PRD_3C......... ZPRA_DPR_REPORT_F01          7372   136
                               ZPRA_DPR_REPORT_F01          9657   181
GT_ZPRA_T_MREC_PRD_3F......... ZPRA_DPR_REPORT_F01          7472   137
GT_ZPRA_T_PRD_TAR............. ZPRA_DPR_REPORT_F01          6601   122
GT_ZPRA_T_PRD_TAR_2C.......... ZPRA_DPR_REPORT_F01          6602   122
GT_ZPRA_T_PRD_TAR_3B.......... ZPRA_DPR_REPORT_F01          6562   121
LT_DLY_PRD_MAIN............... ZPRA_DPR_REPORT_F01          7881   145
                               ZPRA_DPR_REPORT_F01          8226   152
LT_EXP_ASSET.................. ZPRA_DPR_REPORT_F01          6242   115
                               ZPRA_DPR_REPORT_F01          6246   116
                               ZPRA_DPR_REPORT_F01          6248   116
LT_WTD_PI..................... ZPRA_DPR_REPORT_F01          2154    40
                               ZPRA_DPR_REPORT_F01          2155    40
LT_ZPRA_C_PRD_PROF............ ZPRA_DPR_REPORT_F01           663    14
                               ZPRA_DPR_REPORT_F01           666    14
                               ZPRA_DPR_REPORT_F01          2067    38
                               ZPRA_DPR_REPORT_F01          11312  210
                               ZPRA_DPR_REPORT_F01          11597  215
LT_ZPRA_C_PRD_PROF2........... ZPRA_DPR_REPORT_F01           667    14
LT_ZPRA_T_DLY_PRD............. ZPRA_DPR_REPORT_F01          2300    42
                               ZPRA_DPR_REPORT_F01          2333    43
                               ZPRA_DPR_REPORT_F01          2338    43
                               ZPRA_DPR_REPORT_F01          2363    44
                               ZPRA_DPR_REPORT_F01          5404   101
                               ZPRA_DPR_REPORT_F01          6419   119
                               ZPRA_DPR_REPORT_F01          7096   130
                               ZPRA_DPR_REPORT_F01          7192   132
                               ZPRA_DPR_REPORT_F01          7337   135
                               ZPRA_DPR_REPORT_F01          7438   137
                               ZPRA_DPR_REPORT_F01          7536   139
                               ZPRA_DPR_REPORT_F01          7632   140
                               ZPRA_DPR_REPORT_F01          10753  200
LT_ZPRA_T_DLY_PRD_CURRENT..... ZPRA_DPR_REPORT_F01          6389   118
LT_ZPRA_T_MREC_APP............ ZPRA_DPR_REPORT_F01          7572   139
LT_ZPRA_T_MREC_PRD............ ZPRA_DPR_REPORT_F01          7132   131
                               ZPRA_DPR_REPORT_F01          7237   133
                               ZPRA_DPR_REPORT_F01          7373   136
                               ZPRA_DPR_REPORT_F01          7473   137
                               ZPRA_DPR_REPORT_F01          7677   141
LT_ZPRA_T_PRD_PI.............. ZPRA_DPR_REPORT_F01          11227  208
P_ZPRA_C_PRD_PROF............. ZPRA_DPR_REPORT_F01          11431  212
R_PRODUCTION_DATE[]........... ZPRA_DPR_REPORT_F01          6831   126
                               ZPRA_DPR_REPORT_F01          7013   129
                               ZPRA_DPR_REPORT_F01          9746   182
                               ZPRA_DPR_REPORT_F01          9973   186
                               ZPRA_DPR_REPORT_F01          10133  189
```